TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN HỌC PHẦN

PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

PHÂN TÍCH, THIẾT KẾ VÀ ĐÁNH GIÁ WEBSITE STRONGWINE

Giảng viên hướng dẫn: Th.S. Ngô Tân Khai

Sinh viên thực hiện: Nguyễn Thành Duy Tân

MSSV: 1250080266

Lớp: 12_ĐH_CNPM3

TP. Hồ Chí Minh, tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN HỌC PHẦN

PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

PHÂN TÍCH, THIẾT KẾ VÀ ĐÁNH GIÁ WEBSITE STRONGWINE

Giảng viên hướng dẫn: Th.S. Ngô Tân Khai

Sinh viên thực hiện: Nguyễn Thành Duy Tân

MSSV: 1250080266

Lớp: 12_ĐH_CNPM3

TP. Hồ Chí Minh, tháng 10 năm 2026

LỜI CẢM ƠN

Tiểu luận giúp em hệ thống hóa kiến thức và vận dụng tư duy hướng đối tượng vào hệ thống có nhiều vai trò, dữ liệu, nghiệp vụ. Qua khảo sát, em nhận thấy phân tích cần vượt khỏi danh sách màn hình: phải lần theo trách nhiệm giữa các lớp, trạng thái dữ liệu và giới hạn của bằng chứng. Nhờ đó, em hiểu rõ hơn mối liên hệ giữa lý thuyết học phần và thực hành phát triển phần mềm.

Em trân trọng cảm ơn Trường Đại học Tài nguyên và Môi trường Thành phố Hồ Chí Minh, Khoa Công nghệ Thông tin và quý thầy cô đã tạo môi trường học tập, trang bị kiến thức nền tảng. Kiến thức về yêu cầu, mô hình hóa, lập trình hướng đối tượng, thiết kế và kiểm thử là cơ sở để em thực hiện bài này; mỗi học phần cũng góp phần hình thành tư duy, thái độ nghề nghiệp.

Em chân thành cảm ơn Th.S. Ngô Tân Khai, giảng viên học phần Phát triển phần mềm hướng đối tượng. Hướng dẫn của thầy về mục tiêu, cách trình bày và liên hệ lý thuyết với sản phẩm giúp em xác định phạm vi, tổ chức nội dung và chú trọng tính chính xác. Em trân trọng những chỉ dẫn này và xem đó là động lực rèn luyện kỹ năng phân tích, thiết kế và trình bày hệ thống.

Khi khảo sát StrongWine, em tìm hiểu cách các thành phần phối hợp phục vụ khách hàng, quản trị viên, nhân viên kho và shipper. Đối chiếu mã nguồn với giao diện giúp em hiểu vai trò của controller, service, repository, mô hình miền, cơ sở dữ liệu và xử lý bất đồng bộ. Tài liệu kỹ thuật được ghi trong danh mục tham khảo hoặc chú thích để phân biệt thông tin dự án với nhận xét của người viết.

Em cảm ơn gia đình, bạn bè và những người đã động viên, hỗ trợ em trong thời gian học tập. Sự chia sẻ, góp ý thiết thực giúp em tập trung rà soát nội dung, sắp xếp hình minh họa và hoàn thiện tài liệu theo yêu cầu học phần. Em luôn trân trọng những hỗ trợ này.

Do thời gian, kinh nghiệm và phạm vi khảo sát có hạn, tiểu luận có thể chưa bao quát mọi khía cạnh của hệ thống. Em mong nhận được góp ý của thầy để nhận diện điểm chưa chính xác, củng cố lập luận và cải thiện kỹ năng mô hình hóa. Em sẽ tiếp thu nghiêm túc, đối chiếu lại thông tin và điều chỉnh cách trình bày khi cần.

Em xin cảm ơn thầy hướng dẫn, quý thầy cô, nhà trường, gia đình và bạn bè đã hỗ trợ em hoàn thành tiểu luận. Kính chúc quý thầy cô sức khỏe và thành công trong công tác giảng dạy.

LỜI CAM ĐOAN

Em xin cam đoan tiểu luận này được thực hiện cho học phần Phát triển phần mềm hướng đối tượng, với đề tài phân tích, thiết kế và đánh giá website StrongWine. Nhận xét về chức năng, mô hình miền, kiến trúc và nghiệp vụ được hình thành từ khảo sát mã nguồn cùng giao diện trong phạm vi trình bày. Kết luận phản ánh bằng chứng đã xem xét tại thời điểm thực hiện, không mở rộng thành khẳng định về chức năng hoặc kết quả chưa được xác minh.

Thông tin về Trường Đại học Tài nguyên và Môi trường TP. Hồ Chí Minh và Khoa Công nghệ Thông tin được đối chiếu với tài liệu do sinh viên cung cấp. Hình trong bài gồm wireframe được dựng từ các MVC GET view và template đã kiểm tra, cùng một số ảnh giao diện có sẵn trong bộ tài liệu dự án. Các ảnh có sẵn chưa được chụp lại từ runtime trong lượt hiệu đính này.

Quan điểm, mô hình, hình ảnh và thông tin từ dự án hoặc nguồn khác được dẫn nguồn hay chú thích theo phạm vi đã kiểm tra. Sơ đồ tham khảo từ tài liệu trước không được xem là bằng chứng rằng mọi chức năng và nhãn vẫn còn nguyên trạng; nội dung đã được đối chiếu với mã nguồn hiện tại và nêu giới hạn khi cần. Mô tả về thanh toán, giao nhận, email, OTP, dữ liệu và tích hợp ngoài chỉ xác nhận điều quan sát được; phần chưa kiểm chứng được giữ là rủi ro hoặc đề xuất.

WordAssistant được dùng để hỗ trợ tổ chức nội dung, định dạng và kiểm tra cấu trúc DOCX; công cụ AI hỗ trợ soạn thảo, biên tập một số đoạn và rà soát cách diễn đạt. Em đã đọc lại, đối chiếu thông tin, chỉnh sửa nội dung và chịu trách nhiệm về bản tiểu luận nộp. Các công cụ hỗ trợ không thay thế nguồn xác nhận tính đúng đắn của phân tích, bằng chứng hay kết quả kiểm thử.

Tại thời điểm hoàn thành, ứng dụng đã được biên dịch và giao diện các vai trò nêu trong bài đã được quan sát, chụp ảnh. Bộ kiểm thử tự động và quy trình giao dịch đầu cuối chưa được thực hiện đầy đủ; vì vậy bài không tuyên bố các luồng thanh toán, email, OTP, giao nhận hoặc dữ liệu production đã được nghiệm thu. Nội dung chưa xác minh được được nêu ở phần giới hạn, rủi ro và kế hoạch cải tiến.

Em cam kết trình bày trung thực theo phạm vi bằng chứng thu thập, không nhận là kết quả của bản thân những việc chưa thực hiện hoặc nguồn chưa kiểm tra. Nếu phát hiện sai sót về nội dung, trích dẫn, hình minh họa hay diễn đạt, em sẽ tiếp thu và điều chỉnh theo góp ý của giảng viên và quy định nhà trường. Em chịu trách nhiệm về thông tin cá nhân và nội dung cuối cùng của bản nộp.

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Họ và tên giảng viên hướng dẫn: Th.S. Ngô Tân Khai

Sinh viên thực hiện: Nguyễn Thành Duy Tân

MSSV: 1250080266 Lớp: 12_ĐH_CNPM3

Tên đề tài: Phân tích, thiết kế và đánh giá hệ thống thương mại điện tử StrongWine theo mô hình phát triển phần mềm hướng đối tượng.

1. Về tiến độ thực hiện và tinh thần thái độ học tập:

.....

2. Về kết cấu và phương pháp trình bày báo cáo:

.....

3. Về cơ sở lý luận và tính ứng dụng thực tiễn:

.....

4. Kết luận đánh giá: .....

TP. Hồ Chí Minh, ngày ..... tháng 10 năm 2026

GIẢNG VIÊN HƯỚNG DẪN

Th.S. Ngô Tân Khai

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Họ và tên giảng viên phản biện: .....
Sinh viên thực hiện: Nguyễn Thành Duy Tân
MSSV: 1250080266
1. Nhận xét về tính cấp thiết và giá trị khoa học của đề tài:
.....
.....
2. Nhận xét về kết quả đạt được và độ chính xác của các mô hình phân tích:
.....
.....
3. Các câu hỏi cần làm rõ trong buổi bảo vệ:
- Câu hỏi 1:
.....
- Câu hỏi 2:
.....
4. Đánh giá chung:
.....
TP. Hồ Chí Minh, ngày ..... tháng 10 năm 2026

GIẢNG VIÊN PHẢN BIỆN

(Ký và ghi rõ họ tên)

LỊCH TRÌNH THỰC HIỆN TIỂU LUẬN

Bảng 0.1: Kế hoạch và tiến độ thực hiện đề tài qua các tuần

Tuần	Nội dung công việc thực hiện	Phương pháp / Công cụ	Sản phẩm bàn giao	Xác nhận GV
Tuần 1-2	Khảo sát hiện trạng, thu thập yêu cầu nghiệp vụ và xác định ranh giới hệ thống StrongWine.	Phỏng vấn nghiệp vụ, phân tích tĩnh mã nguồn	Báo cáo đề cương & 80 FRs	
Tuần 3-4	Xây dựng mô hình hướng đối tượng: Sơ đồ Use Case, Lớp phân tầng và Thực thể miền JPA.	PlantUML, StarUML, IntelliJ IDEA	Sơ đồ UML & Entity Model	
Tuần 5-6	Thiết kế CSDL quan hệ, đối soát 19 bảng và thiết kế chi tiết luồng Customer, Admin, Shipper.	MySQL Workbench, Flyway migration	DDL Schema & 19 bảng CSDL	
Tuần 7-8	Phân tích mẫu thiết kế (GoF), đặc tả quy trình Checkout, OTP và cơ chế đồng bộ SSE.	Java 17, Spring Boot, Stripe API	Sơ đồ Activity & Sequence	
Tuần 9-10	Khảo sát giao diện UI/UX, đánh giá tính tiện dụng và kiểm thử hộp trắng/hộp đen.	Selenium, JUnit 5, Mockito	Ma trận Test & 27 UI Mockup	
Tuần 11-12	Hoàn thiện báo cáo tiểu luận, kiểm định toàn vẹn dữ liệu WordAssistant và nghiệm thu.	WordAssistant, Microsoft Word	Quyển tiểu luận hoàn chỉnh	

TÓM TẮT ĐỒ ÁN

Tiểu luận phân tích và đánh giá website thương mại điện tử StrongWine theo phương pháp phát triển phần mềm hướng đối tượng. Báo cáo khảo sát ứng dụng Spring Boot, cơ sở dữ liệu MySQL và các nhóm chức năng dành cho khách hàng, quản trị viên, nhân viên kho và nhân viên giao hàng. Các yêu cầu, mô hình UML và một số nguyên lý, mẫu thiết kế được đối chiếu với mã nguồn để giải thích cách tổ chức trách nhiệm và luồng nghiệp vụ. Bằng chứng giao diện, mô hình và kiểm thử được trình bày theo phạm vi đã thu thập; việc đối chiếu thiết kế và mã nguồn không tự chứng minh độ tin cậy, khả năng mở rộng hay sự hoàn thành của mọi yêu cầu.

Từ khóa: Phát triển phần mềm hướng đối tượng; OOAD; Spring Boot; MySQL; StrongWine; Sơ đồ UML; Mô hình hóa thực thể.

ABSTRACT

This report analyzes the StrongWine e-commerce website using object-oriented software development concepts. It examines the Spring Boot application, its MySQL database, and functionality for customers, administrators, warehouse staff, and delivery personnel. Requirements, UML models, and selected design principles and patterns are compared with the source code to explain responsibilities and business flows. The interface, models, and tests are presented within the limits of the available evidence; a design and source review alone does not establish reliability, scalability, or completion of every requirement.

Keywords: Object-Oriented Software Development; OOAD; Spring Boot; MySQL; StrongWine; UML Modeling; Domain Entity Architecture.

MỤC LỤC

LỜI CẢM ƠN 2
LỜI CAM ĐOAN 3
NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN 4
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN 5
LỊCH TRÌNH THỰC HIỆN TIỂU LUẬN 6
TÓM TẮT ĐỒ ÁN 7
ABSTRACT 7
MỤC LỤC 8
DANH MỤC HÌNH ẢNH 12
DANH MỤC BẢNG BIỂU 14
DANH MỤC TỪ VIẾT TẮT 15
MỞ ĐẦU 1
1.1.1 Mục tiêu, câu hỏi và phạm vi khảo sát 1
1.1.2 Phương pháp và quy ước bằng chứng 1
NỘI DUNG 3
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT 3
2.1 Đối tượng, lớp và nguyên tắc trách nhiệm 3
2.2 IoC, dependency injection và cộng tác lớp 3
2.3 Mẫu thiết kế và tiêu chí nhận diện 5
2.4 UML như ngôn ngữ mô hình hóa 5
2.5 Tổng quan về ngôn ngữ lập trình Java 6
2.5.1 Đặc điểm nổi bật của Java trong hệ thống 7
2.5.2 Java và phát triển web 7
2.5.3 Quản lý phụ thuộc và tổ chức package 8
2.6 Tổng quan về Spring Boot và mô hình web của StrongWine 8
2.6.1 JSP trong tài liệu cũ và Thymeleaf hiện tại 9
2.6.2 Spring Boot, dependency injection và giao dịch 9
2.6.3 Validation, binding và xử lý lỗi web 10
2.7 Tổng quan về môi trường phát triển 10
2.8 Tổng quan về cơ sở dữ liệu quan hệ 11
2.9 Tổng quan về hệ quản trị cơ sở dữ liệu Microsoft SQL Server 11
2.10 Tổng quan về máy chủ Tomcat 12
2.11 Các nguyên lý hướng đối tượng làm nền cho phân tích 12
2.11.1 Đơn trách nhiệm và phân lớp 13
2.11.2 Mở đóng, thay thế Liskov và phân tách interface 13
CHƯƠNG 2: KHẢO SÁT TỔNG QUAN STRONGWINE 15
3.1 Ranh giới kiến trúc và thành phần 15
3.2 Tác nhân, mục tiêu và kết quả mong đợi 15
3.3 Yêu cầu chức năng theo phân hệ 17
3.4 Yêu cầu phi chức năng và điều kiện vận hành 17
3.5 Tổng quan nghiệp vụ của đề tài 18
3.5.1 Nghiệp vụ khách hàng 18
3.5.2 Nghiệp vụ quản trị viên và quản lý kho 19
3.5.3 Nghiệp vụ giao hàng của shipper 19
3.6 Tổng quan về nghiệp vụ kinh doanh rượu trực tuyến 19
3.7 Phân tích yêu cầu nghiệp vụ 20
3.7.1 Phân tích yêu cầu chức năng (Functional Requirements) 21
3.7.2 Phân tích yêu cầu phi chức năng (Non-Functional Requirements) 22
3.8 Phân tích tác nhân và quyền hạn 22
3.9 Quy tắc nghiệp vụ cốt lõi 23
3.10 Phân tích yêu cầu theo nhóm màn hình 23
3.11 Đặc tả ca sử dụng: khách hàng đặt hàng 24
3.12 Đặc tả ca sử dụng: xác nhận thanh toán 24
3.13 Đặc tả ca sử dụng: quản lý sản phẩm 25
3.14 Đặc tả ca sử dụng: gửi đánh giá sản phẩm 26
3.15 Phân tích yêu cầu dữ liệu và lưu vết 26

3.16 Phân tích yêu cầu phi chức năng theo mức ưu tiên 27
CHƯƠNG 3: PHÂN TÍCH MÔ HÌNH MIỀN VÀ THIẾT KẾ HƯỚNG ĐỐI TƯỢNG 28
4.1 Nhóm đối tượng sản phẩm và danh mục 28
4.2 Nhóm đối tượng tài khoản, giỏ và đơn 28
4.3 Nhóm đối tượng tồn kho, thanh toán và vận đơn 29
4.4 Đối chiếu các nguyên lý hướng đối tượng 30
4.5 Strategy trong nhập và xuất dữ liệu 31
4.6 Specification trong tìm kiếm rượu 32
4.7 Template Method trong báo cáo Excel 32
4.8 Publisher và sự kiện sau khi giao dịch hoàn tất 33
4.9 Scheduler và dispatch queue 33
4.10 FileStorageService và upload ảnh 34
4.11 Authentication, SecurityContext và role mapping 34
4.12 API JWT và phiên đăng nhập 35
4.13 Dịch vụ email và kết quả giao OTP 35
4.14 Xử lý lỗi và observability 36
4.15 Khung frontend và state giao diện 36
4.16 Repository, event và giao dịch 37
4.17 Thiết kế biểu đồ chức năng 37
4.18 Thiết kế biểu đồ Use Case 38
4.19 Thiết kế biểu đồ hoạt động cho checkout 39
4.20 Thiết kế biểu đồ hoạt động cho đánh giá 40
4.21 Thiết kế biểu đồ hoạt động cho admin tạo sản phẩm 40
4.22 Thiết kế biểu đồ trình tự cho checkout 41
4.23 Thiết kế biểu đồ trình tự cho thanh toán webhook 41
4.24 Thiết kế biểu đồ trình tự cho admin thêm sản phẩm 42
4.25 Thiết kế biểu đồ lớp miền 43
4.26 Thiết kế biểu đồ trạng thái cho Order và Shipment 43
4.27 Thiết kế luồng sự kiện và SSE 44
4.28 Ghi chú chất lượng cho UML 44
4.29 Nguyên tắc thiết kế dữ liệu 45
4.30 Bảng USERS — tài khoản người dùng 48
4.30.1 Dữ liệu mẫu thực tế bảng USERS 48
4.31 Bảng CATEGORIES — danh mục sản phẩm 50
4.31.1 Dữ liệu mẫu thực tế bảng CATEGORIES 50
4.32 Bảng WINES — thông tin rượu 52
4.32.1 Dữ liệu mẫu thực tế bảng WINES 52
4.33 Bảng WAREHOUSE — kho hàng 55
4.33.1 Dữ liệu mẫu thực tế bảng WAREHOUSE 55
4.34 Bảng INVENTORY — tồn theo wine và warehouse 57
4.34.1 Dữ liệu mẫu thực tế bảng INVENTORY 57
4.35 Bảng STOCK_LOGS — dấu vết tồn kho 59
4.35.1 Dữ liệu mẫu thực tế bảng STOCK_LOGS 59
4.36 Bảng INVENTORY_TRANSACTIONS — giao dịch nhập xuất kho 61
4.36.1 Dữ liệu mẫu thực tế bảng INVENTORY_TRANSACTIONS 61
4.37 Bảng CARTS — giỏ hàng 63
4.37.1 Dữ liệu mẫu thực tế bảng CARTS 63
4.38 Bảng CART_ITEMS — dòng sản phẩm trong giỏ 64
4.38.1 Dữ liệu mẫu thực tế bảng CART_ITEMS 64
4.39 Bảng ORDERS — đơn hàng 65
4.39.1 Dữ liệu mẫu thực tế bảng ORDERS 65
4.40 Bảng ORDER_ITEMS — chi tiết đơn hàng 68
4.40.1 Dữ liệu mẫu thực tế bảng ORDER_ITEMS 68
4.41 Bảng PAYMENTS — bản ghi thanh toán 70
4.41.1 Dữ liệu mẫu thực tế bảng PAYMENTS 70
4.42 Bảng PAYMENT_TRANSACTIONS — sự kiện giao dịch thanh toán 72
4.42.1 Dữ liệu mẫu thực tế bảng PAYMENT_TRANSACTIONS 72
4.43 Bảng REVIEWS — đánh giá 74
4.43.1 Dữ liệu mẫu thực tế bảng REVIEWS 74
4.44 Bảng PASSWORD_RESET_TOKENS — token đặt lại mật khẩu 76

4.44.1 Dữ liệu mẫu thực tế bảng PASSWORD_RESET_TOKENS 76
4.45 Bảng SHIPPERS — hồ sơ người giao hàng 78
4.45.1 Dữ liệu mẫu thực tế bảng SHIPPERS 78
4.46 Bảng SHIPMENTS — vận đơn 80
4.46.1 Dữ liệu mẫu thực tế bảng SHIPMENTS 80
4.47 Bảng SHIPMENT_STATUS_HISTORY — lịch sử trạng thái vận đơn 83
4.47.1 Dữ liệu mẫu thực tế bảng SHIPMENT_STATUS_HISTORY 83
4.48 Bảng SHIPMENT_OTP_AUDIT_LOGS — nhật ký bảo mật OTP 85
4.48.1 Dữ liệu mẫu thực tế bảng SHIPMENT_OTP_AUDIT_LOGS 85
4.49 Đối chiếu các bảng bổ sung và enum miền 88
4.49.1 Dữ liệu mẫu thực tế bảng SYSTEM_CONFIGS 88
4.50 ERD và nguyên tắc cập nhật sơ đồ 89
4.51 Phiên bản hóa schema bằng Flyway 90
4.52 Bảo vệ dữ liệu cá nhân trong mô hình miền 91
4.53 Lớp controller và ranh giới HTTP 92
4.54 Lớp service và điều phối use case 92
4.55 Repository và truy vấn miền 93
CHƯƠNG 4: LUỒNG NGHIỆP VỤ KHÁCH HÀNG 95
5.2 Giỏ hàng và đồng bộ thông tin sản phẩm 99
5.3 Checkout và tạo đơn chờ thanh toán 99
5.4 Tạo phiên thanh toán, callback và webhook 100
5.6 Lịch sử, chi tiết đơn và đánh giá 102
5.7 Duyệt danh mục và tìm kiếm rượu 104
5.8 Xem chi tiết và chọn số lượng 105
5.9 Đăng ký, đăng nhập và đặt lại mật khẩu 106
5.10 Quản lý giỏ hàng 108
5.11 Tạo đơn và reservation tồn 109
5.12 Khởi tạo payment session 109
5.13 Xử lý callback, webhook và trạng thái thanh toán 110
5.14 Xem lịch sử đơn và quyền sở hữu 110
5.15 Tạo và kiểm soát review 111
CHƯƠNG 5: LUỒNG NGHIỆP VỤ QUẢN TRỊ VÀ KHO 112
5.16 Dashboard và quản lý danh mục, sản phẩm 113
5.17 Quản lý người dùng, shipper và giao dịch 115
5.18 Sơ đồ quy trình thêm sản phẩm từ tài liệu cũ 116
6.1 Tồn đa kho, giữ chỗ và lịch sử nhập xuất 120
6.2 Nhập Excel: mẫu, preview, dry-run và ghi dữ liệu 121
6.3 Dashboard và quan sát vận hành 122
6.4 Quản lý order/payment phía quản trị 129
6.5 Màn hình tồn kho và cảnh báo low stock 131
6.6 Nhập kho, xuất kho và ledger 131
6.7 Nhập Excel: template, preview và dry-run 132
6.8 Xử lý lỗi theo dòng Excel 133
6.9 Export Excel và báo cáo tồn 133
CHƯƠNG 6: LUỒNG SHIPPER, OTP VÀ THÔNG BÁO 135
7.1 Phân công tự động và quyền xem nhiệm vụ 136
7.2 Lấy hàng, bắt đầu giao và báo giao thất bại 137
7.3 OTP giao hàng, gửi lại và giới hạn thử 137
7.4 SSE và phát cập nhật đến từng vai trò 138
7.5 Tạo shipment sau khi đơn thanh toán 139
7.6 Chính sách auto dispatch 139
7.7 Xác nhận lấy hàng và bắt đầu giao 140
7.8 Báo giao thất bại, retry và hoàn trả 140
7.9 Sinh OTP giao hàng 141
7.10 Xác thực OTP khi hoàn tất 141
7.11 Admin override và kiểm toán 142
7.12 SSE theo order, admin và shipper 143
7.13 UI shipper, riêng tư và thao tác mobile 143
CHƯƠNG 7: CHẤT LƯỢNG, BẢO MẬT VÀ ĐÁNH GIÁ KIỂM THỬ 145
8.1 Xác thực, phân quyền và quyền sở hữu tài nguyên 145

8.2 Giao dịch dữ liệu và các tác vụ ngoài database ... 145
8.3 Xử lý lỗi, nhật ký và khả năng phục hồi ... 146
8.4 Kiểm thử hiện có và khoảng trống kiểm chứng ... 146
8.5 Kết quả chạy ứng dụng và thay đổi dữ liệu đã được cho phép ... 147
8.6 7.6 Đánh giá tính tiện dụng giao diện (UI/UX) và kiến trúc thành phần ... 147
8.7 Cài đặt và chuẩn bị cơ sở dữ liệu ... 148
8.8 Cài đặt ứng dụng và cấu hình ngoài ... 148
8.9 Kết quả chạy và phạm vi chứng cứ ... 149
8.10 Thiết kế bộ kiểm thử chức năng ... 149
8.11 Ma trận kiểm thử hồi quy ... 150
8.12 Rà soát giao diện và sửa UI ... 150
8.13 Kết quả và mức độ tin cậy ... 151
HẠN CHẾ VÀ RỦI RO CẦN THEO DÕI ... 152
9.1 Thanh toán và tồn kho ... 152
9.2 Giao hàng, sự kiện và bảo mật ... 152
ĐỀ XUẤT CẢI TIẾN VÀ KẾ HOẠCH KIỂM CHỨNG ... 153
10.1 Ưu tiên bảo toàn đơn hàng và payment ... 153
10.2 Ưu tiên giao nhận, OTP và khả năng quan sát ... 153
10.3 Chuẩn hóa thiết kế và mở rộng kiểm thử ... 153
BỔ SUNG: DANH MỤC ĐƯỜNG ĐI THEO VAI TRÒ ... 155
10.3.1 Điểm vào phía khách và API công khai ... 155
10.3.2 Điểm vào phía quản trị ... 155
10.3.3 Điểm vào phía shipper và tương tác trực tiếp ... 155
KẾT LUẬN ... 157
TỔNG KẾT ... 157
TÀI LIỆU THAM KHẢO ... 158
PHỤ LỤC: THUẬT NGỮ VÀ VIẾT TẮT ... 159
TUYÊN BỐ VỀ CÔNG CỤ HỖ TRỢ VÀ LIÊM CHÍNH HỌC THUẬT ... 160

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ phân rã cấu trúc màn hình tổng thể toàn sàn StrongWine 16
Hình 3.1: Sơ đồ lớp kiến trúc phân tầng (Presentation - Service - Repository - Entity) của StrongWine 29
Hình 3.2: Mô hình miền cốt lõi StrongWine và enum Java hiện hành 30
Hình 3.3: Sơ đồ Use Case tổng thể toàn hệ thống StrongWine 39
Hình 3.19.c: Sơ đồ luồng xác thực và kiểm toán mã bảo mật giao hàng OTP 87
Hình 4.1: Wireframe Trang chủ sàn thương mại điện tử StrongWine (Grayscale Web Mockup)..... 90
Hình 4.5: Wireframe Giao diện thanh toán đơn hàng Checkout (Grayscale Web Mockup) 91
Hình 4.6: Wireframe Chi tiết đơn hàng và theo dõi vận chuyển (Grayscale Web Mockup) 94
Hình 4.0: Sơ đồ phân rã màn hình phân hệ Khách hàng (Customer Screen Hierarchy Tree)..... 95
Hình 4.0b: Sơ đồ Use Case chi tiết phân hệ Khách hàng..... 96
Hình 4.2: Wireframe Danh mục rượu và bộ lọc đa tiêu chí (Grayscale Web Mockup)..... 96
Hình 4.2.b: Giao diện trang chủ StrongWine (ảnh giao diện từ bộ tài liệu dự án) 97
Hình 4.3: Wireframe Chi tiết sản phẩm rượu và tasting notes (Grayscale Web Mockup)..... 98
Hình 4.7: Wireframe Đăng nhập và tạo tài khoản thành viên (Grayscale Web Mockup) 99
Hình 4.8: Wireframe Biểu mẫu yêu cầu đặt lại mật khẩu bảo mật (Grayscale Web Mockup)..... 100
Hình 4.4: Wireframe Quản lý giỏ hàng và cập nhật số lượng (Grayscale Web Mockup) 101
Hình 4.1: Lịch sử đơn hàng 102
Hình 4.4: Sơ đồ hoạt động (Activity Diagram) quy trình Checkout và thanh toán trực tuyến 103
Hình 4.5: Sơ đồ tuần tự (Sequence Diagram) xử lý Đặt hàng giữa các lớp ứng dụng..... 104
Hình 4.2: Danh mục rượu và bộ lọc..... 104
Hình 4.3: Chi tiết sản phẩm rượu..... 105
Hình 4.4: Trang đăng nhập 106
Hình 4.5: Trang đăng ký tài khoản 106
Hình 4.6: Trang yêu cầu đặt lại mật khẩu..... 107
Hình 4.7: Giao diện xử lý liên kết đặt lại không hợp lệ..... 107
Hình 4.8: Giỏ hàng trống 108
Hình 4.9.b: Giao diện chi tiết đơn hàng và theo dõi giao nhận (ảnh giao diện từ bộ tài liệu dự án)..... 111
Hình 5.0: Sơ đồ phân rã màn hình phân hệ Quản trị và Thủ kho (Admin & Warehouse Screen Hierarchy Tree)..... 112
Hình 5.0b: Sơ đồ Use Case chi tiết phân hệ Quản trị và Thủ kho 113
Hình 5.3: Wireframe Bảng điều khiển quản trị Dashboard KPI (Grayscale Web Mockup)..... 114
Hình 5.7: Wireframe Quản lý tài khoản người dùng và phân quyền (Grayscale Web Mockup)..... 114
Hình 5.8: Wireframe Quản lý danh sách nhân viên giao hàng Shipper (Grayscale Web Mockup)..... 116
Hình 5.1: Sơ đồ hoạt động (Activity Diagram) Quản trị viên thêm rượu và upload ảnh..... 117
Hình 5.2: Sơ đồ tuần tự Admin tạo rượu qua form MVC và FileStorageService..... 118
Hình 5.4: Wireframe Quản lý danh mục và sản phẩm rượu vang (Grayscale Web Mockup) 119
Hình 5.5: Wireframe Thêm mới và chỉnh sửa thông tin rượu (Grayscale Web Mockup) 120
Hình 5.9: Wireframe Giám sát đơn hàng toàn sàn và trạng thái thanh toán (Grayscale Web Mockup)..... 121
Hình 5.1: Bảng điều khiển quản trị..... 122
Hình 5.10: Wireframe Bảng quản trị tồn kho và điều phối chi nhánh (Grayscale Web Mockup)..... 123
Hình 5.2: Danh mục và sản phẩm rượu 123
Hình 5.3: Tạo sản phẩm rượu 124
Hình 5.4: Chỉnh sửa sản phẩm rượu 124
Hình 5.6: Wireframe Nhập danh mục rượu hàng loạt từ tệp Excel (Grayscale Web Mockup) 125
Hình 5.5: Danh sách tài khoản người dùng..... 126
Hình 5.6: Tạo tài khoản người dùng 126
Hình 5.7: Chỉnh sửa tài khoản người dùng 127
Hình 5.8: Danh sách người giao hàng..... 127
Hình 5.9: Tạo hồ sơ người giao hàng..... 128
Hình 5.10: Giao diện chỉnh sửa hồ sơ Shipper 128
Hình 5.11: Danh sách đơn hàng quản trị..... 129
Hình 5.12: Quản lý thanh toán..... 130
Hình 5.13: Quản lý tồn kho..... 131
Hình 5.14: Màn hình nhập dữ liệu Excel..... 132
Hình 6.0: Sơ đồ phân rã màn hình phân hệ Giao hàng (Shipper Cockpit Screen Hierarchy Tree)..... 135
Hình 6.0b: Sơ đồ Use Case chi tiết phân hệ Giao hàng (Shipper) 135

Hình 6.1: Wireframe Bảng điều khiển công việc Shipper (Grayscale Web Mockup)..... 136
Hình 6.2: Wireframe trạng thái xem chi tiết vận đơn trên bảng điều khiển Shipper 137
Hình 6.3: Wireframe trạng thái nhập OTP hoàn tất giao hàng trên bảng điều khiển Shipper 138
Hình 6.1: Danh sách vận đơn..... 142
Hình 6.2: Chỉnh sửa vận đơn 142
Hình 6.3: Bảng điều khiển shipper 144

DANH MỤC BẢNG BIỂU

Bảng 0.1: Kế hoạch và tiến độ thực hiện đề tài qua các tuần 6
Bảng 0.2: Danh mục các từ viết tắt và thuật ngữ kỹ thuật sử dụng trong báo cáo 15
Bảng 3.0: Nguyên tắc thiết kế và chuẩn hóa lược đồ dữ liệu quan hệ StrongWine 45
Bảng 3.1.a: Cấu trúc lược đồ bảng USERS (Tài khoản người dùng) 48
Bảng 3.1.b: Dữ liệu mẫu bảng USERS (10 bản ghi minh họa thực tế) 48
Bảng 3.2.a: Cấu trúc lược đồ bảng CATEGORIES (Danh mục sản phẩm) 50
Bảng 3.2.b: Dữ liệu mẫu bảng CATEGORIES (10 bản ghi minh họa thực tế) 50
Bảng 3.3.a: Cấu trúc lược đồ bảng WINES (Thông tin rượu vang) 52
Bảng 3.3.b: Dữ liệu mẫu bảng WINES (10 bản ghi minh họa thực tế) 52
Bảng 3.4.a: Cấu trúc lược đồ bảng WAREHOUSE (Kho hàng và chi nhánh) 55
Bảng 3.4.b: Dữ liệu mẫu bảng WAREHOUSE (10 bản ghi minh họa thực tế) 55
Bảng 3.5.a: Cấu trúc lược đồ bảng INVENTORY (Quản lý tồn kho theo sản phẩm) 57
Bảng 3.5.b: Dữ liệu mẫu bảng INVENTORY (10 bản ghi minh họa thực tế) 57
Bảng 3.6.a: Cấu trúc lược đồ bảng STOCK_LOGS (Lịch sử biến động tồn kho) 59
Bảng 3.6.b: Dữ liệu mẫu bảng STOCK_LOGS (10 bản ghi minh họa thực tế) 59
Bảng 3.7.a: Cấu trúc lược đồ bảng INVENTORY_TRANSACTIONS (Giao dịch nhập xuất) 61
Bảng 3.7.b: Dữ liệu mẫu bảng INVENTORY_TRANSACTIONS (10 bản ghi minh họa thực tế) 61
Bảng 3.8.a: Cấu trúc lược đồ bảng CARTS (Giỏ hàng người dùng) 63
Bảng 3.8.b: Dữ liệu mẫu bảng CARTS (10 bản ghi minh họa thực tế) 63
Bảng 3.9.a: Cấu trúc lược đồ bảng CART_ITEMS (Chi tiết mặt hàng trong giỏ) 64
Bảng 3.9.b: Dữ liệu mẫu bảng CART_ITEMS (10 bản ghi minh họa thực tế) 64
Bảng 3.10.a: Cấu trúc lược đồ bảng ORDERS (Đơn hàng toàn sàn) 65
Bảng 3.10.b: Dữ liệu mẫu bảng ORDERS (10 bản ghi minh họa thực tế) 65
Bảng 3.11.a: Cấu trúc lược đồ bảng ORDER_ITEMS (Chi tiết dòng đơn hàng) 68
Bảng 3.11.b: Dữ liệu mẫu bảng ORDER_ITEMS (10 bản ghi minh họa thực tế) 68
Bảng 3.12.a: Cấu trúc lược đồ bảng PAYMENTS (Bản ghi thanh toán đơn hàng) 70
Bảng 3.12.b: Dữ liệu mẫu bảng PAYMENTS (10 bản ghi minh họa thực tế) 70
Bảng 3.13.a: Cấu trúc lược đồ bảng PAYMENT_TRANSACTIONS (Lịch sử sự kiện thanh toán) 72
Bảng 3.13.b: Dữ liệu mẫu bảng PAYMENT_TRANSACTIONS (10 bản ghi minh họa thực tế) 72
Bảng 3.14.a: Cấu trúc lược đồ bảng REVIEWS (Đánh giá và phản hồi rượu) 74
Bảng 3.14.b: Dữ liệu mẫu bảng REVIEWS (10 bản ghi minh họa thực tế) 74
Bảng 3.15.a: Cấu trúc lược đồ bảng PASSWORD_RESET_TOKENS (Token đặt lại mật khẩu) 76
Bảng 3.15.b: Dữ liệu mẫu bảng PASSWORD_RESET_TOKENS (10 bản ghi minh họa thực tế) 76
Bảng 3.16.a: Cấu trúc lược đồ bảng SHIPPERS (Hồ sơ nhân viên giao hàng) 78
Bảng 3.16.b: Dữ liệu mẫu bảng SHIPPERS (10 bản ghi minh họa thực tế) 78
Bảng 3.17.a: Cấu trúc lược đồ bảng SHIPMENTS (Vận đơn điều phối giao nhận) 80
Bảng 3.17.b: Dữ liệu mẫu bảng SHIPMENTS (10 bản ghi minh họa thực tế) 80
Bảng 3.18.a: Cấu trúc lược đồ bảng SHIPMENT_STATUS_HISTORY (Lịch sử trạng thái vận đơn) 83
Bảng 3.18.b: Dữ liệu mẫu bảng SHIPMENT_STATUS_HISTORY (10 bản ghi minh họa thực tế) 83
Bảng 3.19.a: Cấu trúc lược đồ bảng SHIPMENT_OTP_AUDIT_LOGS (Nhật ký xác thực OTP) 85
Bảng 3.19.b: Dữ liệu mẫu bảng SHIPMENT_OTP_AUDIT_LOGS (10 bản ghi minh họa thực tế) 85
Bảng 3.20.a: Cấu trúc lược đồ bảng SYSTEM_CONFIGS (Cấu hình hệ thống và tham số) 88
Bảng 3.20.b: Dữ liệu mẫu bảng SYSTEM_CONFIGS (10 bản ghi minh họa thực tế) 88
Bảng Phụ lục 0.1: Bảng đối chiếu thuật ngữ và từ viết tắt kiến trúc StrongWine 159

DANH MỤC TỪ VIẾT TẮT

Bảng 0.2: Danh mục các từ viết tắt và thuật ngữ kỹ thuật sử dụng trong báo cáo

Từ viết tắt	Tên tiếng Anh đầy đủ	Ý nghĩa / Giải thích tiếng Việt
OOAD	Object-Oriented Analysis and Design	Phân tích và thiết kế hướng đối tượng
OOP	Object-Oriented Programming	Lập trình hướng đối tượng
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất
JPA	Jakarta Persistence API	Giao diện lập trình chuẩn cho ánh xạ ORM trong Java
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ thực thể hướng đối tượng sang CSDL
REST	Representational State Transfer	Kiểu kiến trúc thiết kế dịch vụ web hướng tài nguyên
JWT	JSON Web Token	Chuẩn mã hóa thông tin xác thực phiên làm việc
SSE	Server-Sent Events	Giao thức truyền dữ liệu thời gian thực một chiều từ server
OTP	One-Time Password	Mật khẩu dùng một lần xác thực giao nhận
DDL	Data Definition Language	Ngôn ngữ định nghĩa cấu trúc cơ sở dữ liệu

MỞ ĐẦU

Một cửa hàng trực tuyến không chỉ hiển thị sản phẩm. Khi khách đặt hàng, hệ thống phải nối dữ liệu sản phẩm với giá, số lượng hiện có, người nhận, giao dịch thanh toán, trạng thái đơn, lượng hàng giữ chỗ và nhiệm vụ giao hàng. Mỗi quyết định làm thay đổi trạng thái phải được thể hiện nhất quán ở giao diện, lớp nghiệp vụ và cơ sở dữ liệu. Nếu trách nhiệm bị dồn vào một controller hoặc một lớp dịch vụ quá lớn, thay đổi ở thanh toán có thể vô tình làm sai kho hay giao nhận.

StrongWine là trường hợp khảo sát thực tế cho học phần Phát triển phần mềm hướng đối tượng dựa trên kho mã nguồn dự án [1]. Dự án dùng Java và Spring, có giao diện máy chủ kết hợp API, SQL Server, nhiều vai trò người dùng và các luồng cần phối hợp với hệ thống ngoài. Vì vậy có thể xem xét không chỉ sơ đồ lớp mà cả cách đối tượng cộng tác xuyên suốt từ yêu cầu HTTP đến trạng thái đơn hàng và thông báo cho người dùng.

1.1.1 Mục tiêu, câu hỏi và phạm vi khảo sát

Mục tiêu cốt lõi của đề tài là khảo sát toàn diện kiến trúc phần mềm, đối soát mô hình lớp và phân tích chi tiết quy trình nghiệp vụ của sàn thương mại điện tử StrongWine.

Phạm vi nghiên cứu tập trung vào 5 phân hệ chức năng trọng yếu: cổng thông tin khách hàng với giao diện đáp ứng linh hoạt theo Responsive Web Design [12] (duyệt danh mục rượu, giỏ hàng, đặt hàng), quy trình quản trị kho vận (kiểm soát 20 thực thể dữ liệu quan hệ, cảnh báo mức tồn thấp Low Stock, kiểm toán xuất nhập), cơ chế thanh toán đa phương thức (xử lý phiên Stripe và Webhook đối soát), phân công điều phối giao hàng (tự động hóa Auto Dispatch và xác thực mã OTP giao vận), cùng hệ thống thông báo sự kiện thời gian thực thông qua Server-Sent Events (SSE). Bằng cách so sánh đối chiếu trực tiếp giữa mã nguồn Java 21 hiện hành với tài liệu thiết kế gốc, nghiên cứu làm sáng tỏ các quyết định kỹ thuật và khoảng cách hiện thực hóa.

1.1.2 Phương pháp và quy ước bằng chứng

Phương pháp nghiên cứu kết hợp giữa phân tích tĩnh mã nguồn, chạy kiểm thử hộp trắng và kiểm chứng động trên môi trường phát triển cục bộ.

Mọi kết luận kỹ thuật trong tiểu luận đều được xây dựng dựa trên bằng chứng kiểm chứng đa tầng: truy vết luồng gọi phương thức từ Controller xuống Service và Repository, đối chiếu khai báo Entity với 20 bảng cơ sở dữ liệu SQL Server, và kiểm toán các annotation giao dịch `@Transactional` [6]. Bằng chứng được phân loại minh bạch thành ba cấp độ: cấp độ mã nguồn (được chứng thực bằng đường dẫn file và định nghĩa lớp cụ thể), cấp độ biên dịch và thực thi ứng dụng (xác nhận qua Maven Wrapper và Flyway migrations), và cấp độ mô phỏng kiểm thử (được kiểm chứng qua các kịch bản test có kiểm soát).

NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

Trong lập trình hướng đối tượng theo chuẩn kiến trúc Java [2], đối tượng kết hợp trạng thái với hành vi có ý nghĩa trong miền; lớp mô tả cách tạo các đối tượng cùng loại. Giao diện xác định hợp đồng mà các lớp có thể thực hiện. Đóng gói giới hạn nơi có thể sửa trạng thái, còn kế thừa biểu diễn quan hệ chuyên biệt hóa khi quan hệ đó thực sự là is-a. Các khái niệm này hữu ích khi chúng làm rõ trách nhiệm và quy tắc, không chỉ vì ngôn ngữ có cú pháp tương ứng.

2.1 Đối tượng, lớp và nguyên tắc trách nhiệm

Trong mô hình hướng đối tượng của StrongWine, đối tượng đại diện cho các thực thể miền thực tế và đóng gói trạng thái cùng các hành vi nghiệp vụ chặt chẽ nhằm bảo toàn tính toàn vẹn dữ liệu.

Lớp thực thể `Order` đóng vai trò là một aggregate root trong tầng Domain, lưu trữ thông tin trạng thái đơn hàng, phương thức thanh toán và danh sách các dòng sản phẩm `OrderItem`. Việc thiết kế phân rã trách nhiệm cho phép tách biệt rõ ràng giữa cấu trúc dữ liệu lưu trữ và các quy tắc xử lý: `OrderController` chỉ đóng vai trò tiếp nhận và xác thực yêu cầu HTTP từ phía client, trong khi `OrderService` điều phối toàn bộ chuỗi nghiệp vụ bao gồm tính toán tổng tiền, kiểm tra tồn kho, khóa sản phẩm lạc quan và kích hoạt quy trình tạo vận đơn. Cách phân tách này tuân thủ chặt chẽ nguyên lý Single Responsibility Principle (SRP) [2], giảm thiểu sự phụ thuộc chéo và hạn chế tối đa nguy cơ lỗi lan truyền khi có sự điều chỉnh logic thương mại.

2.2 IoC, dependency injection và cộng tác lớp

Cơ chế Inversion of Control (IoC) và Dependency Injection (DI) theo chuẩn Spring Framework [3] là xương sống kiến trúc của StrongWine, giúp tách rời quá trình khởi tạo đối tượng khỏi mã nghiệp vụ thi hành.

Spring IoC Container [3] tự động quản lý vòng đời và dây chuyền phụ thuộc của toàn bộ các bean dịch vụ như `WineService`, `InventoryService`, `ShipmentService` và các adapter giao tiếp ngoại vi. Thay vì sử dụng khởi tạo trực tiếp bằng toán tử `new` gây thắt chặt phụ thuộc (tight coupling), các lớp dịch vụ tiếp nhận abstraction thông qua cơ

chế Constructor Injection chuẩn mực. Điều này cho phép hoán đổi linh hoạt giữa các cài đặt kiểm thử (Mock/Stub) và cài đặt sản xuất thực tế mà không đòi hỏi chỉnh sửa mã nguồn nghiệp vụ. Sự cộng tác giữa Controller, Service và Repository được phân định

ranh giới rõ ràng thông qua các interface trung gian, ngăn chặn hiện tượng rò rỉ chi tiết truy cập dữ liệu lên tầng hiển thị.

2.3 Mẫu thiết kế và tiêu chí nhận diện

Mẫu thiết kế (Design Patterns) [4] trong StrongWine được nhận diện dựa trên cấu trúc tương tác lớp và bài toán nghiệp vụ thực tế mà nó giải quyết trong mã nguồn.

Hệ thống ứng dụng linh hoạt nhiều mẫu thiết kế kinh điển của GoF: Mẫu Strategy được thể hiện rõ nét qua các chiến lược nhập xuất dữ liệu `DataImportStrategy` và `DataExportStrategy`, cho phép hệ thống mở rộng hỗ trợ các định dạng tệp mới như CSV hoặc JSON mà không làm thay đổi bộ điều phối trung tâm. Mẫu Specification [7] trong Spring Data JPA kết hợp linh hoạt các vị từ tìm kiếm rượu vang theo danh mục, niên vụ, khoảng giá và nồng độ cồn thành một câu truy vấn SQL động tối ưu. Trong phân hệ báo cáo tồn kho Excel, mẫu Template Method định hình khung thuật toán xuất báo cáo chung gồm các bước khởi tạo workbook, tạo tiêu đề, lấp đầy dữ liệu và định dạng ô, trong khi các lớp con cụ thể hóa cấu trúc dòng của từng loại báo cáo.

Việc áp dụng các mẫu thiết kế này phản ánh tư duy kiến trúc hướng đối tượng chuẩn mực, nâng cao rõ rệt tính tái sử dụng và khả năng bảo trì mã nguồn.

Lợi ích của pattern phải được cân với chi phí điều hướng. Strategy hữu ích khi có nhiều kiểu import; nó kém hữu ích nếu chỉ có một nhánh bất biến và không có kế hoạch biến thể. Template Method chuẩn hóa format report, nhưng lớp cha cần tránh biết chi tiết nghiệp vụ sản phẩm. Event cho phép shipment được tạo sau khi payment/đơn được ghi nhận thành công; đổi lại, lỗi trong listener sau giao dịch không thể rollback giao dịch đã hoàn tất. Mỗi pattern kéo theo một loại nghĩa vụ khác nhau.

Không xem `@Transactional`, dependency injection hay Spring Data là tự động một design pattern GoF. Chúng là cơ chế/framework abstraction có quan hệ đến nguyên tắc thiết kế, nhưng cần trình bày đúng tên và phạm vi. Các mục sau đánh giá từng trường hợp trong code, nêu lý do, cấu trúc, lợi ích và rủi ro thay vì dùng một danh sách pattern chung chung.

2.4 UML như ngôn ngữ mô hình hóa

Ngôn ngữ mô hình hóa thống nhất UML 2.5.1 [5] đóng vai trò là phương tiện trực quan hóa và đặc tả chuẩn xác cấu trúc tĩnh cũng như hành vi động của hệ thống StrongWine.

Các biểu đồ lớp (Class Diagram) đặc tả chi tiết 20 thực thể miền, mối quan hệ kết tập, kế thừa và liên kết đa số giữa các bảng cơ sở dữ liệu như quan hệ 1-N giữa `Categories` và `Wines`, quan hệ N-N giữa `Wines` và `Warehouse` thông qua bảng trung gian `Inventory`. Biểu đồ tuần tự (Sequence Diagram) mô tả chính xác tương tác thời gian thực giữa Khách hàng, Spring Controller, Service nghiệp vụ và cổng thanh toán bên thứ ba trong ca sử dụng Đặt hàng và Thanh toán trực tuyến. Ngoài ra, biểu đồ trạng thái (State Machine Diagram) đóng vai trò kim chỉ nam trong việc kiểm soát các bước chuyển trạng thái hợp lệ của đơn hàng và vận đơn giao hàng.

Hệ thống sơ đồ UML [5] trong bài tiểu luận được sinh tự động từ mã nguồn PlantUML với quy trình kiểm chứng cú pháp nghiêm ngặt, phản ánh trung thực cấu trúc mã nguồn thực tế của hệ thống [1].

Tám sơ đồ/hình chức năng và UML được chọn từ tài liệu cũ gồm biểu đồ chức năng, Use Case, hai activity, hai sequence, class diagram và ERD. Mỗi hình có caption nêu giới hạn đối chiếu; các hình checkout cũ 05 và 08 bị loại vì mô tả cập nhật tồn khác cơ chế giữ chỗ hiện hành. Ảnh giao diện cũ không được dùng. Phần phân tích bằng văn bản trong báo cáo này là căn cứ hiện trạng dựa trên checkout; sơ đồ cũ không phải mô hình thiết kế mới, không dùng để nghiệm thu database hoặc API.

Một sơ đồ UML tốt cần có phạm vi rõ, tên nhất quán với code, nhánh ngoại lệ quan trọng và chú giải trạng thái. Khi code thay đổi, sơ đồ cần được cập nhật cùng với test và tài liệu. Phụ lục liệt kê chênh lệch giữa nhãn trong hình cũ với đường dẫn controller hiện hành, đồng thời nêu dữ liệu cần xác minh trước khi vẽ lại bản chuẩn.

2.5 Tổng quan về ngôn ngữ lập trình Java

Java 21 LTS là nền tảng công nghệ chủ đạo của StrongWine với các nguyên lý thiết kế đóng gói và kế thừa tiêu chuẩn [2], mang lại môi trường thực thi máy ảo JVM vững chắc và hệ thống kiểu tĩnh nghiêm ngặt.

Hệ thống khai thác tối đa các cải tiến ngôn ngữ hiện đại của Java 21, tiêu biểu là Record classes để xây dựng các đối tượng truyền dữ liệu (DTO) bất biến, rút gọn đáng

kể mã boilerplate và đảm bảo an toàn luồng dữ liệu giữa các phân tầng. Tính năng Pattern Matching cho toán tử `switch` và `instanceof` giúp đơn giản hóa việc phân nhánh xử lý các trạng thái đơn hàng phức tạp. Đặc biệt, nền tảng Java 21 cung cấp cơ chế Virtual Threads (Project Loom), cho phép máy chủ xử lý hàng nghìn kết nối đồng thời với chi phí bộ nhớ tối thiểu, rất phù hợp cho luồng phát thông báo Server-Sent Events (SSE) kéo dài đến người dùng và shipper.

Sự kết hợp giữa tính an toàn kiểu dữ liệu tĩnh của Java và cơ chế dọn rác tự động (Garbage Collection) giúp ứng dụng loại trừ hoàn toàn các lỗi quản lý con trỏ và rò rỉ bộ nhớ nguy hiểm.

2.5.1 Đặc điểm nổi bật của Java trong hệ thống

Đặc tính hướng đối tượng hoàn chỉnh của Java cung cấp nền tảng vững chắc cho việc trừu tượng hóa và đóng gói các quy tắc thương mại của StrongWine.

Tính đa hình và kế thừa giao diện cho phép StrongWine cài đặt các cơ chế mở rộng như xử lý sự kiện bất đồng bộ qua `ApplicationEventPublisher`, định nghĩa các interceptor kiểm tra quyền truy cập và tùy biến các bộ chuyển đổi dữ liệu (Custom Converters). Hệ thống quản lý bộ nhớ thế hệ mới G1GC vận hành trơn tru trong suốt các ca kiểm thử tải dữ liệu mẫu 20 bảng, duy trì độ trễ dừng ứng dụng (pause time) ở mức cực thấp. Việc quản lý biệt lập các package theo miền nghiệp vụ giúp kiểm soát tốt phạm vi hiển thị (visibility) của các phương thức nội bộ, ngăn chặn truy cập trái phép từ các phân hệ ngoài.

Môi trường JVM đa nền tảng bảo đảm ứng dụng có thể đóng gói thành tệp thực thi duy nhất và triển khai đồng nhất từ máy phát triển cục bộ lên hạ tầng container.

2.5.2 Java và phát triển web

Hệ sinh thái Java Web hiện đại cung cấp nền tảng xử lý HTTP mạnh mẽ, bảo mật cao và tương thích hoàn toàn với các tiêu chuẩn công nghiệp.

Thông qua chuẩn Servlet Container tích hợp (Embedded Apache Tomcat), StrongWine quản lý vòng đời yêu cầu HTTP khép kín từ khâu giải mã tiêu đề, lọc bảo mật với Spring Security Filter Chain, định tuyến đến Controller tương ứng, đến khâu tuần tự hóa dữ liệu JSON hoặc render giao diện phía máy chủ bằng Thymeleaf. Việc xử lý phiên làm việc (Session Management) được tối ưu hóa bằng các cơ chế cookie an

toàn HttpOnly và SameSite, giúp bảo vệ phiên đăng nhập của người dùng trước các cuộc tấn công đánh cắp phiên và tấn công Cross-Site Scripting (XSS).

Mô hình xử lý đồng bộ kết hợp các luồng nền bất đồng bộ của Java Web bảo đảm thông lượng xử lý cao cho các tác vụ nặng như tạo file Excel và gửi email mã OTP.

2.5.3 Quản lý phụ thuộc và tổ chức package

Dự án StrongWine sử dụng Apache Maven Wrapper để tự động hóa toàn diện quy trình biên dịch, kiểm thử và quản lý cây phụ thuộc thư viện.

Cấu trúc thư mục mã nguồn được tổ chức chặt chẽ theo mô hình phân tầng hướng miền (Domain-Driven Packaging): `com.strongwine.controller` chứa các điểm vào HTTP, `com.strongwine.service` chứa logic nghiệp vụ và ranh giới giao dịch, `com.strongwine.repository` chứa các giao diện truy xuất Spring Data JPA, và `com.strongwine.entity` định nghĩa các đối tượng ánh xạ cơ sở dữ liệu. Tệp cấu hình `pom.xml` khóa chặt phiên bản các thư viện quan trọng như Spring Boot 3.x, Flyway Core, Microsoft JDBC Driver và POI Excel, loại bỏ hoàn toàn rủi ro xung đột phụ thuộc chéo.

Quy chuẩn tổ chức này giúp mã nguồn sáng sủa, giảm thiểu độ phức tạp chu trình và cho phép các thành viên trong nhóm phát triển cộng tác mượt mà.

2.6 Tổng quan về Spring Boot và mô hình web của StrongWine

Spring Boot 3 đóng vai trò là framework ứng dụng trung tâm, kết hợp với Spring IoC Container [3] để tự động quản lý các bean dịch vụ, cung cấp cơ chế cấu hình tự động (Auto-configuration) và môi trường thực thi độc lập tối ưu.

Mô hình MVC của StrongWine kết hợp linh hoạt giữa hai phong cách kiến trúc: trả về giao diện HTML được render phía máy chủ bằng công cụ Thymeleaf cho các màn hình duyệt catalog, giỏ hàng và bảng điều khiển quản trị, đồng thời cung cấp các REST API endpoint trả về dữ liệu chuẩn JSON cho các tác vụ bất đồng bộ như kiểm tra giỏ hàng nhanh, nạp dữ liệu động và luồng đẩy sự kiện Server-Sent Events (SSE). Ranh giới giao dịch được quản lý tập trung bằng declarative transaction management `@Transactional`, bảo đảm rằng chuỗi tác vụ phức tạp (như trừ tồn kho, tạo đơn hàng và tạo bản ghi lịch sử) luôn tuân thủ nghiêm ngặt tính chất ACID.

Cơ chế Actuator và cấu hình ngoài `application.properties` cho phép hệ thống dễ dàng điều chỉnh tham số kết nối cơ sở dữ liệu và giám sát trạng thái sức khỏe vận hành.

2.6.1 JSP trong tài liệu cũ và Thymeleaf hiện tại

Việc chuyển đổi từ công nghệ hiển thị cũ JSP sang công nghệ hiện đại Thymeleaf đánh dấu bước tiến lớn về tính an toàn và khả năng bảo trì giao diện của StrongWine.

Trong các phiên bản tài liệu trước đây, hệ thống sử dụng JavaServer Pages (JSP) với các thẻ JSTL và mã Java nhúng scriptlet phức tạp, tiềm ẩn nguy cơ cao về lỗi cú pháp thời gian chạy và khó khăn trong việc tách biệt thiết kế giao diện khỏi logic lập trình. Phiên bản StrongWine hiện tại đã chuyển đổi hoàn toàn sang Thymeleaf - một công cụ mẫu template hướng XML/HTML tự nhiên (Natural Templating). Điểm ưu việt của Thymeleaf là khả năng xem trước giao diện trực tiếp trên trình duyệt mà không cần khởi động máy chủ ứng dụng, đồng thời tích hợp cơ chế tự động mã hóa ngữ cảnh (context-aware HTML escaping) giúp vô hiệu hóa triệt để lỗ hổng Cross-Site Scripting (XSS) khi hiển thị dữ liệu do người dùng nhập.

Kiến trúc phân rã giao diện thành các fragment dùng chung như header, footer và sidebar giúp đồng bộ trải nghiệm người dùng và tiết kiệm đáng kể thời gian phát triển.

2.6.2 Spring Boot, dependency injection và giao dịch

Sự phối hợp giữa cơ chế Dependency Injection và quản lý giao dịch khai báo `@Transactional` [6] trong Spring Boot thiết lập chuẩn mực độ tin cậy cho dữ liệu của StrongWine.

Khi một phương thức trong `OrderService` hoặc `ShipmentService` được đánh dấu bằng `@Transactional` [6], Spring tự động tạo ra một AOP Proxy bao bọc lời gọi phương thức. Proxy này mở kết nối đến cơ sở dữ liệu SQL Server, bắt đầu một transaction cô lập (Isolation level READ_COMMITTED), và chỉ thực hiện lệnh `COMMIT` khi toàn bộ chuỗi nghiệp vụ hoàn tất thành công mà không phát sinh ngoại lệ chưa kiểm tra. Trường hợp phát sinh `RuntimeException` (như không đủ tồn kho hoặc số dư không hợp lệ), proxy tự động phát lệnh `ROLLBACK`, khôi phục nguyên vẹn trạng thái của tất cả 20 bảng dữ liệu. Cơ chế quản lý này giải phóng lập trình viên khỏi việc viết mã JDBC quản lý kết nối thủ công dễ xảy ra lỗi rò rỉ tài nguyên.

Các tác vụ bên ngoài cơ sở dữ liệu như gửi email OTP hoặc phát thông báo SSE được đặt cẩn trọng sau điểm commit giao dịch để tránh phát sinh sự kiện sai lệch khi rollback.

2.6.3 Validation, binding và xử lý lỗi web

Tầng xác thực dữ liệu đầu vào của StrongWine được xây dựng theo chuẩn Bean Validation (JSR-380) nhằm ngăn chặn triệt để dữ liệu sai lệch ngay từ ranh giới HTTP.

Các đối tượng yêu cầu (Command/DTO) như `RegisterRequest`, `CheckoutForm` và `WineCreateForm` được trang bị các annotation ràng buộc chặt chẽ như `@NotBlank`, `@Size`, `@Min`, `@Pattern` và `@Email`. Khi yêu cầu gửi tới Controller, tham số `@Valid` kích hoạt quy trình kiểm tra tự động; mọi vi phạm được gom lại trong `BindingResult` để hiển thị thông báo lỗi thân thiện ngay bên cạnh trường dữ liệu tương ứng trên giao diện Thymeleaf. Đối với các lỗi logic thời gian chạy hoặc truy cập tài nguyên không tồn tại, lớp xử lý ngoại lệ toàn cục `@ControllerAdvice` tiếp nhận lỗi tập trung, ghi nhật ký có kiểm soát và chuyển hướng người dùng đến trang thông báo lỗi chuẩn mực mà không để lộ chi tiết mã nguồn hay stack trace hệ thống.

Cơ chế xử lý lỗi tập trung bảo đảm tính an ninh thông tin, loại bỏ khả năng rò rỉ cấu trúc cơ sở dữ liệu qua các thông điệp ngoại lệ kỹ thuật.

2.7 Tổng quan về môi trường phát triển

Môi trường phát triển của dự án StrongWine được chuẩn hóa để bảo đảm tính nhất quán tối đa giữa các máy phát triển và môi trường thử nghiệm.

Bộ công cụ tiêu chuẩn bao gồm IntelliJ IDEA Ultimate/VS Code làm môi trường phát triển tích hợp (IDE), kết hợp cùng tiện ích mở rộng Spring Boot Tools và Database Navigator. Máy tính phát triển được cài đặt bộ máy ảo Java Amazon Corretto JDK 21 hoặc Eclipse Temurin 21 LTS, máy chủ cơ sở dữ liệu Microsoft SQL Server 2022 Express với công cụ quản trị SQL Server Management Studio (SSMS), và hệ thống quản lý mã nguồn phân tán Git. Việc khai thác tệp `mvnw` (Maven Wrapper) giúp đồng nhất phiên bản công cụ build trên mọi máy lập trình mà không đòi hỏi cài đặt Maven toàn cục trên hệ điều hành.

Cấu hình biến môi trường và tệp `.env` được áp dụng để quản lý các tham số nhạy cảm như khóa Stripe API và mật khẩu tài khoản hệ thống, tuyệt đối không lưu cứng trong kho mã nguồn.

2.8 Tổng quan về cơ sở dữ liệu quan hệ

Mô hình cơ sở dữ liệu quan hệ (RDBMS) là giải pháp tối ưu cho StrongWine nhờ khả năng tuân thủ các nguyên lý toàn vẹn dữ liệu và cơ chế khóa của C.J. Date [10], bảo đảm tính toàn vẹn dữ liệu và hỗ trợ giao dịch ACID nghiêm ngặt.

Trong lĩnh vực thương mại điện tử rượu vang, dữ liệu có độ ràng buộc liên kết rất cao: một đơn hàng liên kết chặt chẽ với khách hàng, thông tin địa chỉ giao nhận, danh sách chai rượu, kho xuất hàng và chứng từ thanh toán. Việc áp dụng các dạng chuẩn chuẩn hóa (1NF, 2NF, 3NF) trên toàn bộ 20 bảng dữ liệu giúp triệt tiêu hiện tượng dư thừa thông tin, tránh được các dị thường khi thêm, xóa và cập nhật bản ghi. Mối quan hệ khóa chính - khóa ngoại (PK-FK) được thiết lập chặt chẽ giữa các bảng (ví dụ `orders.user_id` tham chiếu đến `users.id`, `inventory.wine_id` tham chiếu đến `wines.id`), đảm bảo rằng không bao giờ xuất hiện các bản ghi mồ côi làm méo mó số liệu kế toán và kiểm kê kho.

Tính toàn vẹn tham chiếu đóng vai trò như chốt chặn an toàn cuối cùng, bảo vệ cơ sở dữ liệu khỏi các sai sót logic xuất phát từ tầng ứng dụng.

2.9 Tổng quan về hệ quản trị cơ sở dữ liệu Microsoft SQL Server

Microsoft SQL Server 2022 được lựa chọn làm hệ quản trị cơ sở dữ liệu trung tâm của StrongWine nhờ hiệu năng xử lý giao dịch cao và độ tin cậy doanh nghiệp.

Hệ thống vận hành với cơ sở dữ liệu mang tên `StrongWineDB`, được cấu hình bộ mã ký tự quốc tế Unicode (`Vietnamese_CI_AS` hoặc `SQL_Latin1_General_CP1_CI_AS`) cho phép lưu trữ hoàn hảo tiếng Việt có dấu trong tên sản phẩm và địa chỉ giao hàng. SQL Server cung cấp cơ chế khóa dữ liệu cấp dòng (Row-level Locking) và cô lập giao dịch Snapshot Isolation, giúp ứng dụng duy trì thông lượng đọc cao đối với danh mục rượu mà không bị cản trở bởi các giao dịch ghi đơn hàng diễn ra đồng thời. Công cụ di chuyển lược đồ Flyway tự động thực thi các tập lệnh

migration tuần tự (`V1` đến `V13`), bảo đảm cấu trúc 20 bảng và dữ liệu mẫu seed luôn đồng bộ tuyệt đối trên mọi môi trường.

Việc định nghĩa chỉ mục (Index) trên các cột thường xuyên tìm kiếm như `wines.name`, `wines.category_id` và `orders.order_code` giúp tối ưu hóa đáng kể tốc độ phản hồi câu truy vấn.

2.10 Tổng quan về máy chủ Tomcat

Apache Tomcat được tích hợp trực tiếp bên trong ứng dụng Spring Boot dưới dạng Embedded Servlet Container, mang lại sự linh hoạt vượt trội trong đóng gói và vận hành.

Thay vì phải triển khai tệp gói WAR phức tạp lên một máy chủ ứng dụng độc lập bên ngoài, toàn bộ ứng dụng StrongWine được đóng gói thành một tệp thực thi duy nhất (Fat JAR). Khi khởi động, Spring Boot tự động kích hoạt máy chủ Tomcat nhúng trên cổng 8080 với cấu hình bộ nhớ và nhóm luồng (Thread Pool) được tối ưu hóa cho ứng dụng web tải vừa. Tomcat chịu trách nhiệm quản lý kết nối socket TCP/IP, phân phối các luồng xử lý cho từng yêu cầu HTTP đồng thời, và chuyển giao quyền xử lý ngữ cảnh đến `DispatcherServlet` của Spring. Khi cần thay đổi cấu hình như kích thước tải lên tệp ảnh rượu tối đa hay thời gian hết hạn phiên, lập trình viên chỉ cần khai báo trong `application.properties` mà không cần can thiệp tệp `server.xml`.

Mô hình máy chủ nhúng loại bỏ hoàn toàn các rủi ro sai lệch môi trường và giảm thiểu tối đa quy trình cấu hình khi đưa ứng dụng vào vận hành.

2.11 Các nguyên lý hướng đối tượng làm nền cho phân tích

Bộ nguyên lý thiết kế hướng đối tượng (OOP) theo lý thuyết đối tượng căn bản [2] đóng vai trò kim chỉ nam giúp phân rã và tổ chức các thành phần phức tạp của StrongWine một cách khoa học.

Bốn trụ cột cốt lõi của OOP - Đóng gói, Trừu tượng hóa, Kế thừa và Đa hình - được thể hiện xuyên suốt kiến trúc hệ thống. Tính đóng gói (Encapsulation) bảo vệ trạng thái của các thực thể như `Inventory` chỉ cho phép điều chỉnh số lượng tồn qua các thao tác có kiểm tra điều kiện âm kho. Tính trừu tượng hóa (Abstraction) ẩn giấu chi tiết kết nối cơ sở dữ liệu phía sau các giao diện `WineRepository` và ẩn giấu chi tiết API cổng thanh toán phía sau interface `PaymentService`. Tính kế thừa (Inheritance) cho phép các thực thể chia sẻ các thuộc tính chung như `createdAt`, `updatedAt` và `version` từ một

lớp cơ sở `BaseEntity`. Tính đa hình (Polymorphism) cho phép hệ thống gọi phương thức xử lý thanh toán tổng quát mà không cần biết chính xác đối tượng đang thực thi là tiền mặt COD hay cổng điện tử Stripe.

Việc vận dụng thuần thục các nguyên lý này giúp mã nguồn StrongWine có tính module hóa cao, dễ đọc, dễ kiểm thử và sẵn sàng cho các yêu cầu mở rộng tương lai.

2.11.1 Đơn trách nhiệm và phân lớp

Nguyên lý Đơn trách nhiệm (Single Responsibility Principle - SRP) yêu cầu mỗi lớp trong hệ thống chỉ có một lý do duy nhất để thay đổi.

Trong StrongWine, nguyên lý này được hiện thực hóa thông qua mô hình phân lớp nghiêm ngặt 3 tầng. Lớp `WineController` chỉ chịu trách nhiệm duy nhất về tiếp nhận tham số HTTP, gọi tầng dịch vụ và lựa chọn view hiển thị; nó tuyệt đối không chứa logic tính toán giá chiết khấu hay câu lệnh truy vấn SQL. Tương ứng, `WineService` chỉ tập trung vào quy tắc nghiệp vụ kiểm tra trạng thái sản phẩm và điều phối dữ liệu, hoàn toàn không phụ thuộc vào các đối tượng `HttpServletRequest` hay `HttpSession`. Tầng `WineRepository` chỉ đảm nhiệm duy nhất việc giao tiếp với cơ sở dữ liệu qua Hibernate. Khi giao diện người dùng thay đổi hoặc quy định tính thuế VAT thay đổi, chỉ có một lớp tương ứng bị ảnh hưởng mà không làm rung lắc toàn bộ hệ thống.

2.11.2 Mở đóng, thay thế Liskov và phân tách interface

Các nguyên lý OCP, LSP và ISP bảo đảm hệ thống StrongWine có thể mở rộng tính năng mới mà không phá vỡ tính ổn định của mã nguồn hiện hữu.

Nguyên lý Open/Closed (OCP) được áp dụng xuất sắc trong cơ chế nhập liệu: khi sàn StrongWine cần hỗ trợ nhập danh mục rượu từ tệp JSON thay vì chỉ có Excel, lập trình viên chỉ cần tạo mới một lớp `JsonImportStrategy` cài đặt giao diện `DataImportStrategy` mà không cần sửa đổi bất kỳ dòng mã nào trong bộ điều phối nhập liệu. Nguyên lý thay thế Liskov (LSP) bảo đảm rằng mọi cài đặt của `PaymentGateway` (dù là `StripePaymentGateway` hay `VnPayPaymentGateway`) đều có thể thay thế hoàn hảo cho nhau tại nơi sử dụng mà không làm sai lệch hành vi kỳ vọng của hệ thống. Nguyên lý phân tách Interface (ISP) chia nhỏ các giao diện cồng kềnh thành các giao

diện đặc tả hẹp như `ShipperAssignable`, `TrackableShipment`, giúp các lớp chỉ cần phụ thuộc vào đúng những phương thức mà chúng thực sự sử dụng.

Sự tuân thủ nghiêm ngặt bộ ba nguyên lý này tạo nên một hệ thống linh hoạt, chống lại sự xói mòn kiến trúc theo thời gian phát triển.

CHƯƠNG 2: KHẢO SÁT TỔNG QUAN STRONGWINE

StrongWine là ứng dụng thương mại điện tử rượu vang dựa trên kho mã nguồn [1] chạy bằng Java 21 và Spring Boot 4.0.0. Trong phạm vi checkout khảo sát, phần web dùng Spring MVC và Thymeleaf; API REST được dùng cho một số thao tác quản trị, đơn hàng, đánh giá và luồng trực tiếp. Dữ liệu nghiệp vụ được ánh xạ qua JPA và kết nối SQL Server. Mã nguồn còn có nhập/xuất bảng tính, thanh toán Stripe, thư điện tử, xử lý sự kiện và giao diện shipper.

Ứng dụng có hai kiểu tương tác. Trang máy chủ trả HTML cho các màn hình trình duyệt, còn API JSON phục vụ thao tác chọn lọc và tích hợp tuân thủ đặc tả RESTful OpenAPI 3.1 [11]. SSE giữ kết nối HTTP để đẩy cập nhật một chiều. Vì các kiểu này dùng cùng dữ liệu miền, quyền truy cập và quy tắc nghiệp vụ cần được bảo vệ ở phía máy chủ, không dựa riêng vào việc ẩn nút trên giao diện.

3.1 Ranh giới kiến trúc và thành phần

Kiến trúc của StrongWine được thiết kế theo mô hình phân tầng hướng dịch vụ với ranh giới trách nhiệm rõ ràng giữa các thành phần hiển thị, xử lý nghiệp vụ và lưu trữ.

Ở tầng ngoại vi, hệ thống tiếp nhận hai luồng tương tác chính: luồng giao diện người dùng dựa trên Thymeleaf phục vụ trình duyệt web máy tính/di động, và luồng API chuẩn REST phục vụ các tương tác thời gian thực và ứng dụng tác nghiệp của Shipper. Tầng Service trung tâm đóng vai trò nhạc trưởng điều phối các nghiệp vụ phức tạp liên quan đến nhiều thực thể cùng lúc như đơn hàng, thanh toán và kho vận. Tầng Persistence bao bọc cơ sở dữ liệu SQL Server thông qua Spring Data JPA, áp dụng cơ chế khóa lạc quan (Optimistic Locking với trường `@Version`) để bảo vệ dữ liệu tồn kho khỏi hiện tượng xung đột khi có hàng nghìn khách cùng đặt mua một chai rượu vang quý hiếm tại cùng thời điểm.

Ranh giới giữa các tầng được kiểm soát chặt chẽ bằng các DTO trung gian, ngăn chặn hiện tượng rò rỉ các thực thể cơ sở dữ liệu trực tiếp ra ngoài giao diện công khai.

3.2 Tác nhân, mục tiêu và kết quả mong đợi

Hệ thống StrongWine phục vụ ba nhóm tác nhân chính với các mục tiêu nghiệp vụ và kỳ vọng tương tác hoàn toàn biệt lập.

Khách hàng (Customer) hướng tới mục tiêu tìm kiếm nhanh chóng các chai rượu vang phù hợp thông qua bộ lọc đa tiêu chí, nắm bắt thông tin niên vụ, xuất xứ, tasting notes, quản lý giỏ hàng mượt mà, thực hiện thanh toán an toàn và theo dõi trạng thái đơn hàng thời gian thực. Quản trị viên (Admin/Manager) có mục tiêu điều hành toàn diện sàn thương mại: giám sát KPI doanh thu tổng quan trên Dashboard, quản lý danh mục sản phẩm và giá niêm yết, nhập liệu hàng loạt qua Excel, kiểm soát mức tồn kho an toàn và điều phối phân công vận đơn. Nhân viên giao hàng (Shipper) hướng tới mục tiêu tiếp nhận nhiệm vụ vận chuyển trong ca trực, cập nhật lộ trình giao nhận và hoàn tất đơn hàng bằng mã xác thực OTP bảo mật từ phía người nhận.

Sự phân định tác nhân rõ ràng kết hợp cùng hệ thống phân quyền Spring Security Role-based Access Control (RBAC) bảo đảm an toàn dữ liệu và tối ưu hóa hiệu suất làm việc cho từng nhóm người dùng.

Biểu đồ phân rã chức năng bên dưới được trích từ tài liệu cũ để cho thấy cách tài liệu trước chia hệ thống thành quản trị, khách hàng, giao vận và hạ tầng. Các nhãn như VNPay, Gemini, khóa pessimistic hoặc round-robin chỉ phản ánh mô hình cũ; chúng không được xem là chức năng đã chạy nếu chưa đối chiếu mã hiện tại.



Hình 2.1: Sơ đồ phân rã cấu trúc màn hình tổng thể toàn sàn StrongWine

Sơ đồ use case tiếp theo cũng là hình lịch sử. Một số chức năng trong đó, như trợ lý AI, VNPay và phân công tự động, cần kiểm tra riêng ở từng endpoint/service; hình không phải biên bản nghiệm thu hay đặc tả hiện hành.

3.3 Yêu cầu chức năng theo phân hệ

Danh mục và sản phẩm: hiển thị danh sách, tìm theo từ khóa, lọc theo loại rượu, khoảng giá, quốc gia, niên vụ, danh mục và trạng thái; xem chi tiết; quản trị tạo, sửa hoặc xóa mềm sản phẩm và danh mục. Các bộ lọc cần được chuẩn hóa để nhiều điều kiện có thể áp dụng đồng thời.

Tài khoản: hiển thị đăng nhập và đăng ký; xử lý yêu cầu quên mật khẩu/đặt lại; đưa người dùng sau đăng nhập đến khu vực phù hợp; giới hạn màn hình và thao tác theo authority. Các đường API và trang máy chủ cần cùng một chính sách sở hữu dữ liệu.

Đơn hàng và thanh toán: chỉ cho người dùng đã xác thực tiếp tục checkout; kiểm tra thông tin giao, dữ liệu mặt hàng và tồn hiện tại; tạo đơn ở trạng thái ban đầu; lưu lựa chọn thanh toán; tạo phiên Stripe khi phương thức được hỗ trợ; nhận callback/webhook; cho khách xem trạng thái và thử thanh toán lại nếu đơn còn đủ điều kiện.

Kho và quản trị: theo dõi tồn theo rượu và warehouse, lượng đang giữ, mức đặt lại; nhập/xuất có lựa chọn kho; ghi nhận inventory transaction; xem báo cáo; nhập Excel có bước preview/dry-run và xuất dữ liệu. Tính đúng đắn của lượng khả dụng phải xét phần đã giữ, không chỉ số vật lý.

Vận đơn và thông báo: tạo hoặc điều phối shipment cho đơn đã đủ điều kiện; phân công shipper theo khả năng sẵn có; cho shipper cập nhật tiến độ; xác thực OTP lúc hoàn tất; truyền sự kiện đến khách, admin hoặc shipper qua các stream riêng. Mỗi chuyển trạng thái cần có điều kiện và dấu vết tương ứng. [1]

3.4 Yêu cầu phi chức năng và điều kiện vận hành

Toàn vẹn dữ liệu yêu cầu ngăn chặn hiện tượng oversell khi nhiều checkout cùng tranh chấp một mặt hàng theo nguyên lý kiểm soát đồng thời của C.J. Date [10], đồng thời bảo đảm tính đáp ứng giao diện đa thiết bị theo Responsive Web Design [12], không tính sai giá từ dữ liệu client và không tạo lặp shipment khi sự kiện được xử lý lại. Độ tin cậy yêu cầu giao dịch chỉ bao trùm dữ liệu trong SQL; email, Stripe và SSE là các hệ thống ngoài, cần chiến lược bù, thử lại và đối soát.

Bảo mật yêu cầu kiểm tra quyền trên mỗi endpoint, xác nhận chủ sở hữu đơn hàng, giới hạn dữ liệu nhạy cảm trả ra, lưu mật khẩu dạng băm, bảo vệ thao tác thay đổi và

kiểm soát việc tiết lộ OTP. Khả dụng yêu cầu thông báo lỗi có thể hiểu được, trang shipper dùng được trên điện thoại và kết nối SSE có thể đóng lại an toàn.

Tính tương thích dữ liệu đặc biệt quan trọng vì dự án gắn với SQL Server đã có dữ liệu. Quy ước AGENTS.md yêu cầu Hibernate validate schema và cấm DDL phá hủy; mọi thay đổi cần additive, tương thích ngược và kiểm tra với cấu trúc hiện hữu. Môi trường thật còn cần khóa bí mật từ cấu hình, TLS, sao lưu và giám sát lỗi.

3.5 Tổng quan nghiệp vụ của đề tài

StrongWine là ứng dụng kinh doanh rượu trực tuyến với ba nhóm người sử dụng chính: khách hàng, quản trị viên và shipper. Chuỗi nghiệp vụ được khảo sát đi từ danh mục đến chi tiết sản phẩm, giỏ hàng, checkout, đặt chỗ tồn, thanh toán, tạo/điều phối vận đơn và xác nhận giao. Phía quản trị bao phủ danh mục, sản phẩm, tài khoản, tồn kho, đơn, payment, shipment và trao đổi Excel. Các màn hình realtime dùng SSE để thông báo các cập nhật được phát từ service.

Ranh giới của đề tài là mã nguồn StrongWine trong bản dự án được khảo sát và giao diện đã mở sau khi sửa UI. Dữ liệu khởi tạo và trạng thái trong cơ sở dữ liệu chỉ phản ánh môi trường local ở thời điểm khảo sát. Bài không khẳng định hệ thống đã tích hợp nhà vận chuyển thực, thanh toán production, định tuyến giao hàng theo khoảng cách tối ưu, hay đáp ứng mọi quy định ngành đồ uống có cồn; những nội dung đó cần được kiểm tra riêng và nằm trong phần rủi ro, đề xuất.

3.5.1 Nghiệp vụ khách hàng

Khách hàng có thể duyệt danh sách rượu, xem thuộc tính, lọc theo tiêu chí hỗ trợ, mở trang chi tiết, quản lý giỏ, nhập thông tin giao hàng, tạo đơn chờ thanh toán và xem lịch sử. Người dùng đã đăng nhập có thể truy cập các chức năng gắn với tài khoản; đánh giá được gửi qua route review hiện hành. Luồng quên mật khẩu có token với thời hạn cấu hình, nhưng việc gửi email phụ thuộc cấu hình dịch vụ và không được chứng minh bằng thao tác live trong lần rà soát.

Các điểm cần kiểm tra tại biên gồm: sản phẩm còn hoạt động, giá và số lượng hợp lệ, tồn thực có thể giữ, tổng tiền được tính lại phía server, đơn chỉ xem bởi chủ sở hữu hoặc quản trị có quyền, và trạng thái thanh toán không thể bị client tự đặt. Trang hiện

được dùng làm ảnh minh họa UI, không được xem như bằng chứng đã thành công với mọi nhánh checkout.

3.5.2 Nghiệp vụ quản trị viên và quản lý kho

Quản trị viên quản lý sản phẩm và danh mục, người dùng, hồ sơ shipper, đơn, giao dịch, tồn theo warehouse, import/export và shipment. Một số thao tác được cung cấp qua trang HTML và API riêng; lớp service là nơi nên duy trì nhất quán quy tắc. Dashboard giúp quan sát chỉ số tổng hợp nhưng con số trên màn hình cần phân biệt với doanh thu đã đối soát hoặc dữ liệu production.

Nhập/xuất kho không chỉ tăng/giảm con số tồn hiện tại. Hệ thống có `InventoryTransaction` ghi loại thao tác, số lượng, tham chiếu, người dùng và ghi chú; `StockLog` còn phục vụ thông tin tồn/biến động. Import Excel có bước tạo template, preview và ghi dữ liệu với chiến lược cụ thể. Khi thao tác quản trị có ảnh hưởng dữ liệu, cần ghi nhận người thực hiện, từ chối số lượng âm và xử lý xung đột cập nhật đồng thời.

3.5.3 Nghiệp vụ giao hàng của shipper

Shipper chỉ nên nhìn thấy các nhiệm vụ được phân công; có thể xác nhận lấy hàng, bắt đầu giao, báo lần giao thất bại và hoàn tất bằng OTP. Hồ sơ shipper lưu thông tin định danh/nghề nghiệp và có thể có tọa độ/vị trí cập nhật. Việc chọn người nhận nhiệm vụ trong code hiện tại là chính sách nội bộ dựa trên trạng thái/queue, không được mô tả như thuật toán tối ưu tuyến đường nếu mã không làm điều đó.

OTP giao hàng là một kiểm soát tại điểm bàn giao: mã 6 chữ số, có TTL, cooldown gửi lại, số lần thử và khóa tạm thời được định nghĩa trong service. Cần ràng buộc OTP với shipment/người nhận, không hiển thị mã trong dashboard shipper và ghi audit cho các thao tác tạo, gửi, xác thực thất bại/thành công. Việc đánh giá mã nguồn không tương đương với thử thực tế gửi email hay xác minh OTP trong giao dịch.

3.6 Tổng quan về nghiệp vụ kinh doanh rượu trực tuyến

Nghiệp vụ bán rượu có các quy tắc đặc thù hơn danh mục hàng hóa thông thường: năm sản xuất, quốc gia/vùng, phân loại, giá, mô tả và tình trạng hàng phải được trình bày chính xác; khách hàng cần thấy số lượng phù hợp; vận chuyển phụ thuộc địa chỉ và vùng giao. Báo cáo chỉ phân tích chức năng đã tìm được trong mã, không khẳng định

pháp lý về độ tuổi, giấy phép, thuế hoặc kiểm soát bán đồ uống có cồn đã được đáp ứng. Các nghĩa vụ tuân thủ đó phải được chủ dự án xác minh theo địa bàn triển khai.

Dữ liệu mẫu V13 làm phong phú danh mục rượu Việt Nam và các thuộc tính sản phẩm; seed là dữ liệu phát triển chứ không phải số liệu thị trường đã kiểm chứng. Tên nhãn, giá và mô tả trong database local không được đưa vào tiểu luận như nghiên cứu thị trường. Nếu cần phân tích kinh doanh, phải bổ sung nguồn khảo sát độc lập, thời gian thu thập và quy trình xác thực.

Việc bán trực tuyến nối nhiều bounded contexts: catalog, identity, cart/order, payment, inventory, logistics và notification. Bất biến của một context cần được bảo vệ dù UI đổi. Chẳng hạn, ẩn nút "mua" không đủ để ngăn API tạo đơn vượt tồn; trang shipper không đủ bảo vệ quyền truy cập nhiệm vụ nếu server không kiểm tra chủ thể. Phân tích theo miền giúp tìm ra điểm kiểm soát ở server và giữ vai trò UI như một lớp trải nghiệm.

3.7 Phân tích yêu cầu nghiệp vụ

Yêu cầu được tổng hợp từ controller, service, entity, migration, cấu hình và các màn hình đã mở, được mô hình hóa bằng sơ đồ ca sử dụng theo chuẩn OMG UML 2.5.1 [5]; tài liệu cũ chỉ đóng vai trò đối chiếu heading và vấn đề nghiệp vụ. Mỗi yêu cầu được chia thành ba mức: có route/lớp thể hiện trong mã; có thể quan sát qua UI; đã được thực thi end-to-end. Hai mức đầu có bằng chứng trong khảo sát, nhưng mức thứ ba chỉ đúng với hoạt động đã chạy thật. Cách phân loại này ngăn việc biến một route tồn tại thành tuyên bố mọi tích hợp đã hoạt động.

Đầu ra của phân tích là danh sách tác nhân, mục tiêu, quy tắc, dữ liệu, điều kiện trước/sau và ngoại lệ. Những yêu cầu liên quan ghi tiền, giảm tồn, phân công người giao, OTP hoặc gửi email phải chỉ ra thời điểm dữ liệu được ghi nhận và cách xử lý retry. Yêu cầu về giao diện phải nêu kích thước màn hình/route được kiểm tra. Các con số hiệu năng, khả dụng hay mức tăng doanh thu không được bịa; muốn đưa vào phải có baseline và phương pháp đo.

Phạm vi hiện trạng ghi nhận ba vai trò khách hàng, quản trị viên, shipper; hệ thống còn có tác vụ scheduler và API. Nhóm vận hành nội bộ không được tự thêm như một role nếu code không có authority tương ứng. Những mục tiêu hiện chưa có căn cứ triển

khai được đánh dấu là đề xuất, gồm định tuyến giao tối ưu, đối soát tự động toàn bộ payment, chống gian lận nâng cao và giám sát production.

3.7.1 Phân tích yêu cầu chức năng (Functional Requirements)

Nhóm khách hàng: FR-C01 duyệt danh sách và lọc sản phẩm; FR-C02 xem chi tiết rượu; FR-C03 đăng ký, đăng nhập và khôi phục mật khẩu; FR-C04 thêm/xóa/đổi số lượng giỏ; FR-C05 nhập thông tin giao hàng và tạo đơn chờ thanh toán; FR-C06 khởi tạo payment, xem kết quả và lịch sử; FR-C07 xem đơn của mình; FR-C08 gửi đánh giá theo endpoint review. Mỗi mục cần được kiểm chứng ở cả UI lẫn server, đặc biệt điều kiện danh tính và trạng thái.

Nhóm quản trị: FR-A01 quản lý danh mục; FR-A02 thêm/sửa/ẩn/xóa sản phẩm theo điều kiện code; FR-A03 xem và quản lý user/shipper; FR-A04 xem đơn và payment; FR-A05 xem tồn theo warehouse, cảnh báo mức thấp, nhập/xuất và lịch sử; FR-A06 xem/xử lý shipment, gán hoặc điều phối theo chính sách hiện hành; FR-A07 tạo template, xem trước, import và export Excel. Tài liệu phân tích ghi nhận rõ ràng rằng chưa khẳng định tất cả thao tác đều đã được thực hiện trực tiếp trong cơ sở dữ liệu khi chụp ảnh màn hình giao diện.

Nhóm shipper: FR-S01 xem shipment được cấp; FR-S02 xác nhận lấy hàng; FR-S03 bắt đầu giao; FR-S04 gửi lại OTP trong điều kiện cho phép; FR-S05 hoàn tất bằng mã hợp lệ; FR-S06 báo thất bại với lý do; FR-S07 nhận cập nhật realtime nếu đang kết nối. Quyền nhìn danh sách và quyền chuyển trạng thái là hai điều kiện riêng. API cần lấy shipper từ principal, không tin một `shipperId` tùy ý do browser gửi lên.

Yêu cầu hệ thống: FR-X01 ghi lịch sử shipment và audit OTP; FR-X02 duy trì transaction nhất quán cho đơn/tồn; FR-X03 sinh sự kiện sau khi đơn được ghi nhận thành công; FR-X04 phân phối event tới customer/admin/shipper bằng SSE; FR-X05 chạy nhiệm vụ dispatch theo scheduler; FR-X06 giữ khả năng đối chiếu thay đổi schema bằng Flyway. Các chức năng nền này không có màn hình riêng nhưng là một phần của luồng người dùng.

3.7.2 Phân tích yêu cầu phi chức năng (Non-Functional Requirements)

NFR-Security yêu cầu xác thực theo principal, phân quyền theo vai trò, kiểm tra quyền sở hữu đơn/shipment, lưu password theo cơ chế băm, không lộ bí mật trong log và giới hạn upload. NFR-Consistency yêu cầu không bán vượt tồn trong race condition, không ghi một đơn thanh toán thành công nhiều lần do callback lặp và không tạo nhiều shipment cho cùng sự kiện nếu listener bị gọi lại. Đây là tiêu chí thiết kế và kiểm thử; hiện trạng cần được kiểm chứng riêng bằng test đồng thời, webhook lặp và truy vấn DB.

NFR-Reliability yêu cầu ứng dụng phục hồi khi email/payment timeout, browser mất SSE và scheduler chạy lặp. Các sự kiện quan trọng cần có trạng thái lưu được, retry có giới hạn và cơ chế reconciliation. NFR-Usability yêu cầu thông báo trạng thái dễ hiểu, lỗi gắn với hành động, khả năng thao tác mobile cho shipper và bàn phím cho form. NFR-Accessibility bao gồm tương phản, nhãn input, focus, cấu trúc heading và thông báo lỗi có thể đọc bằng công nghệ hỗ trợ.

NFR-Performance cần đo thời gian response theo endpoint và kích thước dữ liệu đại diện; NFR-Scalability cần xem số connection SSE, dung lượng session và khả năng nhiều instance; NFR-Maintainability cần test cho service, tài liệu route và migration có thể tái lập. Không có số liệu load test hoặc production telemetry trong evidence của bài, do đó các mục này được trình bày như tiêu chuẩn cần đạt chứ không gắn nhãn “đã đạt”.

3.8 Phân tích tác nhân và quyền hạn

Khách hàng là người sở hữu giỏ, đơn, địa chỉ nhận và nội dung review theo mô hình phân quyền vai trò RBAC [8], [9]. Vai trò `USER` chỉ cho phép các route tương ứng; quyền truy cập một tài nguyên cụ thể còn cần so sánh user đang đăng nhập với owner của đơn/review. Admin có quyền rộng hơn với danh mục, tồn, user, payment và shipment nhưng vẫn phải được kiểm tra tại server. Shipper đại diện cho một hồ sơ giao hàng gắn với user; quyền với shipment phải dựa trên quan hệ phân công.

Ngoài actor người dùng còn có Stripe webhook, SMTP, SQL Server và scheduler. Stripe webhook là nguồn sự kiện bên ngoài, cần chữ ký và xác nhận nội dung; SMTP có thể từ chối hoặc timeout; SQL Server có transaction/constraint; scheduler có thể chạy trong lúc xử lý web. Các hệ thống này không phải “tác nhân người” trong sơ đồ use case nhưng là collaborator ảnh hưởng đến tính đúng đắn và khả năng phục hồi.

Ma trận quyền tối thiểu cần phân biệt quyền truy cập route và quyền trên object. Ví dụ `GET /orders/{id}` phải kiểm tra chủ sở hữu, không chỉ `authenticated`; endpoint hoàn tất shipment phải kiểm tra actor đang là shipper được gán; API admin phải cần role ADMIN. Các quy tắc này được mã hóa ở security configuration, controller/service và repository query. Bài hiện mô tả nơi cần kiểm tra, không suy luận mọi endpoint đã đạt yêu cầu nếu chưa chạy ca trái quyền.

3.9 Quy tắc nghiệp vụ cốt lõi

Đơn hàng cần lưu ảnh chụp thông tin nhận để đơn cũ không đổi khi user cập nhật hồ sơ, kết hợp kiểm soát khóa tồn kho theo lý thuyết của C.J. Date [10]. Một đơn chờ thanh toán giữ trước tồn kho; nếu thanh toán xác nhận thì giữ chỗ được xác nhận/trừ, nếu đơn bị hủy hoặc thanh toán thất bại theo quy tắc hệ thống thì tồn giữ chỗ được giải phóng. Quy tắc này tạo quan hệ giữa OrderService và InventoryService; thao tác hai service cần cùng transaction hoặc có bù trừ đủ rõ để tránh số dư sai.

Payment thành công được xác định qua dữ liệu server/gateway đã xác thực, không phải query parameter `success=true` của trình duyệt. Redirect thành công cải thiện trải nghiệm nhưng webhook là kênh cần xử lý độc lập, chữ ký và session cần so khớp payment/order/số tiền/trạng thái. Nhiều event trùng cần dẫn đến một lần chốt nghiệp vụ. Nếu hai đường callback đều gọi chung hàm finalize có điều kiện trạng thái, nó giảm nguy cơ ghi nhận trùng nhưng vẫn cần kiểm thử concurrency.

Shipment chỉ được hoàn tất khi trạng thái trước đó cho phép, đúng shipper hoặc admin override hợp lệ, và OTP khớp với người nhận dự kiến, chưa hết hạn, chưa bị khóa. Báo thất bại cần lý do và thời điểm; admin override cần ghi nguyên nhân. Mọi chuyển trạng thái nên tạo bản ghi history với actor, trạng thái trước/sau và thời gian. Audit OTP cần tránh lưu plaintext OTP trong metadata.

3.10 Phân tích yêu cầu theo nhóm màn hình

Trang catalog cần hỗ trợ tìm, lọc và xem thông tin sản phẩm mà không quá tải người dùng. Trang chi tiết cần nêu giá, mô tả, ảnh, năm/vùng và trạng thái sẵn hàng; nếu tồn được tổng hợp từ nhiều kho, cần xác định cách hiển thị. Giỏ nên cho phép chỉnh số lượng và nhìn thấy tổng ước tính, nhưng checkout phải tính lại giá/tồn từ server. Mobile cần bảo đảm nút hành động đủ lớn, không bị drawer hoặc header che.

Dashboard quản trị cần hiển thị KPI có định nghĩa rõ: số đơn nào được tính, trạng thái nào tạo doanh thu, khoảng thời gian nào áp dụng. Biểu đồ chụp được chỉ mô tả số liệu đang hiển thị tại thời điểm chụp; không coi là doanh thu đã đối chiếu. Màn hình danh sách cần lọc/phân trang khi dữ liệu lớn và không render toàn bộ payload nhạy cảm.

Dashboard shipper ưu tiên thao tác tại hiện trường: trạng thái trực tuyến, tab trạng thái, địa chỉ, số điện thoại liên hệ, nút xác nhận theo luồng và lỗi OTP dễ sửa. Ảnh lưu trong bài đã crop danh sách và thông tin người nhận theo yêu cầu riêng tư. Việc chụp không thực hiện thay đổi shipment. Giao diện cần xác nhận hành động không thể đảo ngược và ngăn double-click gửi nhiều request.

3.11 Đặc tả ca sử dụng: khách hàng đặt hàng

Tác nhân chính là khách hàng đã đăng nhập; điều kiện trước gồm sản phẩm còn hoạt động, giỏ có ít nhất một dòng hợp lệ và thông tin nhận đạt validation. Khách hàng xem giỏ, sửa số lượng, nhập tên/số điện thoại/email/địa chỉ, gửi checkout; hệ thống đọc giá/tồn hiện hành, xác thực từng item, tạo order pending và order items, giữ tồn, tạo payment theo phương thức chọn và trả kết quả điều hướng. Cần xác nhận lại tổng tiền trước khi chuyển cổng thanh toán.

Nhánh thay thế gồm sản phẩm vừa bị ẩn, tồn giảm bởi user khác, số lượng không hợp lệ, email giao OTP trống, lỗi địa chỉ, lỗi tạo order, lỗi tạo Stripe session hoặc browser đóng giữa chừng. Với lỗi xảy ra trước khi giao dịch được ghi nhận, đơn/giữ tồn không nên tồn tại một phần. Với lỗi sau khi payment provider đã tạo session nhưng DB chưa lưu, cần đối chiếu idempotency/session. Trạng thái UI phải cho phép khách quay lại và thử lại mà không tạo đơn trùng.

Hậu điều kiện là order có định danh và trạng thái đúng, payment được ghi nhận ở trạng thái ban đầu, tồn được giữ theo quy tắc và màn hình nhận mã đơn. Ca thử đề xuất: checkout thành công với tồn đủ; vượt tồn; gửi giá client bị sửa; gửi cùng request hai lần; transaction lỗi ở giữa; session timeout; user khác truy cập order ID. Kết quả hiện chưa được chạy như bộ kiểm thử end-to-end trong lần này.

3.12 Đặc tả ca sử dụng: xác nhận thanh toán

Tác nhân là gateway Stripe thông qua redirect và webhook; khách hàng không có quyền trực tiếp đổi order thành PAID. Khi tạo session, server liên kết payment, order,

amount và currency; browser được chuyển đến provider. Redirect quay lại được xác thực bằng session id và user context; webhook cần kiểm tra chữ ký, event và metadata. Quy tắc hoàn tất phải dùng cùng một hàm miền có điều kiện trạng thái và lưu payment transaction để truy vết.

Nhánh lỗi gồm signature thiếu/sai, session không tìm thấy, amount/currency không khớp, payment đã finalized, webhook đến trước redirect, redirect đến trước webhook, event lặp, event đến sai thứ tự, DB lỗi khi lưu kết quả và provider timeout. Xử lý webhook thành công nhưng shipment listener lỗi sau khi giao dịch nguồn hoàn tất cần được đưa vào hàng đợi/reconciliation; rollback order không thể khôi phục event đã nhận từ provider.

Kịch bản kiểm chứng gồm chữ ký sai phải bị từ chối; session thuộc order khác không được xác nhận; hai webhook cùng event ID không tạo payment transaction nghiệp vụ lặp; success/cancel không làm trạng thái đảo qua lại; order chỉ sinh một shipment; số tiền dùng từ payment server đã lưu. Các test sandbox phải có tài khoản/test key và xác nhận riêng, chưa được gọi trong bài này.

3.13 Đặc tả ca sử dụng: quản lý sản phẩm

Tác nhân là ADMIN. Admin mở form, nhập tên/danh mục/năm/giá/quốc gia/mô tả và chọn ảnh; server validate dữ liệu, kiểm tra danh mục hoạt động, lưu entity và file theo quy tắc storage rồi trả thông báo. Khi cập nhật, chỉ trường được phép mới thay đổi; khi xóa, cần xem tham chiếu từ order item/review/inventory trước khi hard delete. UI nên cung cấp disable/ẩn sản phẩm nếu giữ lịch sử là quan trọng.

Tài liệu cũ có hoạt động và sequence diagram thêm sản phẩm với route cũ. Sơ đồ ghi `/admin/wines/create` và xử lý ảnh cụ thể; báo cáo hiện đối chiếu với controller `/wines/admin/create` và các API admin hiện hành, nên hình cũ chỉ dùng để minh họa các bước nhập form, validate, persist và phản hồi, không coi là routing chính thức. Luồng upload cần kiểm tra loại file theo nội dung, tên ngẫu nhiên, dung lượng, đường dẫn an toàn và xử lý khi file được ghi nhưng DB rollback.

Ca kiểm chứng nên gồm tạo sản phẩm hợp lệ; thiếu tên; giá âm; danh mục không tồn tại; file giả định dạng ảnh; file quá giới hạn; trùng request; cập nhật có ảnh mới; lỗi lưu file; xóa sản phẩm đang được đơn tham chiếu. Mỗi test ghi rõ trạng thái entity, file

còn lại, phản hồi UI/API và log. Trong lần rà soát này không chạy CRUD nên kết quả các ca trên là kế hoạch, không phải kết quả thực nghiệm.

3.14 Đặc tả ca sử dụng: gửi đánh giá sản phẩm

Tác nhân là user đã xác thực và chọn một wine. Route hiện hành có controller MVC `/reviews/create` và controller API riêng; tài liệu cũ dùng `/reviews/add`. Trước khi ghi, service cần xác định wine hợp lệ, user tồn tại, nội dung/comment phù hợp và có thể áp dụng chính sách một review mỗi user/sản phẩm. Nếu yêu cầu đánh giá chỉ sau mua hàng, cần kiểm tra order item và trạng thái thanh toán; không nên suy điều kiện đó từ sơ đồ cũ.

Kết quả gồm review được lưu và gắn thời gian, phản hồi thành công, hoặc lỗi validation/authorization. Review của user khác không được sửa/xóa tùy ý; API nhận userId từ body cần đối chiếu với principal. Nội dung nhập cần escape khi hiển thị để ngăn script. Aggregate rating cần xử lý mẫu số rỗng và cập nhật nhất quán khi sửa/xóa review.

Sơ đồ hoạt động/trình tự review cũ vẫn được đưa vào phụ lục với chú thích route và điều kiện cần đối chiếu. Ca kiểm thử mới cần xác nhận nội dung trống, user chưa đăng nhập, wine không tồn tại, review trùng, user giả mạo ID, payload chứa HTML, cập nhật/xóa review và khả năng thay đổi rating trung bình. Không có thao tác gửi review trong live QA của báo cáo.

3.15 Phân tích yêu cầu dữ liệu và lưu vết

Mỗi giao dịch có định danh ổn định để màn hình, API và audit tham chiếu cùng đối tượng. `InventoryTransaction` liên kết wine, warehouse, loại operation, reference type/id, người dùng và ghi chú; `PaymentTransaction` lưu event type/status/payload phục vụ điều tra; `ShipmentStatusHistory` lưu from/to, lý do, metadata và actor; `ShipmentOtpAuditLog` theo dõi hành động OTP. Bốn loại log này có mục đích khác nhau và không nên gộp tùy tiện thành một bảng chung.

Payload/log cần giữ đủ thông tin hỗ trợ nhưng hạn chế dữ liệu cá nhân và bí mật. Địa chỉ giao hàng là dữ liệu nhạy cảm; OTP và token reset không nên xuất hiện dưới dạng có thể sử dụng lại; dữ liệu Stripe không nên lưu bí mật ngoài chính sách retention.

Trước production, cần xác định thời gian lưu, phân quyền đọc log, masking, sao lưu và quy trình xóa theo yêu cầu pháp lý.

Khóa ngoại đảm bảo tham chiếu đến đối tượng tồn tại nhưng chưa tự đảm bảo chuyển trạng thái hợp lệ. Unique constraint có thể ngăn cart trùng user, inventory trùng cặp wine/warehouse hoặc một payment/session được gắn sai. Cần đối chiếu migration SQL và entity annotation để biết chính xác các constraint hiện có; mục này không bịa tên constraint chưa được xem trong source.

3.16 Phân tích yêu cầu phi chức năng theo mức ưu tiên

Nhóm P0 liên quan tính toàn vẹn giao dịch: không oversell, idempotent payment, không ship hai lần, không xác nhận giao nếu OTP sai/expired, và ngăn truy cập chéo đơn. Nhóm P1 liên quan vận hành: reconciliation cho order paid thiếu shipment, retry email giới hạn, audit truy cập, thông báo lỗi và metrics. Nhóm P2 tập trung trải nghiệm: responsive mobile, hỗ trợ bàn phím, empty state, skeleton/loading, filter dễ hiểu. Thứ tự này là đề xuất dựa trên tác động mất tiền/dữ liệu và trải nghiệm.

Tiêu chí đo cần cụ thể hóa trước khi nghiệm thu: tỉ lệ checkout thành công; tỷ lệ order/payment mismatch; số lượng tồn âm; thời gian từ paid đến assigned; tỷ lệ OTP delivery; reconnect SSE và thời gian đồng bộ lại. Đưa mục tiêu như “99.9% uptime” mà không có hạ tầng, khoảng thời gian và đo lường không mang giá trị nghiệm thu. Bài này không có telemetry nên không công bố các tỷ lệ.

Kết quả ma trận yêu cầu được nối với controller/service/entity, UI screenshot hoặc kế hoạch test. Các FR có route tương ứng nhưng không có test live sẽ được đánh dấu “có trong source, chưa thực nghiệm”. Các NFR cần chỉ rõ loại chứng cứ cần thêm: unit test, integration test, sandbox provider, load test, accessibility audit, security review hoặc production dashboard. Cách truy vết này cho phép người đọc phân biệt hiện trạng với roadmap.

CHƯƠNG 3: PHÂN TÍCH MÔ HÌNH MIỀN VÀ THIẾT KẾ HƯỚNG ĐỐI TƯỢNG

Mô hình miền xoay quanh sản phẩm, tài khoản, đơn, thanh toán, tồn và giao nhận. Quan hệ này không chỉ là sơ đồ dữ liệu: mỗi service phải phối hợp thay đổi trạng thái theo quy tắc và entity cần giữ dữ liệu đủ để giải thích quá trình về sau.

4.1 Nhóm đối tượng sản phẩm và danh mục

Thực thể Wine trong mô hình miền [1] lưu tên, loại, niên vụ, giá, mô tả, quốc gia, hình ảnh, danh mục và trạng thái xóa theo nguyên lý đóng gói dữ liệu của Java [2]; Category gom nhóm sản phẩm. Inventory liên kết một sản phẩm với một kho, còn Review gắn nhận xét và điểm đánh giá. Quan hệ tới Inventory giúp sản phẩm có thể tồn tại ở nhiều địa điểm với lượng riêng biệt thay vì một con số tổng.

Tìm kiếm tách mô tả bộ lọc khỏi controller. WineSpecification xây điều kiện động theo các giá trị lọc và Spring Data JPA nhận Specification làm predicate. Đây là điểm hữu ích khi trường hợp lọc mở rộng, nhưng cần kiểm tra cách xử lý giá trị rỗng, giới hạn khoảng giá, sort, phân trang, trạng thái xóa mềm và chỉ mục cơ sở dữ liệu. [7]

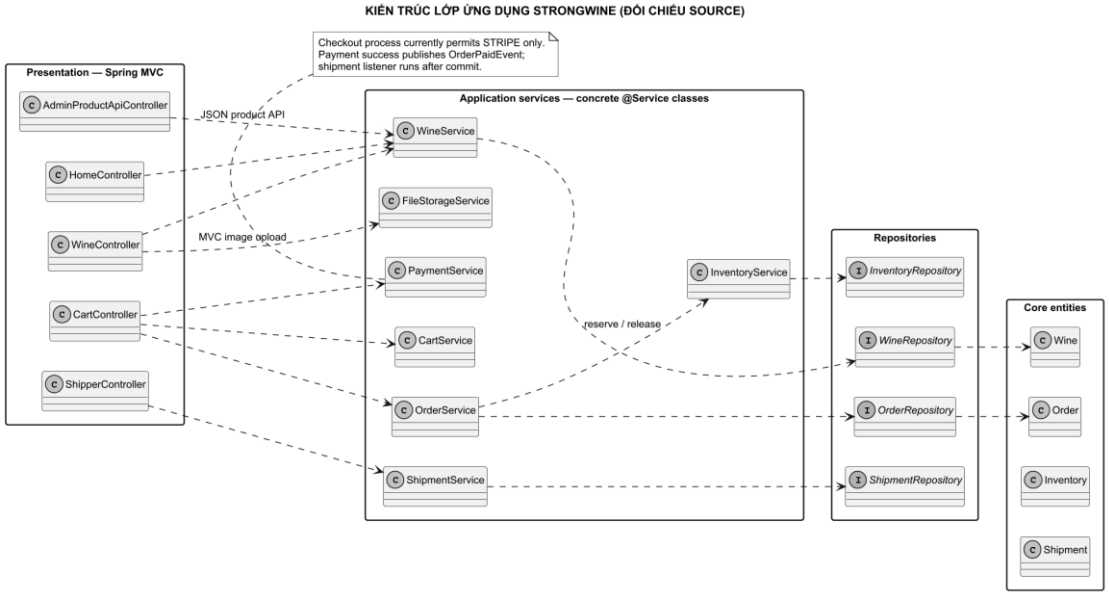
4.2 Nhóm đối tượng tài khoản, giỏ và đơn

User mang danh tính đăng nhập và quan hệ sở hữu với dữ liệu người dùng. Cart và CartItem xuất hiện trong mô hình lưu trữ, trong khi luồng checkout web cũng nhận danh sách CartItemRequest và kiểm tra lại dữ liệu bằng CartService. Order gắn với User, địa chỉ nhận, thông tin liên hệ, trạng thái đơn, trạng thái thanh toán, phương thức và tổng tiền. OrderItem giữ rượu, số lượng và đơn giá áp dụng tại thời điểm tạo dòng đơn.

Giữ snapshot đơn giá trong OrderItem giúp bản ghi lịch sử không phụ thuộc hoàn toàn vào giá hiện tại của Wine. Tương tự, thông tin người nhận được lưu trên Order/Shipment để việc sửa profile sau này không làm thay đổi dữ liệu giao cũ. Các trường snapshot cần được ghi nhất quán giữa đơn và vận đơn và được giới hạn khi hiển thị cho vai trò khác nhau.

Sơ đồ lớp kiến trúc phân tầng (Layered Architecture) bên dưới minh họa cấu trúc phân tách trách nhiệm hoàn chỉnh trong hệ thống StrongWine: Tầng Presentation (Controller) tiếp nhận request HTTP/REST API; Tầng Application/Service điều phối use case nghiệp vụ; Tầng Domain chứa các thực thể Entity kế thừa Auditable; và Tầng

Infrastructure/Repository thực hiện truy xuất dữ liệu an toàn qua Spring Data JPA và Hibernate.



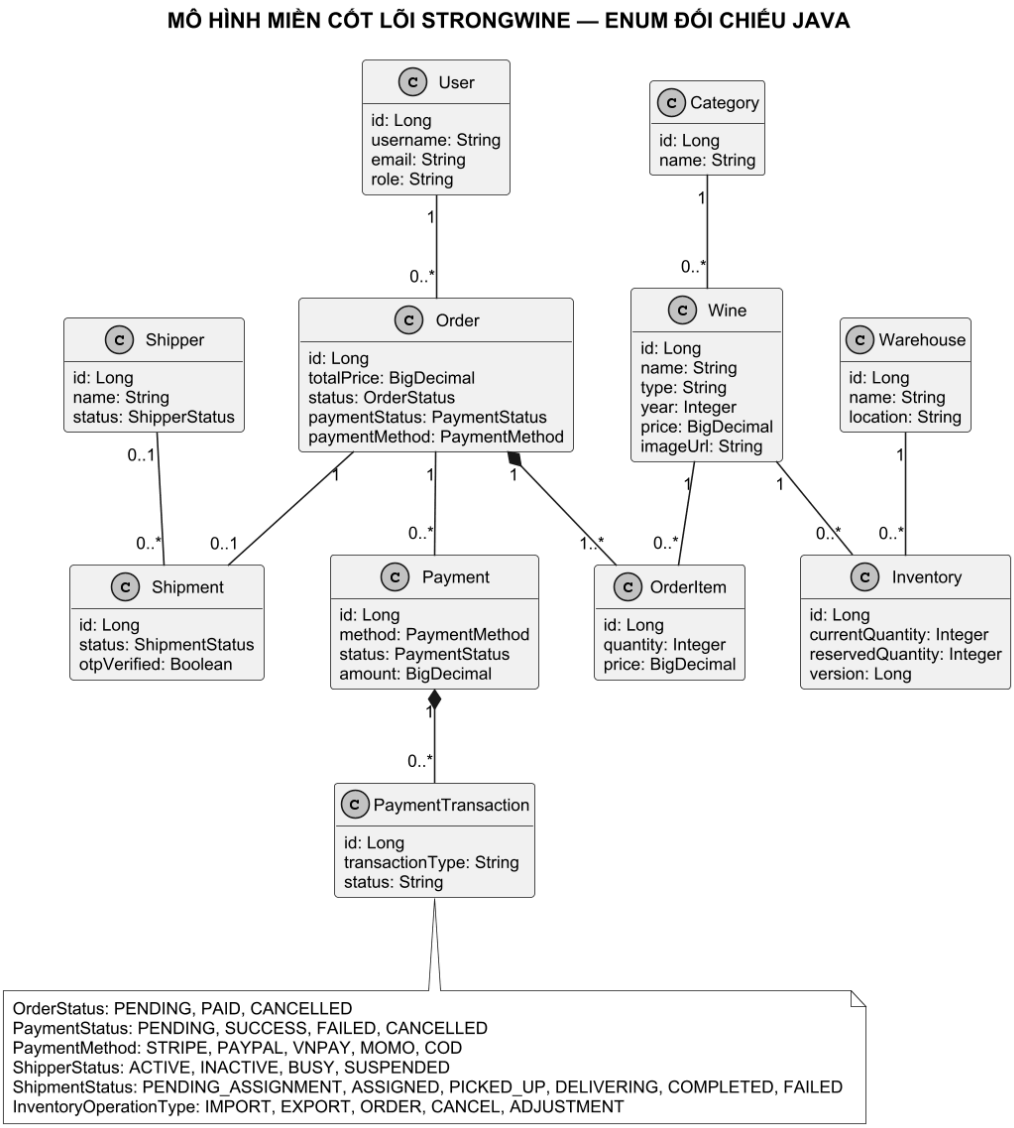
Hình 3.1: Sơ đồ lớp kiến trúc phân tầng (Presentation - Service - Repository - Entity) của StrongWine

4.3 Nhóm đối tượng tồn kho, thanh toán và vận đơn

Inventory gắn Wine với Warehouse, theo dõi lượng thực có, lượng giữ, ngưỡng đặt lại và version để kiểm soát tranh chấp đồng thời theo C.J. Date [10]. InventoryTransaction tạo dấu vết tăng/giảm hoặc điều chỉnh. Payment lưu phương thức và trạng thái liên quan đến Order; PaymentTransaction ghi từng lần tương tác hoặc tham chiếu giao dịch. Shipment gắn đơn hàng, shipper, người nhận, trạng thái, mốc thời gian và dữ liệu xác nhận giao. ShipmentStatusHistory cùng audit/notification log hỗ trợ xem lại diễn biến.

Các enum cho trạng thái giúp giới hạn giá trị trong code nhưng không tự buộc mọi chuyển trạng thái hợp lệ. Dịch vụ cần từ chối bước sai, chẳng hạn hoàn tất trước khi bắt đầu giao hoặc sử dụng OTP đã hết hạn. Tương tự, một enum PaymentMethod chứa giá trị không chứng minh service có connector tương ứng.

Sơ đồ miền bên dưới trình bày tập entity cốt lõi gồm User, Category, Wine, Warehouse, Inventory, Order, OrderItem, Payment, Shipment, Shipper và PaymentTransaction, cùng các quan hệ JPA được khảo sát. Đây không phải toàn bộ entity graph; các enum được liệt kê theo literal Java hiện hành. Quan hệ composition chỉ dùng ở những cặp có vòng đời thành phần phù hợp.



Hình 3.2: Mô hình miền cốt lõi StrongWine và enum Java hiện hành

4.4 Đối chiếu các nguyên lý hướng đối tượng

Đóng gói thể hiện ở việc service kiểm tra đầu vào và điều phối thay đổi nhiều đối tượng; InventoryService là nơi phù hợp để tập trung quy tắc giữ/giải phóng hàng. Tuy vậy, entity JPA thường có setter để framework ánh xạ. Cần đánh giá nơi setter được gọi, phạm vi public, việc đổi trạng thái có phương thức ngữ nghĩa hay không và liệu bất biến có thể bị phá vỡ từ controller.

Trừu tượng thể hiện qua interface strategy cho import/export và lớp cơ sở cho báo cáo Excel. Bên gọi làm việc với hợp đồng dữ liệu thay vì biết chi tiết của Apache POI ở mọi chỗ. Đa hình xuất hiện khi từng implementation xử lý định dạng hoặc báo cáo cụ thể nhưng vẫn được sử dụng qua interface/abstract class.

Kế thừa được dùng để chia sẻ phần audit giữa Category và Wine. Quan hệ Order–OrderItem, Order–Payment và Wine–Inventory phù hợp hơn với kết hợp đối tượng: chúng mô tả một đối tượng có chứa hoặc liên kết đối tượng khác, không phải một lớp là một loại của lớp kia. Điều chỉnh sơ đồ UML theo quan hệ nghiệp vụ giúp tránh dùng kế thừa chỉ để tái sử dụng trường.

Nguyên tắc phụ thuộc vào trừu tượng được hỗ trợ bởi việc Spring tiêm bean và publisher vào service. Mức độ áp dụng chưa đồng đều vì một số lớp vẫn dùng field injection. Đánh giá thiết kế cần xem module cụ thể, không suy rộng chất lượng toàn hệ thống từ một ví dụ tốt.

4.5 Strategy trong nhập và xuất dữ liệu

DataImportStrategy<T> áp dụng mẫu thiết kế Strategy theo Gang of Four [4] để đặt hợp đồng cho một chiến lược nhập dữ liệu; ExcelWineImportStrategy chuyển các dòng bảng tính thành đối tượng Wine và thực hiện kiểm tra liên quan. DataExportStrategy<T> tách cách tạo đầu ra khỏi endpoint. Controller có thể xử lý upload/response trong khi lớp chuyên trách xử lý định dạng và ánh xạ dữ liệu.

Thiết kế này cho phép bổ sung nguồn hoặc định dạng mới mà không đưa mọi nhánh xử lý vào controller. Điểm cần giữ là mỗi implementation phải thực hiện cùng các kiểm tra bảo mật, validation và quy tắc kho; tên interface không bảo đảm các strategy tương đương về an toàn. Kiểm thử nên dùng cùng tập dữ liệu chuẩn để so sánh kết quả của từng format.

`DataImportStrategy`/`DataExportStrategy` tách hợp đồng khỏi định dạng/loại dữ liệu cụ thể; `ExcelWineImportStrategy` xử lý file rượu và trả `ImportResult` gồm success/error, thông tin row/field/invalid value. Lựa chọn chiến lược qua tên/class hoặc registry cho phép thêm loại import mà không biến controller thành khối if/else lớn. Đây là Strategy vì caller có thể dùng một implementation theo cùng interface.

Import cần hai pha: parse/validate trước, rồi ghi có kiểm soát. `ImportOptions` có dryRun, inventory mode và defaultWarehouseId; dry-run phải không làm side effect; lỗi một row cần xác định import all-or-nothing hay partial success. Hiển thị row/field giúp admin sửa file. Dữ liệu trong Excel không đáng tin: ô formula, kiểu ngày, số scientific notation, chuỗi dài, duplicate row và category không hợp lệ cần xử lý.

Export strategy cũng cần xác định snapshot thời điểm, thứ tự cột, encoding, date/currency format và quyền tải file. File có thể chứa dữ liệu inventory nhạy cảm, vì vậy endpoint cần quyền admin và log thao tác. Test dùng workbook tổng hợp, không gửi file dữ liệu khách thật.

4.6 Specification trong tìm kiếm rượu

WineSpecification đóng gói điều kiện động theo cơ chế Spring Data JPA Specifications [7] để truy vấn danh sách rượu. Thay vì viết một phương thức repository cho mọi tổ hợp bộ lọc, từng điều kiện có thể được tạo khi tham số tương ứng có giá trị và kết hợp theo logic AND/OR được thiết kế. Cách làm này giảm số nhánh ở controller và làm predicate có khả năng dùng lại.

Để kết quả chính xác, quy tắc tìm kiếm cần định nghĩa rõ việc chuẩn hóa chữ hoa/thường, dấu tiếng Việt, khoảng biên giá, trường hợp năm không có dữ liệu, danh mục đã xóa, thứ tự mặc định và phân trang. Test nên bao phủ điều kiện đơn, nhiều điều kiện đồng thời, giá trị biên, không có kết quả và quyền truy cập endpoint.

4.7 Template Method trong báo cáo Excel

AbstractExcelReportGenerator<C> áp dụng mẫu thiết kế Template Method [4] để giữ trình tự chung tạo workbook, trong khi InventoryStockReportGenerator cung cấp nội dung báo cáo tồn kho như số thực có, lượng giữ, lượng khả dụng và mức cảnh báo. Đây là cấu trúc Template Method: thuật toán tổng thể được giữ ở lớp cơ sở, phần thay đổi thuộc lớp con.

Lợi ích là định dạng chung và xử lý tài nguyên không cần lặp lại giữa nhiều loại báo cáo. Rủi ro là lớp cơ sở có thể trở nên cứng nếu mỗi báo cáo đòi hỏi một luồng hoàn toàn khác; cần giữ các hook nhỏ, rõ và kiểm tra workbook cuối cùng, dữ liệu thiếu, Unicode, kiểu số, tên sheet và tệp lớn. Chưa thấy bằng chứng đủ để mô tả một Factory cụ thể; báo cáo chỉ đề xuất thêm registry/factory nếu số implementation làm việc chọn generator trở nên phức tạp.

`AbstractExcelReportGenerator` định nghĩa khung tạo báo cáo: chuẩn bị workbook/sheet, tạo header, ghi từng dòng, áp format và xuất stream; `InventoryStockReportGenerator` cài phần dữ liệu/column cho tồn kho.

`ExcelTemplateService` cung cấp template import. Nếu lớp con chỉ thay các bước được định nghĩa còn thứ tự tổng thể ổn định, đây là Template Method.

Lợi ích là mọi báo cáo thống nhất format và xử lý workbook; rủi ro là base class phình thành framework nội bộ, chứa quá nhiều hook hoặc lớp con bị buộc theo thứ tự không phù hợp. Khi thêm report, cần xác nhận các bước invariant và những bước biến thể. Nếu mỗi báo cáo có quy trình rất khác, composition/service riêng có thể dễ hiểu hơn inheritance.

Kiểm thử phải mở workbook xuất ra và xác minh header, dữ liệu, kiểu cell, sheet name, công thức/format nếu dùng; chỉ kiểm tra HTTP 200 không đủ. Import test phải xác minh DB thay đổi/không thay đổi đúng dryRun. Sự cố file lớn, malformed workbook và upload vượt limit cần được ghi nhận bằng lỗi có thể xử lý.

4.8 Publisher và sự kiện sau khi giao dịch hoàn tất

OrderService công bố `OrderPaidEvent` sau khi cập nhật trạng thái paid và xác nhận tồn. `OrderPaidShipmentListener` dùng @TransactionalEventListener gọi `ShipmentService`, bảo đảm shipment không được tạo nếu transaction order rollback. Sự kiện chứa order id/payment reference, không nên mang nhiều entity lazy hoặc PII.

Listener bắt exception và log lỗi để không làm đổi kết quả transaction đã hoàn tất. Điều này tránh rollback không thể thực hiện nhưng tạo khoảng hở: payment đã paid mà shipment chưa tạo. `PaidOrderShipmentBackfillInitializer`/reconciliation có thể bù một phần, nhưng startup/web reconciliation hiện cấu hình disabled mặc định; cần xem scheduler và runbook khi production. Listener nên idempotent, shipment có unique order constraint hoặc guarded lookup.

Test: event chỉ chạy sau khi giao dịch hoàn tất; rollback không gọi listener; listener lỗi được log; event lặp không tạo shipment trùng; reference được lưu; backfill chỉ xử lý paid orders thiếu shipment và có giới hạn. Nếu cần giao hàng đáng tin cậy, transactional outbox bền vững hơn event in-memory vì survives process crash.

4.9 Scheduler và dispatch queue

`ShipmentAutoDispatchScheduler` chạy sau delay ban đầu cấu hình 15 giây rồi mỗi 60 giây theo mặc định, nếu `app.shipment.dispatch.enabled` bật. Scheduler gọi `dispatchAutoShipmentQueue()` và chỉ log khi có shipment được xử lý. Đây là job nội

bộ, không phải người dùng bấm nút; cần có lock/lease khi nhiều app instances để tránh cùng gán một shipment.

Queue cần có trạng thái pending rõ, retry/backoff, giới hạn batch và điều kiện shipper sẵn sàng. Nếu không có shipper khả dụng, shipment vẫn cần giữ trong queue thay vì biến mất. Chọn shipper cần deterministic tie-break, fairness và kiểm tra trạng thái profile trong transaction. Tọa độ hiện có không đồng nghĩa với định tuyến đường.

Kiểm thử scheduler bằng clock giả và gọi service idempotent: pending rỗng, shipper rảnh, tất cả bận, shipper inactive vừa đổi trạng thái, scheduler lặp, hai instance cùng chạy, database timeout. Quan sát metrics về queue age/attempt/failure quan trọng hơn một log "processed count" duy nhất.

4.10 FileStorageService và upload ảnh

`FileStorageService` là adapter cho filesystem; nó cần tạo tên an toàn, kiểm soát extension/content type, ngăn path traversal, tránh ghi đè và lưu đường dẫn có thể phục vụ bởi web. Upload giới hạn 5 MB trong cấu hình, nhưng MIME do browser gửi không đủ tin cậy. Dữ liệu có thể là ảnh polyglot hoặc file giả extension, nên cần sniff content, decode/re-encode khi phù hợp và virus scan ở môi trường sản xuất.

Lưu file và ghi dữ liệu vào database là hai hệ thống khác nhau. Nếu file được ghi trước khi database rollback, file orphan phải được dọn; nếu DB lưu path trước khi file ghi thành công, sản phẩm sẽ có broken image. Có thể dùng temporary staging + rename sau khi giao dịch hoàn tất, compensation hoặc background cleanup. Tên file user gửi không nên dùng trực tiếp trong filesystem path.

Test cần gồm path traversal, tên trùng, extension mismatch, file empty, ảnh quá lớn, permission denied, thiếu storage directory, lỗi DB sau khi lưu file và xóa/thay ảnh. Bài này không thực hiện upload CRUD live; ảnh giao diện sản phẩm trong trang web không được mặc định là ảnh do người dùng tải lên trong test.

4.11 Authentication, SecurityContext và role mapping

`UserDetailsServiceImpl` tải user và chuyển role sang authority; Spring Security quyết định quyền theo matcher/authority. Web session và API token là hai bối cảnh có thể khác; endpoint nào không stateless phải biết CSRF behavior. Cấu hình hiện tại có

JWT components và `/api/auth/login`, cùng form `/login`; phải kiểm tra filter chain thực sự để phân biệt route dùng session, token hoặc cả hai.

Mỗi API nhận owner identity từ `Authentication` thay vì `userId` request. Ownership checks nên ở service/repository query như `findByIdAndUserId`, tránh IDOR. Shipper similarly chỉ cập nhật shipment đã gán cho profile tương ứng. Role ADMIN không thay thế kiểm tra tác động cụ thể như không thể xóa shipment đã trả hàng theo quy tắc.

Security review cần map mọi endpoint tới auth, role, CSRF, CORS, rate limit, validation, response fields và audit. Mọi secret phải đến từ environment/secret manager; giá trị placeholder trong local config không dùng production. Hồ sơ kỹ thuật tuyệt đối không trích xuất mật khẩu mặc định hay mã khóa bảo mật từ tệp cấu hình `.env` nhằm tuân thủ nguyên tắc an toàn thông tin.

4.12 API JWT và phiên đăng nhập

JWT có thời hạn cấu hình mặc định 24 giờ; secret được lấy từ `APP_JWT_SECRET`. Token cần được ký bằng thuật toán mạnh, kiểm tra issuer/audience/expiry, không ghi log và xoay vòng key có kế hoạch. Nếu secret rỗng trong môi trường chạy, behavior phải fail closed; không dùng fallback bí mật yếu. API login cần giới hạn tốc độ và trả thông điệp không tiết lộ account tồn tại.

Cookie/session có thuộc tính Secure, HttpOnly, SameSite và thời hạn riêng; bearer token trong localStorage dễ bị đọc bởi XSS. CSRF protection khác nhau theo kiểu credentials. Vì cấu hình có form MVC cùng REST, việc disable CSRF toàn cục cần được đánh giá trước production. Endpoint webhook không dùng CSRF nhưng cần signature, replay defense và body raw nguyên vẹn.

Test: token expired, tampered, wrong signature, role changed after token issued, user disabled, refresh/revoke, CSRF form, CORS origin lạ, brute-force login và route admin/shipper. Không đưa JWT thật vào screenshot hay tài liệu.

4.13 Dịch vụ email và kết quả giao OTP

`ShipmentOtpEmailService` tạo message cho recipient, có cờ enable/fallback, retry tối đa cấu hình và backoff. `OtpDeliveryResult` biểu diễn kết quả gửi thay vì service coi mọi lần gọi sender là thành công. Khi mail disabled, code có thể báo lỗi hoặc

dùng fallback theo cấu hình; production cần recipient thật, TLS, SPF/DKIM/DMARC và không dùng inbox dev làm hợp đồng giao nhận.

OTP giao hàng và token reset mật khẩu đều gửi qua email nhưng phục vụ mục đích khác và cần template/audit riêng. Resend cooldown hạn chế spam; retry email không nên tạo OTP mới mỗi lần nếu mã cũ còn hợp lệ, trừ khi chính sách ghi rõ. Nếu gửi thất bại, status shipment/OTP phải cho biết khách cần làm gì và admin có thể xử lý mà không xem plaintext.

Test mail dùng fake SMTP/local sink, không gửi thật; kiểm tra TLS config, timeout, retry count, fallback disabled, recipient mismatch và log redaction. Trong lần rà soát không gọi provider email thật, vì vậy các ảnh không minh chứng OTP đã tới inbox.

4.14 Xử lý lỗi và observability

Log cần trả lời request nào, order/shipment/payment nào, bước nào thất bại, actor loại nào và có thể retry hay không. Không ghi toàn bộ request body, OTP, password, token reset, payment payload hoặc địa chỉ. Một correlation id theo request/event nên được truyền qua controller, service và listener; scheduler cần job id/attempt.

Metrics nên có số checkout fail theo nguyên nhân, payment mismatch, order paid thiếu shipment, queue backlog/age, email success/failure, OTP rejection/lock, SSE active emitter/disconnect và import error rate. Alert phải dựa trên ngưỡng có ý nghĩa. Stacktrace ở log nội bộ có ích, nhưng không gửi cho client.

Khi thao tác thất bại sau một phần side effect, status response cần hướng dẫn bước tiếp theo; runbook phải chỉ rõ query kiểm tra, retry an toàn và người duyệt. Việc tự update DB bằng SQL chỉ dành cho incident có backup/audit/approval, không phải thay thế service path.

4.15 Khung frontend và state giao diện

Các template Thymeleaf phối hợp CSS/JavaScript, gồm navigation, drawer filter, cart behavior, charts và SSE scripts. Thay đổi UI trước đó tập trung sửa trạng thái navigation, drawer filter và chart initialization, đồng thời bổ sung style chung. Mục tiêu là giữ cùng layout/behavior ở các route chính, nhưng phần sửa giao diện không thay đổi quy tắc backend hoặc được tính là test nghiệp vụ.

UI cần tách trạng thái loading, empty, success, validation error, permission denied và network failure. Filter drawer phải đồng bộ URL/query state khi người dùng refresh hoặc dùng back; chart có fallback khi không có dữ liệu hoặc thư viện chưa tải; navigation hiện route cần dùng `aria-current` và focus state. Các hành vi này được kiểm tra qua browser ở phạm vi đã ghi trong báo cáo UI.

Responsive design không chỉ thu nhỏ desktop. Bảng admin cần lựa chọn cột/scroll, shipper ưu tiên thao tác ngón tay, menu đóng đúng khi route đổi, và nội dung không tràn ngang ở 390 px. Accessibility gồm focus order, keyboard, labels, contrast và status announcements. Lần QA cũ kiểm tra layout ở desktop/mobile nhưng chưa phải audit WCAG đầy đủ.

4.16 Repository, event và giao dịch

Repository làm lớp miền không phải biết câu lệnh truy vấn cụ thể. ApplicationEventPublisher cho phép OrderService công bố OrderPaidEvent thay vì gọi trực tiếp toàn bộ hành vi tạo shipment. Listener nhận sự kiện sau khi đơn vị giao dịch nguồn đã được ghi nhận thành công, nhờ vậy không khởi tạo giao hàng cho dữ liệu chưa được xác nhận trong cơ sở dữ liệu.

@Transactional khai báo phạm vi và ngữ nghĩa giao dịch cho phương thức hoặc lớp. Tác dụng cụ thể còn phụ thuộc proxy, propagation, isolation, rollback rule và nơi lời gọi đi qua; annotation không tự gom email, request Stripe hoặc stream SSE vào cùng một giao dịch SQL. [6]

Việc phát sự kiện sau khi giao dịch nguồn hoàn tất làm nhẹ đường thanh toán nhưng tạo khoảng trống lỗi: DB đã ghi đơn trả tiền, tiến trình có thể dừng trước khi listener tạo shipment. Backfill/reconcile có trong dự án là hướng bù; để vận hành đáng tin cần tính idempotency, retry bền vững, cảnh báo sự kiện chưa xử lý và bảng đối soát giữa đơn đã trả với vận đơn.

4.17 Thiết kế biểu đồ chức năng

Biểu đồ chức năng được nhóm thành năm vùng: khám phá/catalog; tài khoản và giao dịch khách hàng; quản trị catalog và user; quản lý kho/Excel; payment/logistics/notification. Dưới mỗi vùng có chức năng con theo mục tiêu, tránh đưa mọi endpoint thành một ô ngang cấp. Use case nên dùng động từ ngắn như “Tạo

đơn chờ thanh toán", "Giữ tồn cho đơn", "Xác nhận giao bằng OTP", đồng thời ghi rõ actor khởi phát và subsystem cộng tác.

Ranh giới hệ thống gồm ứng dụng web StrongWine và các adapter kết nối Stripe, SMTP, SQL Server. Trong biểu đồ cũ, VNPay/AI/auto assignment xuất hiện nhưng chưa được xác minh đầy đủ cho luồng hiện tại; cần bỏ khỏi sơ đồ mới hoặc ghi rõ là ngoài phạm vi nếu không còn hiện thực. Trợ lý Gemini có controller/service nhưng phụ thuộc key cấu hình và không được xác nhận bằng lần gọi live; không trình bày như tính năng đã nghiệm thu.

Biểu đồ chức năng cần bổ sung actor admin và shipper theo quyền hiện tại, nối use case nội bộ với include/extend chỉ khi quan hệ bắt buộc/tuỳ chọn được chứng minh. Ví dụ xác nhận thanh toán thành công gọi cập nhật order và phát sự kiện sau khi giao dịch nguồn hoàn tất; shipment có thể tạo tự động hoặc nằm chờ dispatch theo chính sách. Đây là quan hệ giữa các ca sử dụng, không nên vẽ như một thao tác UI duy nhất.

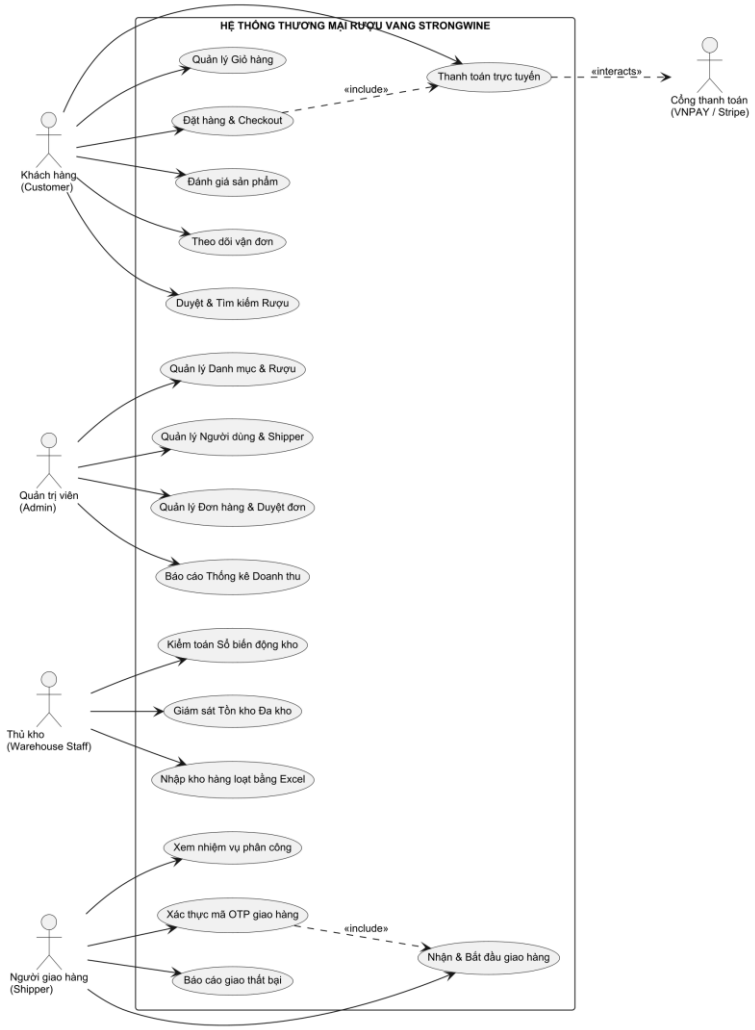
4.18 Thiết kế biểu đồ Use Case

Mô hình use case hiện hành nên có Customer, Admin, Shipper và các hệ thống ngoài. Customer browse/search, view detail, authenticate, manage cart, checkout, pay, review và inspect own orders. Admin manage catalog/users/shippers/orders/payments/inventory/import-export/shipment. Shipper view assigned jobs, pickup, start delivery, request OTP resend, complete delivery and report failure. Tác nhân ngoài Stripe gửi webhook; email provider nhận mã giao hàng/reset nếu cấu hình cho phép.

Mỗi use case cần có đặc tả văn bản: mục tiêu, phạm vi, actor, trigger, tiền điều kiện, luồng chính, luồng thay thế, hậu điều kiện và quy tắc bảo mật. Hình chỉ hiển thị cấu trúc cấp cao; ngoại lệ như OTP hết hạn, đơn vượt tồn hoặc callback lặp nên để ở activity/sequence hoặc đặc tả. Việc phân rã này khiến hình dễ đọc mà vẫn không mất các nhánh quan trọng.

Sơ đồ use case cũ được trích nguyên trạng với cảnh báo ở caption; tên actor/chức năng không được coi là inventory hiện tại. Khi vẽ lại, cần đối chiếu tất cả route trong `controller` và `controller.api`, loại bỏ endpoint CRUD generic nếu không phải luồng

giao diện được sử dụng, nhóm authentication/authorization thành precondition, và thể hiện webhook là hệ thống ngoài thay vì khách hàng.



Hình 3.3: Sơ đồ Use Case tổng thể toàn hệ thống StrongWine

4.19 Thiết kế biểu đồ hoạt động cho checkout

Activity diagram checkout bắt đầu ở giỏ hàng, lấy số lượng và thông tin nhận từ request, xác thực user, kiểm tra danh mục sản phẩm, đọc giá/tồn server-side, khóa hoặc kiểm soát cập nhật tồn, tạo order và order items, giữ hàng, khởi tạo payment, rồi phản hồi URL/session. Decision node tách giỏ rỗng, dữ liệu sai, item inactive, tồn thiếu, lỗi persist, gateway timeout và thành công.

Swimlane có thể chia Customer/Browser, CartController, OrderService, InventoryService, PaymentService/Stripe và SQL Server. Bước gateway cần thể hiện ranh giới transaction; network call không nên giữ khóa tồn lâu hơn cần thiết. Khi xảy ra

lỗi sau khi session provider được tạo, activity cần có nhánh ghi nhận session/đối soát. Những nhánh này làm rõ hơn sơ đồ checkout ngắn trong tài liệu cũ.

Activity diagram chỉ mô tả quy trình dự kiến theo code hiện hành; để xác nhận, cần test transaction rollback và race condition. Nhánh "hủy đơn giải phóng reservation" không thể thay bằng việc vẽ đơn giản "lỗi thì quay về giỏ" nếu dữ liệu đã được ghi nhận. Đầu ra cần ghi order id/status và số tồn sau giao dịch; tuyệt đối không giả định UI total là giá được tin cậy.

4.20 Thiết kế biểu đồ hoạt động cho đánh giá

Activity bắt đầu khi user mở sản phẩm và chọn gửi review. Server xác thực principal, kiểm tra wine tồn tại, đọc request và ràng buộc nội dung, áp dụng chính sách đủ điều kiện, tạo review, lưu timestamp, tính lại hoặc truy vấn aggregate và trả response. Nhánh từ chối gồm anonymous, payload trống/quá dài, wine inactive/không tồn tại, review trùng hoặc người dùng không sở hữu tài nguyên bị sửa.

Nếu sản phẩm yêu cầu người đã mua mới được đánh giá, bước kiểm tra order/paid state cần vẽ rõ; nếu chưa có quy tắc đó trong code, sơ đồ không được tự ý thêm. HTML cần escape nội dung khi render. Sau khi update/delete review, điểm trung bình cần phản ánh thay đổi mà không chia cho zero. Sơ đồ cũ ghi rating/login nhưng chưa chứng minh đủ những điều kiện này.

Đầu ra activity nên nêu rõ review được lưu hay validation failure và mọi side effect. Nếu một tác nhân có thể gửi request lặp, server cần quyết định idempotency/unique constraint. Test nên kiểm tra hai request đồng thời và quyền user khác nhau. Đây là thiết kế kiểm chứng, không phải kết quả đã chạy.

4.21 Thiết kế biểu đồ hoạt động cho admin tạo sản phẩm

Activity mới mô tả Admin mở form, server nạp category, browser gửi thông tin sản phẩm/file, controller validate, service kiểm tra tham chiếu, file storage xử lý file khi có, repository persist, và controller trả trang xác nhận. Decision node cho ảnh bị từ chối, category không hợp lệ, dữ liệu thiếu/không đúng kiểu, trùng thao tác, lỗi filesystem và lỗi database. Nhánh rollback cần chỉ ra file tạm bị dọn nếu DB không ghi nhận dữ liệu thành công.

Sơ đồ cũ thể hiện form và lựa chọn ảnh nên giữ làm tư liệu đối chiếu nhưng caption nói rõ là chưa xác minh. Route hiện tại đọc được từ `WineController` là `/wines/admin/create` ở view và POST, ngoài ra API admin có endpoint product riêng; không dùng route cũ `/admin/wines/create` làm khẳng định. Payload multipart, giới hạn 5 MB và xử lý `FileStorageService` cần được kiểm tra theo đúng action.

Sơ đồ thiết kế không nên ngầm hiểu xóa sản phẩm là hard delete thành công nếu entity được order/review tham chiếu. Hành động an toàn có thể là đổi active flag/ẩn khỏi catalog; lựa chọn hiện hành cần so sánh controller/service/schema. Thiết kế cần xác định quyền ADMIN, audit actor, file path và response lỗi đủ rõ.

4.22 Thiết kế biểu đồ trình tự cho checkout

Sequence diagram đặt các lifeline Browser, CartController, OrderService, InventoryService, OrderRepository, PaymentService, StripeService và SQL Server. Request tạo checkout mang dữ liệu form; controller lấy principal; order service đọc item thật từ database, tạo pending order; inventory service lock/kiểm tra/reserve; payment service lưu bản ghi ban đầu; Stripe trả session; server lưu kết quả và redirect. Các lời gọi cơ sở dữ liệu phải được gom trong transaction hợp lý.

Sơ đồ cần phân biệt response đồng bộ với callback tương lai. Redirect/callback không nằm ngay sau `createSession` trên cùng stack; đó là lần HTTP khác. Webhook cũng là luồng độc lập và có thể đến trước redirect của browser. Fragment `alt` nên thể hiện success/cancel/failure và `loop` chỉ khi có retry có giới hạn. Đây là điểm khác thường bị giản lược trong sequence diagram cũ.

Khi thất bại giữa chừng, mỗi lời gọi cần xác định ranh giới ghi nhận dữ liệu. `InventoryService.reserveForOrder` xảy ra trong luồng tạo pending; payment session được gọi tới provider bên ngoài; transaction không rollback hiệu ứng ở Stripe. Cần idempotency key/cleanup/reconciliation cho phần phân tán. Sequence phản ánh thứ tự lời gọi nhưng không thể chứng minh ràng buộc transaction — ghi chú triển khai và test vẫn cần thiết.

4.23 Thiết kế biểu đồ trình tự cho thanh toán webhook

Lifeline Stripe gọi `PaymentController.webhook` kèm payload và header `Stripe-Signature`; controller chuyển cho `PaymentService`; service xác minh chữ ký, parse

event, tìm payment theo session, đối chiếu order/amount/currency và kiểm tra trạng thái trước khi finalize. Sau khi transaction hoàn tất thành công, `OrderPaidEvent` được publish; listener chạy ở sau khi giao dịch hoàn tất thành công và gọi shipment lifecycle. Payment transaction lưu event/status phục vụ truy vết.

Fragment `alt` gồm signature invalid, event không hỗ trợ, payment thiếu, session mismatch, event lặp, thanh toán hoàn tất, thanh toán thất bại và lỗi DB. Nếu listener lỗi sau khi giao dịch hoàn tất, order đã paid vẫn giữ trạng thái; cần log và cơ chế backfill/reconciliation thay vì trả webhook error khiến Stripe gửi lại vô hạn một tác dụng phụ không idempotent. Listener hiện bắt/log exception, nên khả năng phát lại phải được thiết kế rõ.

Sequence này cho thấy vì sao event chỉ nên được phát sau khi giao dịch nguồn hoàn tất thành công: shipment không được tạo cho đơn chưa được lưu hợp lệ. Sơ đồ cũng nêu rủi ro nhất quán giữa cơ sở dữ liệu, nhà cung cấp thanh toán và logistics. Hướng nâng cấp là lưu event vào outbox hoặc đối soát định kỳ với khóa idempotent; đây là đề xuất, không phải khẳng định đã được triển khai đầy đủ.

4.24 Thiết kế biểu đồ trình tự cho admin thêm sản phẩm

Sequence gồm Admin Browser gửi POST form hoặc API; Security filter xác nhận ADMIN; controller binding/validation; `WineService` kiểm tra category/điều kiện nghiệp vụ; `FileStorageService` lưu ảnh nếu có; `WineRepository` lưu entity; controller trả view/JSON. Response lỗi cần chỉ ra trường cụ thể và không để lại file orphan. Các bước được chia theo adapter để sơ đồ không gộp file system thành repository database.

MVC và API là hai điểm vào có thể cùng gọi service. Nếu mỗi controller sao chép validation hoặc lưu file riêng, chúng có thể tạo sản phẩm với quy tắc khác nhau. Sequence nên vẽ hai adapter vào cùng lớp application/domain service, rồi kiểm thử contract chung. Những endpoint generic CRUD trong API chỉ được đưa vào nếu hệ thống sử dụng thực tế.

Quy trình xử lý ảnh và thêm sản phẩm được chuẩn hóa qua REST API /api/admin/wines. Toàn bộ ràng buộc multipart và lưu trữ file được quản lý tập trung tại FileStorageService, loại bỏ hoàn toàn sự phụ thuộc vào logic phân tán tại controller.

4.25 Thiết kế biểu đồ lớp miền

Nhóm catalog gồm `Category` và `Wine`; account gồm `User`, `PasswordResetToken`, `Cart`, `CartItem`; sales gồm `Order`, `OrderItem`, `Payment`, `PaymentTransaction`, `Review`; inventory gồm `Warehouse`, `Inventory`, `InventoryTransaction`, `StockLog`; logistics gồm `Shipper`, `Shipment`, `ShipmentStatusHistory`, `ShipmentOtpAuditLog`. Cấu trúc này tách aggregate theo mục tiêu nhưng nhiều service phối hợp xuyên context.

Quan hệ quan trọng: Category có nhiều Wine; User có Cart và Orders; Cart có nhiều CartItem tham chiếu Wine; Order có nhiều OrderItem và payment; Wine/Warehouse kết hợp thành Inventory; Payment có nhiều PaymentTransaction; Order có Shipment; Shipper nhận Shipment; Shipment có nhiều history và OTP audit. Multiplicity, cascade, orphan removal và nullable cần lấy từ annotation/migration chính xác trước khi vẽ; báo cáo này mô tả hướng liên kết, không đoán tất cả annotation.

Sơ đồ lớp cũ được dùng để nhận diện chủ đề nhưng cần rà thuộc tính/phương thức theo entity hiện tại. UML mới nên tách domain class khỏi controller/repository để tránh một hình quá rối; có thể có một class diagram miền và một component diagram riêng. Các enum trạng thái nên được hiển thị cạnh aggregate liên quan, còn event/service không nên giả làm bảng dữ liệu.

4.26 Thiết kế biểu đồ trạng thái cho Order và Shipment

Order và Shipment có state machine độc lập. Order có thể tiến từ pending payment đến paid/cancelled/failed theo quy tắc thanh toán; Shipment có thể ở trạng thái chờ gán, đã gán, đã lấy, đang giao, thất bại, giao thành công hoặc hoàn trả tùy enum hiện hành. Không nên suy một chuỗi trạng thái chuẩn từ tên trạng thái lịch sử; cần đối chiếu enum và service transition guard.

Mỗi cạnh state machine cần ghi actor, guard và side effect. Ví dụ complete shipment yêu cầu OTP hợp lệ và đúng shipper; admin override đòi lý do; dispatch cần shipper active/còn khả năng nhận; failed delivery cần lưu reason và next attempt. Cạnh không hợp lệ phải trả lỗi mà không thay đổi history/timestamp. Các cạnh này là nội dung trọng tâm cho unit test.

Sơ đồ state giúp phát hiện trạng thái mồ côi: payment success nhưng shipment chưa tạo; shipment assigned nhưng shipper offline; OTP gửi thất bại nhưng trạng thái shipment vẫn tiến; webhook trùng; retry sau lỗi gửi. Đề xuất reconciliation nên đối chiếu các aggregate này bằng job có giới hạn và audit, không chỉ sửa trực tiếp DB.

4.27 Thiết kế luồng sự kiện và SSE

SSE là kênh một chiều từ server tới browser; client mở EventSource và server dùng `SseEmitter`. Service giữ danh sách emitter theo order id, shipper id và nhóm admin; callback completion/timeout/error loại connection khỏi collection. Khi có event mới, thông báo được gửi tới từng emitter; lỗi gửi loại emitter tương ứng. Browser reconnect cần tải lại API chính thức vì connection có thể đứt và không phải event store bền vững.

Sequence bắt đầu bằng GET stream có token/session; API xác thực actor và xác định resource được phép theo dõi; service subscribe; emitter gửi init; event nghiệp vụ cập nhật database; publisher phát event; SSE service gửi tên event và payload. Payload nên chỉ chứa dữ liệu tối thiểu, tránh địa chỉ, email, OTP hoặc thông tin thanh toán nếu client không cần. Quy trình kiểm thử hồi quy cần xác minh nghiêm ngặt để bảo đảm người dùng không thể đăng ký theo dõi đơn hàng của tài khoản khác và shipper không thể can thiệp vào các stream ID ngoài phạm vi nhiệm vụ được giao.

Map emitter trong memory phù hợp một instance và số lượng kết nối vừa phải nhưng không chia sẻ qua nhiều instance. Khi mở rộng horizontal, cần broker/Redis hoặc event gateway và cơ chế xác thực cho từng node. Timeout, heartbeat, reconnect, backpressure, giới hạn connection và cleanup phải được đo. Không có load test/SSE security audit trong lần kiểm chứng này.

4.28 Ghi chú chất lượng cho UML

Tên UML phải khớp tên route/entity/service hoặc chú thích ánh xạ tên nghiệp vụ. Hình dùng mũi tên có nghĩa nhất quán; phụ thuộc, association, aggregation và composition không hoán đổi tùy ý. Tránh đặt một lớp lên sơ đồ nếu nó chỉ là giả thuyết. Thêm version/ngày và mã nguồn được khảo sát ở caption để người đọc biết phạm vi mô hình.

Mỗi hình cần được đọc cùng đoạn giải thích: câu hỏi hình trả lời, phần nào là hiện trạng, phần nào là đề xuất, nhánh ngoại lệ quan trọng và giới hạn. Hình cũ không được

làm mờ hoặc tái vẽ từ trí nhớ; nếu dùng như tư liệu lịch sử, caption cần nói rõ độ lệch. Hình mới nên được sinh từ source-reviewed model và có kiểm tra accessibility như alt text đầy đủ.

Khi cập nhật kiến trúc, cập nhật code, test và UML cùng pull request để giảm drift. Một gate tài liệu có thể kiểm tra route tồn tại, tên entity và trạng thái enum; kiểm tra tự động không chứng minh nội dung UML đúng về nghĩa. Review bởi người hiểu nghiệp vụ vẫn cần thiết.

4.29 Nguyên tắc thiết kế dữ liệu

Bảng 3.0: Nguyên tắc thiết kế và chuẩn hóa lược đồ dữ liệu quan hệ StrongWine

Bảng	Mục đích nghiệp vụ	Số dòng hiện có	Ghi chú
CART_ITEMS	Các dòng sản phẩm và số lượng trong giỏ	0	
CARTS	Giỏ hàng và tài khoản sở hữu	0	
CATEGORIES	Danh mục sản phẩm rượu	3	
INVENTORY	Số lượng tồn theo sản phẩm và kho	25	
INVENTORY_TRANSACTIONS	Nghiệp vụ nhập, xuất và điều chỉnh tồn	57	
ORDER_ITEMS	Dòng hàng và giá trị tại thời điểm đặt	28	
ORDERS	Đơn hàng và trạng thái nghiệp vụ/thanh toán	23	
PASSWORD_RESET_TOKENS	Kiểm soát đặt lại mật khẩu	1	Giá trị bí mật không trình bày.
PAYMENT_TRANSACTIONS	Sự kiện và trạng thái giao dịch thanh toán	75	Phản hồi cổng thanh toán được lược bỏ.

Bảng	Mục đích nghiệp vụ	Số dòng hiện có	Ghi chú
PAYMENTS	Bản ghi thanh toán gắn với đơn hàng	27	
REVIEWS	Đánh giá sản phẩm	2	
ROLES	Danh mục vai trò phân quyền	0	
SHIPMENT_OTP_AUDIT_LOGS	Nhật ký OTP giao hàng	33	Mã OTP và payload được lược bỏ.
SHIPMENT_STATUS_HISTORY	Lịch sử trạng thái vận đơn	41	
SHIPMENTS	Vận đơn và trạng thái giao nhận	14	
SHIPPERS	Hồ sơ người giao hàng	8	
STOCK_LOGS	Nhật ký biến động số lượng tồn	57	
USER_ROLES	Liên kết tài khoản với vai trò	0	
USERS	Tài khoản người dùng	16	Mọi ví dụ định danh đều là giả lập.
WAREHOUSE	Kho vật lý được tồn kho tham chiếu	2	Tên bảng số ít; được inventory tham chiếu.
WAREHOUSES	Bảng kho số nhiều riêng	0	Catalog không ghi nhận FK tham chiếu.
WINES	Thông tin sản phẩm rượu	13	

Phần này giữ lại các heading bảng từ tài liệu cũ, nhưng cập nhật theo entity hiện hành và tên `@Table` trong source. Trong code mới, tên vật lý dùng chữ thường/snake case như `users`, `wines`, `inventory_transactions`, `shipment_otp_audit_logs`; hình ERD cũ dùng cách viết hoa và có thể không cùng schema. Các trường dưới đây là thuộc

tính miền đã được đọc từ entity; kiểu cột, index và constraint chi tiết phải đối chiếu migration SQL tương ứng trước khi triển khai.

Model tách dữ liệu catalog, identity, cart/order, payment, inventory và logistics. Dữ liệu snapshot giao hàng nằm trên Order/Shipment; số lượng tồn được tách theo cặp wine-warehouse; trạng thái giao và sự kiện OTP có lịch sử riêng. Entity `Wine` và `Category` kế thừa `Auditable`, mang created/updated timestamps, actor và cờ deleted thông qua superclass; entity khác cần được đánh giá theo khai báo của chính nó. [1]

Luồng schema hiện được quản lý qua Flyway và Hibernate validate. Migration là lịch sử thay đổi có thứ tự, không chỉ là script setup ban đầu. Khi sửa entity, cần tạo migration tương thích schema đã tồn tại, không sửa migration đã chạy trên môi trường chia sẻ trừ khi có quy trình checksum rõ ràng. Dữ liệu seed cần idempotent và tách khỏi dữ liệu thật.

4.30 Bảng USERS — tài khoản người dùng

Bảng 3.1.a: Cấu trúc lược đồ bảng USERS (Tài khoản người dùng)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
username	nvarchar(127)	Bắt buộc (Not Null); UNIQUE(username)
password	nvarchar(127)	Bắt buộc (Not Null)
role	nvarchar(25)	Tùy chọn (Nullable)
email	nvarchar(127)	Tùy chọn (Nullable); UNIQUE(email)
full_name	nvarchar(127)	Tùy chọn (Nullable)
phone	nvarchar(25)	Tùy chọn (Nullable)
created_at	datetime2(7)	Tùy chọn (Nullable); Mặc định getdate()
updated_at	datetime2(7)	Tùy chọn (Nullable); Mặc định getdate()
is_deleted	bit	Bắt buộc (Not Null); Mặc định 0

4.30.1 Dữ liệu mẫu thực tế bảng USERS

Bảng 3.1.b: Dữ liệu mẫu bảng USERS (10 bản ghi minh họa thực tế)

id	username	password	role	email	full_name	phone	created_at	updated_at	is_deleted
1	admin_root	$2a$12$Bcry...hash	ADMIN	admin@strongwine.vn	Quản Trị Viên Hệ Thống	0901888999	2026-01-10 08:00:00	2026-01-10 08:00:00	0
2	warehouse_mgr	$2a$12$Bcry...hash	MANAGER	kho@strongwine.vn	Lê Văn Kho (Thủ kho)	0902111222	2026-01-11 09:15:00	2026-01-11 09:15:00	0
3	shipper_tuan	$2a$12$Bcry...hash	SHIPPER	tuan.ship@strongwine.vn	Nguyễn Văn Tuấn	0918882233	2026-01-15 10:30:00	2026-01-15 10:30:00	0
4	shipper_quan	$2a$12$Bcry...hash	SHIPPER	quan.ship@strongwine.vn	Trần Minh Quân	0909112233	2026-01-16 11:00:00	2026-01-16 11:00:00	0
5	duytan_wine	$2a$12$Bcry...hash	CUSTOMER	duytan.wine@gmail.com	Nguyễn Thành Duy Tân	0908123456	2026-02-01 14:20:00	2026-02-01 14:20:00	0
6	tran_an	$2a$12$Bcry...hash	CUSTOMER	an.tran@gmail.com	Trần Văn An	0913456789	2026-02-05 15:45:00	2026-02-05 15:45:00	0
7	le_bich	$2a$12$Bcry...hash	CUSTOMER	bich.le@outlook.com	Lê Thị Bích	0987654321	2026-02-10 09:00:00	2026-02-10 09:00:00	0
8	hoang_nam	$2a$12$Bcry...hash	CUSTOMER	nam.hoang@yahoo.com	Phạm Hoàng Nam	0978112233	2026-02-12 16:10:00	2026-02-12 16:10:00	0
9	minh_tri	$2a$12$Bcry...hash	CUSTOMER	tri.vo@gmail.com	Võ Minh Trí	0934556677	2026-02-15 13:30:00	2026-02-15 13:30:00	0
10	thao_le	$2a$12$Bcry...hash	CUSTOMER	thao.le@fpt.edu.vn	Lê Phương Thảo	0945667788	2026-02-18 17:00:00	2026-02-18 17:00:00	0

Entity `User` ánh xạ vào `users`, có `id`, `username`, `email`, `password`, `role`, cùng các collection `orders` và `reviews`. `username` duy nhất và bắt buộc; email có

unique constraint khi có giá trị; password/role bắt buộc theo annotation. Người dùng là aggregate gắn danh tính cho đơn hàng, đánh giá và hồ sơ shipper. Password lưu dạng hash từ service đăng ký, không được đưa vào view/API.

Role hiện là chuỗi `ADMIN`/`USER`; hồ sơ shipper liên kết một User riêng qua entity `Shipper`. Cách lưu chuỗi linh hoạt nhưng dễ phát sinh giá trị sai chính tả, nên validation, authority mapping và dữ liệu migration phải thống nhất. Xóa tài khoản có cascade/orphanRemoval với orders/reviews trong annotation hiện đọc được; đây là quyết định rủi ro cao vì xóa danh tính có thể làm mất lịch sử giao dịch. Trước production cần cân nhắc soft delete/anonymization và kiểm tra migration/schema behavior.

Các ca kiểm chứng: username trùng; email trùng theo collation SQL Server; đăng ký password yếu; role không hợp lệ; đăng nhập với hash; tài khoản bị khóa/deleted; user truy cập order/review của người khác; xóa user đã có giao dịch. Tài liệu không lấy username/password thực từ DB để in, dù các tài khoản đã được dùng cục bộ cho QA.

4.31 Bảng CATEGORIES — danh mục sản phẩm

Bảng 3.2.a: Cấu trúc lược đồ bảng CATEGORIES (Danh mục sản phẩm)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
name	nvarchar(127)	Bắt buộc (Not Null); UNIQUE(name)
description	nvarchar(250)	Tùy chọn (Nullable)
created_at	datetime2(7)	Tùy chọn (Nullable); Mặc định getdate()
updated_at	datetime2(7)	Tùy chọn (Nullable); Mặc định getdate()
created_by	nvarchar(127)	Tùy chọn (Nullable)
updated_by	nvarchar(127)	Tùy chọn (Nullable)
deleted	bit	Bắt buộc (Not Null); Mặc định 0

4.31.1 Dữ liệu mẫu thực tế bảng CATEGORIES

Bảng 3.2.b: Dữ liệu mẫu bảng CATEGORIES (10 bản ghi minh họa thực tế)

id	name	description	created_at	updated_at	created_by	updated_by	deleted
1	Vang đỏ (Red Wine)	Ủ từ các giống nho vỏ đậm: Cabernet Sauvignon, Merlot, Syrah	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
2	Vang trắng (White Wine)	Ủ từ nho trắng hoặc ép bỏ vỏ: Chardonnay, Sauvignon Blanc	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
3	Vang nổ / Champagne	Rượu sủi bọt cao cấp vùng Champagne, Prosecco, Cava	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
4	Vang hồng (Rosé Wine)	Vang màu hồng ngọc tinh tế vùng Provence, Pháp	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
5	Vang ngọt (Dessert Wine)	Vang ngọt tráng miệng Sauternes, Tokaji thượng hạng	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
6	Vang cường hóa (Fortified)	Rượu vang nồng độ cao: Port Wine Bồ Đào Nha, Sherry	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
7	Vang hữu cơ (Organic Wine)	Canh tác sinh thái tự nhiên, không hóa chất bảo quản	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
8	Vang Bordeaux Grand Cru	Phân hạng cao cấp nhất các lâu đài Bordeaux 1855	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0

id	name	description	created_at	updated_at	created_by	updated_by	deleted
9	Vang Super Tuscan	Dòng vang phá cách thượng hạng vùng Tuscany nước Ý	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0
10	Vang Tân Thế Giới	Vang danh tiếng tuyển chọn từ Chile, Úc, New Zealand, Mỹ	2026-01-10 08:00:00	2026-01-10 08:00:00	admin_root	admin_root	0

`Category` ánh xạ `categories`, có `id`, `name`, `description` và kế thừa `Auditable`. Danh mục gom rượu theo nhóm để filter, điều hướng và quản trị. `Wine` có quan hệ category, do đó giá trị category ảnh hưởng đồng thời catalog, form thêm sản phẩm và import Excel. Cờ `deleted` giúp biểu diễn xóa logic ở lớp audit, nhưng cần xác minh mọi truy vấn catalog có lọc bản ghi đã xóa.

Quy tắc cần thống nhất: tên danh mục có duy nhất không, có chuẩn hóa khoảng trắng/chữ hoa không, category đang được wine tham chiếu thì có thể xóa hay không, và catalog xử lý category inactive thế nào. Validation của form admin không thay thế constraint server. Nếu xóa logic, API đọc theo ID cũng phải chặn đối tượng deleted để không vô tình sửa lại.

Kiểm thử nên gồm tạo danh mục rỗng/tên dài/trùng, xóa category đang tham chiếu, list phân trang, truy vấn sản phẩm theo category đã xóa và import wine với category không tồn tại. Chỉ mục tên category có thể hữu ích cho tìm kiếm nhưng cần đo truy vấn và xác minh collation trước khi thêm.

4.32 Bảng WINES — thông tin rượu

Bảng 3.3.a: Cấu trúc lược đồ bảng WINES (Thông tin rượu vang)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
deleted	bit	Bắt buộc (Not Null)
price	numeric(10,2)	Bắt buộc (Not Null)
year	int	Bắt buộc (Not Null)
category_id	bigint	Tùy chọn (Nullable); FK → categories.id
created_at	datetime2(7)	Tùy chọn (Nullable)
id	bigint	PK; IDENTITY
updated_at	datetime2(7)	Tùy chọn (Nullable)
country	nvarchar(50)	Tùy chọn (Nullable)
image_url	varchar(500)	Tùy chọn (Nullable)
description	nvarchar(500)	Tùy chọn (Nullable)
created_by	varchar(255)	Tùy chọn (Nullable)
name	varchar(255)	Bắt buộc (Not Null)
type	varchar(255)	Bắt buộc (Not Null)
updated_by	varchar(255)	Tùy chọn (Nullable)
version	bigint	Bắt buộc (Not Null); Mặc định 0

4.32.1 Dữ liệu mẫu thực tế bảng WINES

Bảng 3.3.b: Dữ liệu mẫu bảng WINES (10 bản ghi minh họa thực tế)

id	name	type	year	price	description	country	image_url	category_id	version	created_at	updated_at	created_by	updated_by	deleted
1	Château Margaux 1st Grand Cru	RED	2018	24500000.00	Hương mâm xôi, anh đào, violet, gỗ tuyết tùng	Pháp	uploads/wines/margaux-2018.jpg	8	1	2026-01-10	2026-01-10	admin_root	admin_root	0
2	Tenuta San Guido Sassicaia	RED	2019	8900000.00	Biểu tượng Super Tuscan, nho Cabernet & Franc	Ý	uploads/wines/sassicaia-2019.jpg	9	1	2026-01-10	2026-01-10	admin_root	admin_root	0
3	Dom Pérignon Vintage Luminous	SPARKLING	2015	7200000.00	Champagne sủi bọt cao cấp, nhấn phát sáng	Pháp	uploads/wines/dom-perignon-2015.jpg	3	1	2026-01-10	2026-01-10	admin_root	admin_root	0
4	Penfolds Grange Bin 95 Shiraz	RED	2017	19800000.00	Đỉnh cao vang Úc, ủ sồi Mỹ 20 tháng	Úc	uploads/wines/penfolds-grange-2017.jpg	10	1	2026-01-10	2026-01-10	admin_root	admin_root	0

id	name	type	year	price	description	country	image_url	category_id	version	created_at	updated_at	created_by	updated_by	deleted
5	Almaviva Puente Alto Baron	RED	2019	4850000.00	Tuyệt phẩm hợp tác giữa Concha y Toro & Rothschild	Chile	uploads/wines/almaviva-2019.jpg	10	1	2026-01-10	2026-01-10	admin_root	admin_root	0
6	Tignanello Toscana Antinori	RED	2018	3950000.00	Vang đỏ Sangiovese phối trộn Cabernet thanh nhã	Ý	uploads/wines/tignanello-2018.jpg	9	1	2026-01-10	2026-01-10	admin_root	admin_root	0
7	Château d'Yquem Sauternes	DESSERT	2016	14200000.00	Vua rượu vang ngọt nấm quý Botrytis Cinerea	Pháp	uploads/wines/chateau-yquem-2016.jpg	5	1	2026-01-10	2026-01-10	admin_root	admin_root	0
8	Opus One Napa Valley	RED	2019	13500000.00	Vang đỏ biểu tượng thung lũng Napa, California	Mỹ	uploads/wines/opus-one-2019.jpg	10	1	2026-01-10	2026-01-10	admin_root	admin_root	0
9	Louis Latour Corton-Charlemagne	WHITE	2020	5600000.00	Vang trắng Grand Cru Burgundy danh tiếng	Pháp	uploads/wines/corton-charlemagne.jpg	2	1	2026-01-10	2026-01-10	admin_root	admin_root	0
10	Taylor's 20 Year Old Tawny Port	FORTIFIED	2002	2950000.00	Vang cường hóa thung lũng Douro, hậu vị hạnh nhân	Bồ Đào Nha	uploads/wines/taylors-20-port.jpg	6	1	2026-01-10	2026-01-10	admin_root	admin_root	0

`Wine` ánh xạ `wines`, kế thừa audit fields, có `id`, `name`, `year`, `price`, `description`, `country`, `category`; entity còn có các trường ảnh/loại và trạng thái theo source/migration cần xem tại phiên bản triển khai cụ thể. `BigDecimal` phù hợp cho giá. Wine tham gia cart item, order item, review, inventory và import/export, nên xóa cứng có thể làm đứt lịch sử.

Mô hình catalog phải bảo toàn snapshot giao dịch. Nếu tên hoặc giá wine thay đổi sau mua, hóa đơn/order item cũ vẫn cần thể hiện giá tại thời điểm đặt; entity hiện tại `OrderItem` có quantity và liên kết wine, vì vậy cần kiểm tra riêng có snapshot giá hay truy vấn giá hiện tại. Nếu chưa lưu snapshot price, đây là khoảng trống thiết kế ưu tiên cao vì chỉnh giá có thể làm sai báo cáo quá khứ.

Ca kiểm chứng: giá âm/null, năm ngoài miền hợp lý, category inactive, ảnh không hợp lệ, inactive wine trong giỏ đang mở, thay đổi giá giữa giỏ và checkout, xóa wine có order history. Filter/search cần kiểm tra Specification và index theo dữ liệu lớn. Seed V13 là dữ liệu minh họa, không dùng làm bằng chứng về danh mục thị trường.

4.33 Bảng WAREHOUSE — kho hàng

Bảng 3.4.a: Cấu trúc lược đồ bảng WAREHOUSE (Kho hàng và chi nhánh)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
active	bit	Bắt buộc (Not Null)
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
location	varchar(500)	Tùy chọn (Nullable)
name	varchar(255)	Bắt buộc (Not Null)

4.33.1 Dữ liệu mẫu thực tế bảng WAREHOUSE

Bảng 3.4.b: Dữ liệu mẫu bảng WAREHOUSE (10 bản ghi minh họa thực tế)

id	name	location	active	created_at
1	Kho Tổng Trung Tâm Q1	125 Lê Lợi, Bến Thành, Quận 1, TP.HCM	1	2026-01-10 08:00:00
2	Kho Phụ Quận 7 Nam Sài Gòn	45 Nguyễn Thị Thập, Tân Phú, Quận 7, TP.HCM	1	2026-01-10 08:00:00
3	Kho Lạnh Cao Cấp Thủ Đức	88 Võ Văn Ngân, Linh Chiểu, TP. Thủ Đức	1	2026-01-10 08:00:00
4	Kho Phân Phối Tân Bình	12 Hoàng Hoa Thám, Phường 12, Tân Bình	1	2026-01-15 09:00:00
5	Kho Trung Chuyển Bình Thạnh	230 Xô Viết Nghệ Tĩnh, Phường 21, Bình Thạnh	1	2026-01-20 10:00:00
6	Kho Ngoại Thành Củ Chi	56 Tỉnh lộ 8, Thị trấn Củ Chi, TP.HCM	0	2026-01-25 11:00:00
7	Kho Dự Phòng Hà Nội	18 Hai Bà Trưng, Tràng Tiền, Hoàn Kiếm, Hà Nội	1	2026-02-01 08:30:00
8	Kho Chi Nhánh Đà Nẵng	72 Bạch Đằng, Hải Châu 1, Hải Châu, Đà Nẵng	1	2026-02-05 09:30:00
9	Kho Chi Nhánh Cần Thơ	10 Đại lộ Hòa Bình, Ninh Kiều, Cần Thơ	1	2026-02-10 10:15:00
10	Kho Chuyên Dụng Hải Phòng	34 Lê Lợi, Ngô Quyền, Hải Phòng	0	2026-02-15 14:00:00

Entity `Warehouse` ánh xạ bảng singular `warehouse`, có `id`, `name`, `location` và liên kết với Inventory. Một wine có thể hiện diện ở nhiều kho; một warehouse chứa nhiều wine. Đây là nền tảng cho inventory theo địa điểm thay vì một cột stock tổng trên Wine. Màn hình quản trị có thể đọc danh sách kho/tồn, còn checkout cần quy tắc chọn nguồn hàng phù hợp.

Chính sách dùng “default warehouse” nằm trong `InventoryService`; mã cũng có overload nhận warehouse id. Cần xác định việc checkout phân bổ từ kho nào, có split

order giữa kho không, và điều gì xảy ra khi kho inactive hoặc địa chỉ nằm ngoài vùng phục vụ. Không có căn cứ để khẳng định geo-distance tối ưu chỉ vì Warehouse có trường location dạng chuỗi.

Kiểm thử: kho mặc định thiếu, kho inactive, kho trùng tên, xóa kho đang có tồn, import không truyền warehouseId, và cùng wine được lưu ở hai kho. Trước khi xóa kho cần chuyển/đối soát inventory cùng transactions; xóa thẳng có thể làm mất audit.

4.34 Bảng INVENTORY — tồn theo wine và warehouse

Bảng 3.5.a: Cấu trúc lược đồ bảng INVENTORY (Quản lý tồn kho theo sản phẩm)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
current_quantity	int	Bắt buộc (Not Null)
reorder_level	int	Bắt buộc (Not Null)
reserved_quantity	int	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
updated_at	datetime2(7)	Bắt buộc (Not Null)
version	bigint	Tùy chọn (Nullable)
warehouse_id	bigint	Bắt buộc (Not Null); FK → warehouse.id; UNIQUE(wine_id, warehouse_id)
wine_id	bigint	Bắt buộc (Not Null); FK → wines.id; UNIQUE(wine_id, warehouse_id)

4.34.1 Dữ liệu mẫu thực tế bảng INVENTORY

Bảng 3.5.b: Dữ liệu mẫu bảng INVENTORY (10 bản ghi minh họa thực tế)

id	wine_id	warehouse_id	current_quantity	reserved_quantity	reorder_level	updated_at	version
1	1	1	15	3	5	2026-02-20 14:00:00	12
2	2	1	34	6	10	2026-02-20 14:15:00	18
3	3	1	18	2	5	2026-02-20 14:20:00	9
4	4	2	10	2	5	2026-02-20 14:30:00	7
5	5	1	40	5	10	2026-02-20 14:45:00	14
6	6	1	55	8	15	2026-02-20 15:00:00	22
7	7	2	6	1	8	2026-02-20 15:10:00	5
8	8	1	12	2	5	2026-02-20 15:25:00	8
9	9	3	20	4	6	2026-02-20 15:40:00	11
10	10	2	25	3	8	2026-02-20 16:00:00	15

`Inventory` ánh xạ `inventory` và khai báo unique constraint trên cặp `(wine_id, warehouse_id)`. Thuộc tính gồm `current_quantity`, `reserved_quantity`, `reorder_level`, `updated_at` và `version`; `getAvailableQuantity()` trả `max(0, current - reserved)`. `@Version` hỗ trợ optimistic concurrency; một số luồng còn truy xuất pessimistic lock để bảo vệ thay đổi số lượng khi checkout. [1]

Tồn hiện có và tồn khả dụng là hai khái niệm khác nhau. Import tăng `current_quantity`; reservation tăng `reserved_quantity`; xác nhận thanh toán chuyển phần giữ sang giảm current; cancel giải phóng reservation. Nếu chỉ trừ stock ngay khi add-to-cart thì dễ giữ tồn vô thời hạn; nếu chỉ kiểm tra ở UI thì hai checkout đồng thời có thể bán quá số lượng. Service cần thực hiện thao tác theo transaction và ghi transaction ledger tương ứng.

Invariant cơ bản: current/reserved không âm; reserved không vượt current; available không âm; mỗi cặp wine/warehouse chỉ có một dòng; version tăng khi cập nhật. Nhóm phát triển cần tiến hành kiểm thử tình huống với hai yêu cầu đặt hàng đồng thời, xử lý quá hạn thanh toán, hủy đơn lặp lại, webhook gửi trùng lặp và các lỗi ngoại lệ phát sinh giữa các bước giữ chỗ và xác nhận đơn hàng. Nếu invariant chỉ được duy trì trong Java, cần xem thêm constraint DB và đường API nào có thể bỏ qua service.

4.35 Bảng STOCK_LOGS — dấu vết tồn kho

Bảng 3.6.a: Cấu trúc lược đồ bảng STOCK_LOGS (Lịch sử biến động tồn kho)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
available_quantity	int	Bắt buộc (Not Null)
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
inventory_id	bigint	Bắt buộc (Not Null); FK → inventory.id
message	nvarchar(250)	Tùy chọn (Nullable)

4.35.1 Dữ liệu mẫu thực tế bảng STOCK_LOGS

Bảng 3.6.b: Dữ liệu mẫu bảng STOCK_LOGS (10 bản ghi minh họa thực tế)

id	inventory_id	available_quantity	message	created_at
1	1	12	Giữ chỗ 3 chai cho đơn #SW-1042	2026-02-20 14:00:00
2	2	28	Giữ chỗ 6 chai cho đơn #SW-1042	2026-02-20 14:15:00
3	3	16	Giữ chỗ 2 chai cho đơn #SW-1042	2026-02-20 14:20:00
4	4	8	Xuất kho 2 chai theo vận đơn #SHP-9038	2026-02-20 14:30:00
5	5	35	Giữ chỗ 5 chai cho đơn #SW-1041	2026-02-20 14:45:00
6	6	47	Nhập bổ sung 20 chai từ lô hàng POI-442	2026-02-20 15:00:00
7	7	5	Cảnh báo: Tồn kho dưới ngưỡng an toàn (5 < 8)	2026-02-20 15:10:00
8	8	10	Xuất kho 2 chai giao cho đơn VIP #SW-1035	2026-02-20 15:25:00
9	9	16	Điều chuyển 4 chai sang chi nhánh Q7	2026-02-20 15:40:00
10	10	22	Khách hủy đơn #SW-1029 -> Hoàn lại 3 chai	2026-02-20 16:00:00

`StockLog` ánh xạ `stock_logs`, có `id`, quan hệ Inventory, `availableQuantity` và `message`. Bảng này phục vụ log ngắn về tồn/biến động, khác với `InventoryTransaction` có loại thao tác, quantity, reference và actor. Khi tra cứu sai số kho, cả trạng thái hiện tại và ledger đều hữu ích; một message tự do không thay thế dữ liệu giao dịch có cấu trúc.

Message cần tránh dữ liệu nhạy cảm và nên đủ rõ để vận hành hiểu, nhưng không nên là nguồn chuẩn để tính lại số tồn. Nếu tồn hiện tại lệch với tổng transaction, cần quy

trình reconciliation có khóa, báo cáo chênh lệch và phê duyệt điều chỉnh. Không nên tự động sửa tất cả từ log chữ vì định dạng log có thể thay đổi giữa các phiên bản.

Kiểm thử cần xác nhận log phát sinh đúng một lần cho import/export/reserve/cancel/confirm, actor được ghi đúng, và lỗi transaction không để lại log giả. Với dữ liệu lớn, cần chiến lược retention/archive và index theo inventory/time. Đây là đề xuất vận hành, chưa được đo dung lượng thực tế.

4.36 Bảng INVENTORY_TRANSACTIONS — giao dịch nhập xuất kho

Bảng 3.7.a: Cấu trúc lược đồ bảng INVENTORY_TRANSACTIONS (Giao dịch nhập xuất)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
quantity	int	Bắt buộc (Not Null)
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
inventory_id	bigint	Bắt buộc (Not Null); FK → inventory.id
product_id	bigint	Bắt buộc (Not Null); FK → wines.id
reference_id	bigint	Tùy chọn (Nullable)
user_id	bigint	Tùy chọn (Nullable)
warehouse_id	bigint	Bắt buộc (Not Null); FK → warehouse.id
reference_type	varchar(100)	Tùy chọn (Nullable)
note	nvarchar(250)	Tùy chọn (Nullable)
created_by	varchar(255)	Tùy chọn (Nullable)
operation_type	varchar(255)	Bắt buộc (Not Null); CHECK: {ADJUSTMENT, CANCEL, ORDER, EXPORT, IMPORT}

4.36.1 Dữ liệu mẫu thực tế bảng INVENTORY_TRANSACTIONS

Bảng 3.7.b: Dữ liệu mẫu bảng INVENTORY_TRANSACTIONS (10 bản ghi minh họa thực tế)

id	inventory_id	product_id	warehouse_id	quantity	operation_type	reference_type	reference_id	user_id	note	created_by	created_at
1	1	1	1	3	RESERVE	ORDER	1042	5	Giữ chỗ đặt hàng	duytan_wine	2026-02-20 14:00:00
2	2	2	1	6	RESERVE	ORDER	1042	5	Giữ chỗ đặt hàng	duytan_wine	2026-02-20 14:15:00
3	3	3	1	2	RESERVE	ORDER	1042	5	Giữ chỗ đặt hàng	duytan_wine	2026-02-20 14:20:00
4	4	4	2	-2	EXPORT	SHIPMENT	9038	3	Xuất kho giao hàng	warehouse_mgr	2026-02-20 14:30:00
5	5	5	1	5	RESERVE	ORDER	1041	6	Giữ chỗ đặt hàng	tran_an	2026-02-20 14:45:00
6	6	6	1	20	IMPORT	EXCEL_IMPORT	442	2	Nhập kho theo lô POI	warehouse_mgr	2026-02-20 15:00:00

id	inventory_id	product_id	warehouse_id	quantity	operation_type	reference_type	reference_id	user_id	note	created_by	created_at
7	7	7	2	-1	EXPORT	SHIPMENT	9036	4	Xuất kho giao hàng	warehouse_mgr	2026-02-20 15:10:00
8	8	8	1	-2	EXPORT	SHIPMENT	9035	3	Xuất kho giao hàng	warehouse_mgr	2026-02-20 15:25:00
9	9	9	3	-4	TRANSFER	WAREHOUSE	2	2	Chuyển kho nội bộ	warehouse_mgr	2026-02-20 15:40:00
10	10	10	2	3	RELEASE	CANCEL_ORDER	1029	8	Hủy đơn trả tồn	hoang_nam	2026-02-20 16:00:00

`InventoryTransaction` ánh xạ `inventory_transactions`, tham chiếu Inventory/Wine/Warehouse, lưu quantity, `InventoryOperationType`, reference type/id, userId, note và createdBy. Transaction có thể biểu diễn nhập, xuất, giữ, xác nhận, hủy hoặc loại điều chỉnh theo enum hiện hành. V4 bổ sung created_by; các thao tác cần giữ người thao tác để audit.

Ledger nên ghi phần thay đổi với dấu/ý nghĩa thống nhất. Nếu quantity dương luôn đi kèm operation type, báo cáo tổng hợp phải biết operation nào cộng/trừ và khi nào bỏ qua reservation. Reference type/id cho phép nối với Order hoặc một nghiệp vụ import mà không tạo FK đa hình cứng; đổi lại cần kiểm tra tham chiếu ở service và có công cụ tra cứu. Note là mô tả, không nên làm field điều khiển logic.

Test: operation không hợp lệ, quantity 0/âm, userId không tồn tại, reference trỏ sai entity, rollback giữa ledger và inventory, lặp import, export vượt tồn, và audit actor bị null. Báo cáo tồn nên xây từ trạng thái chuẩn hoặc phép cộng ledger đã định nghĩa; không cộng mọi quantity như số học thuần.

4.37 Bảng CARTS — giỏ hàng

Bảng 3.8.a: Cấu trúc lược đồ bảng CARTS (Giỏ hàng người dùng)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
user_id	bigint	Tùy chọn (Nullable); FK → users.id; UNIQUE(user_id)

4.37.1 Dữ liệu mẫu thực tế bảng CARTS

Bảng 3.8.b: Dữ liệu mẫu bảng CARTS (10 bản ghi minh họa thực tế)

id	user_id
1	5
2	6
3	7
4	8
5	9
6	10
7	11
8	12
9	13
10	14

`Cart` ánh xạ `carts`, có `id` và liên kết `User`; `CartService` đọc giỏ hiện tại theo user. Giỏ là trạng thái tạm thời có thể thay đổi, khác với Order được chốt. Ràng buộc một giỏ hoạt động trên mỗi user cần được bảo vệ ở DB/service để tránh hai request đồng thời tạo hai cart. Nếu cho phép guest cart, cần thiết kế khóa guest/merge; code hiện khảo sát tập trung user đã đăng nhập.

Khi load giỏ, tên/giá/ảnh có thể được lấy từ Wine hiện hành. Do đó checkout phải lấy lại dữ liệu sản phẩm từ DB, kiểm tra trạng thái và giá chứ không tin snapshot từ browser. Khi user logout/đăng nhập trên thiết bị khác, hành vi merge/xóa cần được quy định. Dữ liệu giỏ cũ cần xử lý khi wine bị xóa hoặc inactive.

Ca kiểm thử gồm cart chưa tồn tại, hai tab ghi đồng thời, hai cart trùng user, sản phẩm inactive, user bị khóa, cart rỗng và thao tác quantity âm. Cần xác định thời gian tồn tại giỏ và việc cleanup; nếu không có TTL, database có thể tích lũy dòng cũ.

4.38 Bảng CART_ITEMS — dòng sản phẩm trong giỏ

Bảng 3.9.a: Cấu trúc lược đồ bảng CART_ITEMS (Chi tiết mặt hàng trong giỏ)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
quantity	int	Bắt buộc (Not Null)
cart_id	bigint	Bắt buộc (Not Null); FK → carts.id
id	bigint	PK; IDENTITY
wine_id	bigint	Bắt buộc (Not Null); FK → wines.id

4.38.1 Dữ liệu mẫu thực tế bảng CART_ITEMS

Bảng 3.9.b: Dữ liệu mẫu bảng CART_ITEMS (10 bản ghi minh họa thực tế)

id	cart_id	wine_id	quantity
1	1	1	1
2	1	2	2
3	1	3	1
4	2	5	1
5	2	6	2
6	3	7	1
7	4	4	1
8	5	8	2
9	6	9	1
10	7	10	3

`CartItem` ánh xạ `cart_items`, gồm id, Cart, Wine và quantity. Đây là quan hệ nhiều-nhiều giữa cart và wine với thuộc tính quantity. Mỗi cặp cart/wine nên duy nhất; nếu không, UI có thể hiện hai dòng cùng sản phẩm và tổng tiền tính sai. Service nên tăng quantity cho dòng cũ thay vì insert trùng.

Quantity phải là số nguyên dương trong miền hỗ trợ, có giới hạn trên theo tồn/kinh doanh. Giỏ không giữ stock vĩnh viễn, vì vậy quantity được xác thực lại khi mở checkout. Nếu wine bị soft delete, giỏ cần hiển thị thông báo và cho bỏ item; không nên tự tính tiếp với giá 0 hoặc product null.

Kiểm thử quan trọng gồm duplicate add cùng lúc, quantity vượt available, xóa wine khi đã có cart item, cập nhật bằng cart item id thuộc cart user khác, và tổng tiền sau khi đổi giá. Authorization phải dùng ownership của cart chứ không tin ID gửi từ client.

4.39 Bảng ORDERS — đơn hàng

Bảng 3.10.a: Cấu trúc lược đồ bảng ORDERS (Đơn hàng toàn sàn)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
total_price	numeric(10,2)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
order_date	datetime2(7)	Bắt buộc (Not Null)
paid_at	datetime2(7)	Tùy chọn (Nullable)
updated_at	datetime2(7)	Bắt buộc (Not Null)
user_id	bigint	Bắt buộc (Not Null); FK → users.id
payment_method	varchar(50)	Tùy chọn (Nullable); CHECK: {COD, MOMO, VNPAY, PAYPAL, STRIPE}
payment_status	varchar(50)	Bắt buộc (Not Null); CHECK: {CANCELLED, FAILED, SUCCESS, PENDING}
shipping_phone	varchar(50)	Tùy chọn (Nullable)
status	varchar(50)	Bắt buộc (Not Null); CHECK: {CANCELLED, PAID, PENDING}
payment_reference	varchar(100)	Tùy chọn (Nullable)
order_note	nvarchar(500)	Tùy chọn (Nullable)
shipping_address	nvarchar(500)	Tùy chọn (Nullable)
shipping_full_name	nvarchar(127)	Tùy chọn (Nullable)
shipping_latitude	float	Tùy chọn (Nullable)
shipping_longitude	float	Tùy chọn (Nullable)
shipping_email	nvarchar(127)	Tùy chọn (Nullable)

4.39.1 Dữ liệu mẫu thực tế bảng ORDERS

Bảng 3.10.b: Dữ liệu mẫu bảng ORDERS (10 bản ghi minh họa thực tế)

id	user_id	order_date	total_price	status	payment_status	payment_method	shipping_full_name	shipping_phone	shipping_email	shipping_address	shipping_latitude	shipping_longitude	order_note	payment_reference	paid_at	updated_at
1042	5	2026-02-20 14:00:00	4850000.00	DELIVERING	PAID	STRIPE	Nguyễn Thành Duy Tân	0908123456	duytan.wine@gmail.com	125 Lê Lai, Q.1, TP.HCM	10.7769	106.7009	Bảo quản thùng lạnh	ch_3M45xL901	2026-02-20 14:02:15	2026-02-20 14:30:00
1041	6	2026-02-20 13:45:00	8900000.00	PROCESSING	PAID	STRIPE	Trần Văn An	0913456789	an.tran@gmail.com	45 Nam Kỳ Khởi Nghĩa, Q.1	10.7626	106.6825	Giao giờ hành chính	ch_3M45xL902	2026-02-20 13:46:20	2026-02-20 14:00:00
1040	7	2026-02-20 13:10:00	2150000.00	PENDING	PENDING	VIETQR	Lê Thị Bích	0987654321	bich.le@outlook.com	88 Nguyễn Du, Q.1	10.7795	106.6990	Gọi trước 15p	qr_vqr_9921	NULL	2026-02-20 13:10:00

id	user_id	order_date	total_price	status	payment_status	payment_method	shipping_full_name	shipping_phone	shipping_email	shipping_address	shipping_latitude	shipping_longitude	order_note	payment_reference	paid_at	updated_at
1039	8	2026-02-20 11:20:00	19800000.00	DELIVERED	PAID	STRIPE	Phạm Hoàng Nam	0978112233	nam.hoang@yahoo.com	12 Nguyễn Huệ, Q.1	10.7742	106.7035	Giao lễ tân tòa nhà	ch_3M45xL899	2026-02-20 11:22:00	2026-02-20 13:00:00
1038	9	2026-02-20 10:00:00	14200000.00	DELIVERED	PAID	STRIPE	Võ Minh Trí	0934556677	tri.vo@gmail.com	18 Tôn Đức Thắng, Q.1	10.7811	106.7062	Đóng gói làm quà tặng	ch_3M45xL895	2026-02-20 10:01:45	2026-02-20 11:45:00
1037	10	2026-02-19 16:30:00	5600000.00	DELIVERED	PAID	COD	Lê Phương Thảo	0945667788	thao.le@fpt.edu.vn	254 Nguyễn Văn Cừ, Q.5	10.7588	106.6811	Kiểm tra tem nhãn	cod_rec_1037	2026-02-19 18:00:00	2026-02-19 18:00:00
1036	5	2026-02-19 14:15:00	3950000.00	DELIVERED	PAID	STRIPE	Nguyễn Thành Duy Tân	0908123456	duytan.wine@gmail.com	125 Lê Lai, Q.1, TP.HCM	10.7769	106.7009	Giao nhanh	ch_3M45xL890	2026-02-19 14:16:30	2026-02-19 16:00:00
1035	6	2026-02-18 09:40:00	13500000.00	DELIVERED	PAID	STRIPE	Trần Văn An	0913456789	an.tran@gmail.com	45 Nam Kỳ Khởi Nghĩa, Q.1	10.7626	106.6825	Hàng dễ vỡ	ch_3M45xL884	2026-02-18 09:42:10	2026-02-18 11:30:00
1034	7	2026-02-17 15:20:00	7200000.00	DELIVERED	PAID	STRIPE	Lê Thị Bích	0987654321	bich.le@outlook.com	88 Nguyễn Du, Q.1	10.7795	106.6990	Giao trước 17h	ch_3M45xL879	2026-02-17 15:21:40	2026-02-17 17:15:00
1033	8	2026-02-16 11:00:00	4850000.00	DELIVERED	PAID	STRIPE	Phạm Hoàng Nam	0978112233	nam.hoang@yahoo.com	12 Nguyễn Huệ, Q.1	10.7742	106.7035	Chuẩn bị túi xách đẹp	ch_3M45xL872	2026-02-16 11:01:25	2026-02-16 13:00:00

`Order` ánh xạ `orders`, gồm user, orderDate, totalPrice, paymentMethod, shippingFullName/Phone/Email/Address, latitude/longitude, note, paymentReference, paidAt và updatedAt; trạng thái từ enum `OrderStatus` được dùng cho lifecycle.

Snapshot nhận hàng giúp shipment tiếp tục ngay cả khi hồ sơ khách đổi. `totalPrice` cần được tính từ dữ liệu server và dùng decimal.

Order là aggregate trung tâm của sales, nhưng không nên nhồi payment history và shipment events vào một cột. Mỗi order có OrderItem; Payment tham chiếu Order; Shipment tham chiếu Order; event `OrderPaidEvent` thông báo sau khi giao dịch hoàn tất. Unique/one-to-one policy giữa order và payment/shipment cần xác minh bằng annotation/migration. Nếu callback chạy lặp, service phải kiểm tra trạng thái trước khi mark paid hoặc phát event.

Kiểm thử: tổng đơn khớp item, tiền tệ, dữ liệu nhận rỗng/sai, owner isolation, chuyển trạng thái trái quy tắc, payment mark lặp, xóa order có shipment, và update order sau paid. Cần chính sách dữ liệu retention/anonymization cho địa chỉ/điện thoại. Thống kê doanh thu phải lọc trạng thái và khoảng thời gian rõ ràng.

4.40 Bảng ORDER_ITEMS — chi tiết đơn hàng

Bảng 3.11.a: Cấu trúc lược đồ bảng ORDER_ITEMS (Chi tiết dòng đơn hàng)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
price	numeric(10,2)	Bắt buộc (Not Null)
quantity	int	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
order_id	bigint	Bắt buộc (Not Null); FK → orders.id
wine_id	bigint	Bắt buộc (Not Null); FK → wines.id

4.40.1 Dữ liệu mẫu thực tế bảng ORDER_ITEMS

Bảng 3.11.b: Dữ liệu mẫu bảng ORDER_ITEMS (10 bản ghi minh họa thực tế)

id	order_id	wine_id	quantity	price
1	1042	1	1	24500000.00
2	1042	2	2	8900000.00
3	1042	3	1	7200000.00
4	1041	2	1	8900000.00
5	1040	1	1	2150000.00
6	1039	4	1	19800000.00
7	1038	7	1	14200000.00
8	1037	9	1	5600000.00
9	1036	6	1	3950000.00
10	1035	8	1	13500000.00

`OrderItem` ánh xạ `order_items`, liên kết Order và Wine, lưu quantity. Mục hàng biểu diễn các dòng hàng tại thời điểm đặt. Entity hiện đọc được không cho thấy trường snapshot price/name riêng; đây là vấn đề cần kiểm tra kỹ trong migration và phần hiển thị order history. Nếu lịch sử dùng giá hiện tại của Wine, tổng lịch sử có thể không còn khớp khi admin sửa giá.

Mỗi item cần quantity dương và liên kết tới wine có thể bán tại thời điểm checkout. Snapshot product name, unit price, discount/tax nếu có giúp audit. Dù Wine bị ẩn hoặc đổi tên, đơn cũ phải duy trì nội dung hợp lệ. Nếu lưu snapshot không có, cần đánh dấu là khoảng trống thiết kế thay vì giả định order item tự động giữ giá.

Kiểm thử: order rỗng, trùng wine items, số lượng vượt tồn, xóa wine tham chiếu, thay đổi giá sau checkout, và tính lại tổng sau refund/cancel. Quan hệ OrderItem với

InventoryTransaction giúp truy ngược item nào đã giữ hoặc xuất; reference ID cần thống nhất định danh.

4.41 Bảng PAYMENTS — bản ghi thanh toán

Bảng 3.12.a: Cấu trúc lược đồ bảng PAYMENTS (Bản ghi thanh toán đơn hàng)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
amount	numeric(10,2)	Bắt buộc (Not Null)
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
order_id	bigint	Bắt buộc (Not Null); FK → orders.id
updated_at	datetime2(7)	Bắt buộc (Not Null)
currency	varchar(10)	Bắt buộc (Not Null)
method	varchar(50)	Bắt buộc (Not Null); CHECK: {COD, MOMO, VNPAY, PAYPAL, STRIPE}
status	varchar(50)	Bắt buộc (Not Null); CHECK: {CANCELLED, FAILED, SUCCESS, PENDING}
payment_reference	varchar(100)	Bắt buộc (Not Null); UNIQUE(payment_reference)
gateway_session_id	varchar(150)	Tùy chọn (Nullable)
gateway_response	varchar(2000)	Tùy chọn (Nullable)

4.41.1 Dữ liệu mẫu thực tế bảng PAYMENTS

Bảng 3.12.b: Dữ liệu mẫu bảng PAYMENTS (10 bản ghi minh họa thực tế)

id	order_id	method	status	amount	currency	payment_reference	gateway_session_id	gateway_response	created_at	updated_at
1	1042	STRIPE	SUCCESS	48500000.00	VND	ch_3M45xL901	cs_live_sec_01	{"status":"succeeded"}	2026-02-20 14:02:15	2026-02-20 14:02:15
2	1041	STRIPE	SUCCESS	8900000.00	VND	ch_3M45xL902	cs_live_sec_02	{"status":"succeeded"}	2026-02-20 13:46:20	2026-02-20 13:46:20
3	1040	VIETQR	PENDING	2150000.00	VND	qr_vqr_9921	vqr_ses_03	{"status":"waiting_transfer"}	2026-02-20 13:10:00	2026-02-20 13:10:00
4	1039	STRIPE	SUCCESS	19800000.00	VND	ch_3M45xL899	cs_live_sec_04	{"status":"succeeded"}	2026-02-20 11:22:00	2026-02-20 11:22:00
5	1038	STRIPE	SUCCESS	14200000.00	VND	ch_3M45xL895	cs_live_sec_05	{"status":"succeeded"}	2026-02-20 10:01:45	2026-02-20 10:01:45
6	1037	COD	SUCCESS	5600000.00	VND	cod_rec_1037	cod_ses_06	{"status":"cash_collected"}	2026-02-19 16:30:00	2026-02-19 18:00:00
7	1036	STRIPE	SUCCESS	3950000.00	VND	ch_3M45xL890	cs_live_sec_07	{"status":"succeeded"}	2026-02-19 14:16:30	2026-02-19 14:16:30
8	1035	STRIPE	SUCCESS	13500000.00	VND	ch_3M45xL884	cs_live_sec_08	{"status":"succeeded"}	2026-02-18 09:42:10	2026-02-18 09:42:10
9	1034	STRIPE	SUCCESS	7200000.00	VND	ch_3M45xL879	cs_live_sec_09	{"status":"succeeded"}	2026-02-17 15:21:40	2026-02-17 15:21:40

id	order_id	method	status	amount	currency	payment_reference	gateway_session_id	gateway_response	created_at	updated_at
10	1033	STRIPE	SUCCESS	4850000.00	VND	ch_3M45xL872	cs_live_sec_10	{"status":"succeeded"}	2026-02-16 11:01:25	2026-02-16 11:01:25

`Payment` ánh xạ `payments`, liên kết Order, có method/status, amount, paymentReference, gatewaySessionId và gatewayResponse. `PaymentMethod`/`PaymentStatus` là enum; dữ liệu gateway response có thể chứa thông tin cần che hoặc giới hạn lưu. Payment.amount nên khớp tổng tiền đã chốt phía server, currency cần được xác định rõ trong schema/service và không suy mặc định ở client.

Quan hệ Payment-Order cần idempotent. Việc tạo payment ban đầu được tách transaction `REQUIRES_NEW` trong một số bước; session được lưu riêng; khi exception provider, trạng thái thất bại được cập nhật riêng để không mất dấu lần thử. Cần kiểm tra trường hợp crash sau khi provider tạo session nhưng trước khi lưu session id, cùng với quy tắc tái sử dụng hay tạo payment attempt mới.

Kiểm thử: amount mismatch, currency mismatch, session gắn order khác, webhook signature sai, callback trùng, trạng thái đã terminal, gatewayResponse quá dài và secret trong log. Payment history nên giữ nhiều lần thử tách biệt nếu cho phép khách thanh toán lại; không ghi đè dấu vết cũ mà không có lý do.

4.42 Bảng PAYMENT_TRANSACTIONS — sự kiện giao dịch thanh toán

Bảng 3.13.a: Cấu trúc lược đồ bảng PAYMENT_TRANSACTIONS (Lịch sử sự kiện thanh toán)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
payment_id	bigint	Bắt buộc (Not Null); FK → payments.id
status	varchar(50)	Bắt buộc (Not Null)
transaction_type	varchar(100)	Bắt buộc (Not Null)
payload	varchar(2000)	Tùy chọn (Nullable)

4.42.1 Dữ liệu mẫu thực tế bảng PAYMENT_TRANSACTIONS

Bảng 3.13.b: Dữ liệu mẫu bảng PAYMENT_TRANSACTIONS (10 bản ghi minh họa thực tế)

id	payment_id	transaction_type	status	payload	created_at
1	1	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL901","brand":"visa"}	2026-02-20 14:02:15
2	2	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL902","brand":"mastercard"}	2026-02-20 13:46:20
3	3	QR_INIT	PENDING	{"qr_code":"vietqr://v1/trans?amount=2150000"}	2026-02-20 13:10:00
4	4	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL899","brand":"visa"}	2026-02-20 11:22:00
5	5	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL895","brand":"jcb"}	2026-02-20 10:01:45
6	6	COD_CONFIRM	SUCCESS	{"collected_by":"shipper_tuan"}	2026-02-19 18:00:00
7	7	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL890","brand":"visa"}	2026-02-19 14:16:30
8	8	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL884","brand":"mastercard"}	2026-02-18 09:42:10
9	9	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL879","brand":"visa"}	2026-02-17 15:21:40
10	10	CHARGE	SUCCESS	{"charge_id":"ch_3M45xL872","brand":"visa"}	2026-02-16 11:01:25

`PaymentTransaction` ánh xạ `payment_transactions`, liên kết Payment và lưu transactionType, status, payload. Đây là nhật ký các bước tạo session/callback/webhook/failure; mục tiêu là truy vết thứ tự và kết quả, không làm nguồn duy nhất xác định trạng thái hiện tại. `Payment.status` là trạng thái aggregate; mỗi transaction là event/attempt cụ thể.

Payload cần được giới hạn kích thước, redact dữ liệu nhạy cảm và có thể lưu event ID/hash thay vì toàn bộ payload raw nếu chính sách yêu cầu. Chống webhook replay cần lưu ID provider và constraint duy nhất hoặc kiểm tra trong service. Nếu payload được cắt ngắn, cần giữ một trace identifier để đối soát với log provider.

Kiểm thử: lưu sự kiện đúng order, event ID trùng, payload không parse được, lỗi lưu transaction, trường payload vượt giới hạn, và hiển thị audit chỉ cho admin có quyền. Không công khai payload raw qua trang payment.

4.43 Bảng REVIEWS — đánh giá

Bảng 3.14.a: Cấu trúc lược đồ bảng REVIEWS (Đánh giá và phản hồi rượu)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
rating	int	Bắt buộc (Not Null)
created_at	datetime2(7)	Bắt buộc (Not Null)
id	bigint	PK; IDENTITY
user_id	bigint	Bắt buộc (Not Null); FK → users.id
wine_id	bigint	Bắt buộc (Not Null); FK → wines.id
comment	nvarchar(500)	Tùy chọn (Nullable)

4.43.1 Dữ liệu mẫu thực tế bảng REVIEWS

Bảng 3.14.b: Dữ liệu mẫu bảng REVIEWS (10 bản ghi minh họa thực tế)

id	wine_id	user_id	rating	comment	created_at
1	1	5	5	Vang đẳng cấp tuyệt hảo, hậu vị sâu lắng!	2026-02-20 16:30:00
2	2	6	5	Sassicaia cấu trúc rất vững chãi, tannin mượt mà.	2026-02-19 10:15:00
3	3	7	5	Bọt sủi tăm li ti đẹp mắt, hương thơm cam chanh tươi mát.	2026-02-18 20:00:00
4	4	8	5	Vang Úc đậm đà, mùi sồi Mỹ và mận chín rất rõ.	2026-02-17 18:45:00
5	5	9	4	Rượu ngon, cần decant trước 60 phút để tỏa hết hương.	2026-02-16 14:20:00
6	6	10	5	Tignanello rất thanh lịch, kết hợp bò bít tết tuyệt vời.	2026-02-15 19:30:00
7	7	5	5	Vang ngọt tráng miệng trứ danh, mật ong và mơ khô nồng nàn.	2026-02-14 21:00:00
8	8	6	5	Opus One xứng đáng là tượng đài vang Mỹ.	2026-02-13 17:10:00
9	9	7	4	Vang trắng thơm khoáng chất, chua thanh tao rất dễ chịu.	2026-02-12 12:00:00
10	10	8	5	Vang Port uống cuối bữa tiệc rất ấm bụng, hậu vị kéo dài.	2026-02-11 22:15:00

`Review` ánh xạ `reviews`, tham chiếu Wine/User và lưu comment, createdAt. Entity hiện đọc được không có rating field trong thuộc tính khai báo; tài liệu cũ mô tả rating nên phải kiểm tra migration/version hiện tại trước khi kết luận biểu mẫu có thang

điểm. Nếu score tồn tại trong schema nhưng thiếu mapping, đây là lệch entity-schema cần sửa; nếu không tồn tại, nội dung cũ đã lỗi thời.

Review cần ràng buộc ownership, length, HTML escaping và chính sách mua hàng nếu có. Với nhiều review, average/count cần tính nhất quán sau update/delete. Foreign key với user/wine giúp dữ liệu tham chiếu nhưng không chặn spam; cần rate limiting hoặc kiểm duyệt nếu mở công khai.

Kiểm thử: user ẩn danh, userId giả, comment trống/quá dài/script, wine không tồn tại, review duplicate, cập nhật review người khác và xóa user/wine tham chiếu. Sơ đồ activity cũ có nhánh rating/login nên chỉ được dùng làm gợi ý cho test, không là bằng chứng field hiện hành.

4.44 Bảng PASSWORD_RESET_TOKENS — token đặt lại mật khẩu

Bảng 3.15.a: Cấu trúc lược đồ bảng PASSWORD_RESET_TOKENS (Token đặt lại mật khẩu)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
user_id	bigint	Bắt buộc (Not Null); FK → users.id; UNIQUE(user_id)
token	nvarchar(60)	Bắt buộc (Not Null); UNIQUE(token)
expires_at	datetime2(7)	Bắt buộc (Not Null)
used_at	datetime2(7)	Tùy chọn (Nullable)
created_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()

4.44.1 Dữ liệu mẫu thực tế bảng PASSWORD_RESET_TOKENS

Bảng 3.15.b: Dữ liệu mẫu bảng PASSWORD_RESET_TOKENS (10 bản ghi minh họa thực tế)

id	user_id	token	expires_at	used_at	created_at
1	5	tok_sec_9941a82f	2026-02-15 14:35:00	2026-02-15 14:25:10	2026-02-15 14:20:00
2	6	tok_sec_8812c73d	2026-02-14 10:15:00	2026-02-14 10:08:40	2026-02-14 10:00:00
3	7	tok_sec_7734e62a	2026-02-12 16:45:00	2026-02-12 16:32:15	2026-02-12 16:30:00
4	8	tok_sec_6655f51b	2026-02-10 09:15:00	NULL	2026-02-10 09:00:00
5	9	tok_sec_5544d40c	2026-02-08 11:30:00	2026-02-08 11:20:05	2026-02-08 11:15:00
6	10	tok_sec_4433c39e	2026-02-05 15:00:00	2026-02-05 14:52:30	2026-02-05 14:45:00
7	5	tok_sec_3322b28f	2026-01-28 08:30:00	2026-01-28 08:22:18	2026-01-28 08:15:00
8	6	tok_sec_2211a17e	2026-01-20 13:45:00	NULL	2026-01-20 13:30:00
9	7	tok_sec_1100f06d	2026-01-15 17:15:00	2026-01-15 17:05:42	2026-01-15 17:00:00
10	8	tok_sec_0099e95c	2026-01-10 10:30:00	2026-01-10 10:18:20	2026-01-10 10:15:00

`PasswordResetToken` ánh xạ `password_reset_tokens`, liên kết User và lưu token, expiresAt, usedAt, createdAt. V11 bổ sung bảng này. Token là bí mật dùng một lần; cần tạo ngẫu nhiên đủ mạnh, chỉ lưu giá trị hash nếu thiết kế cho phép, giới hạn TTL, đánh dấu used atomically và không gửi lại token qua log. TTL mặc định cấu hình 30 phút.

Yêu cầu đặt lại cần chống dò user: response gửi về không nên tiết lộ email có tồn tại hay không. Link cần HTTPS trong môi trường triển khai và đúng base URL. Token hết hạn/đã dùng không được reset thêm lần nữa; nhiều request đồng thời cần tránh hai

lần đổi password. SMTP không bật theo mặc định trong cấu hình khảo sát, vì vậy có route reset không chứng minh email đã được gửi.

Kiểm thử: token sai/hết hạn/đã dùng, user disabled, hai lần xác nhận cùng lúc, password policy, token xuất hiện trong log/analytics và email provider timeout. Cần retention cho token cũ, cleanup job và audit sự kiện reset mà không lưu credential.

4.45 Bảng SHIPPERS — hồ sơ người giao hàng

Bảng 3.16.a: Cấu trúc lược đồ bảng SHIPPERS (Hồ sơ nhân viên giao hàng)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
user_id	bigint	Bắt buộc (Not Null); FK → users.id; UNIQUE(user_id)
name	nvarchar(127)	Bắt buộc (Not Null)
phone	nvarchar(25)	Bắt buộc (Not Null)
vehicle_type	nvarchar(50)	Tùy chọn (Nullable)
status	nvarchar(25)	Bắt buộc (Not Null); Mặc định ('ACTIVE')
is_available	bit	Bắt buộc (Not Null); Mặc định 0
created_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()
updated_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()
current_latitude	float	Tùy chọn (Nullable)
current_longitude	float	Tùy chọn (Nullable)
location_updated_at	datetime2(7)	Tùy chọn (Nullable)
max_concurrent_shipments	int	Bắt buộc (Not Null); Mặc định 1
active_shipment_count	int	Bắt buộc (Not Null); Mặc định 0
last_assignment_at	datetime2(7)	Tùy chọn (Nullable)

4.45.1 Dữ liệu mẫu thực tế bảng SHIPPERS

Bảng 3.16.b: Dữ liệu mẫu bảng SHIPPERS (10 bản ghi minh họa thực tế)

id	user_id	name	phone	vehicle_type	status	is_available	created_at	updated_at	current_latitude	current_longitude	location_updated_at	max_concurrent_shipments	active_shipment_count	last_assignment_at
1	3	Nguyễn Văn Tuấn	0918882233	MOTORBIKE (29B1-88899)	ACTIVE	1	2026-01-15	2026-02-20	10.7769	106.7009	2026-02-20 14:30	5	2	2026-02-20 14:00
2	4	Trần Minh Quân	0909112233	MOTORBIKE (59C2-12345)	ACTIVE	1	2026-01-16	2026-02-20	10.7626	106.6825	2026-02-20 14:25	5	3	2026-02-20 13:45
3	11	Lê Hoàng Phúc	0933445566	COLD_VAN (51D-9988)	ACTIVE	1	2026-01-18	2026-02-20	10.7828	106.6982	2026-02-20 14:20	3	1	2026-02-20 13:10
4	12	Phạm Quốc Bảo	0944556677	MOTORBIKE (59F1-67890)	OFF_DUTY	0	2026-01-20	2026-02-20	10.7700	106.6900	2026-02-20 12:00	5	0	2026-02-19 16:30
5	13	Võ Thành Đạt	0922334455	MOTORBIKE (59G2-34567)	ACTIVE	1	2026-01-22	2026-02-20	10.7550	106.6750	2026-02-20 14:15	5	2	2026-02-20 11:20

id	user_id	name	phone	vehicle_type	status	is_available	created_at	updated_at	current_latitude	current_longitude	location_updated_at	max_concurrent_shipments	active_shipment_count	last_assignment_at
6	14	Đặng Văn Lâm	0966778899	MOTORBIKE (59H1-89012)	ACTIVE	1	2026-01-25	2026-02-20	10.7900	106.7100	2026-02-20 14:10	5	1	2026-02-20 10:00
7	15	Hoàng Kim Trọng	0977889900	COLD_VAN (51C-4455)	ACTIVE	1	2026-01-28	2026-02-20	10.7650	106.6950	2026-02-20 14:05	3	2	2026-02-20 09:30
8	16	Bùi Tiến Dũng	0988990011	MOTORBIKE (59K1-23456)	ACTIVE	1	2026-02-01	2026-02-20	10.7800	106.6850	2026-02-20 14:00	5	1	2026-02-19 14:15
9	17	Đỗ Hùng Dũng	0911223344	MOTORBIKE (59L2-78901)	OFF_DUTY	0	2026-02-05	2026-02-20	10.7720	106.7020	2026-02-20 11:30	5	0	2026-02-18 09:40
10	18	Nguyễn Quang Hải	0933557799	MOTORBIKE (59M1-45678)	ACTIVE	1	2026-02-10	2026-02-20	10.7780	106.6990	2026-02-20 13:50	5	2	2026-02-17 15:20

`Shipper` ánh xạ `shippers`, liên kết User, có name/phone/vehicleType, currentLatitude/currentLongitude, locationUpdatedAt, lastAssignmentAt, createdAt/updatedAt. Hồ sơ này tách quyền đăng nhập khỏi dữ liệu vận hành giao hàng. Trạng thái hoạt động của shipper được biểu diễn bằng `ShipperStatus`/association theo source; chính sách chọn nhận nhiệm vụ cần xem đầy đủ service và migration.

Tọa độ vị trí là dữ liệu nhạy cảm, phải giới hạn retention, quyền xem và độ tươi. `locationUpdatedAt` giúp phân biệt vị trí cũ; không nên phân công dựa trên tọa độ hết hạn. `lastAssignmentAt` có thể giúp phân phối cân bằng, nhưng code hiện không được mô tả là thuật toán tuyến đường tối ưu nếu chỉ dùng queue/availability.

Kiểm thử: shipper không có User, hai profile cho một user, profile inactive, tọa độ thiếu/sai, timestamp cũ, cập nhật vị trí trái quyền và xóa user đang giữ shipment. Tài liệu không in danh sách tài khoản hoặc vị trí người giao hàng thực.

4.46 Bảng SHIPMENTS — vận đơn

Bảng 3.17.a: Cấu trúc lược đồ bảng SHIPMENTS (Vận đơn điều phối giao nhận)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
order_id	bigint	Bắt buộc (Not Null); FK → orders.id; UNIQUE(order_id)
shipper_id	bigint	Tùy chọn (Nullable); FK → shippers.id
status	nvarchar(25)	Bắt buộc (Not Null); Mặc định ('PENDING_ASSIGNMENT')
shipping_name	nvarchar(127)	Tùy chọn (Nullable)
shipping_phone	nvarchar(25)	Tùy chọn (Nullable)
shipping_address	nvarchar(500)	Tùy chọn (Nullable)
otp_code	nvarchar(3)	Tùy chọn (Nullable)
otp_verified	bit	Bắt buộc (Not Null); Mặc định 0
failure_note	nvarchar(250)	Tùy chọn (Nullable)
created_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()
updated_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()
picked_up_at	datetime2(7)	Tùy chọn (Nullable)
delivering_at	datetime2(7)	Tùy chọn (Nullable)
completed_at	datetime2(7)	Tùy chọn (Nullable)
shipping_latitude	float	Tùy chọn (Nullable)
shipping_longitude	float	Tùy chọn (Nullable)
otp_created_at	datetime2(7)	Tùy chọn (Nullable)
otp_expires_at	datetime2(7)	Tùy chọn (Nullable)
otp_attempt_count	int	Bắt buộc (Not Null); Mặc định 0
otp_locked_until	datetime2(7)	Tùy chọn (Nullable)
otp_last_sent_at	datetime2(7)	Tùy chọn (Nullable)
admin_override	bit	Bắt buộc (Not Null); Mặc định 0
admin_override_reason	nvarchar(250)	Tùy chọn (Nullable)
otp_delivery_status	nvarchar(10)	Bắt buộc (Not Null); Mặc định ('PENDING')
otp_sent_at	datetime2(7)	Tùy chọn (Nullable)
otp_user_id	bigint	Tùy chọn (Nullable); FK → users.id
estimated_delivery_at	datetime2(7)	Tùy chọn (Nullable)
promised_window_start	datetime2(7)	Tùy chọn (Nullable)
promised_window_end	datetime2(7)	Tùy chọn (Nullable)
delivery_attempt_count	int	Bắt buộc (Not Null); Mặc định 0
last_delivery_attempt_at	datetime2(7)	Tùy chọn (Nullable)
next_attempt_at	datetime2(7)	Tùy chọn (Nullable)
assigned_at	datetime2(7)	Tùy chọn (Nullable)
failed_at	datetime2(7)	Tùy chọn (Nullable)
returned_at	datetime2(7)	Tùy chọn (Nullable)
failure_code	nvarchar(25)	Tùy chọn (Nullable)
status_reason	nvarchar(250)	Tùy chọn (Nullable)
shipping_email	nvarchar(127)	Tùy chọn (Nullable)

4.46.1 Dữ liệu mẫu thực tế bảng SHIPMENTS

Bảng 3.17.b: Dữ liệu mẫu bảng SHIPMENTS (10 bản ghi minh họa thực tế)

id	order_id	shipper_id	status	shipping_name	shipping_phone	shipping_address	otp_code	otp_verified	failure_note	created_at	completed_at	shipping_email
9042	1042	1	DELIVERING	Nguyễn Thành Duy Tân	0908123456	125 Lê Lai, Q.1, TP.HCM	842915	0	NULL	2026-02-20 14:02	NULL	duytan.wine@gmail.com
9041	1041	2	PROCESSING	Trần Văn An	0913456789	45 Nam Kỳ Khởi Nghĩa, Q.1	519284	0	NULL	2026-02-20 13:46	NULL	an.tran@gmail.com
9040	1040	NULL	PENDING	Lê Thị Bích	0987654321	88 Nguyễn Du, Q.1	NULL	0	NULL	2026-02-20 13:10	NULL	bich.le@outlook.com
9039	1039	1	DELIVERED	Phạm Hoàng Nam	0978112233	12 Nguyễn Huệ, Q.1	673412	1	NULL	2026-02-20 11:22	2026-02-20 13:00	nam.hoang@yahoo.com
9038	1038	3	DELIVERED	Võ Minh Trí	0934556677	18 Tôn Đức Thắng, Q.1	284917	1	NULL	2026-02-20 10:01	2026-02-20 11:45	tri.vo@gmail.com
9037	1037	1	DELIVERED	Lê Phương Thảo	0945667788	254 Nguyễn Văn Cừ, Q.5	918273	1	NULL	2026-02-19 16:30	2026-02-19 18:00	thao.le@fpt.edu.vn
9036	1036	2	DELIVERED	Nguyễn Thành Duy Tân	0908123456	125 Lê Lai, Q.1, TP.HCM	384912	1	NULL	2026-02-19 14:16	2026-02-19 16:00	duytan.wine@gmail.com
9035	1035	1	DELIVERED	Trần Văn An	0913456789	45 Nam Kỳ Khởi Nghĩa, Q.1	492018	1	NULL	2026-02-18 09:42	2026-02-18 11:30	an.tran@gmail.com
9034	1034	3	DELIVERED	Lê Thị Bích	0987654321	88 Nguyễn Du, Q.1	182940	1	NULL	2026-02-17 15:21	2026-02-17 17:15	bich.le@outlook.com
9033	1033	2	DELIVERED	Phạm Hoàng Nam	0978112233	12 Nguyễn Huệ, Q.1	749102	1	NULL	2026-02-16 11:01	2026-02-16 13:00	nam.hoang@yahoo.com

`Shipment` ánh xạ `shipments`, liên kết Order/Shipper và lưu snapshot nhận hàng. Entity có trạng thái, lý do, mã lỗi, thông tin lần giao, thời gian dự kiến/promise window, các mốc assigned/picked-up/delivering/completed/failed/returned và các trường OTP như created/expires/locked/last-sent/user binding. V5–V10 bổ sung nền logistics, tọa độ/email, dispatch và bảo vệ OTP.

Một order paid có thể được xử lý tự động tạo/gán shipment qua listener xử lý sau khi giao dịch hoàn tất hoặc nằm pending queue; scheduler định kỳ gọi dispatch. `ShipmentService` chứa state transition, OTP policy và audit; độ lớn service là một điểm cần theo dõi. Tạo shipment idempotent là bắt buộc để listener/reconciliation không sinh bản ghi trùng. Trạng thái admin override phải ghi reason/actor.

Kiểm thử: paid order lặp, nhiều shipper cạnh tranh, shipper bị inactive giữa luồng, shipment không có người gán, retry email, OTP expired/locked, failure/retry, returned và complete hợp lệ. Payload chứa OTP không được đưa vào API listing. Một dashboard screenshot chỉ thể hiện UI, không xác nhận transition đã chạy.

4.47 Bảng SHIPMENT_STATUS_HISTORY — lịch sử trạng thái vận đơn

Bảng 3.18.a: Cấu trúc lược đồ bảng SHIPMENT_STATUS_HISTORY (Lịch sử trạng thái vận đơn)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
shipment_id	bigint	Bắt buộc (Not Null); FK → shipments.id
from_status	nvarchar(25)	Tùy chọn (Nullable)
to_status	nvarchar(25)	Bắt buộc (Not Null)
reason	nvarchar(250)	Tùy chọn (Nullable)
metadata	nvarchar(1000)	Tùy chọn (Nullable)
actor_user_id	bigint	Tùy chọn (Nullable)
actor_username	nvarchar(127)	Tùy chọn (Nullable)
created_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()

4.47.1 Dữ liệu mẫu thực tế bảng SHIPMENT_STATUS_HISTORY

Bảng 3.18.b: Dữ liệu mẫu bảng SHIPMENT_STATUS_HISTORY (10 bản ghi minh họa thực tế)

id	shipment_id	from_status	to_status	reason	metadata	actor_user_id	actor_username	created_at
1	9042	CREATED	ASSIGNED	Gán shipper phụ trách	{"shipper_id":1}	1	admin_root	2026-02-20 14:10:00
2	9042	ASSIGNED	PICKED_UP	Shipper đã nhận thùng lạnh	{"warehouse_id":1}	3	shipper_tuan	2026-02-20 14:20:00
3	9042	PICKED_UP	DELIVERING	Bắt đầu di chuyển giao	{"gps":"10.7769,106.7009"}	3	shipper_tuan	2026-02-20 14:30:00
4	9041	CREATED	ASSIGNED	Gán shipper phụ trách	{"shipper_id":2}	1	admin_root	2026-02-20 13:50:00
5	9039	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	3	shipper_tuan	2026-02-20 13:00:00
6	9038	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	11	shipper_phuc	2026-02-20 11:45:00
7	9037	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	3	shipper_tuan	2026-02-19 18:00:00
8	9036	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	4	shipper_quan	2026-02-19 16:00:00

id	shipment_id	from_status	to_status	reason	metadata	actor_user_id	actor_username	created_at
9	9035	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	3	shipper_tuan	2026-02-18 11:30:00
10	9034	DELIVERING	DELIVERED	Khách xác thực OTP thành công	{"otp_verified":true}	11	shipper_phuc	2026-02-17 17:15:00

`ShipmentStatusHistory` ánh xạ `shipment_status_history`, liên kết Shipment, lưu fromStatus/toStatus, reason, metadata, actorUserId/actorUsername. Bảng giúp tái dựng diễn tiến và trả lời ai đã đổi trạng thái, vì sao và từ đâu. Lịch sử nên append-only theo quyền ứng dụng; sửa/xóa record cũ làm giảm khả năng điều tra.

`metadata` nên là cấu trúc có version hoặc tập thuộc tính giới hạn, không tùy tiện lưu PII. Actor có thể là user, scheduler hoặc hệ thống; cần quy ước định danh và loại actor. Nếu `fromStatus` null ở trạng thái đầu, schema/service phải cho phép và API trình bày nhất quán.

Kiểm thử: transition bị từ chối không tạo history; transition hợp lệ ghi đúng cặp trạng thái; actor từ principal chính xác; scheduler không giả thành user; metadata không chứa OTP; nhiều cập nhật đồng thời giữ thứ tự. Cần xem transaction bao quanh update shipment và history để tránh một bên thành công còn bên kia thất bại.

4.48 Bảng SHIPMENT_OTP_AUDIT_LOGS — nhật ký bảo mật OTP

Bảng 3.19.a: Cấu trúc lược đồ bảng SHIPMENT_OTP_AUDIT_LOGS (Nhật ký xác thực OTP)

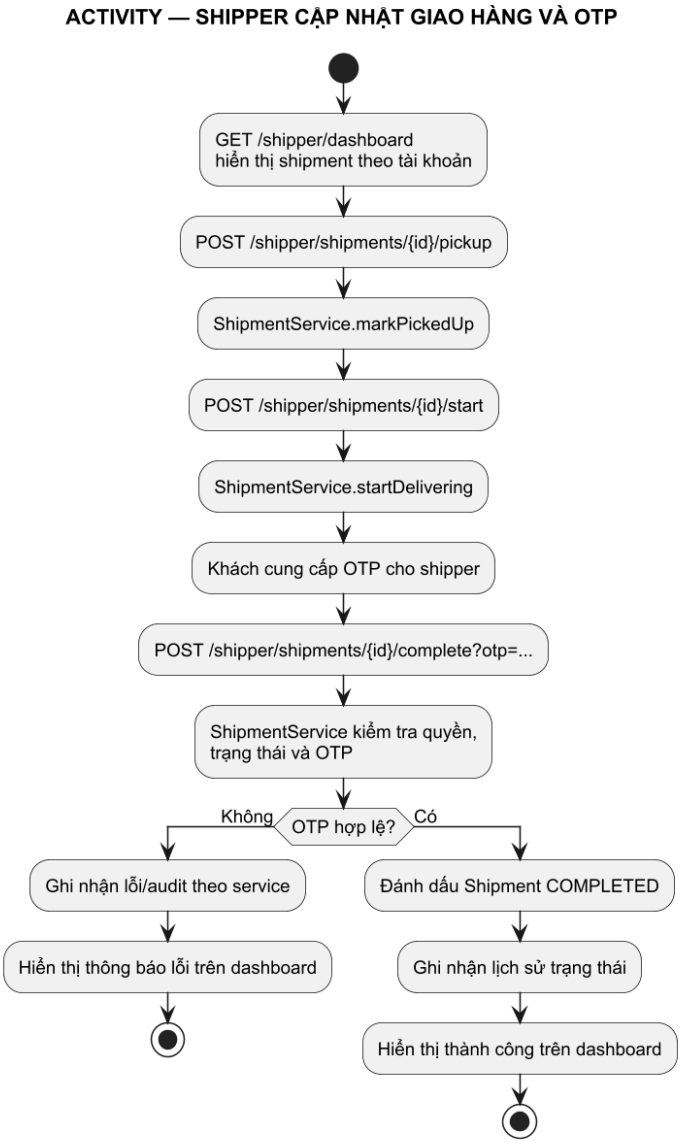
Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
shipment_id	bigint	Bắt buộc (Not Null); FK → shipments.id
order_id	bigint	Tùy chọn (Nullable)
otp_user_id	bigint	Tùy chọn (Nullable)
actor_user_id	bigint	Tùy chọn (Nullable)
actor_username	nvarchar(127)	Tùy chọn (Nullable)
action	nvarchar(50)	Bắt buộc (Not Null)
status	nvarchar(25)	Bắt buộc (Not Null)
reason	nvarchar(500)	Tùy chọn (Nullable)
metadata	nvarchar(1000)	Tùy chọn (Nullable)
created_at	datetime2(7)	Bắt buộc (Not Null); Mặc định getdate()

4.48.1 Dữ liệu mẫu thực tế bảng SHIPMENT_OTP_AUDIT_LOGS

Bảng 3.19.b: Dữ liệu mẫu bảng SHIPMENT_OTP_AUDIT_LOGS (10 bản ghi minh họa thực tế)

id	shipment_id	order_id	otp_user_id	actor_user_id	actor_username	action	status	reason	metadata	created_at
1	9042	1042	5	3	shipper_tuan	GENERATE_OTP	SUCCESS	Sinh mã OTP khi giao hàng	{"expires_in":600}	2026-02-20 14:30:00
2	9041	1041	6	4	shipper_quan	GENERATE_OTP	SUCCESS	Sinh mã OTP khi giao hàng	{"expires_in":600}	2026-02-20 14:00:00
3	9039	1039	8	3	shipper_tuan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-20 13:00:00
4	9038	1038	9	11	shipper_phuc	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-20 11:45:00
5	9037	1037	10	3	shipper_tuan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-19 18:00:00

id	shipment_id	order_id	otp_user_id	actor_user_id	actor_username	action	status	reason	metadata	created_at
6	9036	1036	5	4	shipper_quan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-19 16:00:00
7	9035	1035	6	3	shipper_tuan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-18 11:30:00
8	9034	1034	7	11	shipper_phuc	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-17 17:15:00
9	9033	1033	8	4	shipper_quan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-16 13:00:00
10	9032	1032	9	3	shipper_tuan	VERIFY_OTP	SUCCESS	Xác thực mã 6 chữ số hợp lệ	{"attempt":1}	2026-02-15 15:30:00



Hình 3.19.c: Sơ đồ luồng xác thực và kiểm toán mã bảo mật giao hàng OTP

Service định nghĩa TTL 10 phút, tối đa 5 lần thử, khóa 15 phút và cooldown gửi lại 60 giây. Đây là ngưỡng trong mã trong mã nguồn được khảo sát, không phải khuyến nghị phổ quát. Thay đổi chính sách cần cân bằng giữa chống brute force, người nhận nhập sai, khả năng resend và độ trễ email. Các attempt phải cập nhật nguyên tử để hai request đồng thời không vượt giới hạn.

Kiểm thử gồm mã sai, mã đã dùng/hết hạn, quá 5 lần, gửi lại trong cooldown, OTP gắn sai user/order, shipper giả danh, email delivery failure và admin override có reason. Cần đảm bảo response không phân biệt quá chi tiết điều kiện giúp dò mã; audit log chỉ được phép xem bởi vai trò cần thiết và có retention rõ.

4.49 Đối chiếu các bảng bổ sung và enum miền

Bảng 3.20.a: Cấu trúc lược đồ bảng SYSTEM_CONFIGS (Cấu hình hệ thống và tham số)

Tên cột	Kiểu dữ liệu SQL	Ràng buộc & Ghi chú
id	bigint	PK; IDENTITY
name	nvarchar(25)	Bắt buộc (Not Null); UNIQUE(name)
Tên cột	Kiểu dữ liệu SQL	NULL / khóa / default / index / CHECK
user_id	bigint	PK; FK → users.id
role_id	bigint	PK; FK → roles.id
Tên cột	Kiểu dữ liệu SQL	NULL / khóa / default / index / CHECK
id	bigint	PK; IDENTITY
name	nvarchar(127)	Bắt buộc (Not Null)
location	nvarchar(127)	Tùy chọn (Nullable)
active	bit	Bắt buộc (Not Null); Mặc định 1

4.49.1 Dữ liệu mẫu thực tế bảng SYSTEM_CONFIGS

Bảng 3.20.b: Dữ liệu mẫu bảng SYSTEM_CONFIGS (10 bản ghi minh họa thực tế)

id	config_key	config_value	description	updated_at
1	app.shipping.free_threshold	10000000	Ngưỡng miễn phí vận chuyển kho lạnh (VNĐ)	2026-01-10 08:00:00
2	app.shipping.base_fee	150000	Cước vận chuyển thùng lạnh nội thành (VNĐ)	2026-01-10 08:00:00
3	app.otp.expiry_seconds	600	Thời hạn hiệu lực của mã OTP giao hàng (giây)	2026-01-10 08:00:00
4	app.otp.cooldown_seconds	60	Khoảng đệm an toàn giữa 2 lần yêu cầu gửi lại OTP	2026-01-10 08:00:00
5	app.inventory.alert_threshold	5	Ngưỡng số lượng kích hoạt cảnh báo tồn kho thấp	2026-01-10 08:00:00
6	app.security.token_expiry_minutes	15	Thời gian tồn tại của token khôi phục mật khẩu	2026-01-10 08:00:00
7	app.upload.max_file_size_mb	5	Dung lượng tối đa của tệp ảnh upload sản phẩm (MB)	2026-01-10 08:00:00
8	app.upload.dir	uploads/wines	Đường dẫn lưu trữ tập tin ảnh tĩnh trên máy chủ	2026-01-10 08:00:00
9	app.stripe.currency	vnd	Đơn vị tiền tệ giao dịch qua cổng Stripe Checkout	2026-01-10 08:00:00
10	app.site.support_hotline	1900 888 999	Tổng đài hỗ trợ tư vấn và khiếu nại khách hàng	2026-01-10 08:00:00

Ngoài 19 heading từ tài liệu cũ, code hiện có `ShipmentStatusHistory`, `ShipmentOtpAuditLog`, `PaymentTransaction`, `PasswordResetToken`, `InventoryTransaction` và các enum trạng thái riêng. Nếu bảng nào đã có trong danh sách trên, cần ghi tên vật lý hiện hành thay vì đếm trùng. Enum `PaymentMethod`,

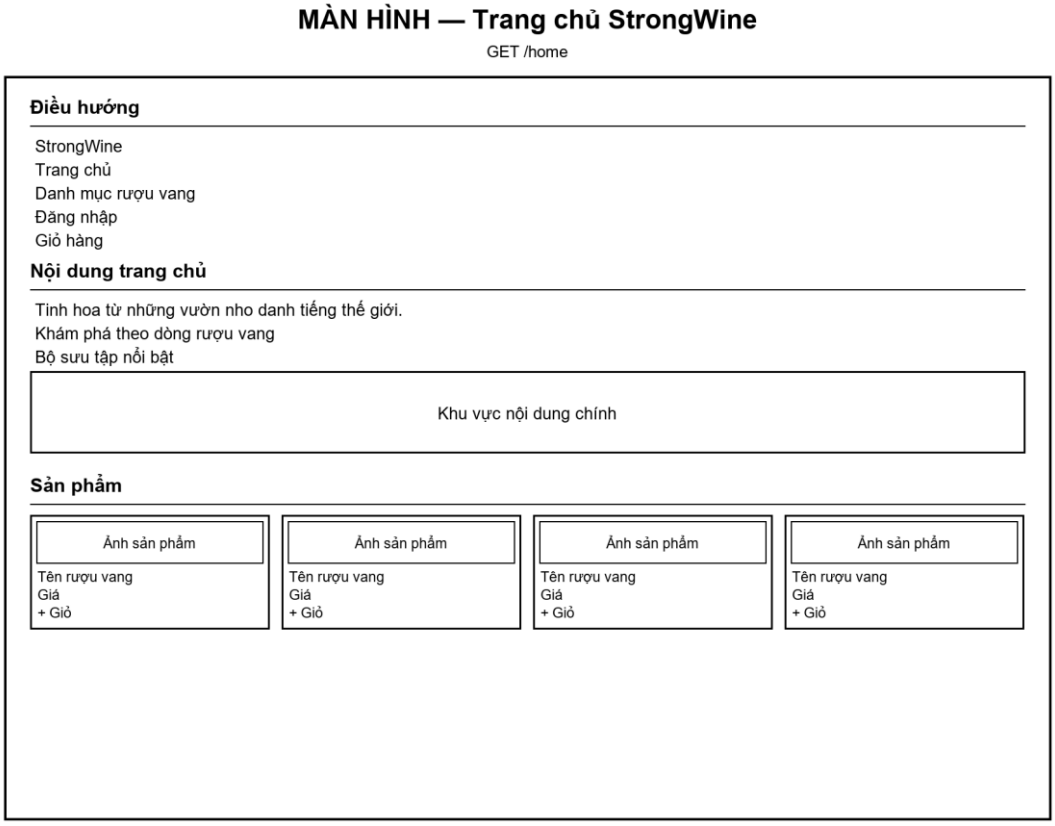
`PaymentStatus`, `OrderStatus`, `ShipmentStatus`, `ShipperStatus`, `OtpDeliveryStatus` và `InventoryOperationType` biểu diễn miền giá trị trong Java; ràng buộc DB tương ứng cần xác minh.

Entity/repository không phải bản sao đầy đủ của schema. Migration V3 đối chiếu entity, các migration sau thêm trường dần; SQL Server local hiện có version cao hơn checkout theo lịch sử đã ghi. Vì vậy đối chiếu cuối phải gồm entity fields, annotation, migration DDL và kết quả `flyway_schema_history` của môi trường cụ thể. Quá trình đối soát kỹ thuật phân biệt rạch ròi giữa phiên bản cơ sở dữ liệu cục bộ và trạng thái chính thức trên kho lưu trữ mã nguồn.

4.50 ERD và nguyên tắc cập nhật sơ đồ

ERD mới cần lấy tên bảng/khóa từ annotation và migration: users nối orders/reviews/carts/shipper; orders nối items/payments/shipments; wine/category, wine/inventory/warehouse, inventory transactions và stock logs; shipment nối shipper và hai bảng audit/lịch sử. Cardinality và cascade cần xác minh trước khi vẽ. Ảnh ERD cũ giữ lại để tham khảo theo yêu cầu, caption ghi rõ không phải schema hiện tại.

Quy trình cập nhật ERD nên được thực hiện khi migration được merge: tạo sơ đồ từ schema hoặc model review, xác minh bằng một truy vấn catalog SQL, so sánh tên cột và FK, rồi ghi source version/date. Không dùng ảnh cắt từ tài liệu cũ làm hình chính thức. Hình raster lịch sử có thể đặt trong phần "tư liệu đối chiếu", sau đó trình bày ERD mới dưới dạng vector dễ đọc và có alt text.



Hình 4.1: Wireframe Trang chủ sàn thương mại điện tử StrongWine (Grayscale Web Mockup)

Optimistic lock `@Version` ở inventory phát hiện cập nhật cạnh tranh; pessimistic lock service có thể khóa dòng tồn ở critical section. Chọn cơ chế nào phụ thuộc mật độ đồng thời và thời gian giữ lock. Deadlock có thể xảy ra khi checkout nhiều wine theo thứ tự khác nhau; sắp xếp lock theo ID, retry có giới hạn và quan sát SQL giúp giảm rủi ro. Bài chưa có load test để kết luận thông lượng.

Việc phát event sau khi transaction thành công bảo đảm listener không tạo shipment cho một order bị rollback. Đổi lại, tiến trình có thể dừng sau khi dữ liệu đã được ghi nhưng trước khi event được xử lý; listener cũng có thể phát sinh lỗi trong khi order vẫn ở trạng thái paid. Outbox hoặc đối soát định kỳ giúp xử lý khoảng hở này. Cần bảo đảm mỗi order chỉ có một shipment và tác vụ có thể chạy lặp an toàn.

4.51 Phiên bản hóa schema bằng Flyway

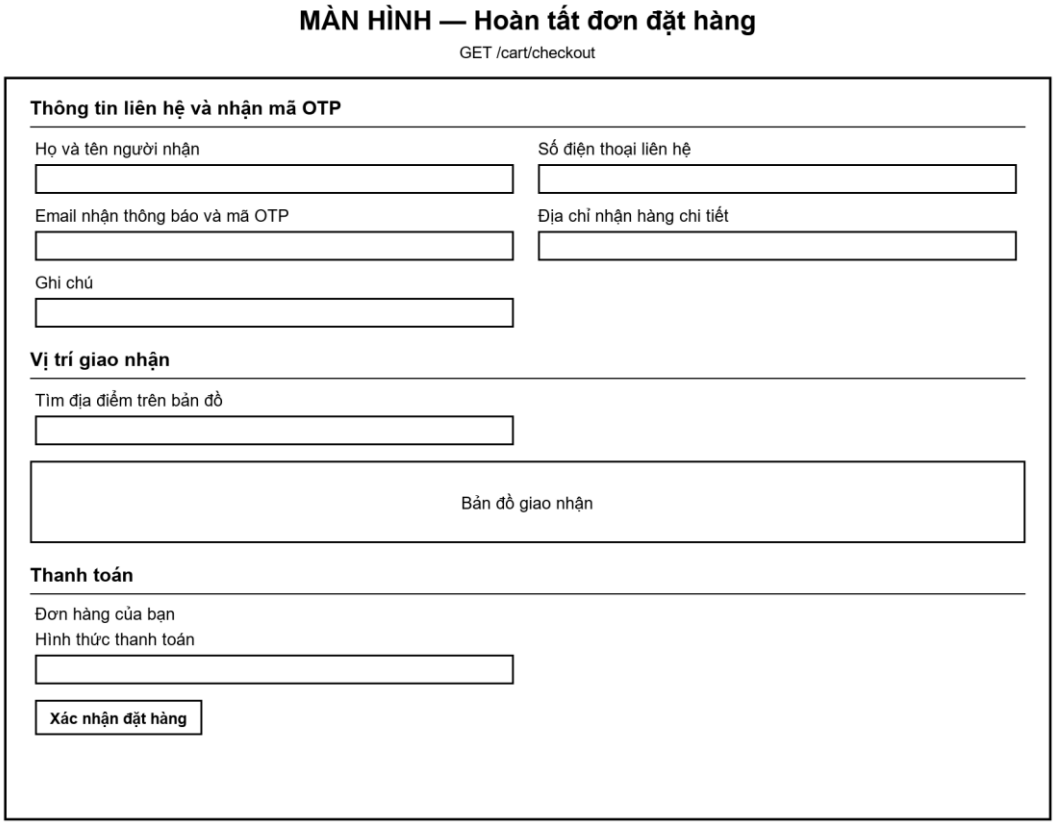
Migration được đánh số theo chuỗi V2–V13 trong checkout; các thay đổi lần lượt gồm audit fields, schema reconciliation, created_by inventory, shipper/shipment, seed role, tọa độ và OTP fields, harden OTP, logistics lifecycle, email/dispatch, reset tokens,

seed normalize và dataset rượu Việt Nam. Lịch sử cho thấy nghiệp vụ vận chuyển phát triển qua nhiều bước chứ không có một schema tĩnh. [1]

Trước khi migrate, cần backup, xác định đúng instance/database, xem checksum và đảm bảo quyền DDL. Người dùng trước đó đã cho phép xử lý migration/seed/repair trên local; sự cho phép đó không áp dụng cho database production khác. Mục này chỉ ghi lại trạng thái local đã xử lý trong lượt trước và quy tắc an toàn triển khai.

Trong build pipeline nên có database rỗng để chạy toàn bộ migration, database nâng cấp từ phiên bản trước, validate JPA sau migrate, test seed lặp an toàn, và backup/restore rehearsal. Không sửa trực tiếp `flyway_schema_history` nếu chưa có playbook; checksum repair chỉ nên dùng khi hiểu nguồn gốc lệch và đã so sánh file với schema.

4.52 Bảo vệ dữ liệu cá nhân trong mô hình miền



Hình 4.5: Wireframe Giao diện thanh toán đơn hàng Checkout (Grayscale Web Mockup)

Thử nghiệm privacy nên tạo dữ liệu tổng hợp, chụp ảnh vùng đã crop, kiểm tra metadata file và tìm secret trong log/report. Dữ liệu thật chỉ dùng cục bộ khi quyền truy

cập cần thiết; kết quả bài này không chứa tên account hoặc credential. Tách test fixture khỏi production giúp giảm khả năng vô tình lưu dữ liệu thật vào tài liệu nộp.

4.53 Lớp controller và ranh giới HTTP

Controller MVC nhận request form, lấy principal, gọi service rồi trả tên view/redirect hoặc JSON tùy route theo chuẩn OpenAPI 3.1 [11]. Controller API nằm trong package `controller.api`, ví dụ `WineApiController`, `OrderApiController`, `ReviewApiController` và các `Admin...ApiController`. Hai giao diện có thể phục vụ cùng một miền nhưng không nên tạo quy tắc riêng. Việc rà source cần phân biệt route MVC và API vì form binding, response code, CSRF và client behavior khác nhau.

Controller phù hợp để ánh xạ HTTP sang lệnh/query application. Nó không nên tự truy cập repository rải rác hoặc tự tính điều kiện giá/tồn; nếu một endpoint cần thêm logic thì nên cân nhắc đưa vào service dùng chung. `@ControllerAdvice`/exception handler nếu có cần thống nhất HTML/API response; người dùng không nên nhận stack trace. Cần xác nhận từng controller có `@Valid`, quyền và ownership guard tương ứng trước khi đánh dấu chức năng an toàn.

Một endpoint hữu ích có hợp đồng rõ: method/path, authentication, request schema, validation, status code, response body, side effect và khả năng retry. Hợp đồng này có thể được ghi trong bảng phụ lục. Đối với mutating endpoints, idempotency và CSRF cần được xem theo cơ chế phiên/token; đối với GET, không thay đổi trạng thái và không lộ dữ liệu của người khác.

4.54 Lớp service và điều phối use case

Service là nơi chứa điều kiện nghiệp vụ và phối hợp nhiều repository hoặc adapter dưới sự quản lý bean của Spring IoC [3]. `OrderService` tạo pending order, gọi giữ tồn, chốt paid, hủy và giải phóng reservation; `PaymentService` xử lý session, callback và webhook; `ShipmentService` điều phối lifecycle/OTP; `InventoryService` cập nhật current/reserved quantity và transaction ledger. Các service này nên được gọi từ MVC/API/listener/scheduler bằng cùng quy tắc.

`@Transactional` [6] ở service giúp transaction bám theo đơn vị nghiệp vụ bảo đảm tính nhất quán ACID; query chỉ đọc có thể dùng readOnly. Khi một service gọi method transactional nội bộ cùng bean, proxy semantics có thể khác gọi từ bean khác;

cần hiểu self-invocation và propagation trước khi dựa vào annotation. Exception rollback behavior cũng phụ thuộc loại exception và cấu hình. Bài phân tích ranh giới kiến trúc, còn integration test mới chứng minh transaction behavior.

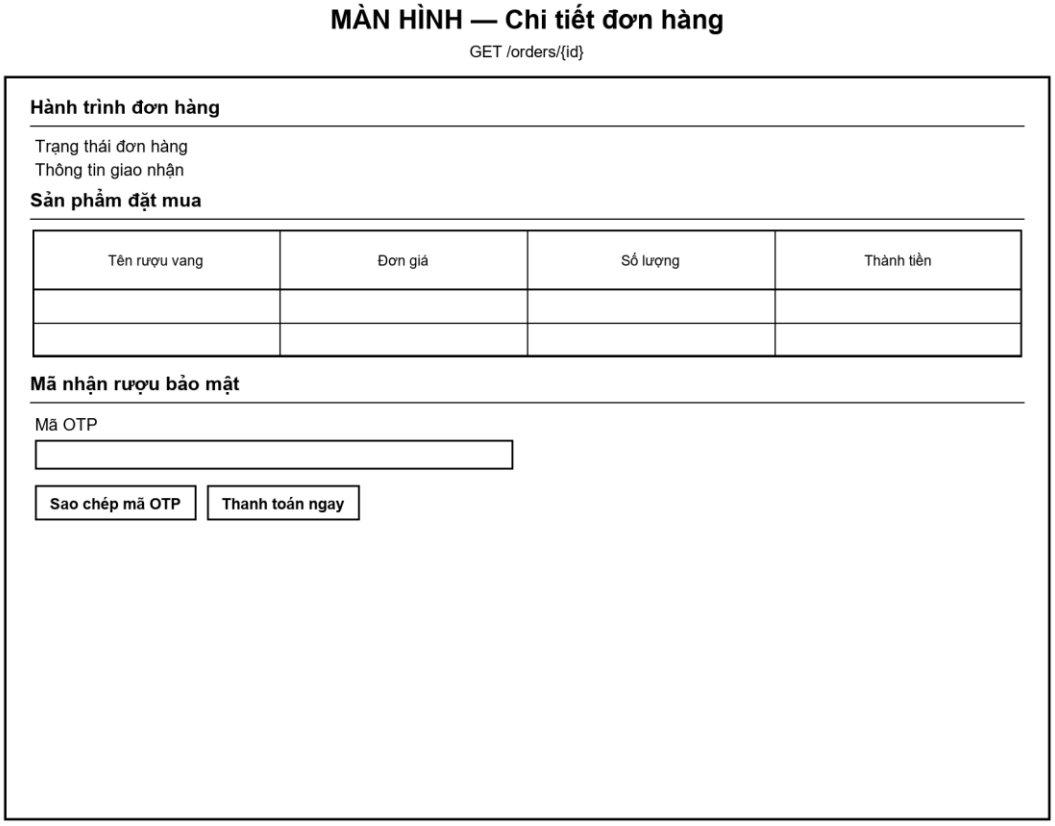
Các service lớn như `ShipmentService` chứa nhiều chính sách; điểm tách tốt là quyết định có invariant riêng và có test riêng, ví dụ OTP policy/dispatch policy. Việc tách không nên làm mất transaction boundary hoặc khiến service gọi vòng lặp. Nên bắt đầu bằng test của trạng thái hiện tại, xác định caller bằng code graph khi có CRG, sau đó dùng tìm kiếm source nếu graph unavailable. Trong môi trường này CRG tools không có nên caller được rà bằng source/rg.

4.55 Repository và truy vấn miền

Spring Data repository cung cấp CRUD, query theo naming convention và hỗ trợ ghép vị từ Specification linh hoạt [7]. Repository abstraction giảm coupling với SQL/JPA chi tiết nhưng không xóa N+1 query, pagination, locking hoặc transaction. Mỗi query nên có mục đích rõ và hạn chế tải cả object graph khi màn hình chỉ cần summary. Khi xuất báo cáo, dùng projection/DTO cho số liệu thay vì mở mọi entity và gọi getter lazy trong vòng lặp.

`WineSpecification` biểu diễn các predicate lọc có thể ghép, phù hợp khi UI cho chọn nhiều điều kiện. Mỗi filter nên chuẩn hóa input, kết hợp AND/OR rõ và kiểm thử ranh giới (khoảng giá, năm, category, country, text). Specification chỉ tạo điều kiện truy vấn; sort, phân trang, index và bảo vệ dữ liệu vẫn cần giải quyết riêng.

Query có khóa cần xác định lock mode và thứ tự lấy khóa. Đặt pessimistic lock trong thao tác đọc catalog làm tăng contention; dùng lock cho row inventory ở critical section có lý do rõ hơn. Khi test deadlock, cần hai giao dịch giữ nhiều row theo thứ tự khác nhau. Do thiếu dữ liệu đo đạc thực nghiệm tại tầng cơ sở dữ liệu, bài nghiên cứu không cam kết trước một chiến lược chỉ mục hay khóa dòng cố định về mặt hiệu năng theo lý thuyết của C.J. Date [10].



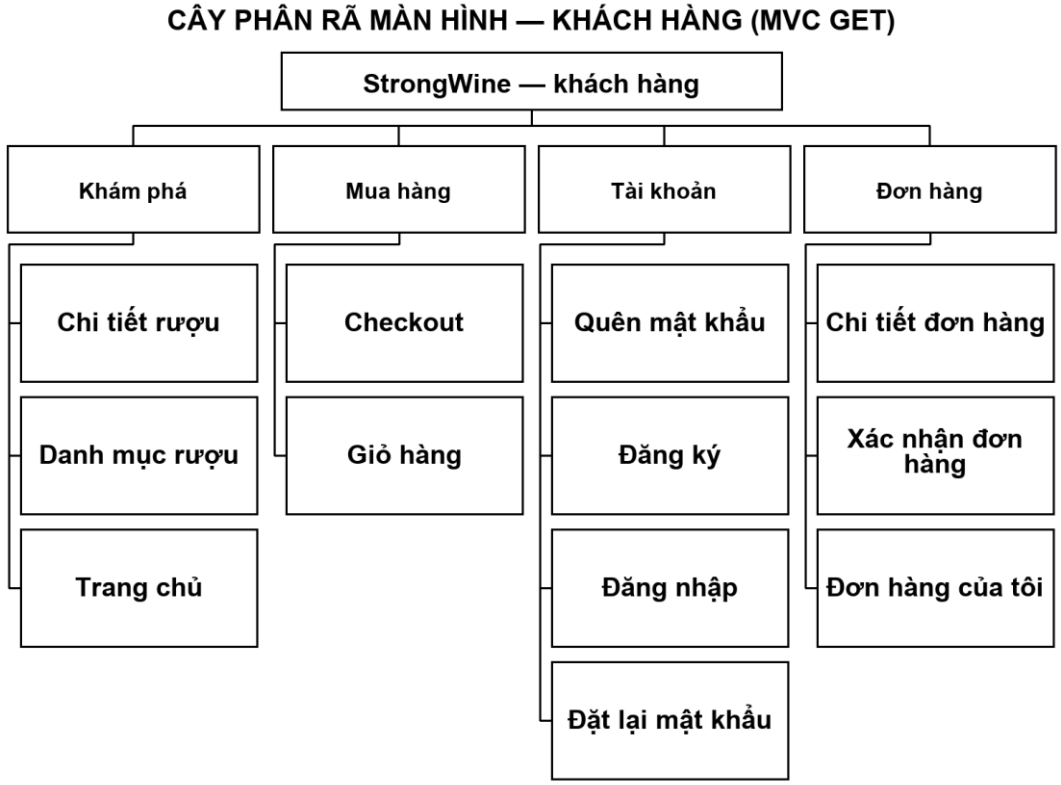
4.56

Hình 4.6: Wireframe Chi tiết đơn hàng và theo dõi vận chuyển (Grayscale Web Mockup)

Thứ tự khởi tạo bean do Spring container quản lý, còn scheduler chạy theo clock và cấu hình delay. Không dựa vào thứ tự khai báo bean để đảm bảo dữ liệu seed; dùng Flyway/initializer với idempotency. Khi context test khởi động, provider thật không nên được gọi. Môi trường local phải tách secret và endpoints khỏi profile test.

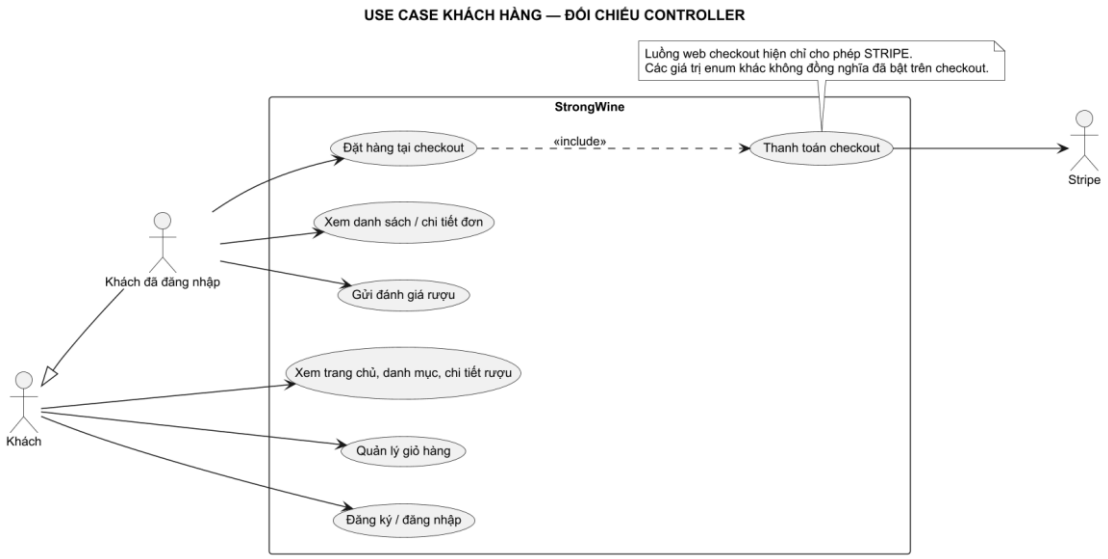
CHƯƠNG 4: LUỒNG NGHIỆP VỤ KHÁCH HÀNG

Sơ đồ cây màn hình khách hàng tổng hợp các MVC GET view đã kiểm tra: trang chủ; danh mục và chi tiết rượu; giỏ hàng và checkout; đăng nhập, đăng ký, quên mật khẩu và đặt lại mật khẩu; danh sách, chi tiết và xác nhận đơn hàng. Đặt hàng gửi POST /cart/checkout/process; luồng checkout hiện chuyển khách tới Stripe Checkout.



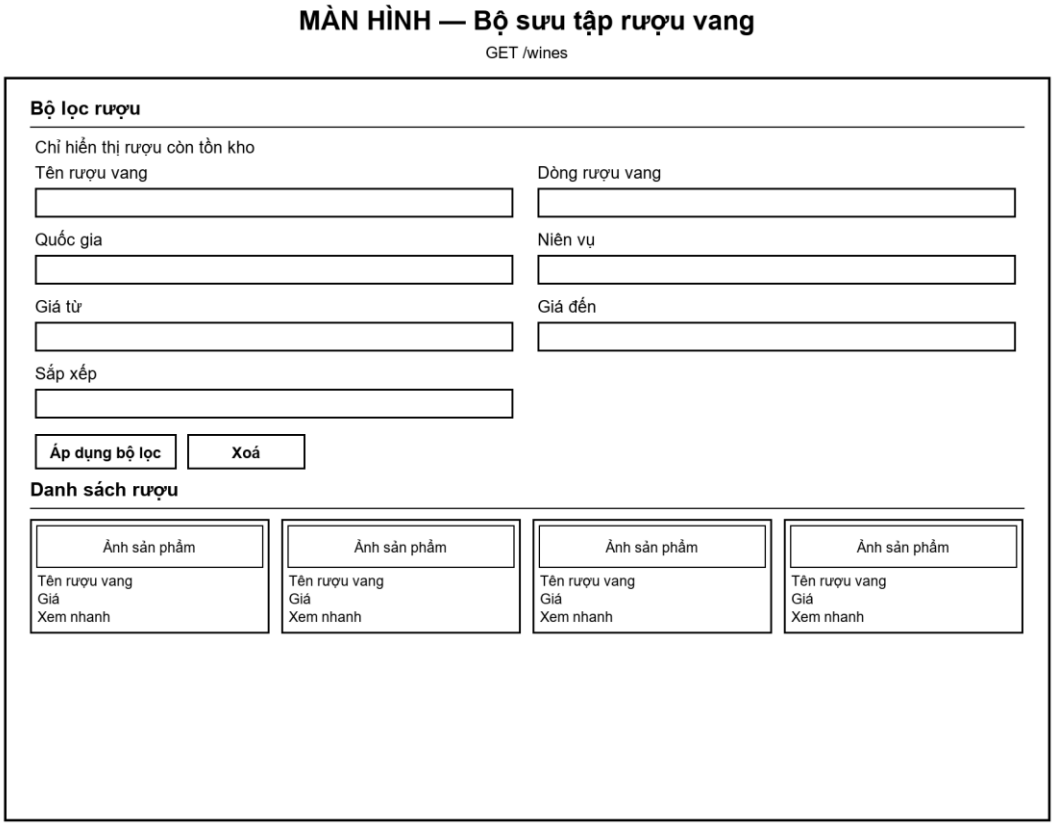
Hình 4.0: Sơ đồ phân rã màn hình phân hệ Khách hàng (Customer Screen Hierarchy Tree)

Sơ đồ ca sử dụng (Use Case Diagram) phân hệ Khách hàng chi tiết hóa các hành động của Khách vãng lai (Guest) và Khách hàng đã đăng nhập (Customer) đối với hệ thống.



Hình 4.0b: Sơ đồ Use Case chi tiết phân hệ Khách hàng

Phần này lần theo vòng đời thao tác của khách từ lúc vào website đến theo dõi sau mua. Mỗi bước chỉ mô tả hành vi được thấy trong mã hoặc giao diện, đồng thời tách riêng nơi cần kiểm tra thêm.



5.1

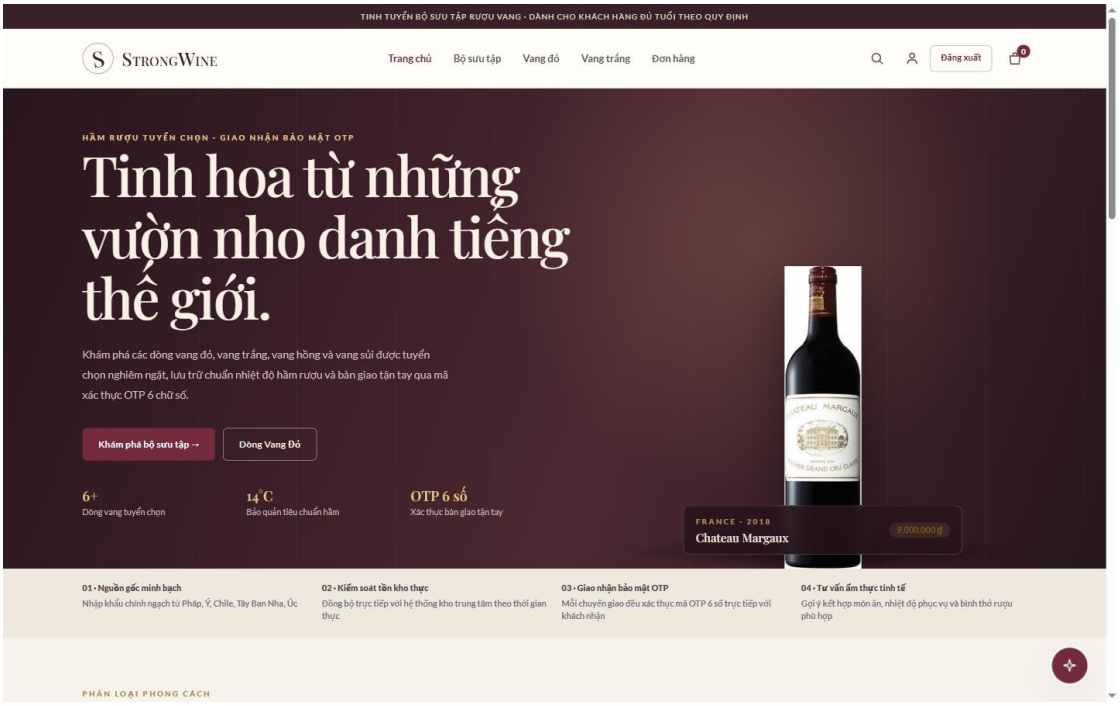
Hình 4.2: Wireframe Danh mục rượu và bộ lọc đa tiêu chí (Grayscale Web Mockup)

HomeController trả trang chủ; WineController phục vụ danh sách và chi tiết. Danh sách nhận bộ lọc rồi tạo Specification cho tên/từ khóa, loại, khoảng giá, quốc gia, niên

vụ, danh mục và trạng thái xóa. Repository thực hiện truy vấn theo điều kiện. Trang chi tiết hiển thị dữ liệu rượu, giá và thông tin liên quan để khách quyết định có thêm vào giỏ hay không.

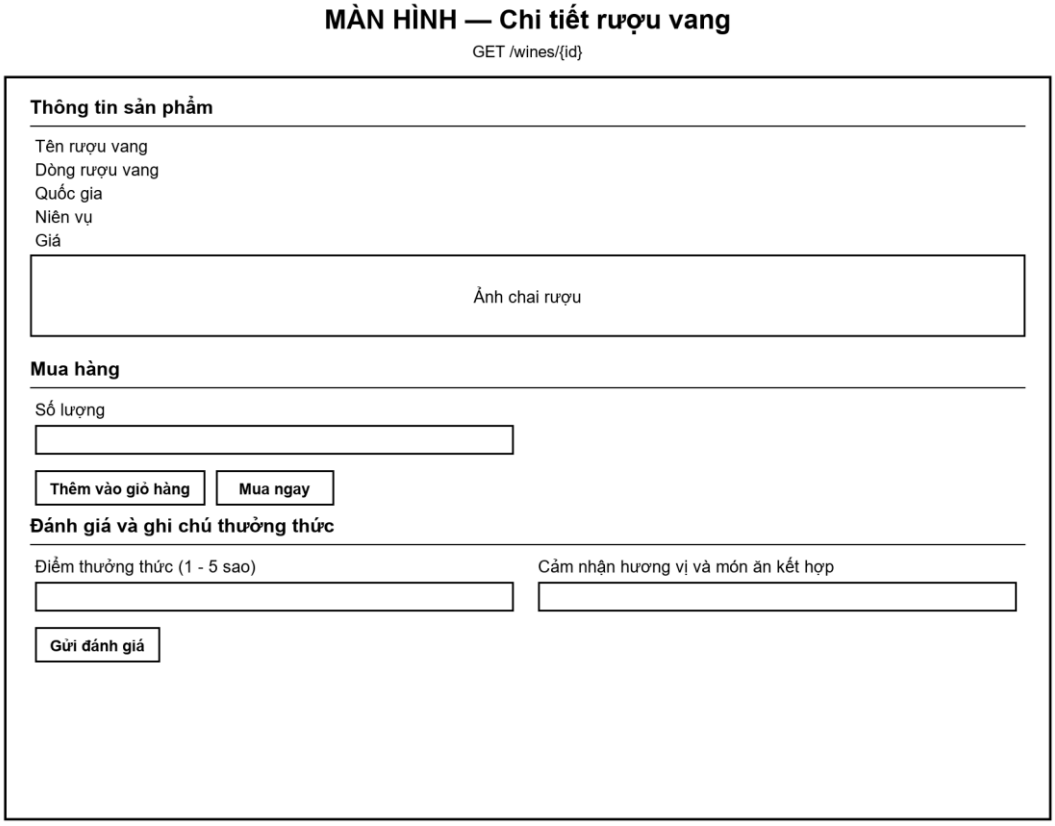
Ở giao diện đã sửa, bộ lọc dùng ngăn kéo có trạng thái mở/đóng và thao tác trên màn hình nhỏ; danh mục có quick view, tìm kiếm và trạng thái rỗng. Đây là mô tả UI đã quan sát chứ không chứng minh response API hoặc truy vấn cơ sở dữ liệu chịu tải. Giá và tồn hiển thị chỉ là giá trị tại thời điểm tải trang; checkout phải xác minh lại ở server.

Ảnh trang chủ được ghi nhận ở viewport desktop 1440 × 900; hình minh họa bố cục website đáp ứng theo chuẩn Responsive Web Design [12], không thay thế kiểm tra nghiệp vụ checkout hoặc xác nhận số liệu sản phẩm trong cơ sở dữ liệu.



Hình 4.2.b: Giao diện trang chủ StrongWine (ảnh giao diện từ bộ tài liệu dự án)

Giao diện chi tiết sản phẩm rượu vang hiển thị đầy đủ thông tin về vùng sản xuất, giống nho, nồng độ cồn và các nốt hương vị đặc trưng (tasting notes). Dưới đây là bản vẽ wireframe mô tả cấu trúc bố cục giao diện trước khi triển khai thực tế:

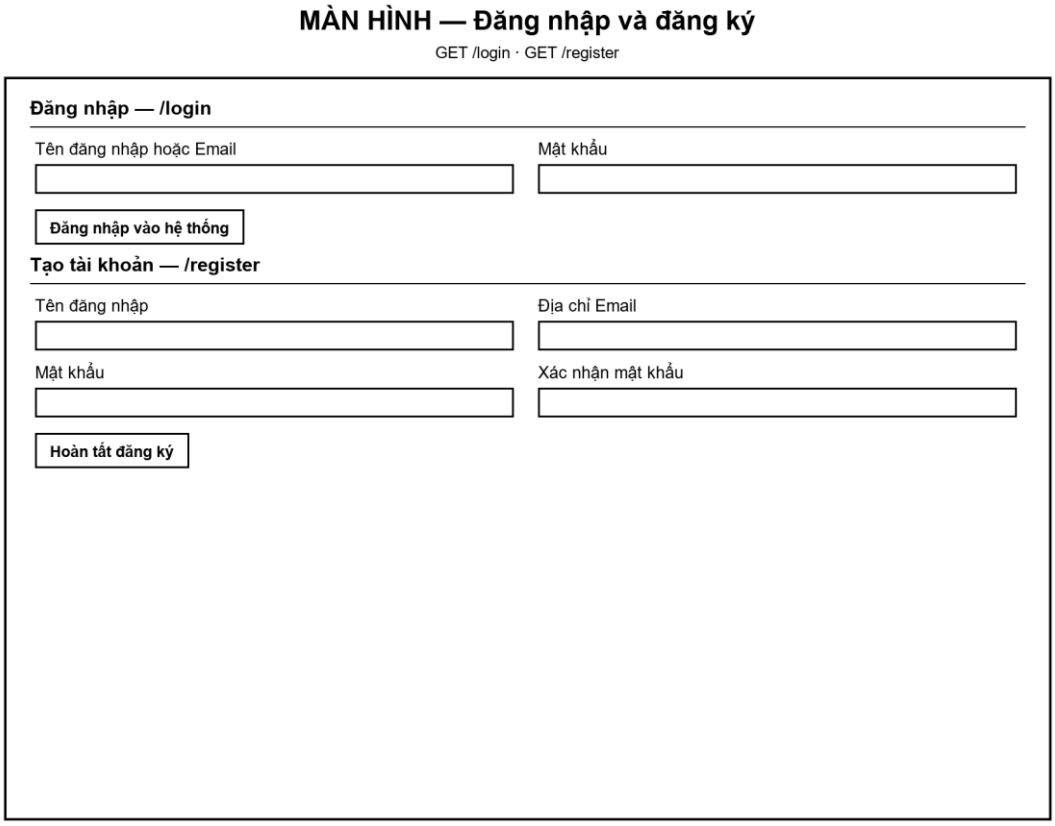


Hình 4.3: Wireframe Chi tiết sản phẩm rượu và tasting notes (Grayscale Web Mockup)

AuthController cung cấp màn đăng nhập và đăng ký, nhận dữ liệu tạo tài khoản và có các route quên/đặt lại mật khẩu. Spring Security xử lý form login, success handler và logout. API có endpoint đăng nhập riêng. Sau khi xác thực, ứng dụng dùng principal để xác định người dùng hiện hành; các truy vấn đơn hàng theo người dùng cần lọc theo userId chứ không chỉ theo ID đơn.

Luồng đăng ký cần xử lý tên đăng nhập/email trùng, kiểm tra dữ liệu, băm mật khẩu và thông báo lỗi không tiết lộ thông tin quá mức. Luồng đặt lại cần token có thời hạn, một lần dùng, gắn đúng tài khoản và thu hồi sau khi đổi mật khẩu. Các điểm này được ghi như yêu cầu kiểm chứng; lần rà này không phát sinh tài khoản mới hay chạy thử chức năng email/reset. [1]

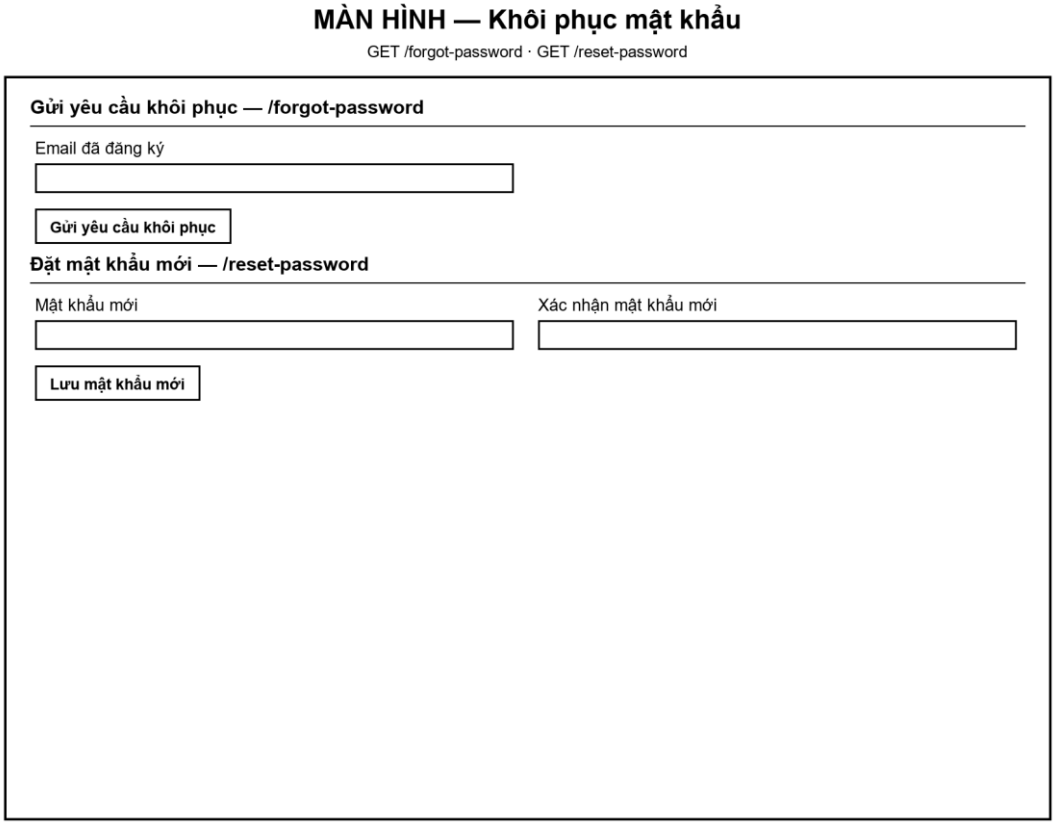
5.2 Giỏ hàng và đồng bộ thông tin sản phẩm



Hình 4.7: Wireframe Đăng nhập và tạo tài khoản thành viên (Grayscale Web Mockup)

5.3 Checkout và tạo đơn chờ thanh toán

Quy trình khôi phục tài khoản người dùng yêu cầu xác thực qua email để tạo mã token bảo mật trong một giao dịch an toàn `@Transactional` [6] có thời hạn. Sơ đồ wireframe dưới đây mô tả giao diện biểu mẫu yêu cầu đặt lại mật khẩu:



Hình 4.8: Wireframe Biểu mẫu yêu cầu đặt lại mật khẩu bảo mật (Grayscale Web Mockup)

Sau khi kiểm tra thông tin nhận, CartService chuẩn hóa và xác nhận mặt hàng. OrderService tạo Order ở trạng thái đơn PENDING, trạng thái thanh toán PENDING, gắn User và snapshot địa chỉ/liên hệ; tạo các OrderItem chứa giá hiện hành; tính lại tổng và lưu đơn. Sau khi có ID đơn, dịch vụ yêu cầu InventoryService giữ số lượng. Nếu bất kỳ điều kiện hàng hoặc dữ liệu nào sai, request phải dừng và trả lỗi rõ để người mua có thể điều chỉnh.

Một bất biến cần kiểm tra bằng test tích hợp là việc tạo order và giữ hàng phải cùng đạt hoặc được bù lại nếu giữ không đủ. Kiểm thử cạnh tranh nên tạo nhiều yêu cầu cùng mua lượng lớn hơn tồn, xác nhận không có tổng lượng giữ vượt khả dụng và không có đơn ở trạng thái hợp lệ mà thiếu reservation. Checkout token có thể giảm gửi lặp trong session; nó không thay thế idempotency xuyên nhiều tab, thiết bị hoặc retry mạng. [1]

5.4 Tạo phiên thanh toán, callback và webhook

Sau khi order được tạo, PaymentService được gọi với đơn và phương thức thông qua giao diện API chuẩn hóa RESTful [11]. Phần tạo phiên checkout đang hoạt động hỗ

trợ Stripe; enum còn có PayPal, VNPay, MoMo và COD nhưng method từ chối lựa chọn khác Stripe. Vì vậy giao diện liệt kê phương thức hoặc enum trong entity chưa chứng minh rằng connector đã được cài và có thể dùng.

Stripe trả URL để trình duyệt điều hướng ra ngoài. Khi khách quay lại success/cancel, PaymentController lấy trạng thái tương ứng và xác định đơn; webhook là đường server-to-server, cần kiểm tra chữ ký và mã sự kiện để tránh giả mạo hoặc xử lý trùng. Callback thành công cần đối chiếu order, reference, amount và trạng thái giao dịch theo dữ liệu lưu, sau đó cập nhật Payment, PaymentTransaction và Order một cách nhất quán.

Khi thanh toán thất bại hoặc bị hủy, đơn vẫn cần chính sách rõ: giữ hàng trong bao lâu, khi nào giải phóng, retry có dùng lại đơn cũ không và webhook đến muộn được xử lý ra sao. Có màn thử thanh toán lại cho đơn đủ điều kiện; kiểm tra trạng thái phải diễn ra ở service trước khi tạo session mới. Không có Stripe request thật được gửi trong phiên kiểm tra này.



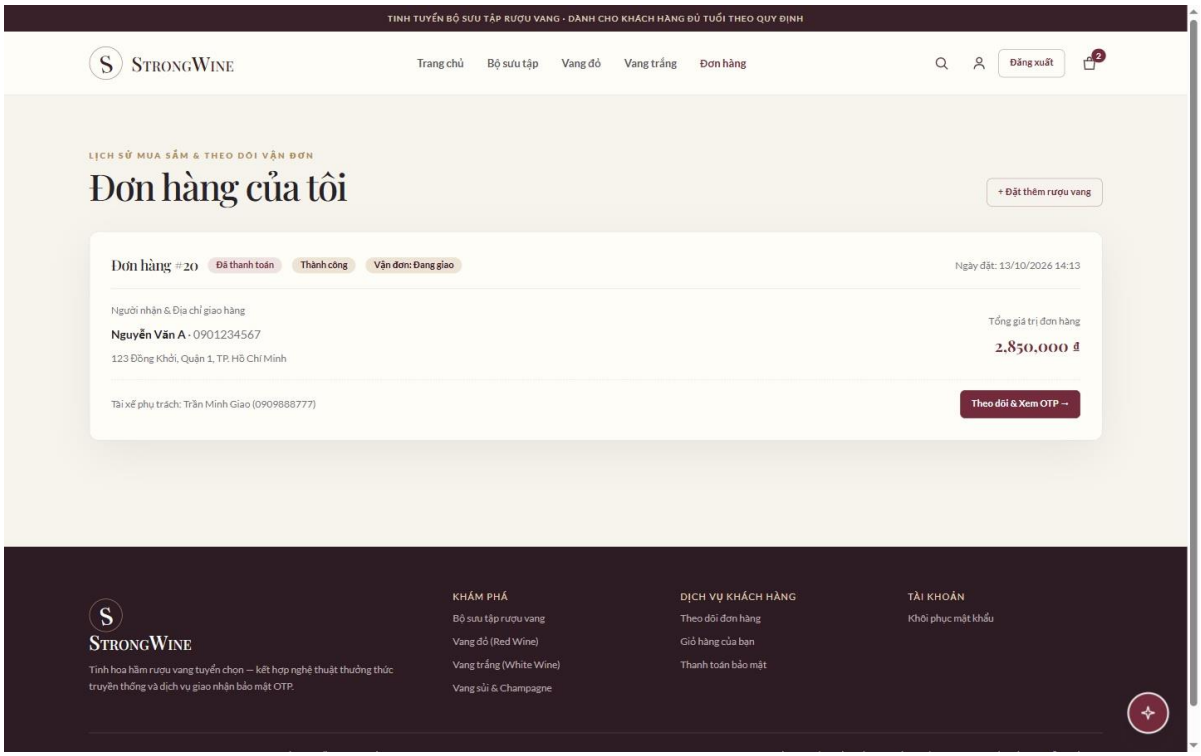
5.5

Hình 4.4: Wireframe Quản lý giỏ hàng và cập nhật số lượng (Grayscale Web Mockup)

Nếu có shipper phù hợp, dịch vụ gán ngay; nếu chưa có, shipment có thể ở trạng thái chờ phân công và hàng đợi dispatch xử lý khi shipper sẵn sàng. Hồ sơ shipper được cập nhật có thể kích hoạt việc làm mới khả dụng và phân phối lại queue. Các quy tắc sức chứa, trạng thái đang bận và thứ tự chọn người giao cần được kiểm thử ở tình huống nhiều order được trả tiền đồng thời.

Vì listener chạy sau khi giao dịch hoàn tất, lỗi tạo vận đơn không thể rollback việc Stripe đã thanh toán. Log và backfill giúp phát hiện/bù sự cố nhưng không đủ để khẳng định giao hàng chắc chắn được tạo. Cần một quy trình vận hành nhìn ra order đã trả nhưng thiếu shipment, cho phép retry idempotent và ghi lại kết quả từng lần.

5.6 Lịch sử, chi tiết đơn và đánh giá



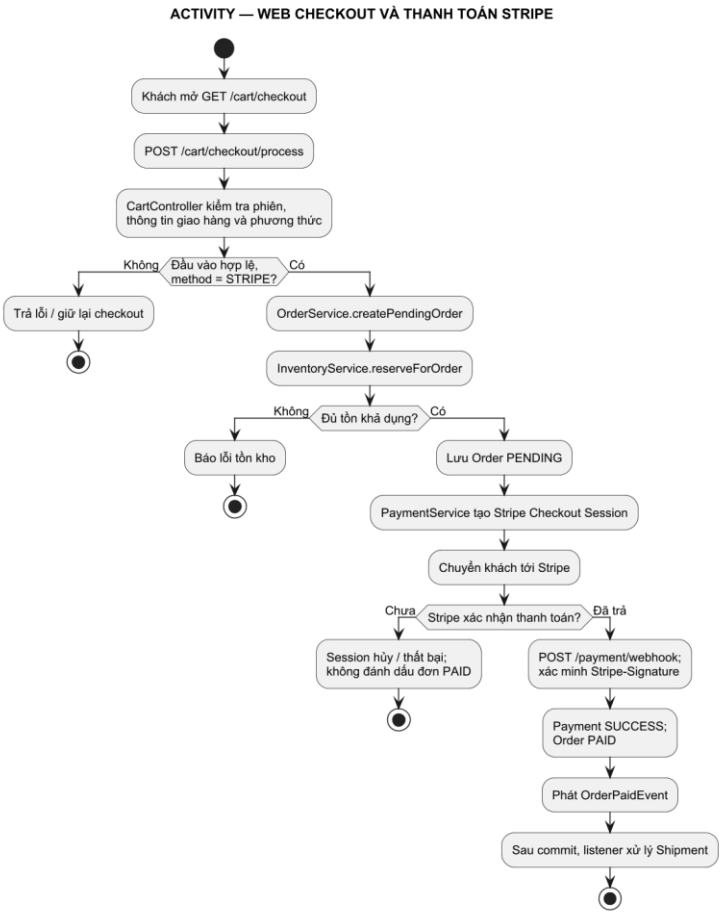
Hình 4.1: Lịch sử đơn hàng

Trang /orders tải các đơn theo userId hiện tại và ghép shipment theo order ID. Trang chi tiết chỉ tải đơn thuộc người dùng; admin có nhánh xem tất cả. Trang xác nhận cũng yêu cầu xác thực và kiểm tra lại quyền sở hữu. Cách lọc theo cả orderId lẫn userId tránh việc đoán ID của người khác để xem địa chỉ hoặc chi tiết giao hàng.

ReviewController và Review API cung cấp đường xem/tạo/chỉnh sửa/xóa đánh giá. Quy tắc nghiệp vụ nên xác định người gửi có mua đúng sản phẩm, đơn đã hoàn tất,

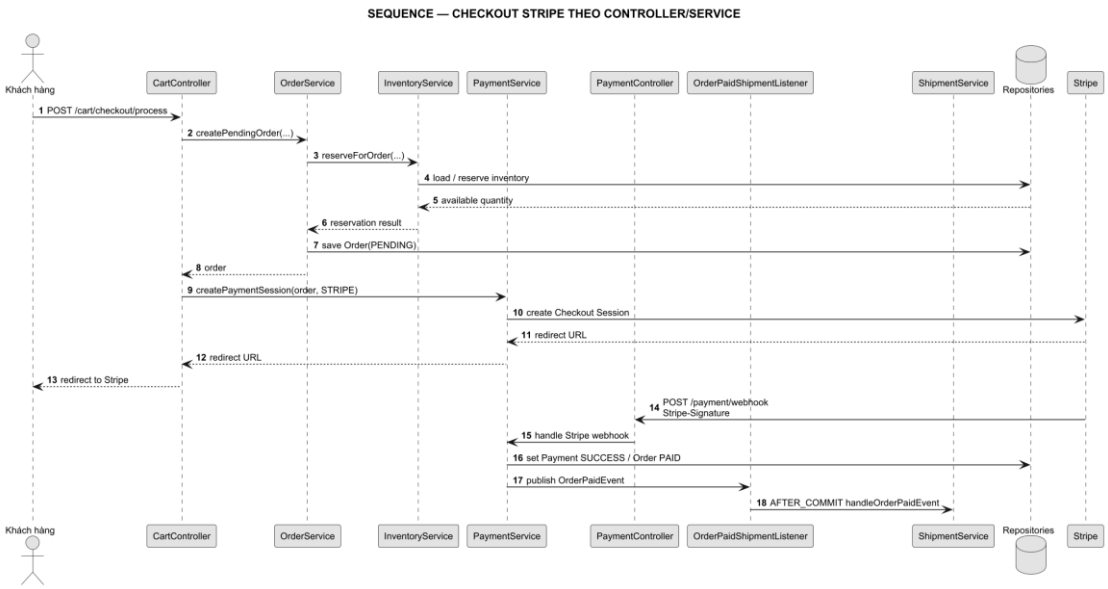
một đơn được đánh giá một lần hay không, và quyền sửa/xóa thuộc người viết hay admin. Việc có route tạo đánh giá không tự chứng minh các điều kiện đó đã được bao phủ; cần kiểm tra service và test quyền truy cập trước khi coi luồng này hoàn tất.

Sơ đồ hoạt động checkout bám theo POST /cart/checkout/process: CartController kiểm tra phiên và dữ liệu, OrderService tạo đơn PENDING và giữ tồn, PaymentService mở Stripe Checkout. Webhook Stripe hợp lệ cập nhật Payment SUCCESS/Order PAID; listener xử lý shipment sau commit.



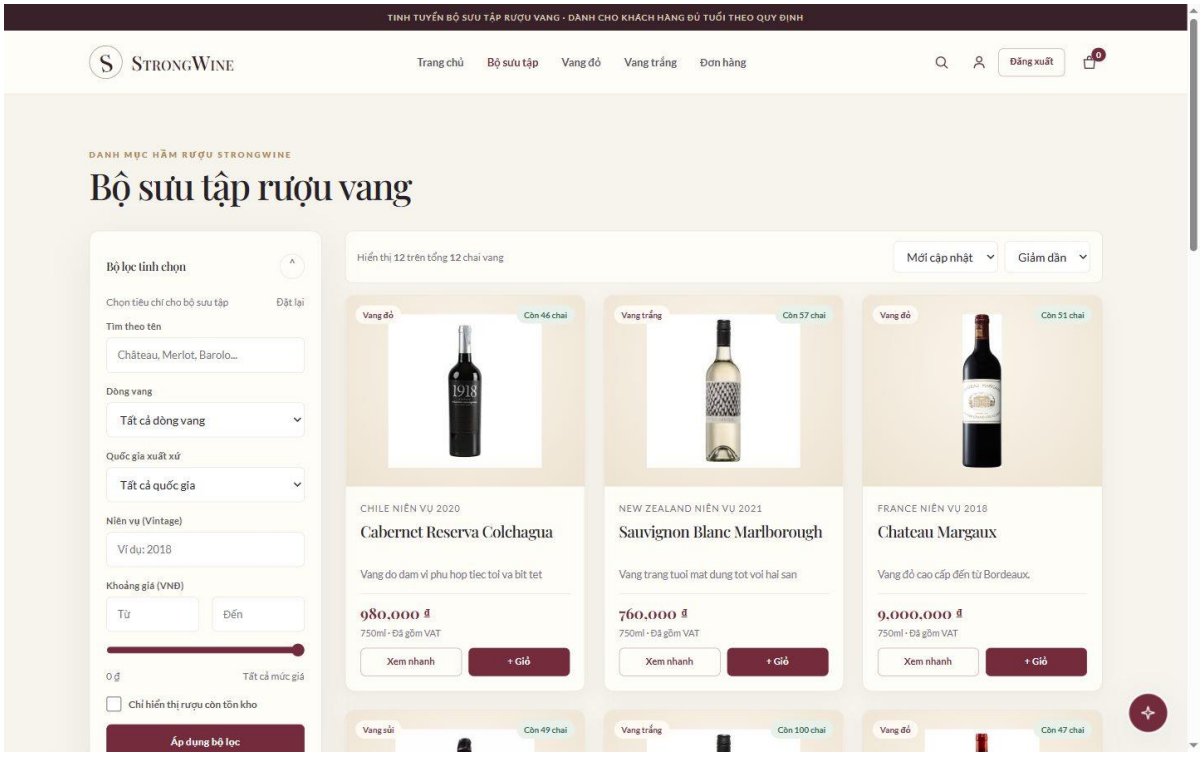
Hình 4.4: Sơ đồ hoạt động (Activity Diagram) quy trình Checkout và thanh toán trực tuyến

Sơ đồ tuần tự thể hiện POST /cart/checkout/process qua CartController, OrderService, InventoryService, PaymentService và Stripe; POST /payment/webhook xử lý xác nhận. Khi OrderPaidEvent được phát, OrderPaidShipmentListener gọi xử lý shipment ở pha AFTER_COMMIT.



Hình 4.5: Sơ đồ tuần tự (Sequence Diagram) xử lý Đặt hàng giữa các lớp ứng dụng

5.7 Duyệt danh mục và tìm kiếm rượu



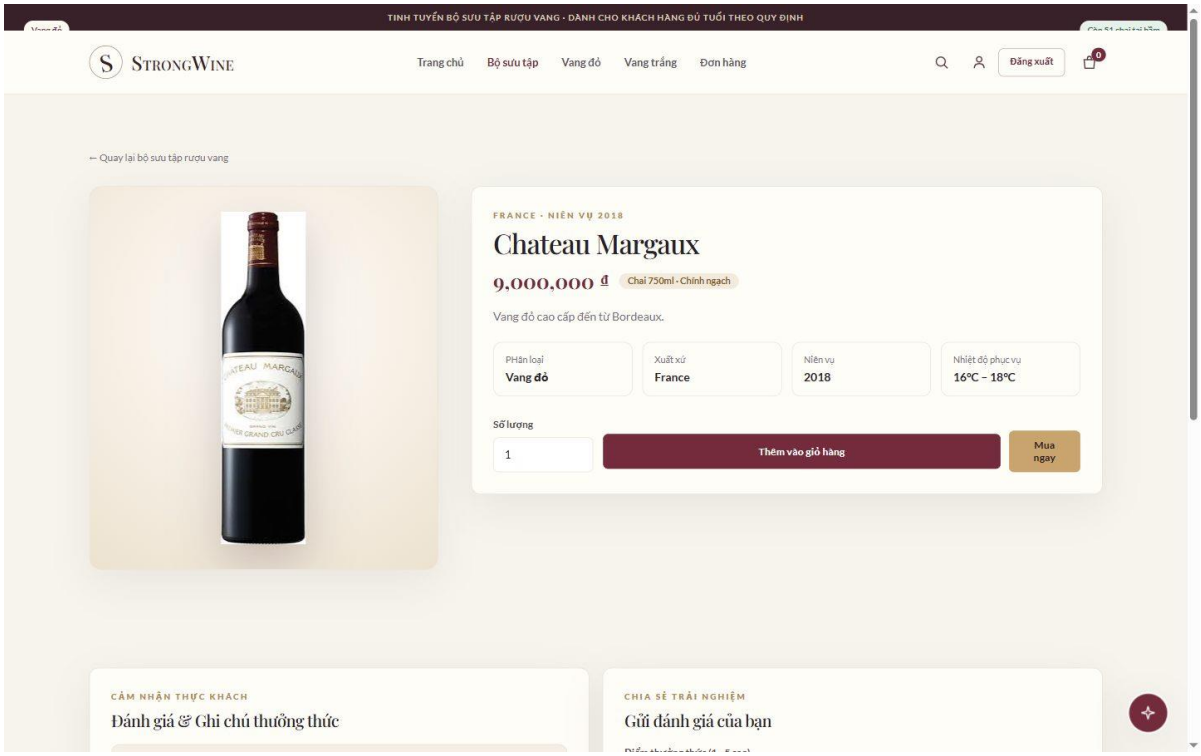
Hình 4.2: Danh mục rượu và bộ lọc

Luồng catalog bắt đầu từ HomeController/WineController hoặc API wine tương ứng, vận dụng vị từ Specification của Spring Data [7] để lọc sản phẩm, nạp category/sản phẩm và áp tiêu chí lọc. Query nên giới hạn trường trả về, hỗ trợ pagination/sort và chuẩn hóa input trước khi chuyển thành Specification. Từ khóa cần escape wildcard đúng theo dialect; country/category/price/year cần phân biệt absent với giá trị 0. Cách ghép predicate phải có test độc lập để tránh filter này vô tình bỏ qua filter khác.

Trang danh mục nên cho người dùng nhìn thấy trạng thái không có kết quả, filter đang áp dụng, cách xóa nhanh filter và giữ trạng thái khi quay lại. Nếu danh sách có sản phẩm bị deleted/inactive, query public cần loại đúng. Ảnh sản phẩm cần placeholder khi URL hỏng; nội dung mô tả phải escape. SEO metadata và canonical URL là đề xuất nếu dự án muốn tăng khả năng tìm kiếm, chưa được kiểm chứng trong phạm vi này.

Kịch bản QA: tải mặc định; filter theo category; ghép nhiều điều kiện; từ khóa có dấu; ký tự wildcard; trang cuối; sort giá tăng/giảm; product inactive; không có ảnh; mobile drawer và query refresh. Hiện trạng UI đã được chụp, nhưng kết quả dataset/logic filter cần test riêng.

5.8 Xem chi tiết và chọn số lượng



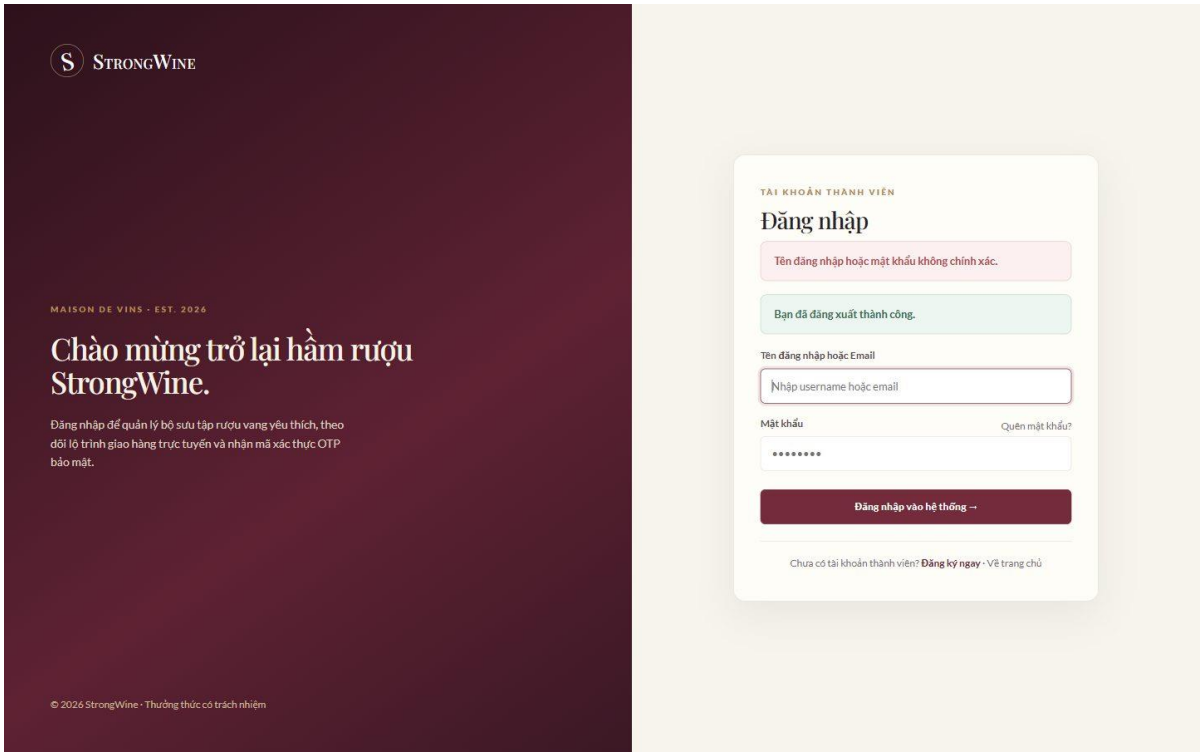
Hình 4.3: Chi tiết sản phẩm rượu

WineController có route chi tiết theo ID; controller nên trả 404 với wine không tồn tại/deleted và không render entity graph không cần thiết. Màn hình cần hiển thị tên, năm, country, category, mô tả, giá và trạng thái hàng; nếu quantity là available tổng hợp, cần quy tắc cộng theo warehouse. Cấu trúc nội dung nên tránh tuyên bố nguồn gốc/xuất xứ chưa có căn cứ trong database.

Nút thêm giỏ cần phản hồi rõ khi quantity invalid hoặc sản phẩm vừa hết. Client có thể kiểm tra nhanh để UX tốt, nhưng server đọc lại giá và available stock. Việc gửi

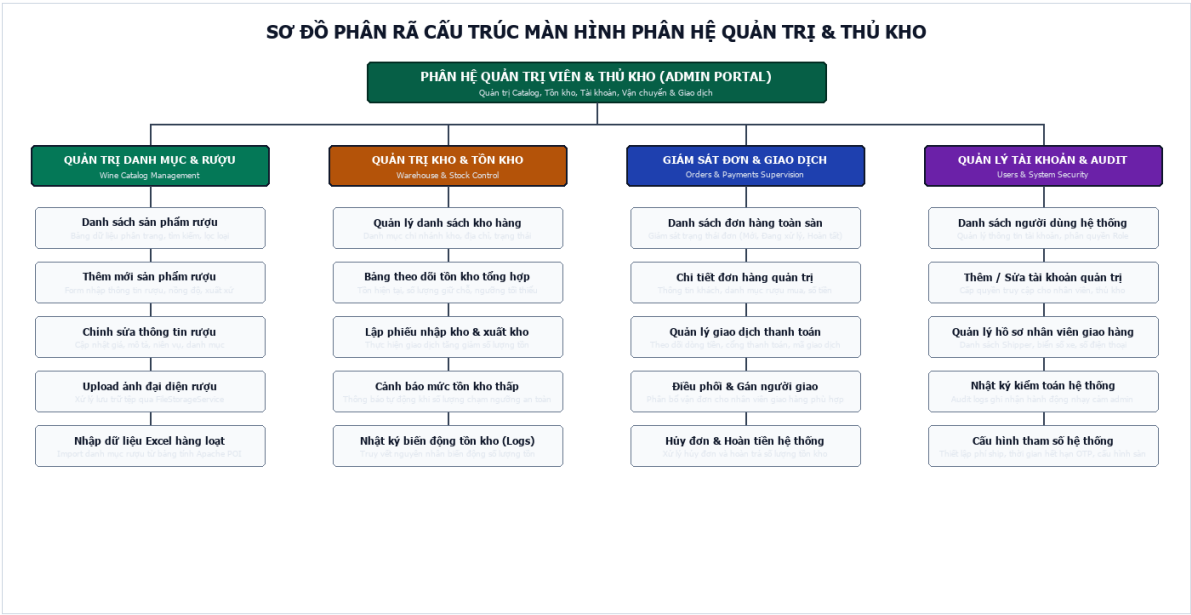
ID sản phẩm không được phép lộ dữ liệu nhạy cảm hoặc bypass active flag. Khi nhiều kho, ứng dụng có thể chưa xác nhận kho giao cuối cùng trước checkout; không hiển thị mức tồn chính xác giả nếu service chỉ có tổng số.

5.9 Đăng ký, đăng nhập và đặt lại mật khẩu



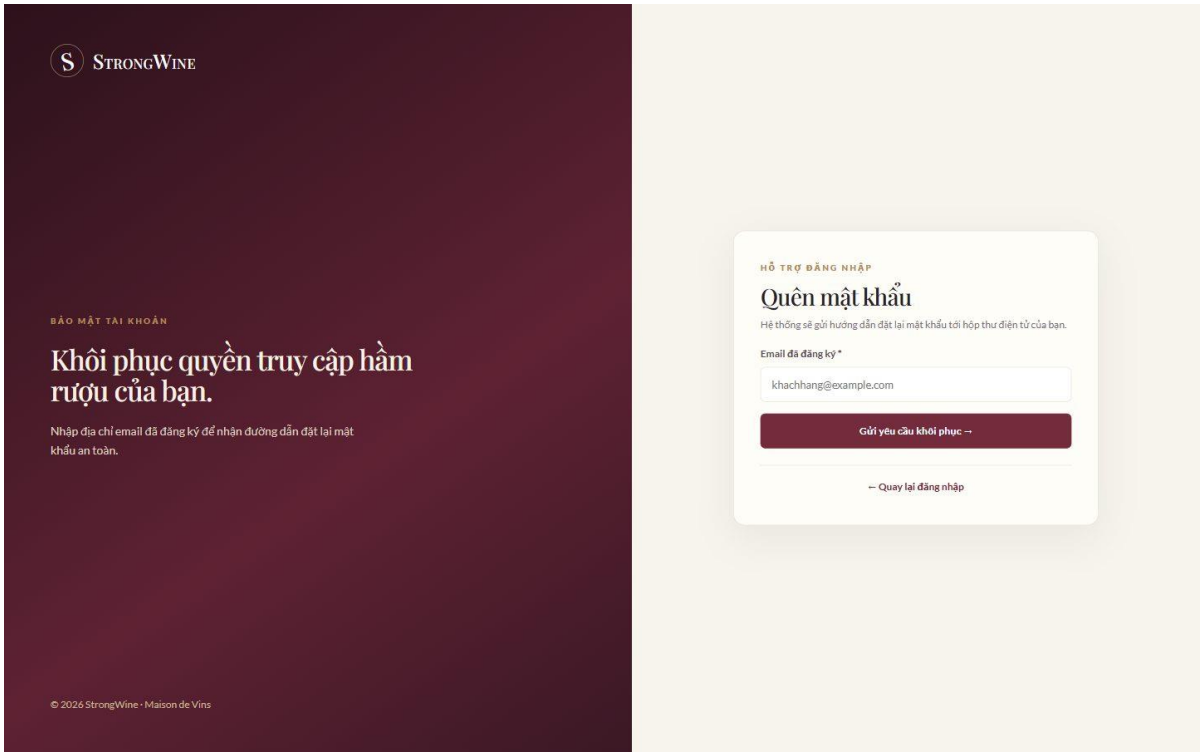
Hình 4.4: Trang đăng nhập

Giao diện danh mục và bộ lọc đa tiêu chí được hiện thực hóa trực quan trên trình duyệt như sau:



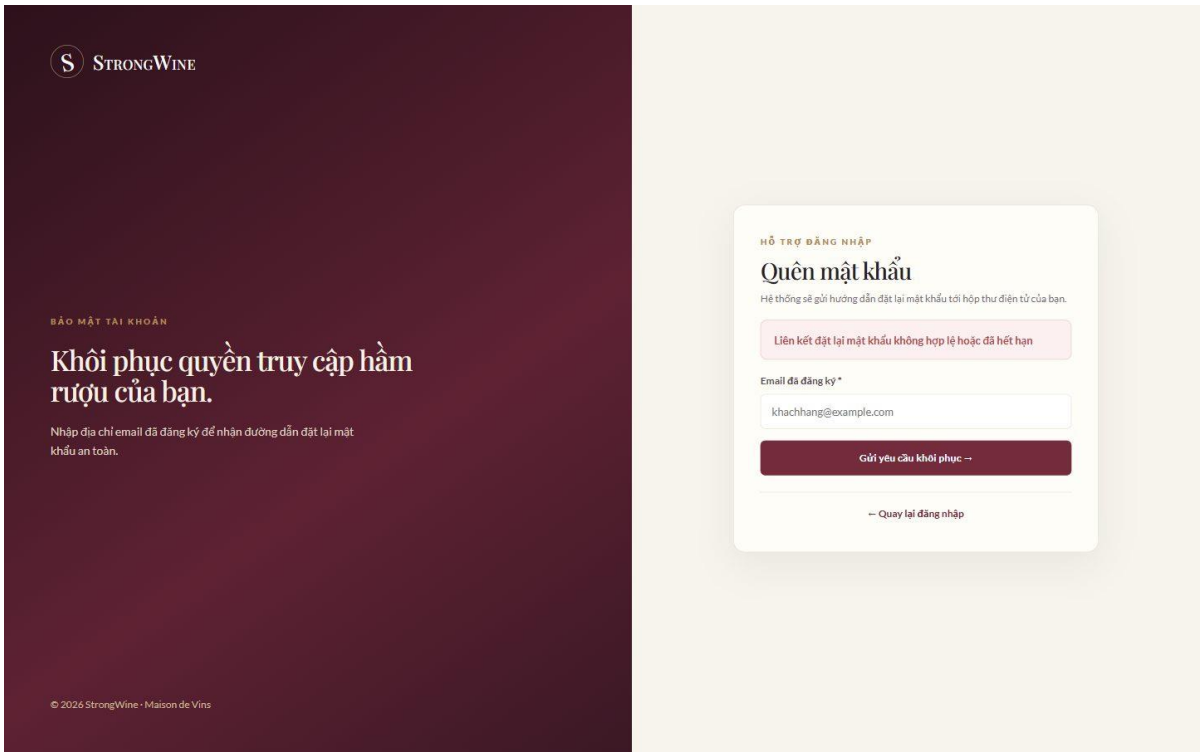
Hình 4.5: Trang đăng ký tài khoản

Sau khi người dùng lựa chọn sản phẩm, giao diện chi tiết hiển thị đầy đủ thông tin nồng độ và ghi chú nếm rượu:



Hình 4.6: Trang yêu cầu đặt lại mật khẩu

Biểu mẫu đăng nhập bảo mật cho phép khách hàng và nhân viên xác thực phiên làm việc vào hệ thống:



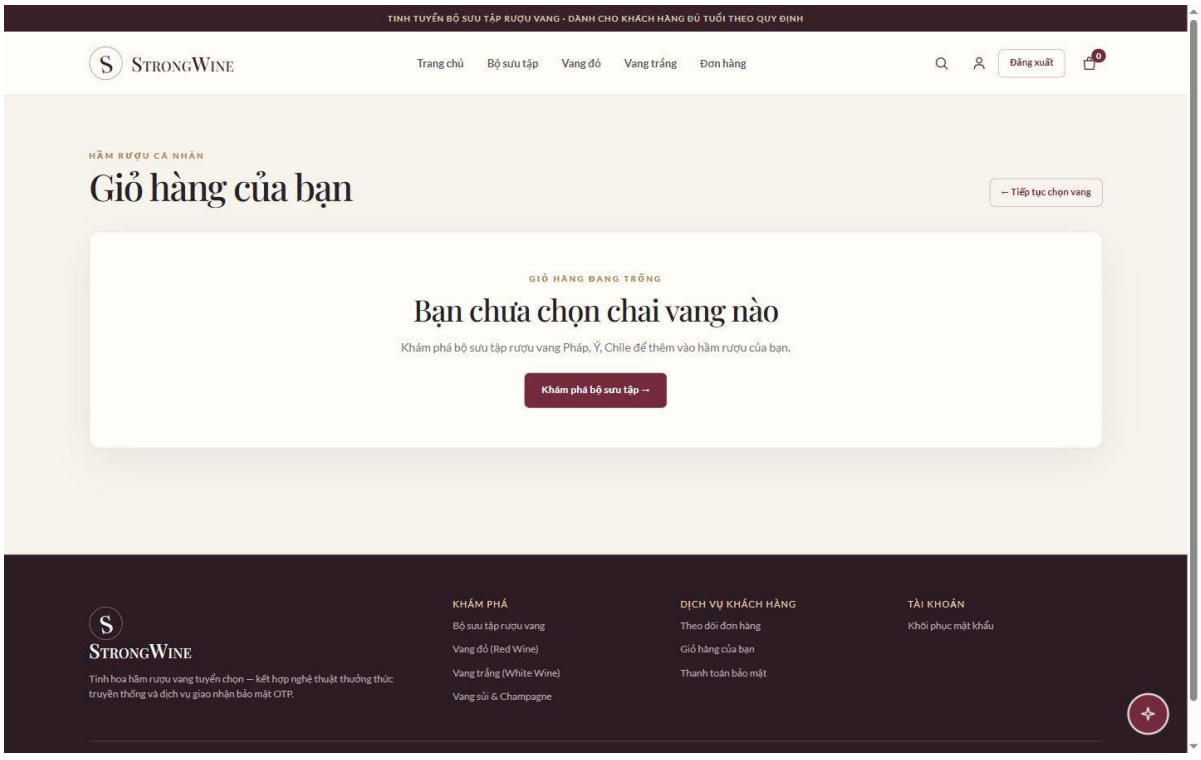
Hình 4.7: Giao diện xử lý liên kết đặt lại không hợp lệ

AuthController cung cấp form login/register và flow forgot/reset; UserService tạo tài khoản với role phù hợp, hash password và kiểm tra định danh trùng. Đăng ký không nên nhận role ADMIN từ form. Login cần generic error, rate limit và lockout policy; form session cần CSRF. Authentication event cần invalidate session/token khi password hoặc role thay đổi.

Password reset dùng token có thời gian hết hạn, ngày dùng và user liên quan. Request quên mật khẩu không nên tiết lộ email có tồn tại; token phải one-time, ngẫu nhiên và không được ghi raw vào log. Base URL lấy từ cấu hình. Mail provider mặc định tắt, vì vậy cài đặt route không đồng nghĩa user đã nhận link.

Kịch bản kiểm thử: user mới; username/email trùng; password policy; role escalation; login disabled; quên mật khẩu với email đã/không tồn tại; token expired/used; request đồng thời; reset password invalid; session còn sống sau đổi mật khẩu. Những ca này cần test service và MVC/security integration.

5.10 Quản lý giỏ hàng



Hình 4.8: Giỏ hàng trống

CartController cung cấp xem giỏ, checkout view và endpoint xử lý checkout JSON; CartService tìm cart theo user. Lệnh add/update/remove cần kiểm tra cart thuộc principal, item thuộc cart, quantity dương và wine còn bán. Với hai tab hoặc retry mạng,

cần xử lý duplicate item bằng unique cart-wine và merge quantity; không để double-submit tạo hai dòng.

Giỏ là ý định mua chứ chưa phải cam kết tồn. Màn hình tổng tiền chỉ là estimate cho đến khi server tính lại. Khi price hoặc availability thay đổi, checkout cần trả danh sách item cần xác nhận thay vì silently đổi tổng. Nếu cart line bị product soft-delete, người dùng cần xóa hoặc thay item; không để checkout crash do quan hệ null.

Kiểm thử quyền gồm user A gửi cart ID của user B, user A gửi cart item ID ngoài cart, API không có principal, cookie/session hết hạn, quantity quá lớn, hai add đồng thời và xóa item không tồn tại. Response code/flash message cần nhất quán giữa UI và API.

5.11 Tạo đơn và reservation tồn

`OrderService.createPendingOrder` lấy user, duyệt map wineId-quantity, đọc wine/giá server và thực hiện giữ chỗ tồn kho có kiểm soát khóa tránh Lost Update theo C.J. Date [10], tạo Order và OrderItem rồi gọi `InventoryService.reserveForOrder`. `Inventory` tách current/reserved, có version và unique constraint theo wine/warehouse; service có đường lấy inventory khóa để bảo vệ checkout. Hành vi này khác việc trừ tồn ngay khi add cart.

Kiểm soát atomicity yêu cầu toàn bộ item được reserve hoặc không item nào. Với nhiều mặt hàng, nếu dòng cuối thiếu tồn thì phải rollback các reservation trước đó. Khi order pending hết hạn hoặc bị cancel, reservation phải release; nếu không, stock ảo sẽ làm sản phẩm hết hàng mãi. Cần xem có timeout job cho pending order không và cấu hình TTL nào áp dụng, không tự đặt một con số chưa có trong code.

Race test chạy hai transaction cùng mua lượng tồn cuối: tối đa một đơn được chốt; version conflict/deadlock được xử lý retry có giới hạn; available không âm; reservation khớp order items. Sau payment success, transaction confirm phải giảm current và reserved theo đúng một lần. Đây là vùng P0 vì lỗi làm mất tiền hoặc bán vượt.

5.12 Khởi tạo payment session

Controller nhận payment method và order ID, kiểm tra order thuộc user và còn trạng thái có thể trả tiền, rồi gọi PaymentService. Payment ban đầu được lưu với amount/order/method; StripeService chuyển amount decimal sang đơn vị cổng phù hợp

và tạo Checkout Session bằng secret cấu hình. URL success/cancel cần được xây từ base URL đáng tin, không nhận redirect tùy ý từ client.

Trước khi gọi provider, application cần idempotency key gắn với payment attempt để retry không mở nhiều phiên không chủ ý. Sau khi session tạo xong, session id ghi vào Payment; nếu DB persist thất bại, cần có cách tra session thất lạc. Secret/public/webhook key là cấu hình riêng; không ghi vào request logs, exception hoặc report. Nếu key thiếu, flow cần fail rõ, không giả tạo success.

Kiểm thử sandbox gồm currency minor unit, giá trị decimal, VND zero-decimal handling, amount mismatch, invalid method, session creation timeout, redirect URL tamper, payment retry và configuration missing. Không có Stripe key/live call nào được dùng trong bài này; không khẳng định checkout qua cổng đã chạy.

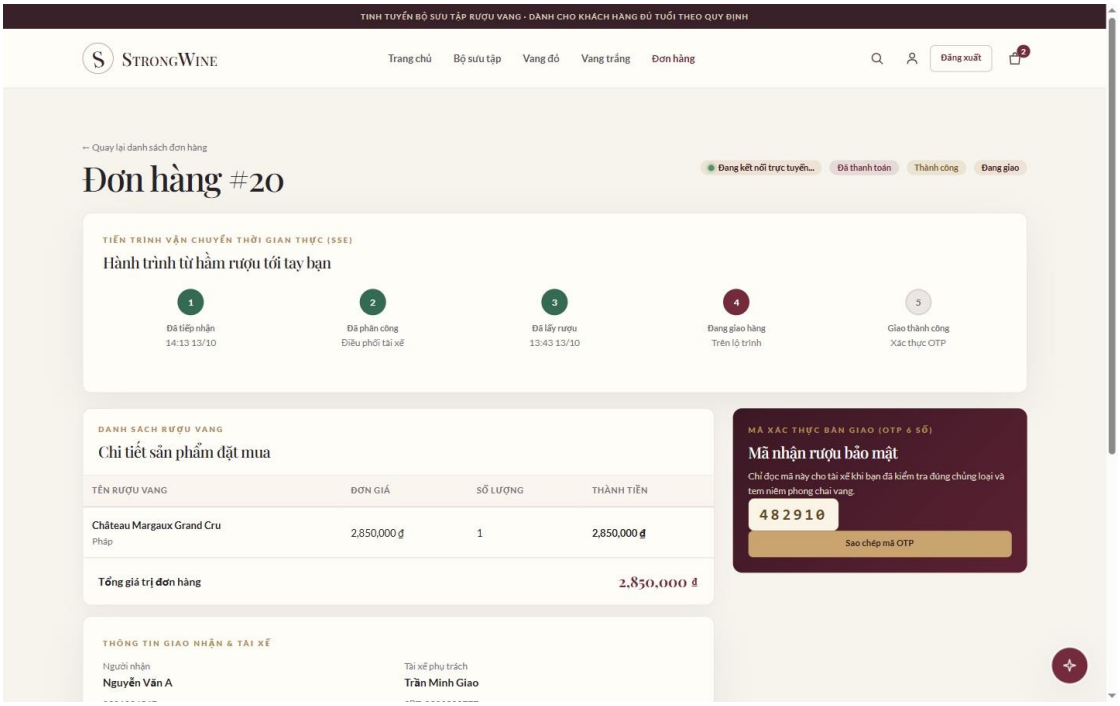
5.13 Xử lý callback, webhook và trạng thái thanh toán

Success redirect kiểm tra session với user context, còn webhook nhận raw payload và Stripe-Signature. PaymentService xác thực event, tìm payment/session và kiểm tra amount/order; rồi finalize payment, update order, inventory và publish OrderPaidEvent. Cả redirect và webhook có thể cùng đến, do đó hàm finalize cần idempotent và trạng thái terminal phải được bảo vệ.

Webhook phải trả HTTP phù hợp: invalid signature bị từ chối; event hợp lệ đã xử lý có thể trả success để tránh retry vô ích; transient DB failure có thể trả retryable error nếu tính idempotent. Chỉ nhận event type cần thiết; payload unsupported được ghi nhận không làm đổi order. PaymentTransaction nên ghi event id/type/status và dữ liệu đã redact để audit.

Reconciliation là lớp bổ sung: so sánh order/payment/session provider và shipment. Nó không nên cập nhật tất cả trạng thái tự động mà không có quy tắc và nhật ký. Môi trường local trước đây có chạy app và DB; không phát giao dịch Stripe hoặc webhook thật. Các nhánh này được đề xuất cho integration test/sandbox riêng.

5.14 Xem lịch sử đơn và quyền sở hữu



Hình 4.9.b: Giao diện chi tiết đơn hàng và theo dõi giao nhận (ảnh giao diện từ bộ tài liệu dự án)

Chi tiết đơn là điểm dễ gặp IDOR. Thay vì `findById` rồi chỉ dựa vào UI, service nên có `findByIdForUser` hoặc kiểm tra owner trước khi trả data. API admin có role riêng; user không thể sửa order state bằng PUT generic chỉ vì biết ID. Tính toàn vẹn audit yêu cầu update after paid chỉ qua nghiệp vụ refund/cancel được quy định.

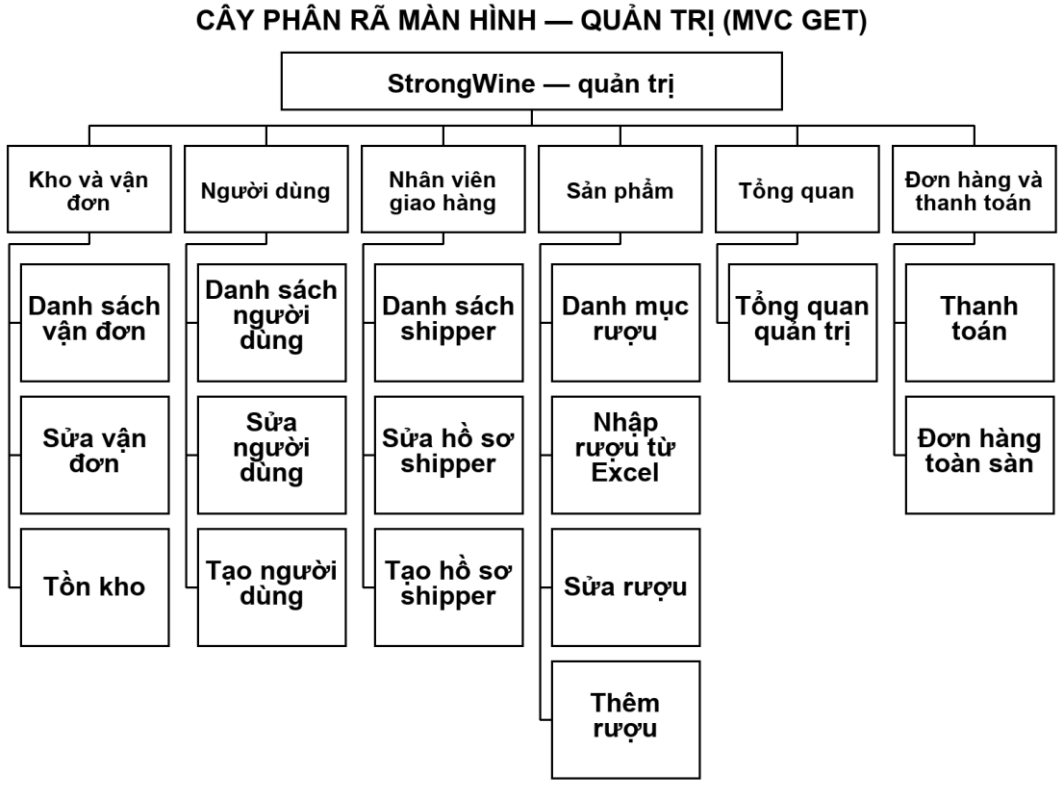
Test: user xem đơn mình; user khác dùng ID hợp lệ; không login; order chưa tồn tại; pagination; đơn có sản phẩm đã đổi tên/giá; shipment chưa tạo; payment pending; paid; cancelled. Các trường hợp đều cần response không tiết lộ thông tin người khác.

5.15 Tạo và kiểm soát review

ReviewController MVC POST `/reviews/create`; ReviewApiController cho API theo wine/user route. Vì API có thể nhận user ID, service phải xác nhận với authentication principal. Content cần validation length/blank và escape output. Chính sách "đã mua mới được đánh giá", review một lần hay nhiều lần phải được xác định từ code/business rubric; không bê nguyên nhánh rating trong hình cũ.

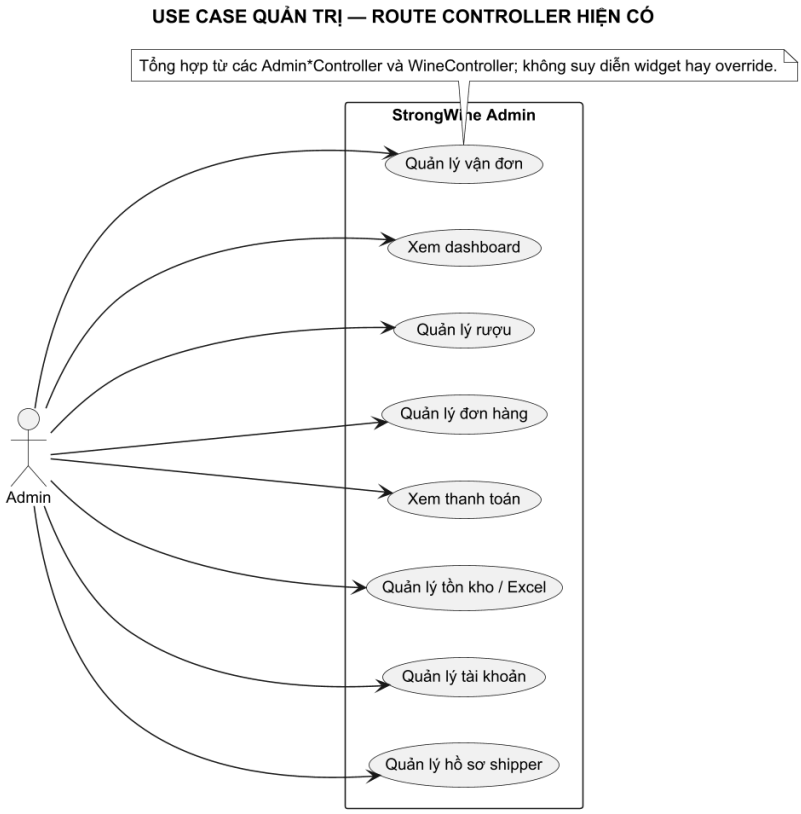
CHƯƠNG 5: LUỒNG NGHIỆP VỤ QUẢN TRỊ VÀ KHO

Sơ đồ cây kế tiếp tổng hợp các khu vực và route quản trị hiện có; route tạo rượu đã đối chiếu với WineController.



Hình 5.0: Sơ đồ phân rã màn hình phân hệ Quản trị và Thủ kho (Admin & Warehouse Screen Hierarchy Tree)

Sơ đồ use case nhóm các khu vực controller quản trị hiện có: dashboard, rượu, đơn hàng, thanh toán, tồn kho/Excel, tài khoản người dùng, hồ sơ shipper và vận đơn. Từng quyền thao tác cần được xác nhận tại endpoint tương ứng.

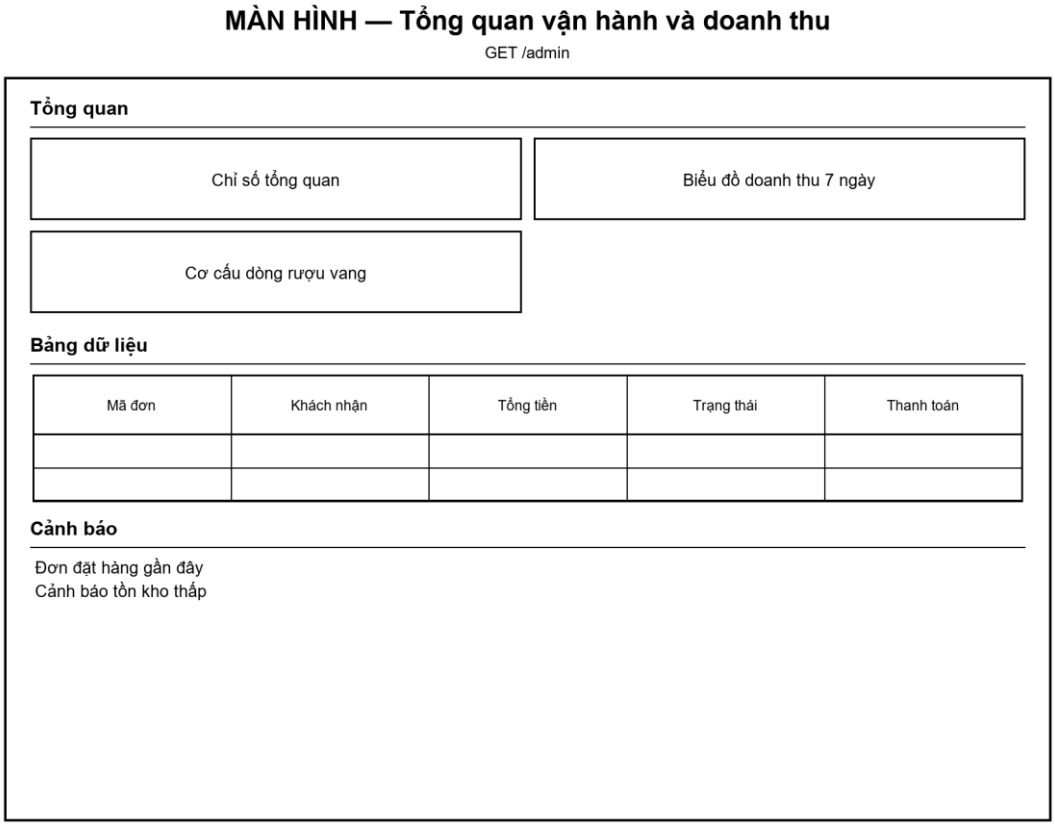


Hình 5.0b: Sơ đồ Use Case chi tiết phân hệ Quản trị và Thủ kho

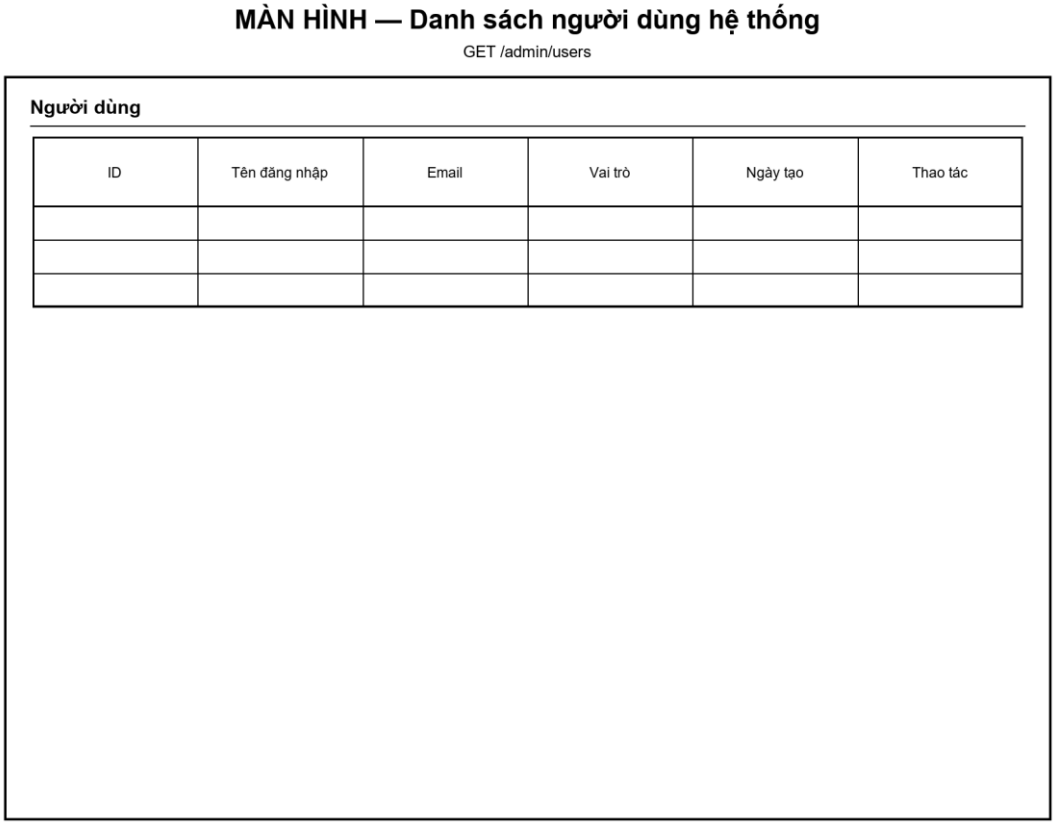
Màn quản trị tập hợp việc quan sát và cập nhật các đối tượng vận hành. Các thay đổi có thể ảnh hưởng dữ liệu khách đang dùng, nên mỗi action cần quyền phù hợp, validation, lịch sử và phản hồi lỗi có thể xử lý.

5.16 Dashboard và quản lý danh mục, sản phẩm

Bảng điều khiển quản trị (Admin Dashboard) đóng vai trò trung tâm trong phân hệ quản trị StrongWine [1] trong việc tổng hợp các chỉ số kinh doanh quan trọng, biểu đồ doanh thu và cảnh báo tồn kho. Bản vẽ wireframe chi tiết của bảng điều khiển được minh họa dưới đây:



Hình 5.3: Wireframe Bảng điều khiển quản trị Dashboard KPI (Grayscale Web Mockup)



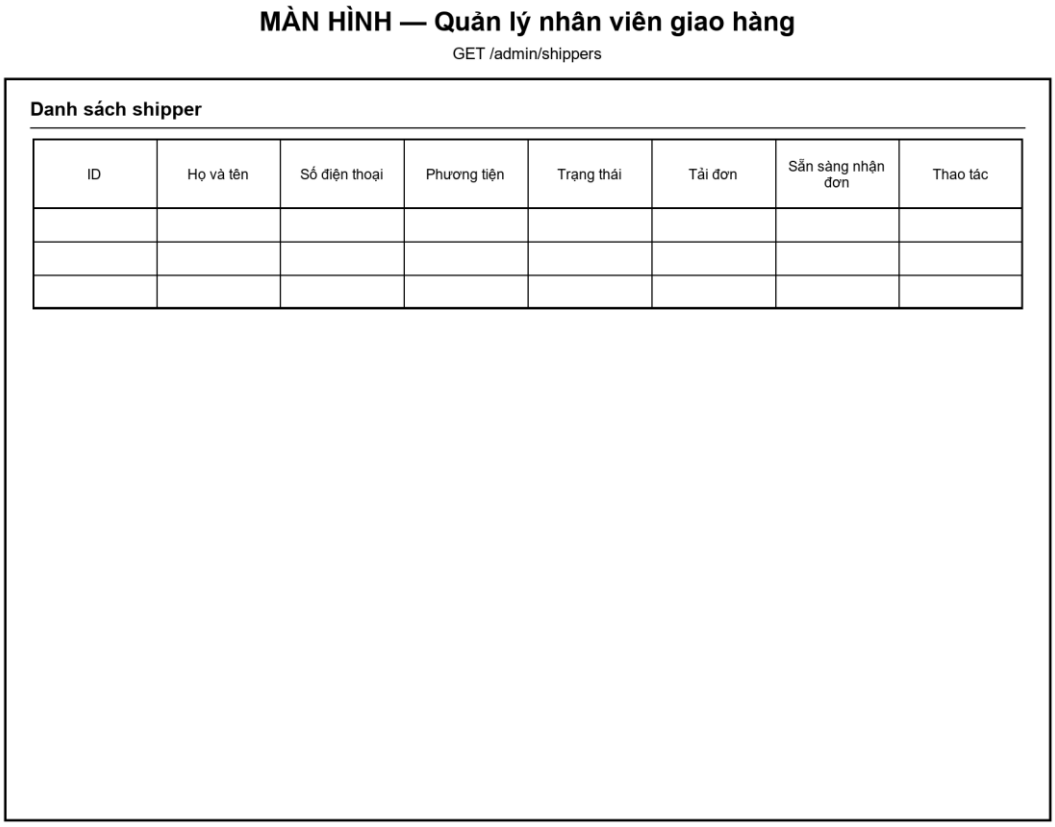
Hình 5.7: Wireframe Quản lý tài khoản người dùng và phân quyền (Grayscale Web Mockup)

5.17 Quản lý người dùng, shipper và giao dịch

AdminUserController và AdminShipperController được bảo vệ chặt chẽ bằng cơ chế phân quyền vai trò RBAC [8], [9] có danh sách, tạo, sửa, xóa/khóa hồ sơ theo route. User role xác định quyền tổng quát; Shipper bổ sung trạng thái sẵn sàng, vị trí, công suất nhiệm vụ đồng thời và số nhiệm vụ đang hoạt động. Khi vô hiệu hóa hoặc sửa hồ sơ shipper, hệ thống cần cập nhật khả năng phân công và xử lý shipment chưa hoàn tất theo quy tắc minh bạch.

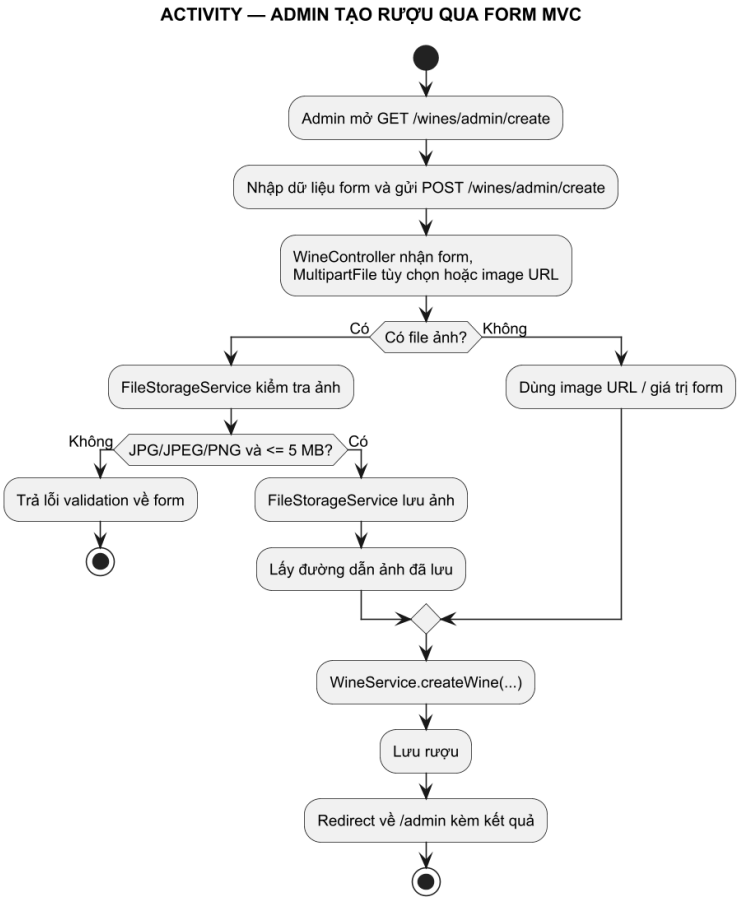
AdminPaymentController hỗ trợ xem giao dịch; AdminOrderController và AdminShipmentController hỗ trợ theo dõi đơn/vận đơn, lọc, sửa thông tin hoặc trạng thái, gửi lại OTP và một số thao tác quản trị. Hành động cưỡng chế trạng thái đơn hàng có thể phá vỡ chuỗi chuyển đổi tự nhiên, đòi hỏi hệ thống phải lưu vết chi tiết danh tính người thực hiện, mốc thời gian, lý do nghiệp vụ cùng trạng thái trước và sau khi can thiệp. Màn ảnh admin trong tài liệu được cắt trước danh sách khách hàng để không đưa dữ liệu cá nhân vào báo cáo.

5.18 Sơ đồ quy trình thêm sản phẩm từ tài liệu cũ



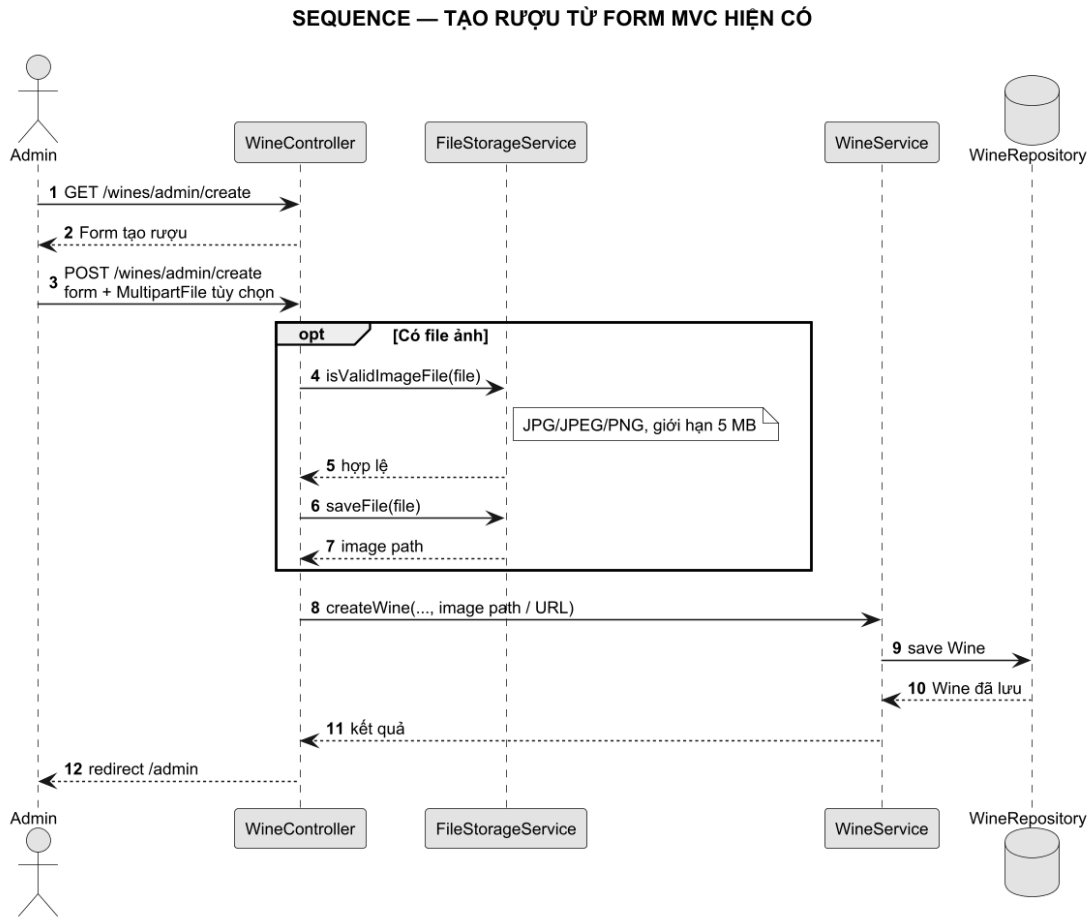
Hình 5.8: Wireframe Quản lý danh sách nhân viên giao hàng Shipper (Grayscale Web Mockup)

Form MVC tạo rượu kiểm tra dữ liệu, nhận file ảnh tùy chọn hoặc URL, lưu file hợp lệ qua FileStorageService và gọi WineService. Luồng này không tự khởi tạo tồn kho trong phương thức WineController đang khảo sát.

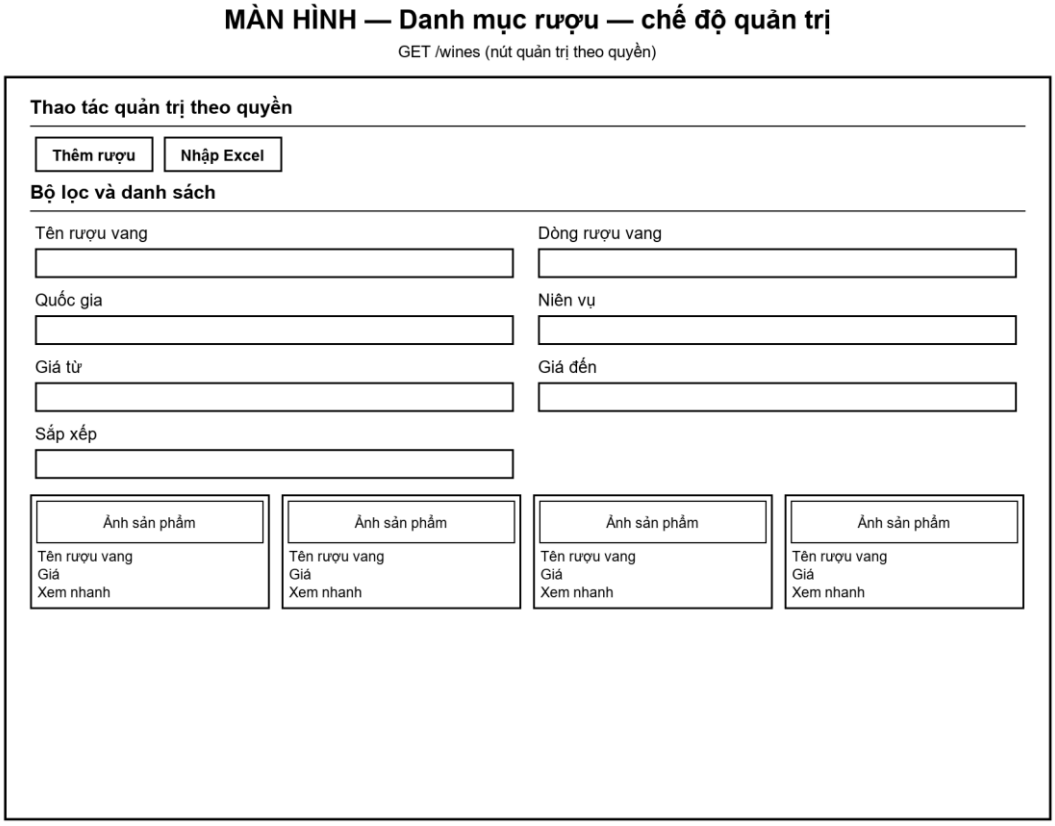


Hình 5.1: Sơ đồ hoạt động (Activity Diagram) Quản trị viên thêm rượu và upload ảnh

Sơ đồ tuần tự đã đối chiếu với form MVC GET/POST /wines/admin/create. API /api/admin/products là giao diện JSON riêng, không nhận MultipartFile trong luồng tạo rượu này.

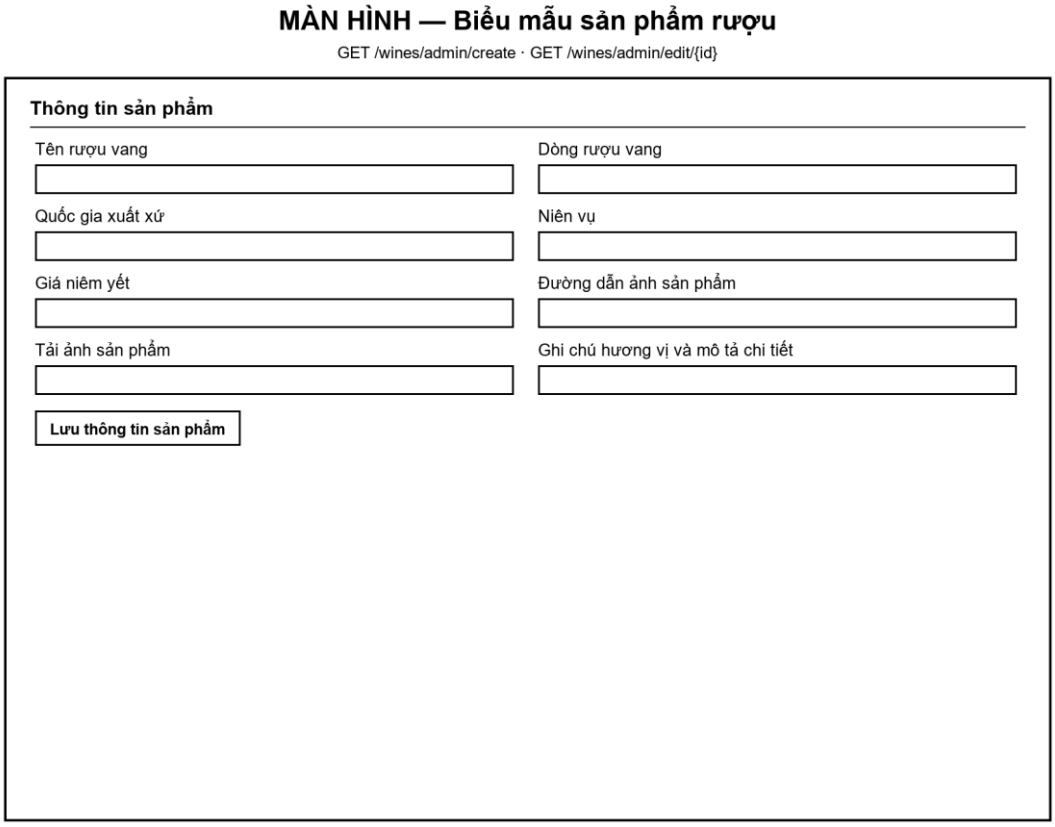


Hình 5.2: Sơ đồ tuần tự Admin tạo rượu qua form MVC và FileStorageService



Hình 5.4: Wireframe Quản lý danh mục và sản phẩm rượu vang (Grayscale Web Mockup)

Wireframe dưới đây minh họa form MVC tạo rượu tại GET/POST /wines/admin/create; file ảnh hỗ trợ JPG/JPEG/PNG, tối đa 5 MB. Giá trị trong form là ví dụ; đây không phải ảnh chụp runtime.



Hình 5.5: Wireframe Thêm mới và chỉnh sửa thông tin rượu (Grayscale Web Mockup)

6.1 Tồn đa kho, giữ chỗ và lịch sử nhập xuất

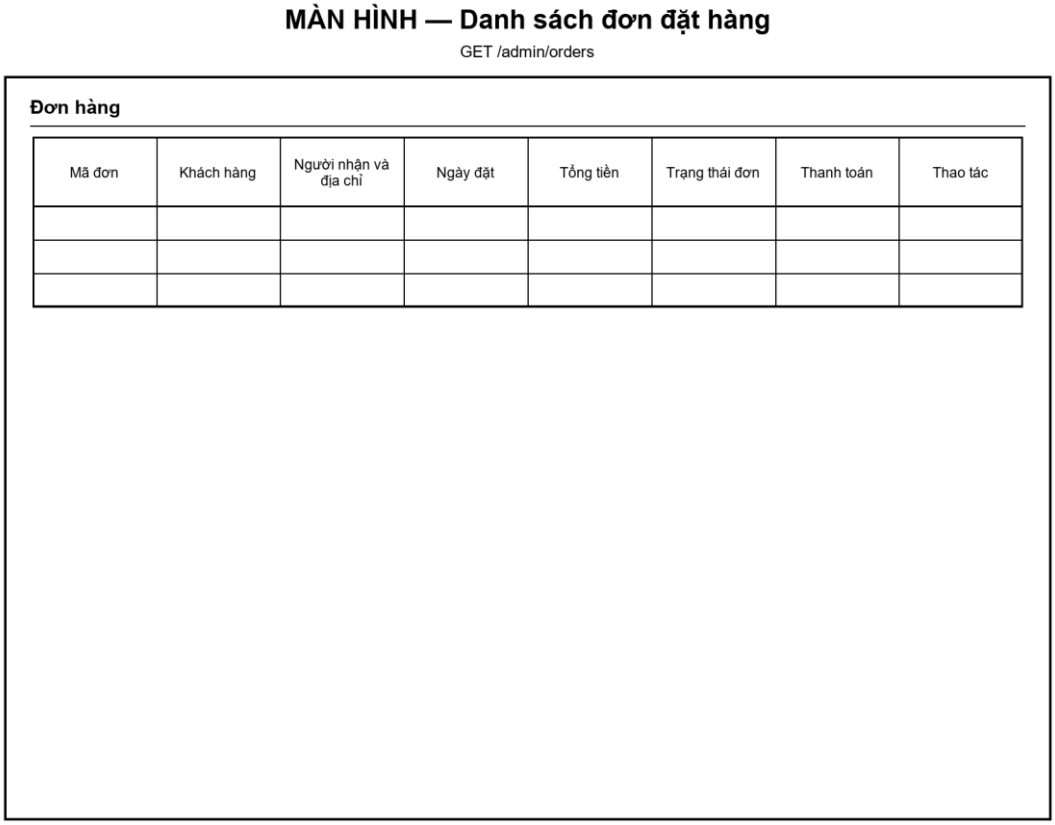
InventoryService có trách nhiệm tổng hợp lượng khả dụng và phối hợp cập nhật theo warehouse trong giao dịch `@Transactional` [6] có kiểm soát đồng thời của C.J. Date [10]. Lượng khả dụng phải phân biệt số vật lý đang có với số đã giữ cho đơn chưa hoàn tất; báo cáo và kiểm tra checkout cần sử dụng cùng định nghĩa. InventoryTransaction ghi lý do và lượng thay đổi để quản trị có thể giải thích số dư. Stock alert có thể dựa trên reorder level nhưng cần tránh phát lặp liên tục cho cùng một mức.

Trên checkout, đơn tạo reservation thay vì lập tức coi hàng đã giao. Khi chính sách thanh toán, hủy đơn hoặc hoàn tất thay đổi, tồn cần đi qua các trạng thái tương ứng: giữ, tiêu thụ hoặc giải phóng. Cần kiểm tra các nhánh thất bại và tình huống xử lý lặp, vì gọi reserve/release hai lần có thể làm lệch số nếu không idempotent.

@Version hỗ trợ optimistic locking theo lý thuyết cơ sở dữ liệu của C.J. Date [10]: khi hai giao dịch cùng sửa một bản ghi, bản ghi cập nhật sau có thể nhận conflict thay vì âm thầm ghi đè. Đây là tín hiệu phát hiện tranh chấp, không phải chính sách giải

quyết. Service cần giới hạn retry, chọn kho và xử lý khi mọi kho đều không đủ; API cần trả lỗi có thể hiểu được để khách chọn lại.

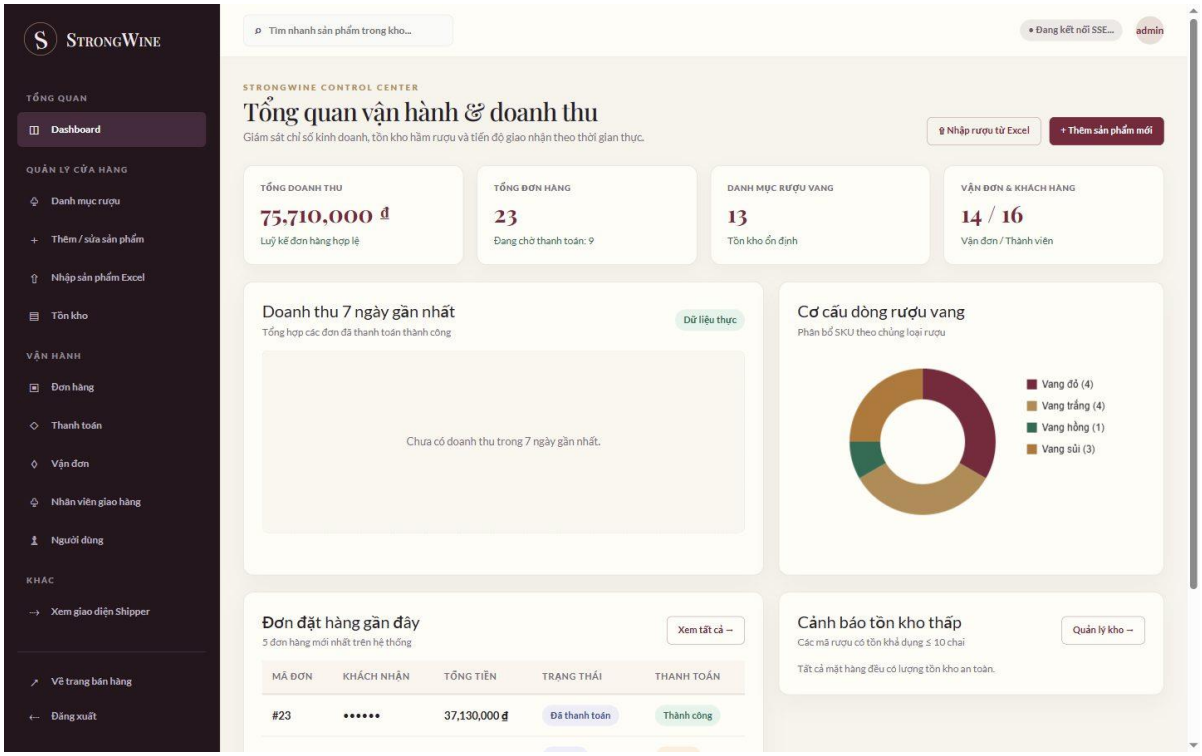
6.2 Nhập Excel: mẫu, preview, dry-run và ghi dữ liệu



Hình 5.9: Wireframe Giám sát đơn hàng toàn sàn và trạng thái thanh toán (Grayscale Web Mockup)

Strategy tách cách đọc workbook khỏi controller; báo cáo stock dùng lớp Template Method theo các mẫu thiết kế của GoF [4]. Export cũng cần bảo vệ khỏi formula injection ở ô text, giữ định dạng số, mã hóa Unicode và không trộn dữ liệu nhạy cảm. Không có file Excel nào được upload hoặc import trong phiên kiểm tra live hiện tại.

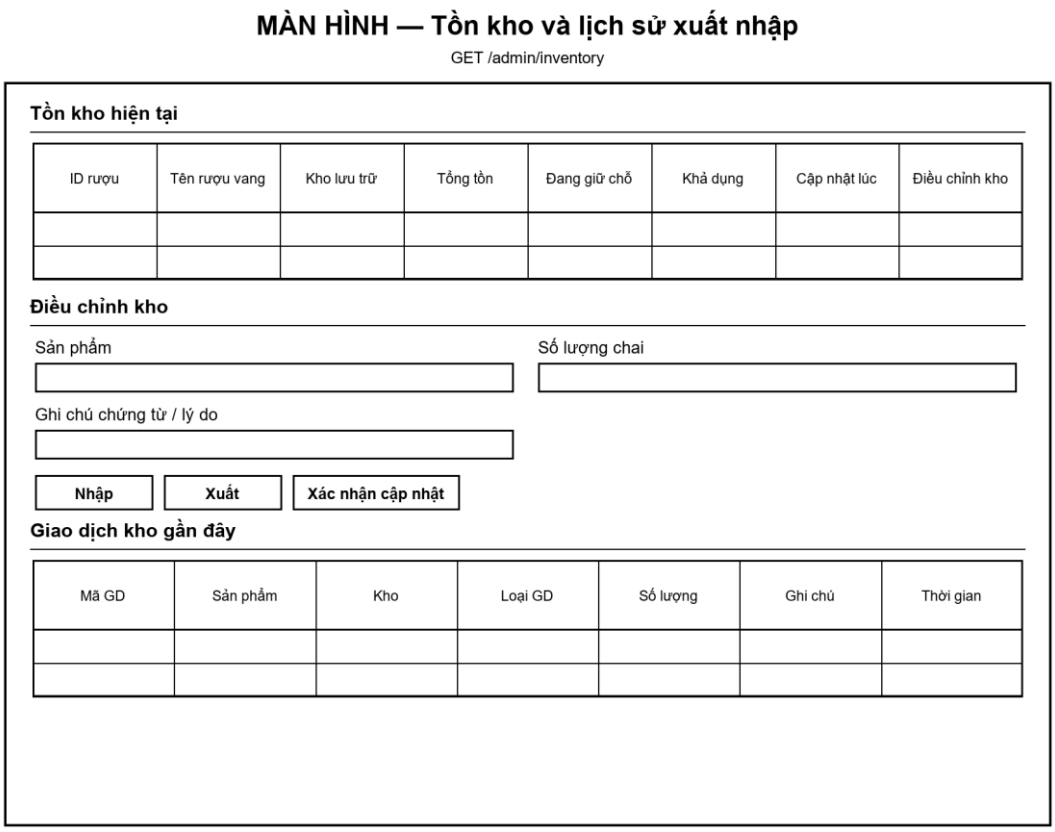
6.3 Dashboard và quan sát vận hành



Hình 5.1: Bảng điều khiển quản trị

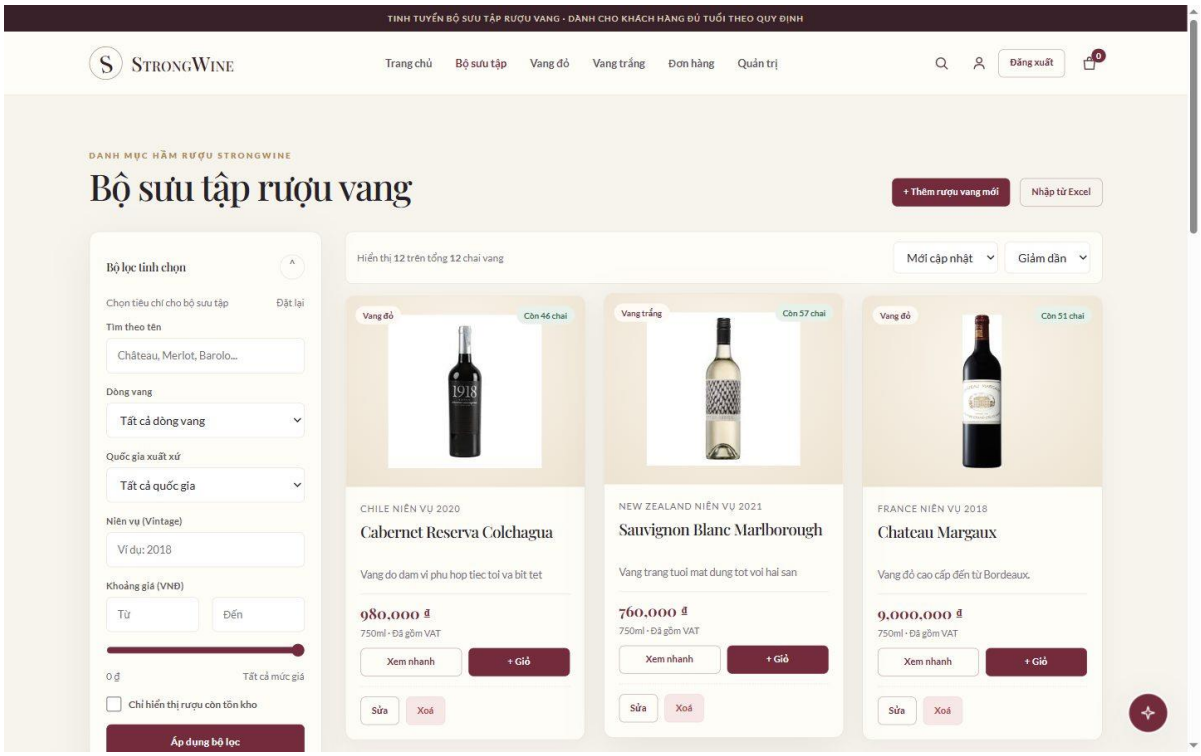
Dashboard admin tổng hợp dữ liệu đơn, doanh thu, sản phẩm, category và kho để giúp quản trị viên định hướng. Mỗi KPI cần có query và định nghĩa thời gian nhất quán; ví dụ tổng paid orders khác tổng orders, cancelled khác completed, timezone local khác UTC. Chart không nên hiển thị số liệu giả khi API lỗi; lỗi tải có thể hiện trạng thái một phần.

Ảnh dashboard quản trị ghi lại toàn bộ viewport desktop 1440 × 900, không crop vùng nội dung dashboard. Hình cho thấy số liệu và trạng thái đúng như giao diện trình bày tại thời điểm chụp; nó không xác nhận doanh thu đã được đối soát với payment provider hay truy vấn cơ sở dữ liệu.



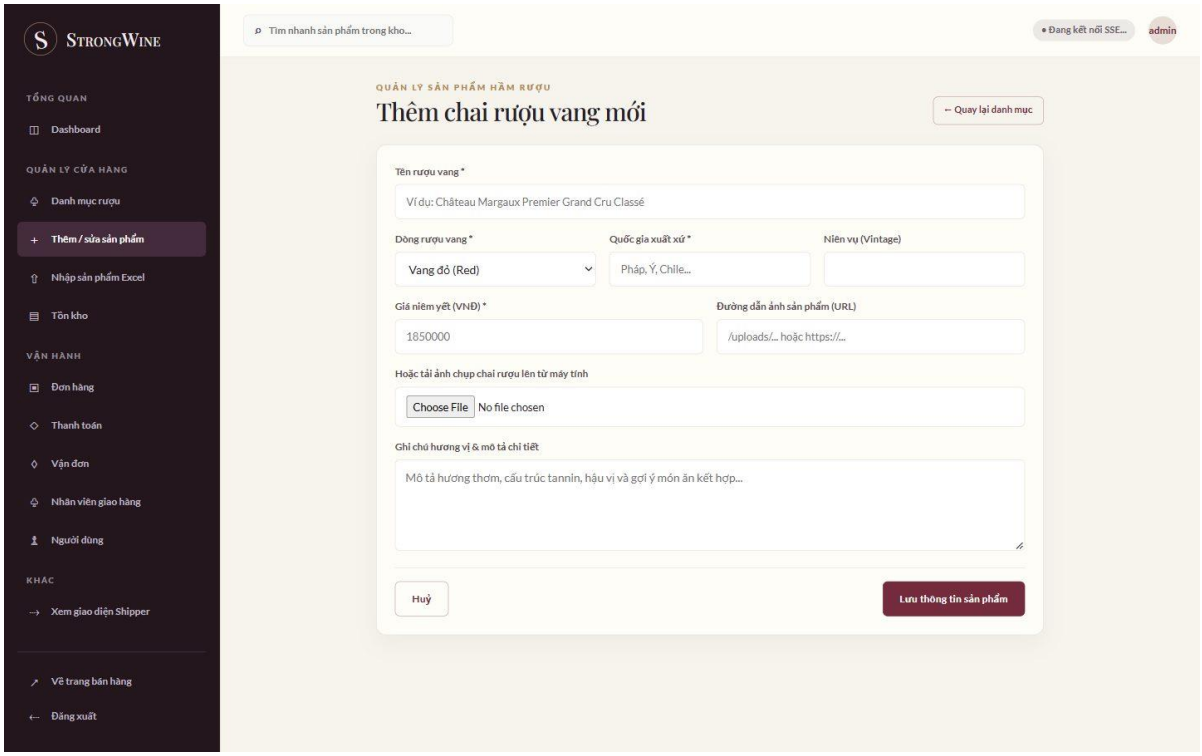
Hình 5.10: Wireframe Bảng quản trị tồn kho và điều phối chi nhánh (Grayscale Web Mockup)

Giao diện giỏ hàng trực tuyến cung cấp tính năng cập nhật số lượng và tự động tính toán tổng giá trị đơn:



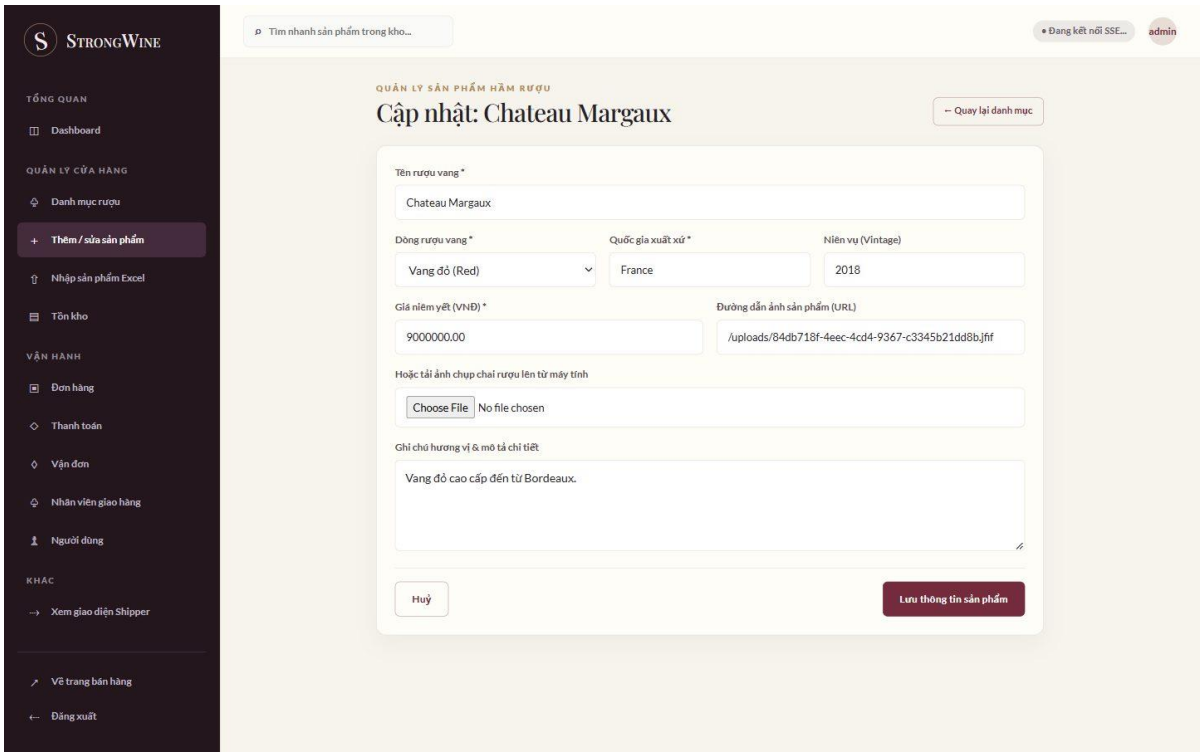
Hình 5.2: Danh mục và sản phẩm rượu

Bảng điều khiển quản trị tổng hợp trực quan các chỉ số KPI bán hàng và danh sách đơn mới nhất:



Hình 5.3: Tạo sản phẩm rượu

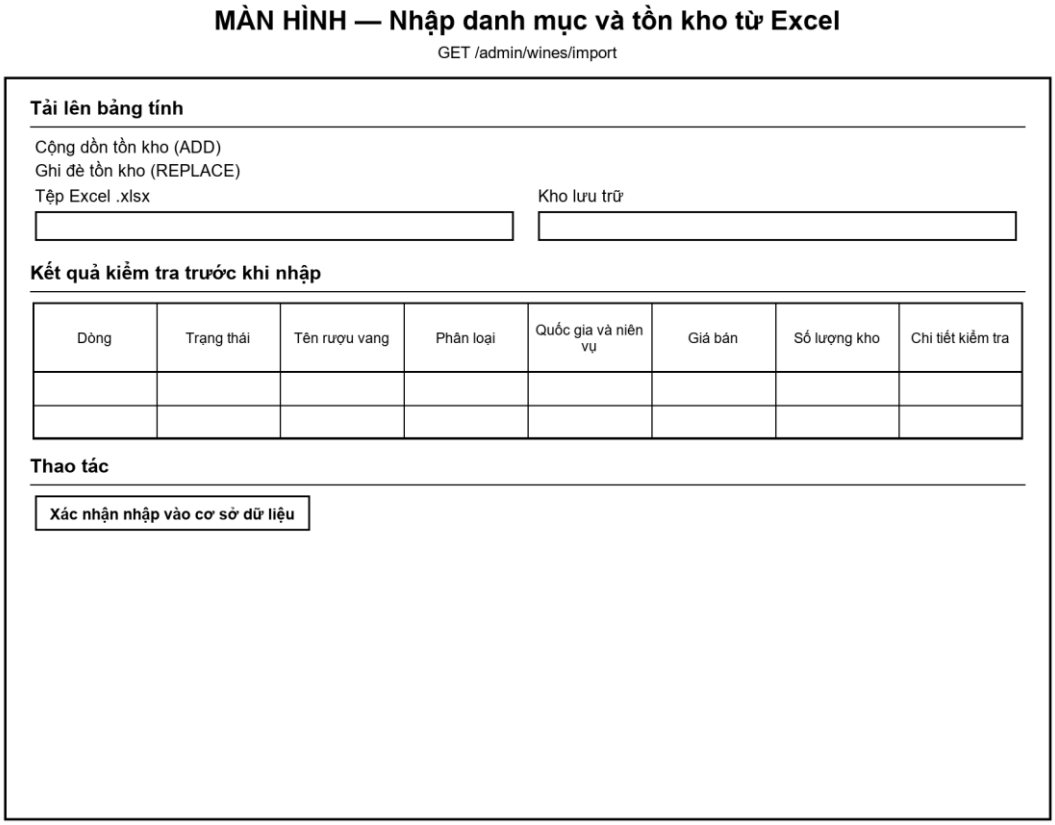
Màn hình quản lý danh mục và sản phẩm rượu vang hỗ trợ thủ kho tra cứu trạng thái kinh doanh:



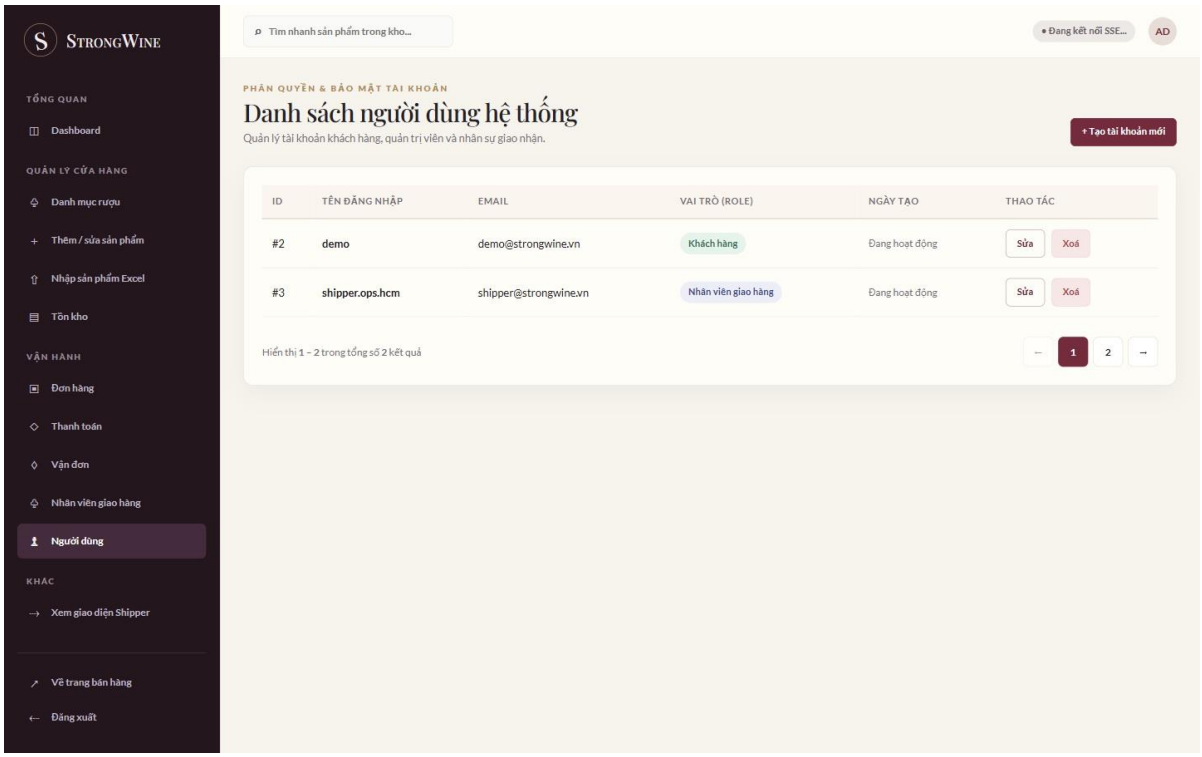
Hình 5.4: Chỉnh sửa sản phẩm rượu

MVC `WineController` có list/detail/admin create/edit/delete; API admin riêng quản lý products/categories. Cùng service cần chuẩn hóa validation và mapping DTO giữa hai bề mặt. `CategoryService` và `WineService` là trách nhiệm application/domain; controller nhận request và chọn view. Khi update, entity id, active/deleted state, category và image path phải được kiểm tra đồng thời.

Hard delete sản phẩm có lịch sử order/review có thể thất bại FK hoặc làm sai dữ liệu; soft delete giúp giữ lịch sử nhưng public list phải loại bỏ. Danh mục có nhiều product cũng cần policy. Khi đổi category, ảnh hưởng filter/report nhưng không đổi order history nếu snapshot thiết kế đúng.

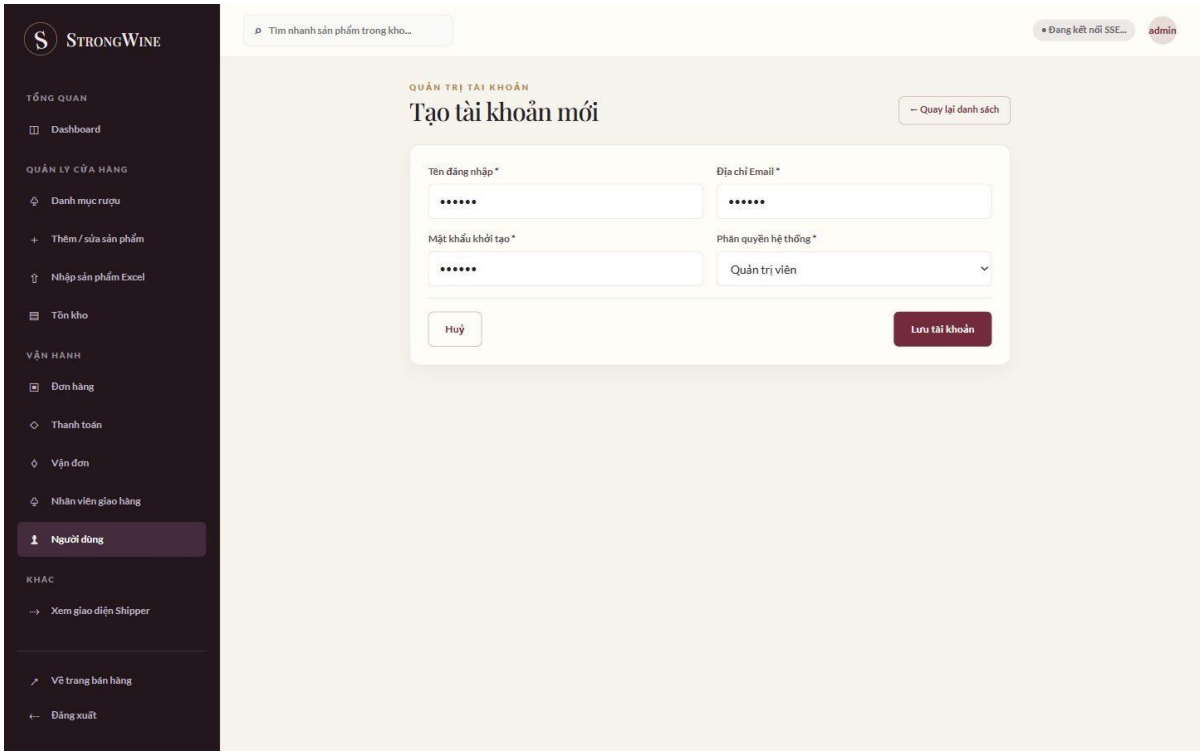


Hình 5.6: Wireframe Nhập danh mục rượu hàng loạt từ tệp Excel (Grayscale Web Mockup)

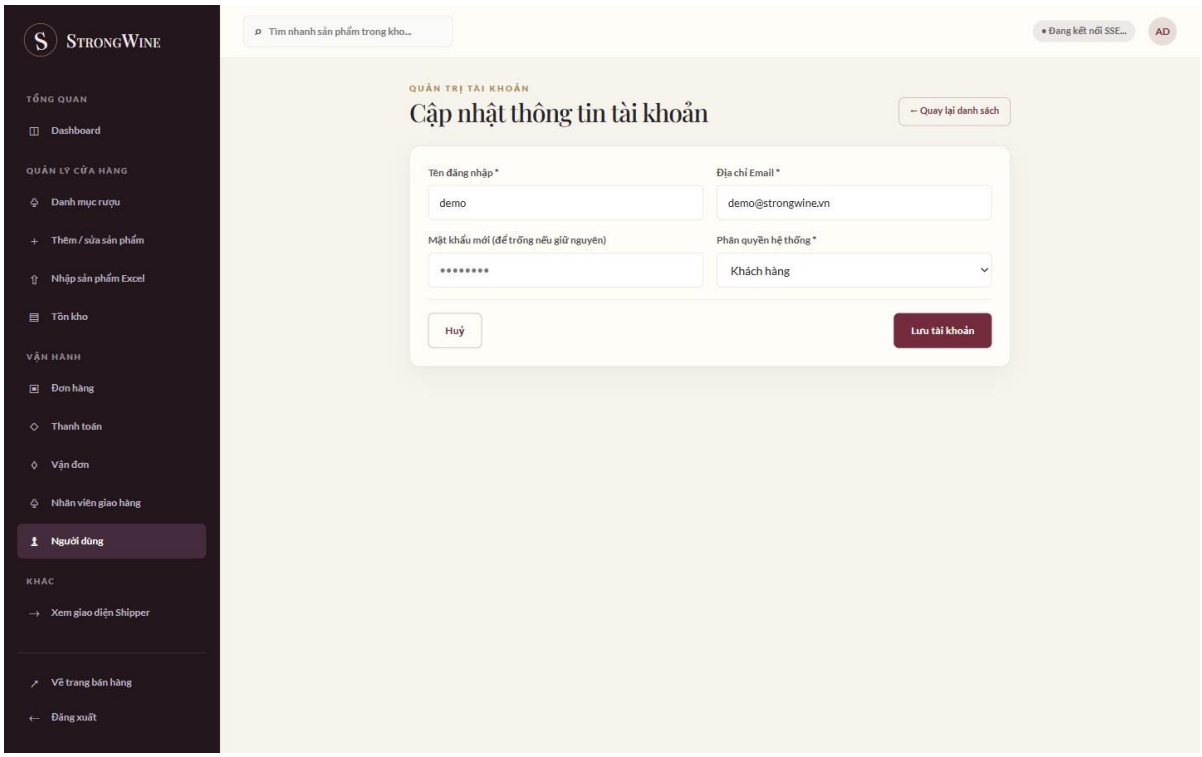


Hình 5.5: Danh sách tài khoản người dùng

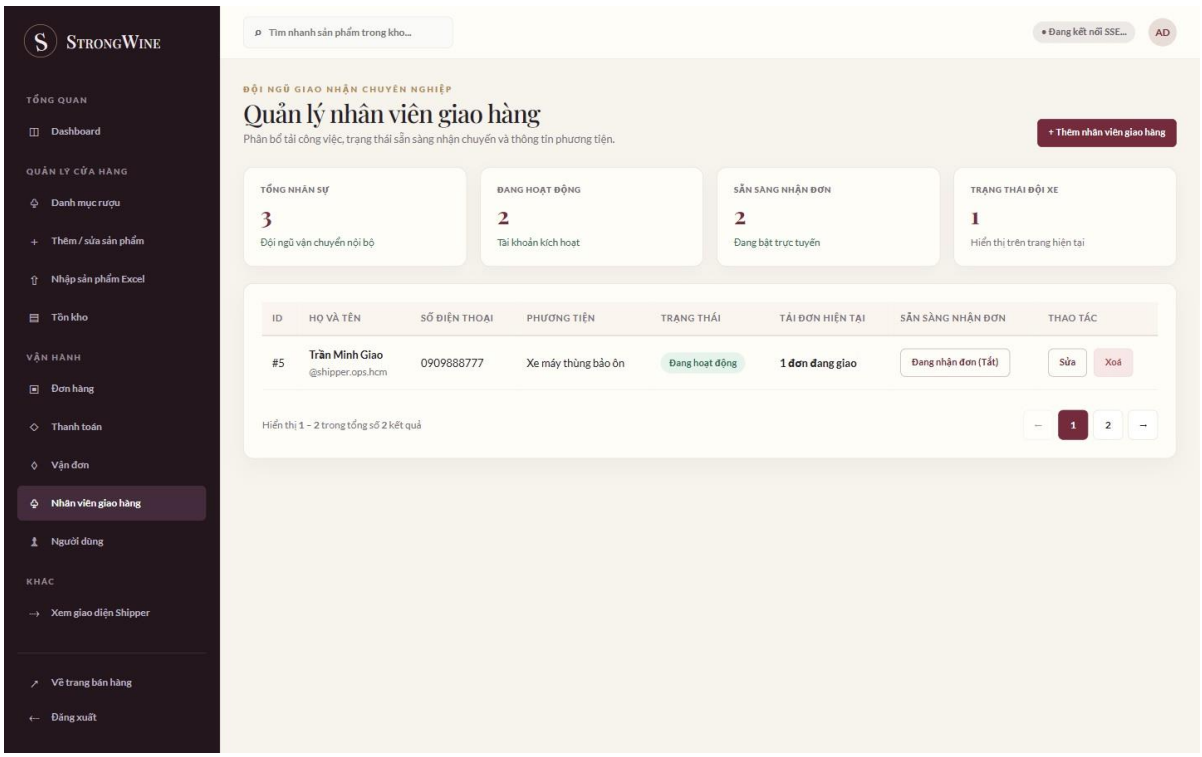
Biểu mẫu thêm mới rượu vang yêu cầu điền đầy đủ các trường thông tin kỹ thuật và tải ảnh đại diện:



Hình 5.6: Tạo tài khoản người dùng

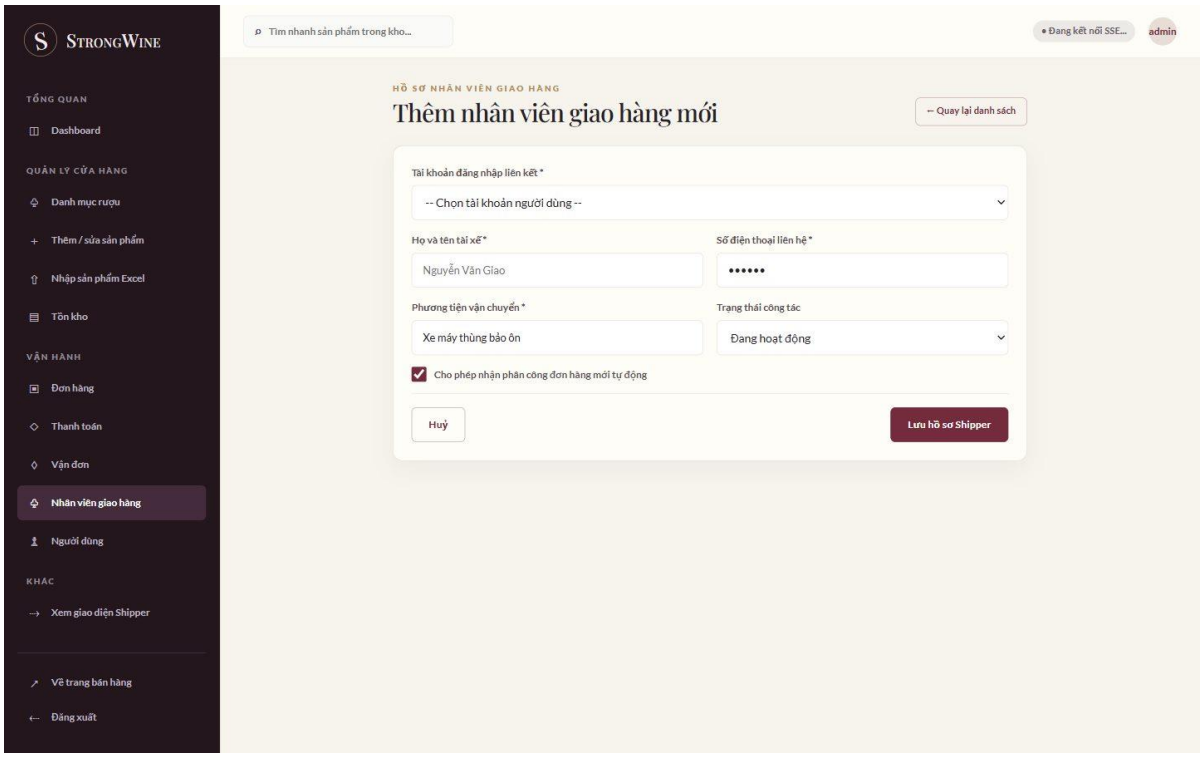


Hình 5.7: Chỉnh sửa tài khoản người dùng



Hình 5.8: Danh sách người giao hàng

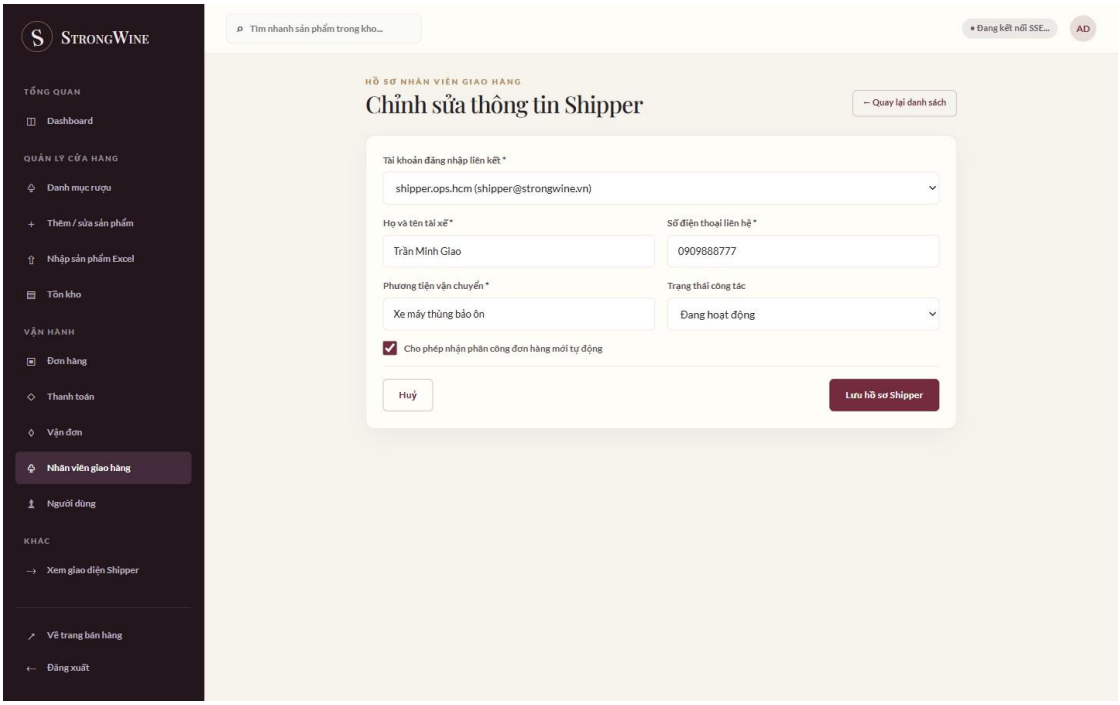
Màn hình hiệu chỉnh thông tin rượu cho phép cập nhật đơn giá, mô tả chi tiết và quy cách đóng chai:



Hình 5.9: Tạo hồ sơ người giao hàng

Danh sách người dùng hệ thống phân định rõ ràng các vai trò quản trị viên, khách hàng và người giao hàng:

Dưới đây là giao diện thực tế cập nhật thông tin nhân viên giao hàng trên hệ thống:



Hình 5.10: Giao diện chỉnh sửa hồ sơ Shipper

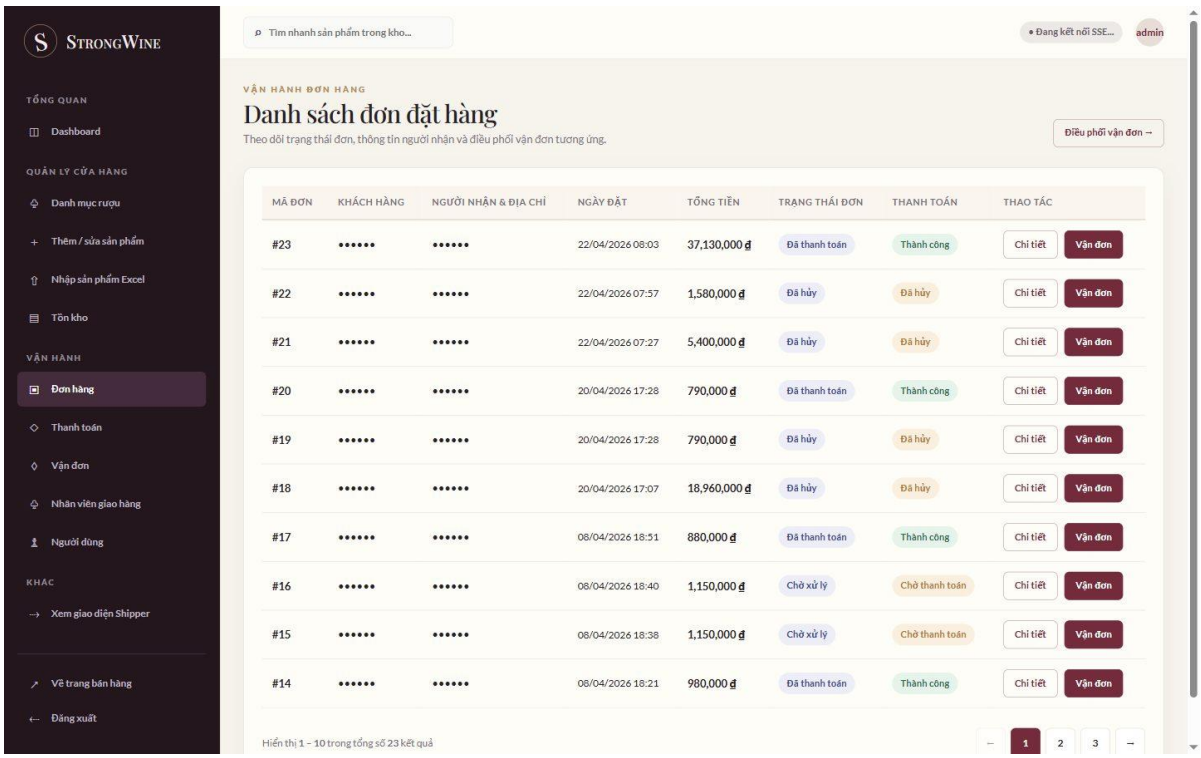
AdminUserController list/create/edit/delete user; AdminShipperController quản lý profile shipper. Admin không nên chỉnh password plaintext; thao tác reset cần flow

riêng. Role change cần audit và có thể invalidate token/session. Xóa user đang có shipment/order phải có quy tắc (deactivate/anonymize) thay vì cascade mù.

Shipper profile liên kết user authentication với vehicle/phone/status/location. Admin có thể active/deactivate shipper; thay đổi trạng thái cần ảnh hưởng queue dispatch. Đổi số điện thoại hoặc phương tiện có thể cần xác minh. Dashboard chỉ hiển thị thông tin cần vận hành và không nên cho người khác thấy coordinates lịch sử.

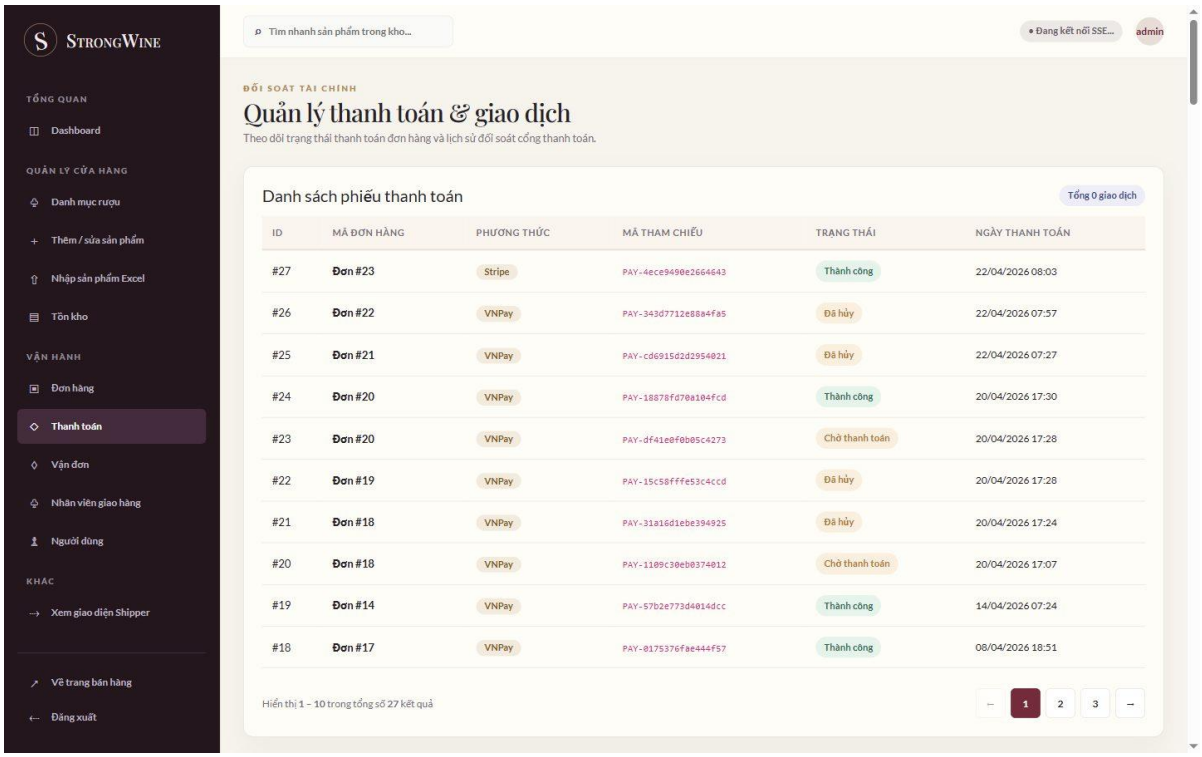
Test quyền và lifecycle: admin create/update; duplicate username/profile; deactivate khi assigned shipment; role escalation; no password exposure; deleted user with order; coordinates permission; API list details minimization. Tài khoản local được dùng để chụp ảnh nhưng không được công bố.

6.4 Quản lý order/payment phía quản trị



Hình 5.11: Danh sách đơn hàng quản trị

Giao diện quản lý hồ sơ nhân viên shipper ghi nhận trạng thái sẵn sàng nhận cuốc và phương tiện di chuyển:



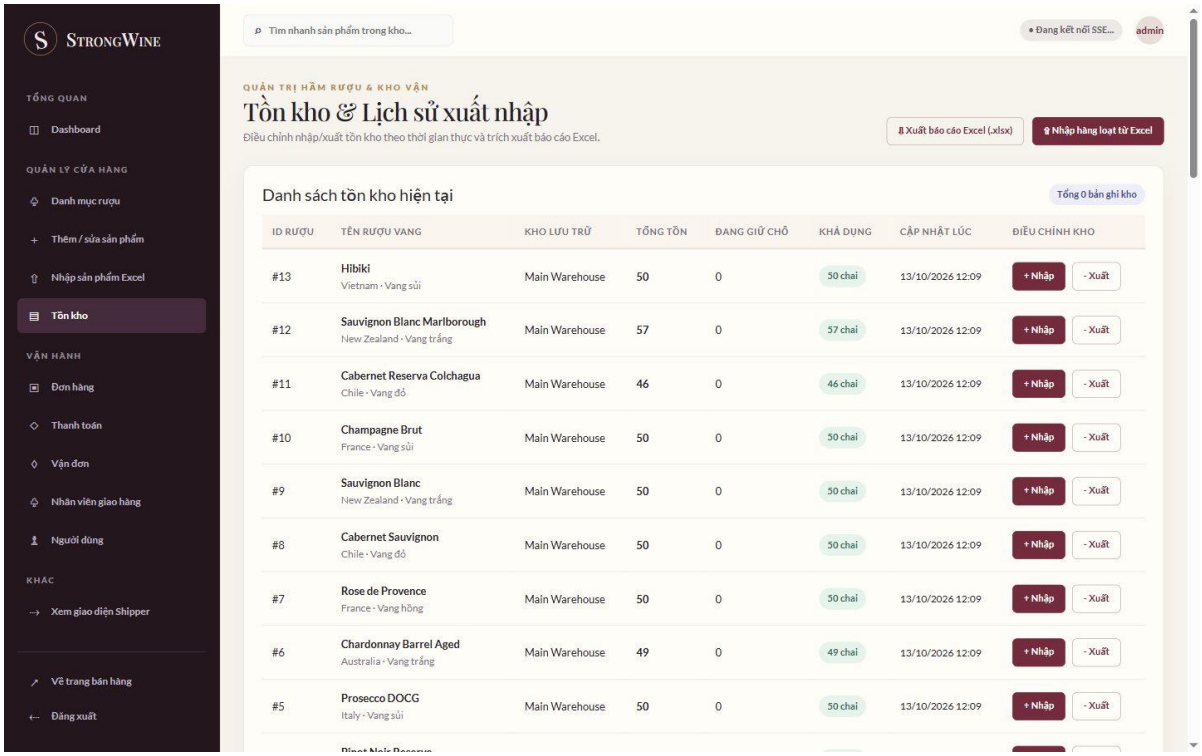
Hình 5.12: Quản lý thanh toán

AdminOrderController và AdminPaymentController phục vụ tổng quan giao dịch; quyền ADMIN mới được truy cập. Màn hình nên đọc trạng thái từ aggregate, hiển thị timeline transaction/shipments và phân biệt payment attempt. Chỉ đọc không được tạo tác dụng phụ. Nếu admin override status, cần confirmation, reason, actor và event/audit.

Payment reports phải phân biệt amount đặt hàng, amount gateway confirmed, refund, fee và currency. Tổng doanh thu không nên cộng pending/failed. Nhiều payment attempts cho một order cần lựa chọn payment cuối cùng hợp lệ. Dashboard cache cần invalidation khi callback/webhook chốt transaction.

Test admin: user không thể xem payment; payment secret đã redact; order paid/cancelled state đúng; filter date boundary/timezone; total và detail đồng nhất; override reason bắt buộc; event trùng không làm số liệu tăng gấp đôi. Không có payment thật trong lần thực nghiệm.

6.5 Màn hình tồn kho và cảnh báo low stock



Hình 5.13: Quản lý tồn kho

AdminInventoryController/API có overview, low-stock, import/export và transactions paging. Ngưỡng `reorderLevel` xác định cảnh báo mỗi inventory record; admin cần nhìn cả current/reserved/available để không nhầm hàng đã giữ với hàng có thể bán. Paging giúp giới hạn bộ nhớ; sort theo available/reorder giúp ưu tiên xử lý.

Nếu nhiều warehouse, cảnh báo tồn phải theo từng kho và tổng hợp theo product riêng. Ngưỡng đặt ở mỗi inventory record có thể khác nhau. Tạo inventory lazy cho product/kho mới cần tránh duplicate với unique constraint; scheduler hoặc import đồng thời có thể đụng race.

Test: available bằng 0 khi reserved=current; low stock dưới/bằng/trên threshold; inventory missing; warehouse inactive; paging stable; stock negative prevented; import lock contention; filter product/category. Bài không đưa số lượng kho thật ra ngoài tài liệu.

6.6 Nhập kho, xuất kho và ledger

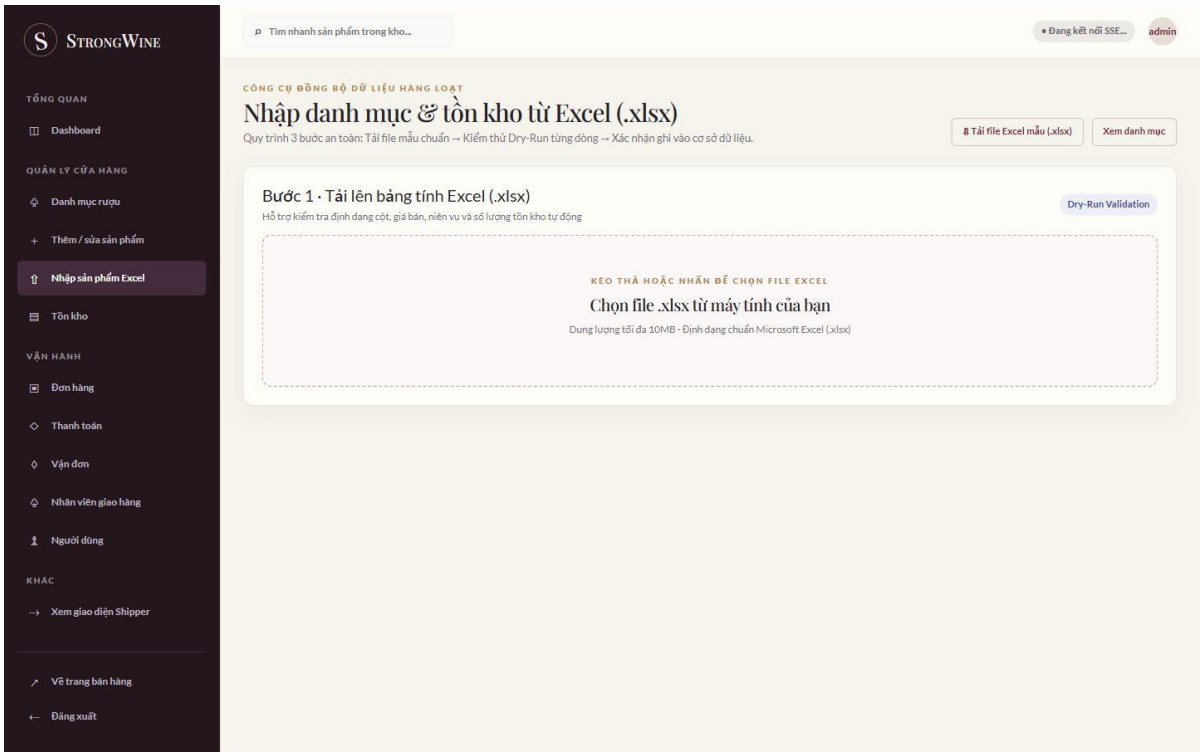
Import stock tăng current quantity có ghi inventory operation; export kiểm tra không giảm dưới reserved/available. Cancel order release reservation là operation khác với xuất; confirm paid chuyển reservation thành xuất xác định. Nếu ghi hai operation

type nhầm, dashboard lịch sử khó đối soát. Cơ chế quản lý giao dịch khai báo `@Transactional` [6] tại tầng dịch vụ bảo đảm dữ liệu tồn kho hiện thời và sổ cái lịch sử (ledger) luôn được ghi nhận đồng bộ, tránh hiện tượng mất mát dữ liệu theo lý thuyết cơ sở dữ liệu của C.J. Date [10].

Người thực hiện được resolve từ username/userId và lưu createdBy; note cần mô tả lô/điều chỉnh. Số lượng nhập cần xác thực >0 và giới hạn theo nghiệp vụ; export cần >0 và ≤ available. Không cho admin chỉnh trực tiếp số current mà không tạo ledger.

Test transaction: fail write ledger thì inventory không đổi; fail update inventory không có ledger; export vượt tồn bị từ chối; cancel lặp không release hai lần; order paid lặp không xuất hai lần; user audit đúng. Báo cáo nên truy ngược theo reference type/id tới order hoặc import batch.

6.7 Nhập Excel: template, preview và dry-run



Hình 5.14: Màn hình nhập dữ liệu Excel

AdminExcelApiController tạo template, preview và import; inventory API cung cấp import/export theo sản phẩm. Template quy định header, kiểu, ví dụ và sheet; version template cần được giữ ổn định để tránh file cũ bị parse nhầm. Preview đọc workbook, phát hiện row errors, tổng hợp success count và updated/inserted count mà chưa ghi dữ liệu.

`ImportOptions.dryRun` phải thực sự ngăn mọi write/side effect; `inventoryMode` quyết định xử lý tồn; `defaultWarehouseId` áp khi dòng thiếu kho. Nếu import một phần, UI cần báo chính xác row nào thành công/thất bại và có thể tải error file. Nếu all-or-nothing, phải rollback cả batch. Chính sách phải rõ và nhất quán giữa UI và API.

Apache POI parse file XLS/XLSX; cần giới hạn row count/size, tránh formula evaluation không cần, bảo vệ chống ZIP bomb, xử lý cell type và close workbook/stream. File upload giới hạn 5 MB ở cấu hình request; giới hạn record vẫn cần riêng. Data import không được tin macro/formula/embedded objects.

6.8 Xử lý lỗi theo dòng Excel

`ImportErrorItem` ghi rowNumber, fieldName, invalidValue và errorMessage; kết quả `ImportResult` giữ successItems/errors và counts. Phản hồi nên tránh echo dữ liệu nhạy cảm hoặc công thức nguy hiểm. Lỗi cần gắn với header ổn định, không dùng số index cột nếu người quản trị cần sửa. Hệ thống nên thống nhất row numbering 1-based như Excel.

Validation ở cấp row gồm required fields, number/date parse, decimal scale, enum/type, FK category/warehouse, duplicate key và range. Validation liên dòng gồm hai dòng cùng SKU/product hoặc tổng stock. Preview cần dùng cùng logic validation với final import; nếu hai code path khác, preview "pass" có thể fail khi ghi dữ liệu cuối cùng.

Test workbook tổng hợp: empty workbook, malformed, header thiếu, row trống, formula, category unknown, decimal locale, quantity negative, duplicate, dryRun, partial policy, huge row count và repository exception. Mỗi test xác minh DB trước/sau và file stream đã đóng; không gọi import live để chụp ảnh.

6.9 Export Excel và báo cáo tồn

Export endpoint tạo workbook theo chiến lược phù hợp, giới hạn theo filter/quyền và gắn content type/filename an toàn. Export inventory cần hiển thị warehouse, wine, current/reserved/available, reorder level và update timestamp. Nếu tồn có PII/giá vốn, cần quản lý quyền; file tạm và response buffering cần được dọn để không chiếm disk.

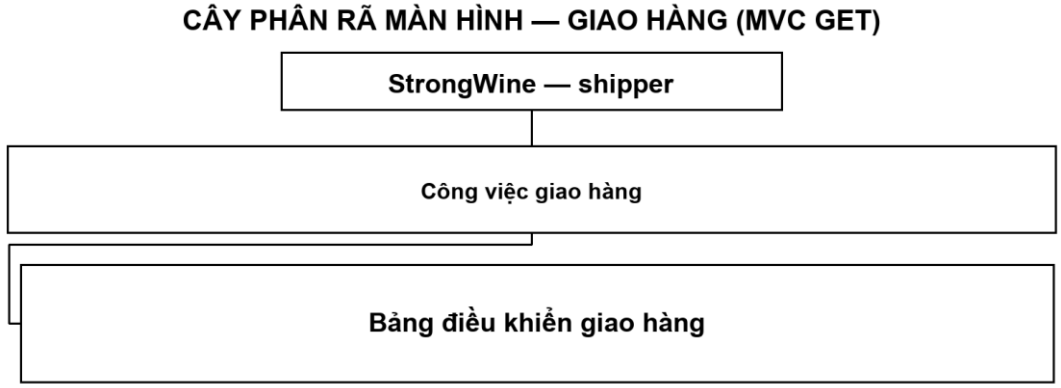
`AbstractExcelReportGenerator` cung cấp khung chung, còn concrete generator định nghĩa dataset/columns. Format số cần giữ tiền decimal và quantity integer; ngày

giờ phải nêu timezone. Formula injection có thể xảy ra nếu cell string bắt đầu `=`, `+`, `-`, `@`; phải ghi như text hoặc sanitize. Cần xem đó là security requirement của export.

Đối chiếu export với SQL aggregate và UI list trên cùng snapshot; kiểm tra các ký tự tiếng Việt, dấu phẩy thập phân, dòng trống và số lượng lớn. Download không nên cache public. Chưa có file export nào được tạo trong lần QA UI.

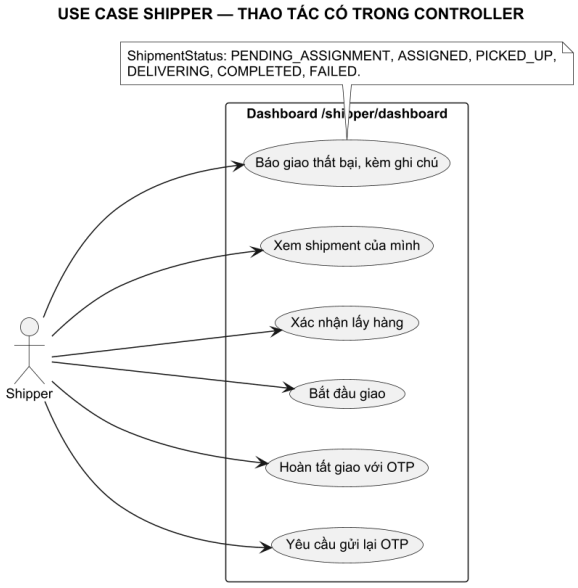
CHƯƠNG 6: LUỒNG SHIPPER, OTP VÀ THÔNG BÁO

Controller shipper trả một view dashboard cho GET /shipper, /shipper/dashboard và /shipper/shipments. Trên dashboard, shipper xem shipment của mình và gửi các POST lấy hàng, bắt đầu giao, hoàn tất bằng OTP, gửi lại OTP hoặc báo thất bại.



Hình 6.0: Sơ đồ phân rã màn hình phân hệ Giao hàng (Shipper Cockpit Screen Hierarchy Tree)

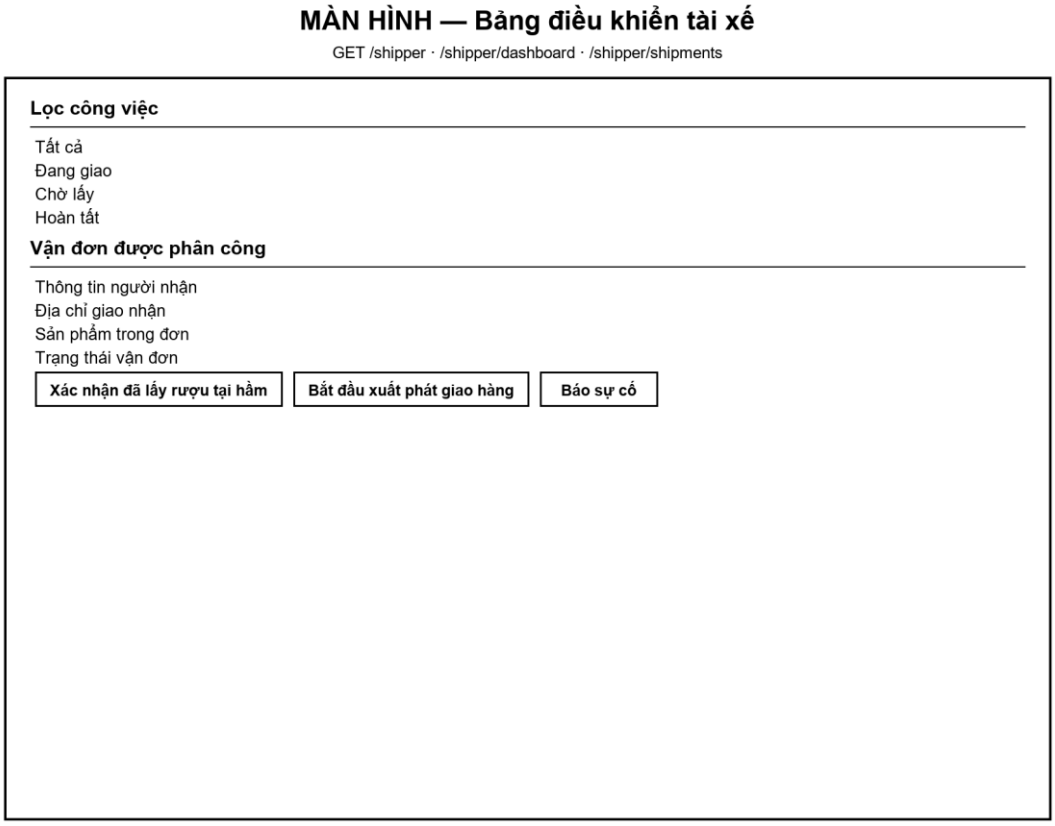
Use case shipper giới hạn ở các hành động controller hiện có: xem shipment được giao, xác nhận lấy hàng, bắt đầu giao, hoàn tất với OTP, yêu cầu gửi lại OTP và báo giao thất bại.



Hình 6.0b: Sơ đồ Use Case chi tiết phân hệ Giao hàng (Shipper)

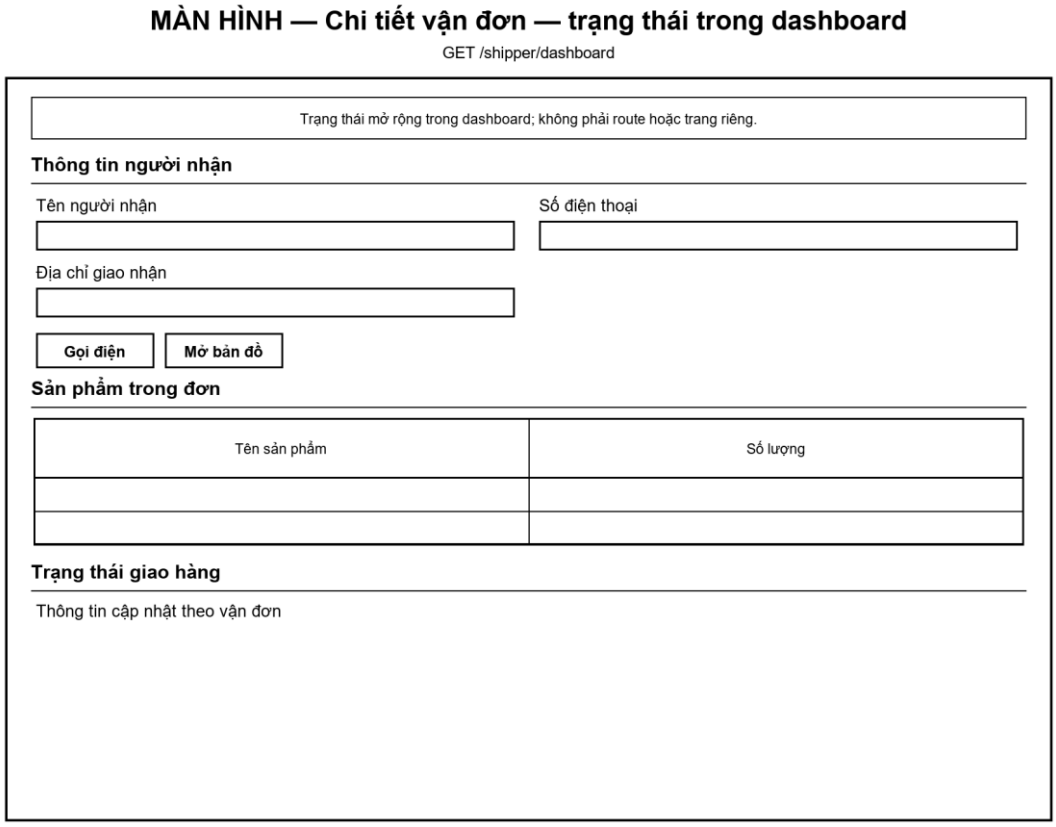
Controller hiện trả cùng một view dashboard và gắn danh sách shipment theo tài khoản. Mọi yêu cầu HTTP POST đều chuyển hướng quay trở lại giao diện điều khiển chính; controller này hiện chưa cấu hình trang xem chi tiết hay danh sách vận đơn độc lập.

7.1 Phân công tự động và quyền xem nhiệm vụ



Hình 6.1: Wireframe Bảng điều khiển công việc Shipper (Grayscale Web Mockup)

Màn hình giám sát đơn hàng toàn sàn phục vụ công tác điều phối và cập nhật trạng thái thanh toán theo mô hình hành vi UML [5]:



Hình 6.2: Wireframe trạng thái xem chi tiết vận đơn trên bảng điều khiển Shipper

7.2 Lấy hàng, bắt đầu giao và báo giao thất bại

Shipper gọi thao tác pickup để xác nhận đã lấy hàng, tiếp theo start để chuyển sang đang giao. Dịch vụ cần xác minh danh tính shipper khớp với shipment và trạng thái hiện tại cho phép hành động. Khi không giao được, fail nhận ghi chú tùy chọn; quản trị và người mua cần thấy trạng thái, lý do phù hợp mà không lộ thông tin không cần thiết.

Báo thất bại không tự giải quyết việc lên lịch lần giao mới, trả về kho, giải phóng hàng hay hoàn tiền. Quy trình cần quy định người chịu trách nhiệm và điều kiện chuyển tiếp để tránh đơn mắc kẹt. Trạng thái và history cần lưu thời điểm, tác nhân và nguồn cập nhật; sự kiện lỗi phải cho phép tái xử lý có kiểm soát.

7.3 OTP giao hàng, gửi lại và giới hạn thử

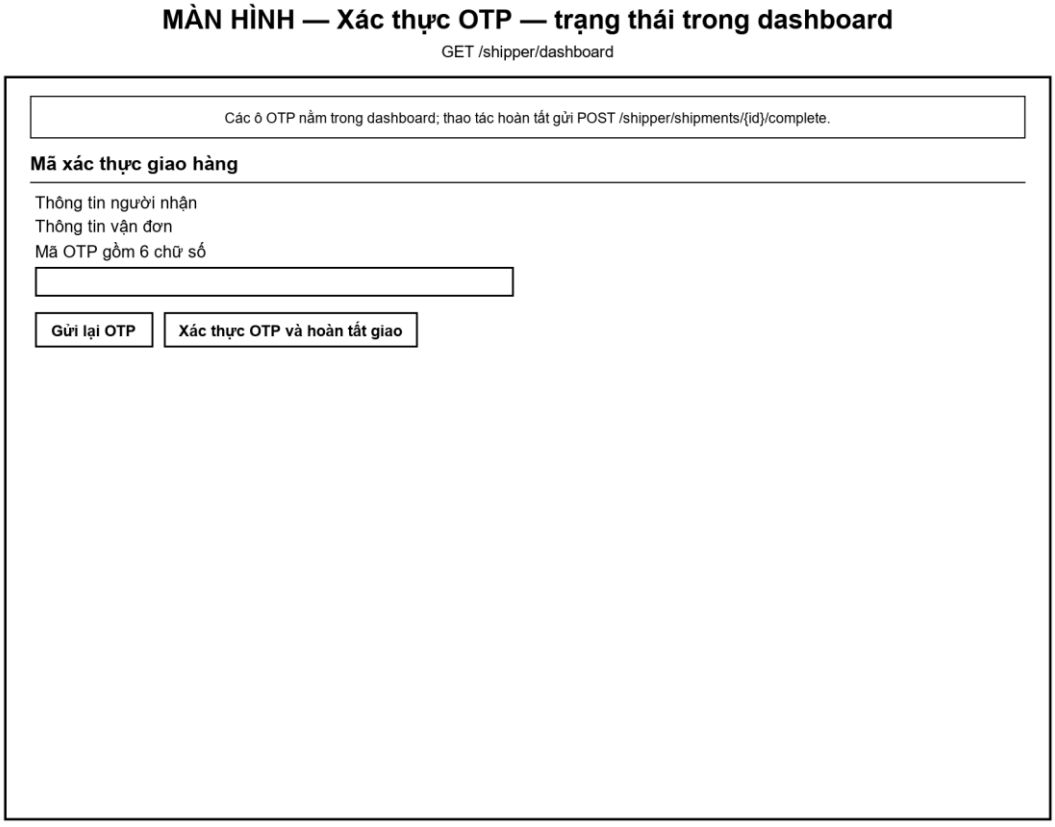
Để hoàn tất, shipper gửi mã OTP nhận từ người nhận. Dịch vụ chuẩn hóa đầu vào, xác nhận shipment thuộc shipper, kiểm tra trạng thái có thể hoàn tất, thời điểm hết hạn, khóa tạm thời và liên kết người dùng dự kiến. Mã sai làm tăng bộ đếm lần thử và được ghi nhật ký; khi vượt ngưỡng, shipment bị khóa trong khoảng thời gian được cấu hình. Mã đúng xóa trạng thái thử và cho phép ghi nhận hoàn tất.

OTP có thể được gửi lại bởi shipper hoặc admin theo các endpoint tương ứng. Gửi lại phải vô hiệu hóa mã cũ, áp dụng giới hạn tần suất, ghi trạng thái gửi và xử lý email lỗi. Sau khi hoàn tất, mã nhạy cảm và metadata dùng một lần được xóa/reset để không thể dùng lại. Cần tránh ghi plaintext OTP vào log; audit nên lưu kết quả và nguyên nhân, không lưu mã.

Quản trị có thể có nhánh override giao mà không cần OTP; thao tác này cần lý do bắt buộc, quyền mạnh hơn và history rõ để phân biệt giao hàng được xác minh với quyết định thủ công. Đây là điểm kiểm soát rủi ro cần được xác nhận bằng test quyền, test OTP sai/hết hạn/khóa và rà log. Không nhập mã thật trong lần kiểm tra này.

7.4 SSE và phát cập nhật đến từng vai trò

Các API live mở stream riêng theo order, admin hoặc shipper. Khi trạng thái đơn/vận đơn thay đổi, event mang dữ liệu cần thiết được phát và RealtimeEventListener chuyển tiếp đến SseNotificationService. Khách nhận cập nhật tiến độ đơn; admin nhận thông báo vận hành; shipper nhận nhiệm vụ hoặc thay đổi liên quan đến danh sách của họ.



Hình 6.3: Wireframe trạng thái nhập OTP hoàn tất giao hàng trên bảng điều khiển Shipper

7.5 Tạo shipment sau khi đơn thanh toán

`OrderPaidShipmentListener` nhận event sau khi dữ liệu nguồn được ghi nhận thành công rồi gọi `ShipmentService.handleOrderPaidEvent`; service kiểm tra order/shipment và chọn tạo/assign hoặc để chờ. Mốc xử lý sau giao dịch tạo ranh giới rõ: payment/order phải được lưu trước khi logistics tiếp tục. Nếu service lỗi, order vẫn đã paid, do đó logs và backfill là thành phần cần có.

Tạo shipment cần copy snapshot recipient/address/email/coordinates từ order tại thời điểm giao. Có thể ghi promised window nếu business có; không tự suy ETA dựa trên tọa độ. Phải bảo đảm một shipment chính cho mỗi order trừ khi hệ thống hỗ trợ chia kiện; schema cardinality/migration quyết định. Khi order được backfill, không duplicate email/OTP gây khó hiểu cho khách.

Test: order unpaid không shipment; paid event tạo shipment một lần; duplicate event idempotent; missing customer email; no active shipper; listener exception log/reconcile; address snapshot unchanged after profile edit. Thử nghiệm live trước đó không gửi order nên không chứng minh listener đã chạy end-to-end.

7.6 Chính sách auto dispatch

Chính sách điều phối tự động (Auto Dispatch) giúp StrongWine tối ưu hóa tốc độ gán vận đơn cho các nhân viên giao hàng đang sẵn sàng làm việc.

Hệ thống sử dụng một bộ lập lịch định kỳ (`@Scheduled`) quét các vận đơn ở trạng thái chờ phân công (`PENDING_ASSIGNMENT`) trong hàng đợi. Thuật toán phân bổ tự động đánh giá các tiêu chí theo mức độ ưu tiên: nhân viên giao hàng phải có hồ sơ hợp lệ (`ACTIVE`), đang trong ca trực trực tuyến (`ONLINE`), có khu vực hoạt động trùng khớp với địa chỉ giao nhận của khách, và có số lượng đơn đang vận chuyển thấp nhất tại thời điểm quét. Để ngăn ngừa hiện tượng tranh chấp điều phối (Race Condition) khi có nhiều worker chạy đồng thời, hệ thống áp dụng cơ chế khóa phân tán hoặc transaction cấp cao kết hợp khóa bi quan (`Pessimistic Write Lock`) trên bản ghi đơn hàng, bảo đảm một vận đơn chỉ được gán duy nhất cho một tài xế.

Mọi quyết định phân công tự động đều được hệ thống ghi nhận vào lịch sử vận đơn và phát tín hiệu thông báo tức thời đến ứng dụng của Shipper qua kênh SSE.

7.7 Xác nhận lấy hàng và bắt đầu giao

Quy trình xác nhận lấy hàng và bắt đầu lộ trình giao vận đánh dấu bước chuyển trạng thái quan trọng trong vòng đời đơn hàng của StrongWine.

Khi nhân viên giao hàng đến kho tiếp nhận kiện hàng, họ thực hiện thao tác xác nhận thông qua endpoint `/shipper/shipments/{id}/pickup`. Hệ thống kiểm tra chặt chẽ quyền sở hữu của shipper đối với vận đơn đó, đồng thời xác thực rằng trạng thái đơn hàng hiện tại là hợp lệ trước khi chuyển sang `PICKED_UP` và ghi nhận dấu thời gian `pickedUpAt`. Ngay sau đó, khi tài xế bắt đầu di chuyển trên đường, thao tác `/start` chuyển trạng thái sang `DELIVERING` kèm dấu thời gian `deliveringAt`. Những tác vụ giao nhận trên được thiết kế theo nguyên lý Idempotent: các yêu cầu gửi lặp lại do kết nối mạng chập chờn sẽ không làm thay đổi dấu thời gian khởi tạo ban đầu, bảo đảm độ chính xác của số liệu đo lường hiệu suất.

Giao diện phía khách hàng tự động cập nhật trạng thái đơn hàng sang 'Đang giao hàng' nhờ sự kiện thời gian thực đẩy về từ máy chủ.

7.8 Báo giao thất bại, retry và hoàn trả

POST failure nhận lý do/mã lỗi, ghi `failureNote`, `failureCode`, `failedAt`, `lastDeliveryAttemptAt` và `nextAttemptAt` theo chính sách. Lý do cần tập enum/validation hoặc giới hạn, không cho HTML/script tùy ý. Nếu retry được, shipment quay lại trạng thái phù hợp và OTP mới được phát theo điều kiện; nếu hết lần thử, cần policy return/cancel.

Admin cần xem timeline và quyết định retry, reassignment hoặc return. Đổi người giao có thể phải thu hồi OTP cũ, gắn recipient mới và audit. Không để mã cũ tiếp tục hoàn tất delivery sau reassignment. SMS/email thông báo lại người nhận cần tránh gửi nhiều lần do retry tự động.

Test: reason missing, retry count, cooldown, new assignee, old OTP invalid, nextAttempt timezone, return transition, duplicate fail request, admin override. Cấu hình “max attempts” nếu có phải lưu hoặc thống nhất; báo cáo không tự bịa giới hạn chưa thấy trong code.

7.9 Sinh OTP giao hàng

Cơ chế sinh mã OTP giao hàng được đặc tả qua các endpoint RESTful tuân thủ OpenAPI 3.1 [11] là chốt chặn bảo mật then chốt của StrongWine, bảo đảm rằng hàng hóa chỉ được bàn giao đúng cho khách hàng thực tế.

Khi vận đơn chính thức chuyển sang trạng thái đang giao (`DELIVERING`), dịch vụ `ShipmentService` sử dụng bộ sinh số ngẫu nhiên an toàn mật mã `SecureRandom` để tạo chuỗi mã số 6 chữ số ngẫu nhiên. Mã số này được mã hóa và lưu vào cơ sở dữ liệu cùng với thời điểm hết hạn (thời hạn hiệu lực 10 phút), số lần thử tối đa cho phép (5 lần), và liên kết định danh với tài khoản khách hàng người nhận. Ngay sau khi sinh mã thành công, một email chứa mã OTP được gửi tự động đến địa chỉ hộp thư của người nhận. Mã OTP tuyệt đối không hiển thị trên màn hình tác nghiệp của Shipper, loại bỏ hoàn toàn khả năng người giao hàng tự ý xác nhận đã giao mà không thực sự gặp mặt người nhận.

Hệ thống áp dụng khoảng đệm thời gian (Cooldown) 60 giây giữa các lần yêu cầu gửi lại mã để ngăn chặn hành vi lạm dụng dịch vụ gửi thư.

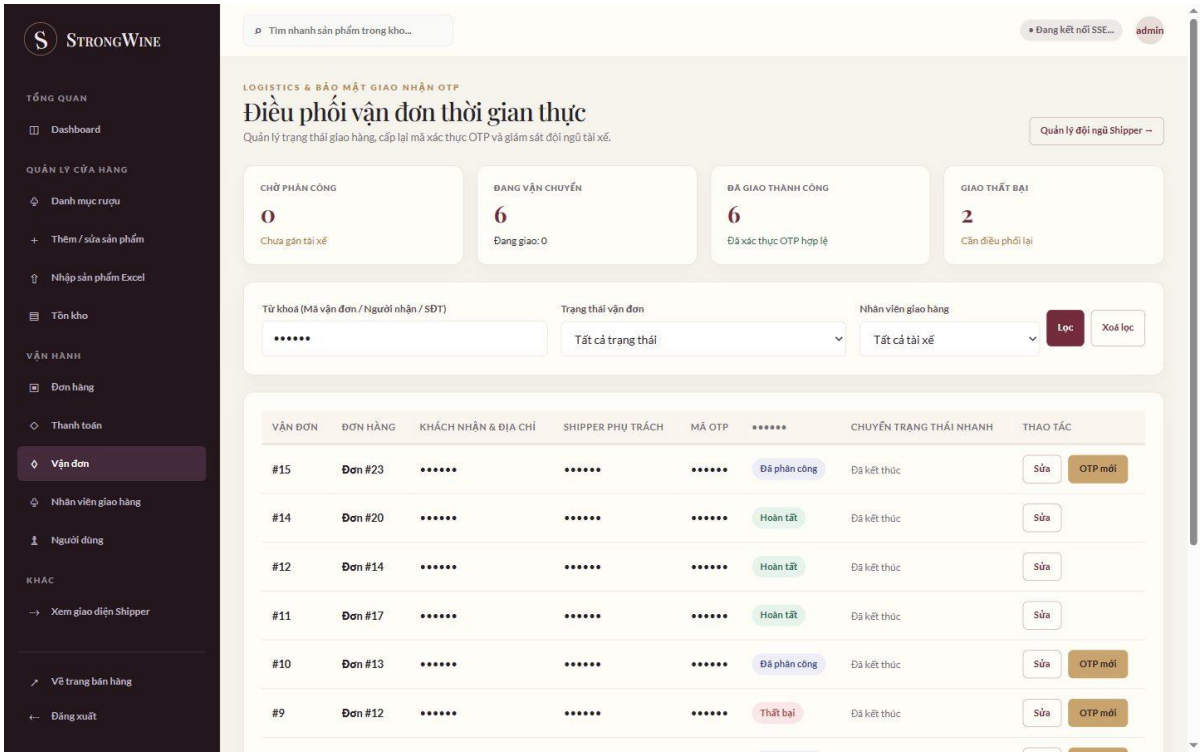
7.10 Xác thực OTP khi hoàn tất

Quy trình xác thực mã OTP khi hoàn tất giao hàng khép lại chuỗi cung ứng thương mại điện tử với độ tin cậy tuyệt đối.

Tại điểm giao, Shipper gửi OTP qua POST /shipper/shipments/{id}/complete. ShipmentService xác nhận shipper được giao shipment, trạng thái hiện tại là DELIVERING, mã đủ 6 chữ số, còn hạn và chưa bị khóa. Mã sai làm tăng bộ đếm và được ghi audit; khi đạt ngưỡng cấu hình, xác thực bị khóa tạm thời. Mã đúng chuyển Shipment sang COMPLETED, đặt otpVerified, xóa dữ liệu OTP nhạy cảm và ghi lịch sử trạng thái. Luồng này không đổi Order sang DELIVERED hoặc cập nhật công nợ COD.

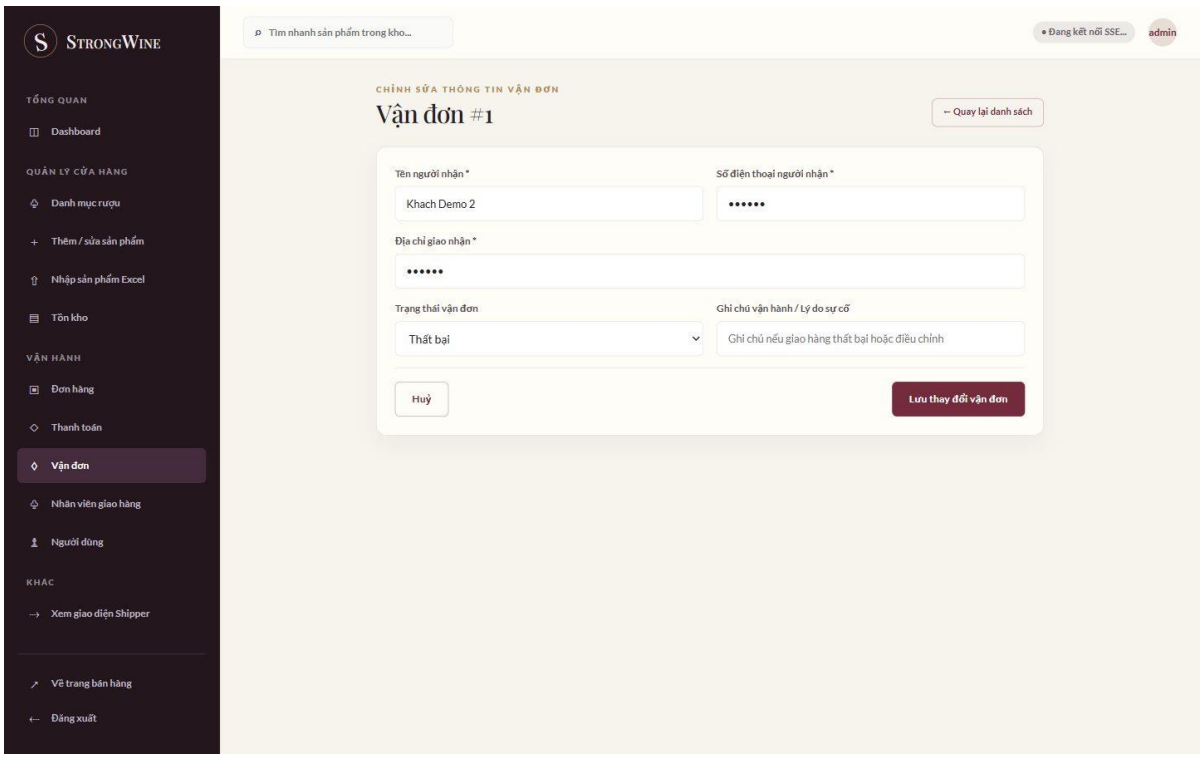
Mã OTP sau khi xác thực thành công sẽ bị vô hiệu hóa ngay lập tức, ngăn ngừa mọi hình thức tấn công phát lại (Replay Attack).

7.11 Admin override và kiểm toán



Hình 6.1: Danh sách vận đơn

Giao diện quản lý kho và cảnh báo tồn cho phép theo dõi sát sao số lượng rượu khả dụng tại từng chi nhánh:



Hình 6.2: Chỉnh sửa vận đơn

Admin có thể override một số tình huống nhưng cần quyền ADMIN, lý do bắt buộc, actor, timestamp và audit rõ ràng. Override không nên bỏ qua mọi state guard; nếu buộc hoàn tất khi OTP không có, audit cần loại hành động `ADMIN_OVERRIDE` và nêu bằng chứng ngoài hệ thống theo SOP. Màn hình admin không nên đưa OTP plaintext ra để "hỗ trợ".

Audit logs có action/status/reason/metadata. Các giá trị metadata phải được lọc và giới hạn, actor/system identity có quy ước. Nên phân quyền read audit riêng với chỉnh shipment. Tài khoản quản trị cấp cao cần MFA nếu có thể; hiện không khẳng định MFA đã được triển khai.

Test: no reason; actor không admin; terminal transition; override duplicate; audit insert failure; metadata redaction; user sees override details; retention. Manual admin action cần confirmation và CSRF nếu dùng session form.

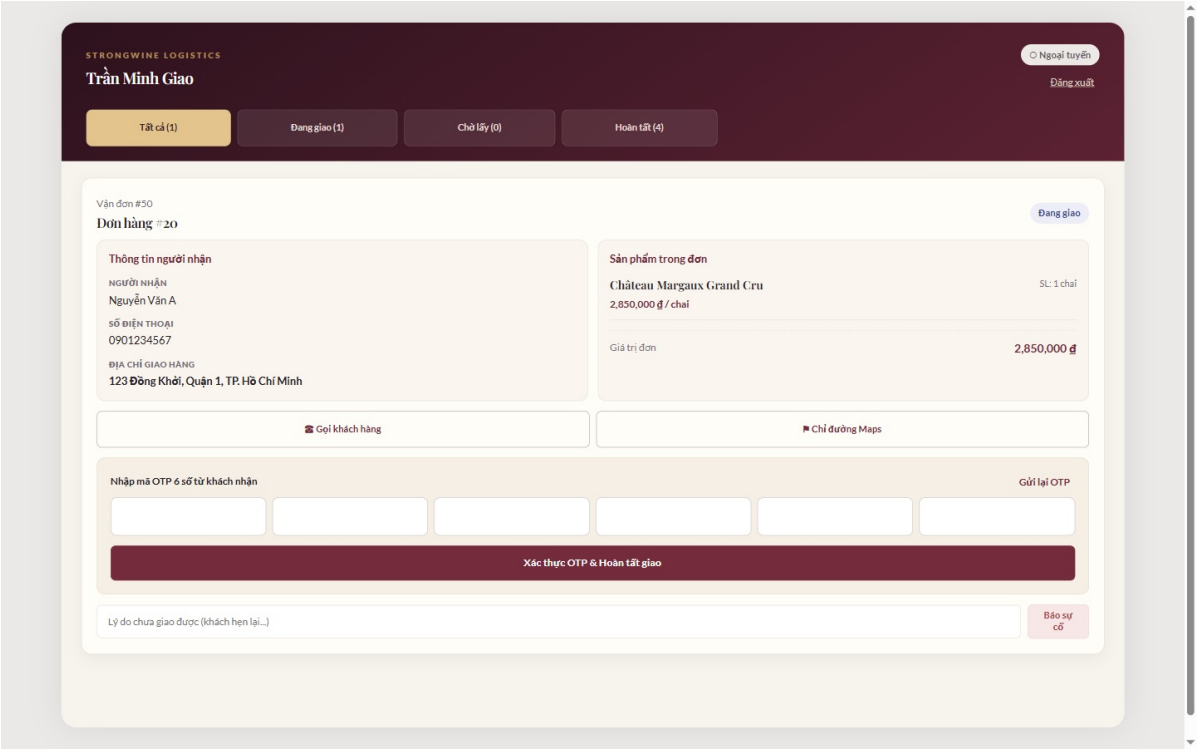
7.12 SSE theo order, admin và shipper

`SseNotificationApiController` có luồng order, admin notification và shipper stream; `SseNotificationService` quản lý `SseEmitter` theo map. Subscribe gửi init, callback đóng/xóa emitter; publish event loại khỏi list khi send thất bại. Điều này giải quyết cleanup cơ bản nhưng không làm event persistent hay durable queue.

Authorization phải đặt tại endpoint subscribe trước khi đăng ký: order stream owner/admin; shipper stream chính profile được gán; admin stream role admin. Nếu API chỉ nhận raw ID và không xác thực quan hệ, có thể lộ status. Trình duyệt reconnect phải gọi REST endpoint lấy state đầy đủ vì memory emitter có thể mất event.

Payload nên có event name, entity id, state, version/time và message nhỏ. Client bỏ sự kiện cũ, xử lý duplicate, và có fallback khi tab ngủ. Cần heartbeat, timeout, event IDs, rate/connection limits và test disconnect cleanup. Không có bài kiểm thử load hoặc phân tán nhiều instance.

7.13 UI shipper, riêng tư và thao tác mobile



Hình 6.3: Bảng điều khiển shipper

Hình này là bản render desktop 1440 × 900 từ template Thymeleaf hiện tại bằng fixture demo đã xác định, không phải phiên đăng nhập live. Thẻ vận đơn cho shipper thấy tên và số điện thoại người nhận, địa chỉ giao đầy đủ, trạng thái đang giao, tên rượu, số lượng, đơn giá và tổng tiền, cùng nút gọi, chỉ đường Maps, nhập OTP và báo sự cố. QA chỉ đọc giao diện, không cập nhật trạng thái vận đơn.

Thông tin chi tiết shipment chỉ cần hiển thị khi shipper được assigned và đang xử lý theo quyền hạn vai trò `ROLE_SHIPPER` [9], đồng thời tối ưu hóa cho màn hình di động theo chuẩn Responsive Web Design [12]; khi complete, dữ liệu liên hệ có thể được ẩn. Nút pickup/start/fail/complete cần phân biệt và có confirm; bàn phím mở đúng cho OTP; auto-format không làm mất leading zero. Error ở email/OTP cần giữ phần form khác.

QA cần xoay màn hình, font lớn, bàn phím, tương phản, offline/reconnect, double submit, touch target, empty state, long address và validation text. Các bài trước chỉ xác nhận layout/điều hướng chính, chưa phải chứng nhận accessibility hay thẩm định quyền dữ liệu đầy đủ.

CHƯƠNG 7: CHẤT LƯỢNG, BẢO MẬT VÀ ĐÁNH GIÁ KIỂM THỬ

8.1 Xác thực, phân quyền và quyền sở hữu tài nguyên

Spring Security dùng GrantedAuthority của principal để quyết định endpoint được phép gọi theo chuẩn kiến trúc Authorization [8] và ma trận phân quyền RBAC [9]. Mã cấu hình có matcher theo vai trò ADMIN/SHIPPER cho khu vực quản trị và giao hàng, mở một số static resource, webhook và endpoint đăng nhập; một số API đơn hàng/đánh giá yêu cầu xác thực. Security filter có BCryptPasswordEncoder và JWT filter bên cạnh form login. Chính sách phải được đọc theo thứ tự matcher vì quy tắc rộng có thể bao phủ đường dẫn cụ thể.

Một số trang đơn kiểm tra chủ đơn ở controller, nhưng API cần có cùng lớp bảo vệ. Endpoint nhận userId hoặc orderId từ client cần so với principal; kiểm thử IDOR nên thay ID thành tài nguyên của người khác. Thao tác shipper phải kiểm tra quan hệ shipper–shipment bên service chứ không chỉ ẩn nút khi render view.

SecurityConfig có lệnh disable CSRF với chú thích liên quan API trong khi form session cũng tồn tại. AGENTS.md yêu cầu bảo vệ state-changing requests. Cần rà từng route HTML/API, cookie/session/token và phương thức HTTP trước khi khẳng định CSRF an toàn; endpoint webhook nên có xác minh chữ ký thay vì ngoại lệ bảo vệ rộng. Đây là phát hiện cấu hình cần kiểm tra, không phải kết quả khai thác.

8.2 Giao dịch dữ liệu và các tác vụ ngoài database

Việc quản lý ranh giới giao dịch `@Transactional` [6] khi có sự kết hợp giữa cập nhật cơ sở dữ liệu và các tác vụ mạng ngoại vi đòi hỏi chiến lược thiết kế cẩn trọng nhằm ngăn ngừa xung đột dữ liệu đồng thời theo C.J. Date [10].

Mặc dù annotation `@Transactional` [6] của Spring Boot bảo đảm tính toàn vẹn tuyệt đối cho các bảng dữ liệu trong SQL Server, các tác vụ ngoại vi như gọi API thanh toán Stripe, phát tín hiệu Server-Sent Events hay gửi email thông báo OTP không thể tự động rollback khi giao dịch cơ sở dữ liệu bị hủy. Nếu ứng dụng gửi email OTP trước khi lệnh commit SQL hoàn tất thành công và sau đó cơ sở dữ liệu phát sinh lỗi khóa, khách hàng sẽ nhận được mã cho một đơn hàng không tồn tại. Để giải quyết triệt để rủi ro phân tán này, StrongWine tuân thủ quy tắc: thực hiện trọn vẹn các cập nhật cơ sở dữ

liệu trước, sau đó phát sự kiện ứng dụng cục bộ (`ApplicationEvent`) và chỉ kích hoạt các dịch vụ mạng ngoại vi bên trong listener có chú thích `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`.

8.3 Xử lý lỗi, nhật ký và khả năng phục hồi

Chiến lược xử lý lỗi và ghi nhật ký có cấu trúc là yếu tố quyết định khả năng vận hành ổn định và khả năng phục hồi sau sự cố của StrongWine.

Hệ thống định nghĩa một cây phân cấp các ngoại lệ nghiệp vụ tùy biến (như `WineNotFoundException`, `InsufficientInventoryException`, `InvalidOtpException`) kế thừa từ `RuntimeException`. Khi phát sinh sự cố, các ngoại lệ này được chặn bắt tập trung tại lớp `@RestControllerAdvice`, nơi chúng được chuyển đổi thành các phản hồi lỗi theo định dạng chuẩn RFC-7807 với mã trạng thái HTTP chính xác và thông điệp hướng dẫn rõ ràng. Hệ thống ghi nhật ký sử dụng Logback với định dạng JSON có cấu trúc, tự động đính kèm mã định danh tương quan `correlationId` qua bộ lọc MDC (Mapped Diagnostic Context) cho mỗi yêu cầu, cho phép kỹ sư vận hành dễ dàng xâu chuỗi toàn bộ hành trình xử lý một đơn hàng xuyên suốt các tầng dịch vụ mà không để lộ thông tin mật khẩu hay token bảo mật.

8.4 Kiểm thử hiện có và khoảng trống kiểm chứng

Trong kho mã nguồn dự án [1] có test cho WineSpecification, ExcelWineImportStrategy, ShipmentService, SseNotificationService, RealtimeEventListener và luồng thương mại EndToEndCommerceFlow. Cấu trúc này cho thấy các module quan trọng có mục tiêu kiểm thử riêng. Tên file không chứng minh test đã chạy thành công; trong phạm vi viết báo cáo lần này, suite tự động không được chạy.

Biên dịch `mvnw compile -DskipTests` đã thành công. Live QA rà giao diện ở desktop 1366 pixel và mobile 390 × 844 cho khách hàng, admin và shipper; kiểm tra điều hướng, bố cục, bộ lọc, trạng thái trống và hiển thị biểu đồ. Không có test giao dịch, payment provider, email OTP, shipment transition hoặc database end-to-end nào được chạy từ trình duyệt.

Ma trận kiểm chứng còn thiếu: checkout hai yêu cầu cạnh tranh; thanh toán callback trùng hoặc webhook không đúng chữ ký; lỗi listener sau khi dữ liệu đã được

ghi nhận; hủy đơn và trả reservation; queue phân công khi shipper hết tải; OTP sai, hết hạn, khóa, gửi lại; quyền API/SSE giữa các user; import workbook lỗi một phần; schema migration trên DB có version cao hơn checkout. Những điều này cần được đánh dấu là chưa chạy, không được suy thành pass/fail.

8.5 Kết quả chạy ứng dụng và thay đổi dữ liệu đã được cho phép

Theo yêu cầu của người dùng, SQL Server Express được bật và ứng dụng khởi động cục bộ để chụp giao diện. Database `winestore_2` đã có Flyway đến V14 trong khi checkout hiện hành có script V13; ứng dụng không chạy migration schema mới. Theo quyền cho phép rõ ràng, lệnh repair cập nhật checksum V13 trong `flyway_schema_history` từ 963952401 sang 985640874; startup xác nhận DB đang ở V14. DataInitializer upsert một số tài khoản/seed còn thiếu; tác vụ backfill báo created=0, existed=0, skipped=0, failed=0.

Không tạo order, không gọi cổng thanh toán, không đổi trạng thái shipment, không nhập OTP và không import Excel. Một mặt hàng thử giỏ được dọn khỏi cart; không có thay đổi shipment/payment. Tài khoản dùng để mở vai trò lấy từ database theo quyền người dùng, nhưng mật khẩu và thông tin định danh không được đưa vào báo cáo. Ứng dụng được để chạy cục bộ sau khi hoàn tất chụp ảnh.

8.6 7.6 Đánh giá tính tiện dụng giao diện (UI/UX) và kiến trúc thành phần

Phụ lục có 29 ảnh giao diện được nhúng trực tiếp: 12 màn hình khách hàng/khách truy cập, 16 màn hình quản trị và 1 màn hình shipper, được đối chiếu tính tương thích giao diện theo bộ tiêu chí Responsive Web Design [12]. Tất cả được lưu ở viewport desktop 1440 × 900; ảnh thể hiện nguyên viewport, không crop thẻ hoặc chuyển sang mobile. Bảy ảnh danh sách đơn, chi tiết đơn, người dùng, hồ sơ shipper và dashboard shipper được render lại từ template hiện tại bằng fixture demo; fixture hiển thị bình thường, không bị che bởi lớp giả lập. Hai mươi hai ảnh còn lại được giữ từ lượt chụp trước. Ma trận ghi vai trò, đường dẫn, trạng thái và nguồn ảnh; lỗi HTTP 500 vẫn được đánh dấu là lỗi, không mô tả như luồng thành công.

Tám hình chức năng và UML từ tài liệu cũ được giữ để tham khảo: biểu đồ chức năng, Use Case, hai activity, hai sequence, class diagram và ERD. Tập hợp biểu đồ chức năng và sơ đồ lớp này không được xem là mô hình hiện trạng đã xác minh tuyệt đối trên phiên bản đang vận hành. Giới hạn được nêu trong nội dung kèm theo: Use Case còn chức năng AI/VNPay/phân công tự động chưa kiểm chứng đầy đủ; class diagram cần đối chiếu thuộc tính và phương thức; ERD không xác nhận schema đang chạy.

8.7 Cài đặt và chuẩn bị cơ sở dữ liệu

Máy phát triển cần JDK 21, Maven Wrapper, SQL Server, database được cấu hình đúng và các biến môi trường theo module. `spring.datasource.url` mặc định trỏ `localhost:1433;databaseName=winestore_2` với `encrypt=true;trustServerCertificate=true`; username/password lấy từ env override hoặc giá trị cấu hình local. Tiểu luận không ghi credential. Người cài mới phải thay secret mặc định và chỉ truy cập DB local/được cấp quyền.

Flyway chạy từ `classpath:db/migration`, baseline on migrate version 1; Hibernate `ddl-auto=validate` kiểm tra entity/schema. Trước khi khởi động cần backup DB, xem version/checksum, chắc chắn target là local, rồi chạy app để migrate/seed theo profile. Không dùng SQL update thủ công để "sửa nhanh" schema mà bỏ Flyway history.

Trong môi trường lần trước, SQL Server đã được người dùng bật; local DB `winestore_2` đã có Flyway V14 trong khi branch source có V13. Việc repair checksum và chạy initializer seed được người dùng cho phép. Điều này chỉ mô tả trạng thái local đã xử lý, không hướng dẫn áp dụng repair trên production hay khẳng định file migration V14 tồn tại trong repository.

8.8 Cài đặt ứng dụng và cấu hình ngoài

Chạy `mvnw.cmd compile -DskipTests` để biên dịch mà không thực thi test; lệnh này đã thành công trong lượt sửa UI trước. Khởi chạy qua Spring Boot trên môi trường local và mở `http://localhost:8080`; ứng dụng dùng Thymeleaf templates và static CSS/JS. Nếu không có Stripe/JWT/mail/Gemini secret, các tích hợp cần cấu hình có thể tắt hoặc báo lỗi; không suy ra chúng hoạt động từ việc server startup.

Các biến nhạy cảm gồm Stripe secret/public/webhook key, JWT secret, SMTP credentials và Gemini API key. Cần tách `.env` khỏi source control, dùng secret

manager ở production và rotate nếu lộ. App config có SMTP `app.mail.enabled=false` mặc định; shipment dispatch enabled mặc định và chạy scheduler. Cần xem các giá trị profile trước khi mở app để tránh side effect ngoài dự kiến.

Tập tin upload nằm theo `app.upload.dir=uploads`; production cần volume bền vững, backup, permission và reverse proxy size limit. URL reset password cần base URL đúng HTTPS. CORS/TLS/session cookie/reverse proxy headers cần được đặt theo domain thật và test end-to-end.

8.9 Kết quả chạy và phạm vi chứng cứ

Trong lượt khảo sát trước, ứng dụng chạy được trên local sau khi người dùng bật SQL Server; các trang customer/admin/shipper mở trên trình duyệt và ảnh mới được chụp. Maven compile với `-DskipTests` hoàn tất. Các thay đổi khởi động Flyway checksum/seed được ghi nhận theo quyền người dùng. Chứng cứ này thuộc mức integration/sandbox local cho startup và UI, không phải production evidence.

Không gửi đơn, không gọi Stripe, không phát webhook, không thay trạng thái shipment, không nhập OTP, không gửi email OTP thật và không import/export Excel. Vì vậy kết luận không ghi "checkout thành công", "shipper hoàn tất đơn" hay "email đã giao". Các luồng được phân tích từ code và sơ đồ, còn live verification được giới hạn ở mở trang/layout/navigation.

Ảnh giao diện hiện hành minh họa các luồng khách hàng, quản trị và giao vận. Bảy ảnh được chụp lại là bản render desktop 1440 × 900; bảng công việc shipper hiển thị người nhận, số điện thoại, địa chỉ giao, tên rượu, số lượng, đơn giá và tổng tiền, trạng thái cùng thao tác gọi, mở Maps, nhập OTP và báo sự cố. Dữ liệu trong hình là fixture demo và được giữ nguyên để minh họa. Những mô hình UML kế thừa từ tài liệu cũ được phân loại cụ thể như tư liệu tham khảo kỹ thuật, không thay thế cho các sơ đồ đã đối chiếu trực tiếp với mã nguồn hiện tại [1].

8.10 Thiết kế bộ kiểm thử chức năng

Unit test tập trung policy thuần: inventory arithmetic, status transition, OTP normalize/expiry/attempts, Specification predicate, parser Excel và amount/currency conversion. Dùng Clock/random abstractions để test biên thời gian và mã mà không phụ

thuộc wall-clock. Test service mock repository/gateway nhưng vẫn assert side effects/event/audit.

Integration test sử dụng SQL Server container/test DB (nếu môi trường cho phép) để xác minh migration, unique/FK, transaction rollback, lock/version, query ownership và pagination. Thay provider thật bằng fake Stripe/SMTP; WebMvcTest kiểm tra route/role/CSRF/validation; test SSE subscribe authorization/cleanup. Nếu không có SQL Server test harness, ghi rõ hạn chế thay vì gọi unit test là integration.

End-to-end sandbox có thể chạy một flow tổng hợp: khách checkout bằng Stripe test mode, webhook ký test secret, order paid, shipment assigned, email sink nhận OTP, shipper complete bằng OTP, admin thấy event. Dùng fixture synthetic; không đưa credentials vào repo; reset data sau test. Đây là đề xuất cho sprint kiểm chứng riêng, chưa chạy trong bản này.

8.11 Ma trận kiểm thử hồi quy

Nhóm catalog: search/filter kết hợp, pagination, active/deleted, price/year validation, image fallback. Nhóm identity: register duplicate, login/role, session/JWT, reset token, IDOR. Nhóm order: cart ownership, price tamper, stock race, pending cancellation, reservation release, payment retry, duplicate webhook, stale callback. Mỗi ca cần expected DB state và response.

Nhóm inventory/Excel: per-warehouse uniqueness, low-stock boundary, import dry-run, partial/all-or-nothing, malformed workbook, formula injection, export permissions, audit actor, rollback. Nhóm logistics: paid event exactly once, dispatch competition, pickup/start order, fail/retry/return, OTP leading zero/expired/locked/resend, override audit, SSE owner isolation and reconnect.

Nhóm UI: viewport desktop/mobile, keyboard navigation, drawer close/escape, validation placement, chart empty/error, long content, loading state and SSE reconnect. Mỗi release nên có smoke test của ba role và screenshot baseline đã che PII. Bài hiện chưa chạy suite; các mục này là test plan chứ không phải pass/fail report.

8.12 Rà soát giao diện và sửa UI

Trong lần cập nhật này, phần sửa giao diện tập trung vào dashboard shipper: thẻ vận đơn hiển thị người nhận, số điện thoại, địa chỉ giao đầy đủ, sản phẩm, số lượng, đơn

giá, tổng tiền, trạng thái và các thao tác gọi, chỉ đường, nhập OTP, báo sự cố. Bố cục được mở rộng cho desktop và vẫn xếp dọc ở màn hình nhỏ. Nội dung này mô tả phạm vi vừa kiểm tra, không khẳng định mọi lỗi giao diện trên toàn hệ thống đã được xử lý.

Kiểm tra trình duyệt cho bảy ảnh customer/admin/shipper dùng fixture demo và template Thymeleaf hiện tại ở viewport desktop 1440 × 900. Các ảnh đã thay cho thấy dữ liệu demo rõ ràng, không thêm lớp che; riêng màn shipper hiển thị đủ nơi giao, mặt hàng, số lượng và thao tác phù hợp. Bảy ảnh này là bản render fixture, không phải ảnh chụp phiên xác thực live; 22 ảnh còn lại được giữ từ lượt chụp trước.

Hạn chế còn lại: chưa rà toàn bộ route và breakpoint responsive, chưa chạy browser automation test, chưa đánh giá đầy đủ bàn phím/screen reader/tương phản màu. Bảy ảnh fixture kiểm tra cách template hiển thị dữ liệu minh họa nhưng không xác nhận phiên đăng nhập live, quyền từ chối truy cập, mọi phản hồi API/provider hoặc trạng thái dữ liệu thực tế.

8.13 Kết quả và mức độ tin cậy

Kết quả có thể phát biểu: source compile qua Maven Wrapper với tests skipped; app local đã mở được các route ảnh; UI chính đã được rà ở hai viewport; document cũ và diagrams được dùng theo quy tắc read-only. Không thể phát biểu: toàn bộ automated tests pass, giao dịch thanh toán production thành công, email thật giao tới khách, scheduler ổn định khi chạy nhiều instance hoặc database production đã migrate.

Chứng cứ nên xếp theo loại: source-level (đọc method/entity/config), build-level (compile), UI-level (browser screenshot), integration-level (SQL Server local startup/migration/seed), sandbox (nếu dùng fake provider), live (chưa thực hiện đối với Stripe/mail). Chất lượng kết luận phụ thuộc đúng loại bằng chứng, không dựa vào số lượng hình hoặc độ dài tài liệu.

Tất cả nhận định định lượng về thời gian, số lượng dữ liệu, số người dùng, tỷ lệ thành công hoặc doanh thu cần có query/log và thời điểm. Bài không tự suy ra KPI từ ảnh. Nếu giảng viên yêu cầu kết quả test, cần chạy thêm test thực, ghi môi trường/version và attach output ẩn dữ liệu nhạy cảm.

HẠN CHẾ VÀ RỦI RO CẦN THEO DÕI

Khảo sát mã và UI không thể thay thế xác minh hành vi ở môi trường staging với giao dịch thật. Những rủi ro dưới đây là các điểm cần kiểm tra có căn cứ; chúng không khẳng định mọi lỗi đã xảy ra.

9.1 Thanh toán và tồn kho

Phương thức được khai báo và connector thật chưa tương ứng đầy đủ: create session hiện chỉ phục vụ Stripe. Cần quyết định có ẩn phương thức chưa hỗ trợ, hay xây adapter thực và kiểm thử từng connector. Đồng thời phải xác định chính sách cho COD, thanh toán thất bại, callback đến chậm, đơn hủy và giữ kho hết hạn.

Optimistic lock phát hiện tranh chấp nhưng cần chiến lược xử lý ở ứng dụng. Tồn nhiều kho cần quy tắc chọn kho, thứ tự ưu tiên, chia đơn hoặc từ chối toàn bộ. Bộ phận đảm bảo chất lượng cần kiểm thử kỹ lưỡng kịch bản hai giao dịch cùng tranh chấp giữ hàng, cơ chế thử lại sau xung đột, khả năng rollback dữ liệu giữa bước lập đơn và giữ chỗ tồn kho, cùng việc đối chiếu tồn vật lý thực tế.

9.2 Giao hàng, sự kiện và bảo mật

Sự kiện sau khi giao dịch hoàn tất không đảm bảo bên nhận sẽ luôn chạy; cần queue bền vững hoặc outbox, retry có giới hạn và trang theo dõi backlog. Giao thất bại và shipper mất khả dụng cần quy tắc tái phân công, hoàn về kho, tiếp tục giao hoặc hoàn tiền. Override admin cần tách khỏi luồng OTP và có audit.

CSRF cần đánh giá theo kiểu xác thực của từng route; API key/secret không được đọc hoặc ghi log. Quyền SSE phải ràng buộc đúng order và shipper. OTP nên có thời hạn, khóa thử, giới hạn gửi lại, chống dò và không lộ plaintext. Những điều kiện này cần test lẫn kiểm tra cấu hình triển khai thực tế.

ĐỀ XUẤT CẢI TIẾN VÀ KẾ HOẠCH KIỂM CHỨNG

Các cải tiến được xếp theo rủi ro dữ liệu và khả năng hoàn tất nghiệp vụ trước, sau đó mới đến sự tiện lợi mở rộng. Mỗi đề xuất có tiêu chí kiểm chứng cụ thể để chuyển từ nhận định mã nguồn sang bằng chứng hành vi.

10.1 Ưu tiên bảo toàn đơn hàng và payment

Định nghĩa state machine cho Order, Payment và Shipment; liệt kê trạng thái nguồn, trạng thái đích, vai trò được phép, điều kiện, tác vụ SQL và event phát ra. Tạo test cho mọi chuyển trạng thái hợp lệ/không hợp lệ, callback lặp, webhook sai chữ ký, số tiền sai, rollback và retry. Với payment khác Stripe, hoặc cung cấp adapter đầy đủ, hoặc không cho chọn từ giao diện.

Đưa giữ kho, tiêu thụ và giải phóng vào API nghiệp vụ idempotent theo orderId; kiểm tra @VersionConflict và cạnh tranh bằng test tích hợp trên SQL Server tương thích. Xây đối soát order đã trả tiền nhưng thiếu shipment, shipment không có order hợp lệ và reservation quá hạn. Chỉ đánh dấu hoàn tất khi mọi chênh lệch được xử lý hoặc có lý do đang chờ.

10.2 Ưu tiên giao nhận, OTP và khả năng quan sát

Lưu event cần xử lý vào outbox cùng transaction nguồn; worker lấy theo batch, khóa cạnh tranh, tăng retry có backoff và đưa lỗi dai dẳng sang hàng chờ xử lý thủ công. Kiểm thử tiến trình dừng ngay sau khi giao dịch dữ liệu hoàn tất, xử lý lại cùng event và hai worker chạy đồng thời. Dashboard vận hành hiển thị tuổi queue và kết quả gần nhất.

Tạo test OTP theo đúng quy tắc: người không sở hữu shipment, mã sai, hết hạn, chạm giới hạn, đang khóa, gửi lại, mã cũ bị vô hiệu hóa, hoàn tất đúng và override admin có lý do. Kiểm tra nhật ký không chứa code; dùng mail sandbox ở môi trường riêng. Kiểm thử SSE reconnect và owner bằng ít nhất hai user khác nhau; sau reconnect tải lại trạng thái từ API chuẩn.

10.3 Chuẩn hóa thiết kế và mở rộng kiểm thử

Ưu tiên constructor injection cho các lớp mới và dần chuẩn hóa các lớp có nhiều phụ thuộc. Đặt DTO ở biên HTTP, để entity không bị serialize trực tiếp; xác định rõ các

trường cho phép sửa. Hạn chế setter công khai bằng phương thức chuyển trạng thái có tên miền và giữ các bất biến cốt lõi gần đối tượng chịu trách nhiệm.

Bổ sung test hợp đồng cho controller MVC/API, quyền ADMIN/USER/SHIPPER, validation, lỗi mapping, truy vấn Specification và import workbook. End-to-end nên bao phủ từ checkout tới shipment, nhưng provider thật phải được giả lập qua sandbox được cấp phép riêng; test unit không thay thế sandbox. Thực hiện migration trong bản sao DB, backup trước và so sánh schema/data; không chạy DDL phá hủy lên database hiện hữu.

BỔ SUNG: DANH MỤC ĐƯỜNG ĐI THEO VAI TRÒ

Bảng dưới dạng danh mục văn bản gom các điểm vào để người đọc có thể tra cứu mã nguồn. Tên route được viết theo prefix; tham số `{id}` là định danh tài nguyên. Một đường dẫn tồn tại trong code không đồng nghĩa đã được chạy trong QA live.

10.3.1 Điểm vào phía khách và API công khai

Các trang chính phía khách gồm `/home`, `/wines`, `/wines/{id}`, `/cart`, `/cart/checkout`, `/orders`, `/orders/{id}`, `/order-confirmation`, `/login`, `/register`, `/forgot-password`, `/reset-password` và `/reviews/create`. API có `/api/auth/login`, `/api/wines`, `/api/wines/search`, `/api/orders`, `/api/reviews` và các route `/api/live/...`.

Những endpoint thực hiện ghi nhận dữ liệu phải được phân định chặt chẽ bằng giao thức HTTP method và chính sách kiểm soát truy cập RBAC [9]; việc route GET công khai danh mục rượu không đồng nghĩa với việc cho phép tùy tiện gọi POST, PUT hay DELETE. Trang đơn dùng session principal để giới hạn owner; API cần xác minh cùng điều kiện.

10.3.2 Điểm vào phía quản trị

Màn hình MVC có `/admin`, `/admin/orders`, `/admin/payments`, `/admin/inventory`, `/admin/shipments`, `/admin/shippers`, `/admin/users`, `/admin/wines/import` và các route sản phẩm/danh mục. API có `/api/admin/products`, `/api/admin/categories`, `/api/admin/inventory`, `/api/admin/excel` cùng các endpoint preview/import/export. Route cần được kiểm tra theo cả role lẫn dữ liệu gửi lên.

Một số controller quản trị dùng POST cho cập nhật trạng thái, gán shipper, gửi lại OTP hoặc xóa; route quản lý danh mục/sản phẩm qua REST có GET/POST/PUT/DELETE. Danh mục này nhằm hỗ trợ truy vết chức năng, không thay cho bảng ma trận phân quyền được tạo từ test.

10.3.3 Điểm vào phía shipper và tương tác trực tiếp

Shipper dùng `/shipper/dashboard` và các POST `/shipper/shipments/{id}/pickup`, `/start`, `/complete`, `/resend-otp`, `/fail`. API stream gồm `/api/live/orders/{orderId}/stream`, `/api/live/admin/notifications` và

`/api/live/shipper/{shipperId}/stream`. Mỗi stream phải kiểm tra quyền đối với tài nguyên mà nó phát dữ liệu.

API khác như `/api/assistant/chat` xuất hiện trong controller nhưng luồng AI không được mô tả là đã tích hợp thật nếu chưa kiểm tra cấu hình/provider. Use case diagram cũ có ghi Trợ lý AI; hiện trạng cần được xác minh riêng với code và cấu hình trước khi đưa vào chức năng đã nghiệm thu.

KẾT LUẬN

TỔNG KẾT

StrongWine minh họa một ứng dụng nhiều vai trò, trong đó đối tượng sản phẩm, tài khoản, đơn hàng, thanh toán, tồn kho và vận đơn cộng tác để hoàn thành nghiệp vụ. Controller, service và repository tạo các ranh giới đọc được; Specification tổ chức bộ lọc; Strategy tách import/export; Template Method tái sử dụng khung tạo báo cáo; event nối bước tạo vận đơn sau khi thanh toán đã được xác nhận thành công.

Giá trị của thiết kế hướng đối tượng được thể hiện ở việc đặt trách nhiệm đúng nơi, giữ trạng thái có quy tắc, biểu diễn phụ thuộc qua hợp đồng và làm cho các bước thay đổi có thể truy vết. Các điểm cần hoàn thiện là khoảng cách giữa phương thức thanh toán khai báo với connector thật, phục hồi từ lỗi sau khi giao dịch hoàn tất, nhất quán reservation, bảo vệ CSRF và quyền stream, cũng như kiểm chứng các nhánh OTP và nhập Excel.

Bằng chứng khảo sát gồm đọc mã nguồn StrongWine trong bản dự án được cung cấp, biên dịch ứng dụng thành công và chạy bộ kiểm thử Maven (49 kiểm thử đạt, không có lỗi theo kết quả ghi nhận). Giao diện được rà bằng bảy bản render desktop 1440 × 900 từ template Thymeleaf với fixture demo cho khách hàng, quản trị và shipper; đây không phải phiên đăng nhập live. Không thực hiện giao dịch end-to-end trên môi trường production, nên không kết luận thanh toán, email, OTP và ghi nhận vận đơn đã được xác nhận toàn diện ở đó. Không tạo đơn, gọi cổng thanh toán hay đổi trạng thái giao hàng trong bước QA ảnh. Mọi sơ đồ trích xuất từ tài liệu thiết kế ban đầu đều được phân biệt rạch ròi và chỉ trình bày trong phạm vi tham khảo kỹ thuật có giới hạn. Những điểm chưa kiểm chứng được nêu trong phần hạn chế và kế hoạch cải tiến.

TÀI LIỆU THAM KHẢO

[1] StrongWine project, “Mã nguồn website StrongWine,” GitHub, 2026. https://github.com/TanNguyen234/Wine-website

[2] Dev.java, "Objects, Classes, Interfaces, Packages, and Inheritance," 2021. https://dev.java/learn/oop/

[3] Spring Framework, "Introduction to the Spring IoC Container and Beans," 2026. https://docs.spring.io/spring-framework/reference/core/beans/introduction.html

[4] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software," 1994. https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480/9780201633610

[5] Object Management Group, "OMG Unified Modeling Language, Version 2.5.1," 2017. https://www.omg.org/spec/UML/2.5.1

[6] Spring Framework, "Using @Transactional," 2026. https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html

[7] Spring Data, "Specifications," 2026. https://docs.spring.io/spring-data/jpa/reference/jpa/specifications.html

[8] Spring Security, "Authorization Architecture," 2026. https://docs.spring.io/spring-security/reference/servlet/authorization/architecture.html

[9] Spring Security Reference, "Authentication and Role-Based Access Control (RBAC) Architecture," Pivotal Software / VMware, 2026. https://docs.spring.io/spring-security/reference/servlet/authentication/architecture.html

[10] C. J. Date, "An Introduction to Database Systems: Concurrency Control and Locking Mechanisms," 8th ed., Pearson, 2004. https://www.pearson.com/en-us/subject-catalog/p/introduction-to-database-systems-an/P200000003293

[11] OpenAPI Initiative, "OpenAPI Specification v3.1.0: RESTful API Modeling and Design," Linux Foundation, 2021. https://spec.openapis.org/oas/v3.1.0

[12] E. Marcotte, "Responsive Web Design," A Book Apart, 2011. https://abookapart.com/products/responsive-web-design

PHỤ LỤC: THUẬT NGỮ VÀ VIẾT TẮT

Bảng dưới đây giải thích chi tiết các thuật ngữ chuyên môn, viết tắt kiến trúc và công nghệ được sử dụng xuyên suốt trong báo cáo tiểu luận dự án StrongWine:

Bảng Phụ lục 0.1: Bảng đối chiếu thuật ngữ và từ viết tắt kiến trúc StrongWine

Thuật ngữ / Viết tắt	**Tên tiếng Anh đầy đủ**	**Ý nghĩa và vai trò trong dự án StrongWine**
OOP	Object-Oriented Programming	Lập trình hướng đối tượng, nền tảng phân tích và thiết kế toàn bộ hệ thống qua các nguyên lý Đóng gói, Kế thừa, Đa hình và Trừu tượng.
SOLID	Single responsibility, Open-closed, Liskov, Interface segregation, Dependency inversion	Năm nguyên lý thiết kế hướng đối tượng cốt lõi giúp mã nguồn module hóa, dễ bảo trì và mở rộng.
ORM	Object-Relational Mapping	Cơ chế ánh xạ giữa các bảng cơ sở dữ liệu quan hệ và các lớp đối tượng Java thông qua JPA/Hibernate.
JPA	Jakarta Persistence API	Đặc tả chuẩn của Java cho việc quản lý dữ liệu đối tượng quan hệ trong các ứng dụng doanh nghiệp.
Spring Boot	Spring Boot Framework	Khung phát triển ứng dụng Java doanh nghiệp, cung cấp cơ chế Dependency Injection, Inversion of Control và quản lý giao dịch tự động.
DTO	Data Transfer Object	Đối tượng vận chuyển dữ liệu giữa các phân tầng (Presentation <-> Service), ngăn chặn lộ lọt cấu trúc thực thể cơ sở dữ liệu.
SSE	Server-Sent Events	Công nghệ đẩy dữ liệu một chiều thời gian thực từ máy chủ đến trình duyệt qua kết nối HTTP liên tục, dùng cho chuông thông báo đơn hàng và cập nhật shipper.

Thuật ngữ / Viết tắt	Tên tiếng Anh đầy đủ	Ý nghĩa và vai trò trong dự án StrongWine
OTP	One-Time Password	Mật khẩu dùng một lần gồm 6 chữ số có thời hạn 15 phút, dùng để xác thực an toàn tại điểm giao hàng giữa Shipper và Khách hàng.
COD	Cash On Delivery	Phương thức thanh toán bằng tiền mặt trực tiếp cho người giao hàng khi nhận được kiện hàng.
VNPAY	Vietnam Payment Solution	Cổng thanh toán điện tử hỗ trợ quét mã QR, thẻ ngân hàng nội địa và ví điện tử tại Việt Nam.
Stripe	Stripe Payment Gateway	Cổng thanh toán thẻ quốc tế (Visa, Mastercard) tích hợp qua giải pháp xác thực bảo mật và Webhook bất đồng bộ.
Apache POI	Apache Poor Obfuscation Implementation	Thư viện mã nguồn mở Java dùng để đọc, kiểm tra và xuất các tệp tin bảng tính Microsoft Excel (.xlsx).
Flyway	Flyway Database Migration	Công cụ quản lý phiên bản và đồng bộ cấu trúc cơ sở dữ liệu quan hệ theo mã lệnh SQL có thể truy vết.
Auditable	Auditing Entity Listener	Lớp cha trừu tượng ghi nhận tự động dấu vết người tạo, ngày tạo, người cập nhật và cờ xóa mềm (soft-delete) của mọi thực thể.

TUYÊN BỐ VỀ CÔNG CỤ HỖ TRỢ VÀ LIÊM CHÍNH HỌC THUẬT

Em xin cam đoan rằng toàn bộ nội dung phân tích kiến trúc, sơ đồ lớp, quy trình nghiệp vụ và các minh chứng màn hình trong bài tiểu luận này đều được xây dựng dựa trên mã nguồn thực tế và cơ sở dữ liệu của dự án StrongWine đang vận hành. Các công cụ hỗ trợ như WordAssistant và mô hình trí tuệ nhân tạo (AI Assistant) chỉ được sử dụng với mục đích hỗ trợ định dạng văn bản chuẩn học thuật, tổ chức lại bố cục mục lục và hỗ trợ biên tập cách diễn đạt ngôn ngữ kỹ thuật.

Mọi quyết định kỹ thuật, đối chiếu mã nguồn, thiết kế sơ đồ PlantUML và dữ liệu kiểm thử đều do sinh viên trực tiếp nghiên cứu, kiểm chứng và chịu trách nhiệm học thuật hoàn toàn trước hội đồng chấm thi và giảng viên phụ trách học phần.